



Spring Cloud Data Flow Reference Guide

Sabby Anandan • Marius Bogoevici • Eric Bottard • Mark Fisher • Ilayaperumal Gopinathan • Gunnar Hillert • Mark Pollack
• Patrick Peralta • Glenn Renfro • Thomas Risberg • Dave Syer • David Turanski • Janne Valkealahti • Oleg Zhurakousky
• Jay Bryant • Vinicius Carvalho • Chris Schaefer • Damien Vitrac • Christian Tzolov

Preface

1. About the documentation
2. Getting help

Getting Started

3. Getting Started - Local
4. Getting Started - Cloud Foundry
5. Getting Started - Kubernetes
 - 5.1. Application and Server Properties
 - 5.1.1. Memory and CPU Settings
 - 5.1.2. Environment Variables
 - 5.1.3. Liveness and Readiness Probes
 - 5.1.4. Using `SPRING_APPLICATION_JSON`
 - 5.1.5. Private Docker Registry
 - 5.1.6. Annotations
 - 5.1.7. Entry Point Style
 - 5.1.8. Deployment Service Account
 - 5.1.9. Image Pull Policy
 - 5.1.10. Deployment Labels
 - 5.1.11. Tolerations
 - 5.1.12. Secret Key References
 - 5.1.13. Config Map Key References
 - 5.1.14. Pod Security Context
 - 5.1.15. Service Ports
 - 5.1.16. StatefulSet Init Container
 - 5.1.17. Init Containers

Applications

6. Available Applications

Architecture

7. Introduction

Configuration

8. Maven

8.1. Wagon

9. Configuration - Local

9.1. Feature Toggles

9.2. Database

9.2.1. MySQL

9.2.2. MariaDB

9.2.3. PostgreSQL

9.2.4. SQL Server

9.2.5. Db2

9.2.6. Oracle

9.2.7. Adding a Custom JDBC Driver

9.3. Deployer Properties

9.4. Logging

9.5. Streams

9.6. Tasks

9.7. Security

9.7.1. Enabling HTTPS

Using Self-Signed Certificates

Self-Signed Certificates and the Shell

9.7.2. Authentication using OAuth 2.0

9.7.3. Customizing Authorization

Authorization - Shell and Dashboard Behavior

Securing the Spring Boot Management Endpoints

9.7.4. Setting up UAA Authentication

Requirements

Prepare UAA for JWT

Download + Start UAA

Start Skipper

Start Spring Cloud Data Flow

Role Mappings

9.7.5. LDAP Authentication

LDAP Role Mapping

LDAP Security and UAA Example Application

9.7.6. Spring Security OAuth2 Resource/Authorization Server Sample

9.7.7. Data Flow Shell Authentication

9.8. About Configuration

9.8.1. Enabling Shell Checksum values

9.8.2. Reserved Values for URLs

10. Configuration - Cloud Foundry

10.1. Feature Toggles

10.2. Deployer Properties

10.3. Tasks

10.4. Application Names and Prefixes

10.5. Custom Routes

10.6. Docker Applications

10.7. Application-level Service Bindings

10.8. Configuring Service binding parameters

10.9. User-provided Services

10.10. Database Connection Pool

10.11. Maximum Disk Quota

10.11.1. PCF's Operations Manager

10.12. Scale Application

10.13. Managing Disk Use

10.14. Application Resolution Alternatives

10.15. Security

10.15.1. Authentication

Pivotal Single Sign-On Service

Cloud Foundry UAA

10.16. Configuration Reference

10.17. Debugging

10.18. Spring Cloud Config Server

10.18.1. Stream, Task, and Spring Cloud Config Server

10.18.2. Sample Manifest Template

10.18.3. Self-signed SSL Certificate and Spring Cloud Config Server

10.19. Configure Scheduling

11. Configuration - Kubernetes

11.1. Feature Toggles

11.2. Deployer Properties

11.3. Tasks

11.4. General Configuration

11.4.1. Using ConfigMap and Secrets

11.5. Database Configuration

11.6. Monitoring and Management

11.6.1. Inspecting Server Logs

11.6.2. Streams

11.6.3. Tasks

11.7. Scheduling

- 11.7.1. Entry Point Style

- 11.7.2. Environment Variables

- 11.7.3. Image Pull Policy

- 11.7.4. Private Docker Registry

- 11.7.5. Namespace

- 11.7.6. Service Account

- 11.8. Debug Support

- 11.8.1. Enabling Debugging Manually

- 11.8.2. Enabling Debugging with Patching

Shell

- 12. Shell Options

- 13. Listing Available Commands

- 14. Tab Completion

- 15. White Space and Quoting Rules

- 15.1. Quotes and Escaping

- 15.1.1. Shell rules

- 15.1.2. Property files rules

- 15.1.3. DSL Parsing Rules

- 15.1.4. SpEL Syntax and SpEL Literals

- 15.1.5. Putting It All Together

Streams

- 16. Introduction

- 16.1. Stream Pipeline DSL

- 16.2. Stream Application DSL

- 16.3. Application properties

- 17. Stream Lifecycle

- 17.1. Register a Stream App

- 17.1.1. Register Supported Applications and Tasks

- Spring Cloud Stream App Starters

- Spring Cloud Task App Starters

- 17.1.2. Whitelisting application properties

- 17.1.3. Creating and Using a Dedicated Metadata Artifact

- 17.1.4. Using the Companion Artifact

- 17.1.5. Creating Custom Applications

- 17.2. Creating a Stream

- 17.2.1. Application Properties

- 17.2.2. Common Application Properties

- 17.3. Deploying a Stream

- 17.3.1. Deployment Properties

- 17.4. Destroying a Stream
- 17.5. Undeploying a Stream
- 17.6. Validating a Stream
- 17.7. Updating a Stream
- 17.8. Force update of a Stream
- 17.9. Stream versions
- 17.10. Stream Manifests
- 17.11. Rollback a Stream
- 17.12. Application Count
- 17.13. Skipper's Upgrade Strategy

18. Stream DSL

- 18.1. Tap a Stream
- 18.2. Using Labels in a Stream
- 18.3. Named Destinations
- 18.4. Fan-in and Fan-out

19. Stream Java DSL

20. Stream Applications with Multiple Binder Configurations

21. Function Composition

22. Examples

- 22.1. Simple Stream Processing
- 22.2. Stateful Stream Processing
- 22.3. Other Source and Sink Application Types

Stream Developer Guide

Stream Monitoring

Tasks

- 23. Introduction
- 24. The Lifecycle of a Task
 - 24.1. Creating a Task Application
 - 24.1.1. Task Database Configuration
 - 24.2. Registering a Task Application
 - 24.3. Creating a Task Definition
 - 24.3.1. Automating the Creation of Task Definitions
 - 24.4. Launching a Task
 - 24.4.1. Application properties
 - Application Properties With Sensitive Information on Kubernetes
 - 24.4.2. Common application properties
 - 24.5. Limit the number concurrent task launches
 - 24.6. Reviewing Task Executions
 - 24.7. Destroying a Task Definition

24.8. Validating a Task

24.9. Stopping a Task Execution

24.9.1. Stopping a Task Execution that was started outside of Spring Cloud Data Flow

25. Subscribing to Task/Batch Events

26. Composed Tasks

26.1. Configuring the Composed Task Runner

26.1.1. Registering the Composed Task Runner

26.1.2. Configuring the Composed Task Runner

Configuration Options

26.2. The Lifecycle of a Composed Task

26.2.1. Creating a Composed Task

Task Application Parameters

26.2.2. Launching a Composed Task

Passing properties to the child tasks

Passing arguments to the composed task runner

Launching a Composed Task using Custom Composed Task Runner

Exit Statuses

26.2.3. Destroying a Composed Task

26.2.4. Stopping a Composed Task

26.2.5. Restarting a Composed Task

27. Composed Tasks DSL

27.1. Conditional Execution

27.2. Transitional Execution

27.2.1. Basic Transition

27.2.2. Transition With a Wildcard

27.2.3. Transition With a Following Conditional Execution

27.3. Split Execution

27.3.1. Split Containing Conditional Execution

27.3.2. Establishing the proper thread count for splits

28. Launching Tasks from a Stream

28.1. Launching a Composed Task From a Stream

29. Sharing Spring Cloud Data Flow's Datastore with Tasks

29.1. A Common DataStore Dependency

29.2. A Common Data Store

29.2.1. Simple Task Launch

29.2.2. Composed Task Runner

29.2.3. Launching a task externally from Spring Cloud Data Flow

30. Scheduling Tasks

30.1. The Scheduler

30.2. Enabling Scheduling

30.3. The Lifecycle of a Schedule

30.3.1. Scheduling a Task Execution

30.3.2. Deleting a Schedule

30.3.3. Listing Schedules

31. Continuous Deployment

31.1. Task Launch Lifecycle

31.1.1. Launch a Task With No Changes

31.1.2. Launch a Task With Changes That Is Not Currently Running

31.1.3. Launch a Task With Changes While Another Instance Is Running

Task Developer Guide

Task Monitoring

Dashboard

32. Introduction

33. Apps

33.1. Bulk Import of Applications

34. Runtime

35. Streams

35.1. Working with Stream Definitions

35.2. Creating a Stream

35.3. Deploying a Stream

35.4. Accessing Stream Logs

35.5. Creating Fan-In/Fan-Out Streams

35.6. Creating a Tap Stream

35.7. Import/Export Streams

36. Tasks

36.1. Apps

36.1.1. View Task App Details

36.1.2. Create a Task Definition

36.2. Definitions

36.2.1. Creating Composed Task Definitions

36.2.2. Launching Tasks

36.2.3. Import/Export Tasks

36.3. Executions

36.4. Execution Detail

36.4.1. Stop Executing Tasks

37. Jobs

37.1. Job Execution Details

37.2. Step Execution Details

37.3. Step Execution Progress

38. Scheduling

39. Auditing

Samples

40. Links

REST API Guide

41. Overview

41.1. HTTP verbs

41.2. HTTP Status Codes

41.3. Headers

41.4. Errors

41.5. Hypermedia

42. Resources

42.1. Index

42.1.1. Accessing the index

Request Structure

Example Request

Response Structure

Example Response

Links

42.2. Server Meta Information

42.2.1. Retrieving information about the server

Request Structure

Example Request

Response Structure

42.3. Registered Applications

42.3.1. Listing Applications

Request Structure

Request Parameters

Example Request

Response Structure

42.3.2. Getting Information on a Particular Application

Request Structure

Request Parameters

Path Parameters

Example Request

Response Structure

42.3.3. Registering a New Application

Request Structure

- Request Parameters

- Path Parameters

- Example Request

- Response Structure

42.3.4. Registering a New Application with version

- Request Structure

- Request Parameters

- Path Parameters

- Example Request

- Response Structure

42.3.5. Registering Applications in Bulk

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.3.6. Set the default application version

- Request Structure

- Path Parameters

- Example Request

- Response Structure

42.3.7. Unregistering an Application

- Request Structure

- Path Parameters

- Example Request

- Response Structure

42.3.8. Unregistering all Applications

- Request Structure

- Example Request

- Response Structure

42.4. Audit Records

42.4.1. List All Audit Records

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.4.2. Retrieve Audit Record Detail

- Request Structure

- Path Parameters

- Example Request

Response Structure

42.4.3. List all the Audit Action Types

Request Structure

Example Request

Response Structure

42.4.4. List all the Audit Operation Types

Request Structure

Example Request

Response Structure

42.5. Stream Definitions

42.5.1. Creating a New Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.5.2. List All Stream Definitions

Request Structure

Request Parameters

Example Request

Response Structure

42.5.3. List Related Stream Definitions

Request Structure

Request Parameters

Example Request

Response Structure

42.5.4. Retrieve Stream Definition Detail

Request Structure

Path Parameters

Example Request

Response Structure

42.5.5. Delete a Single Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.5.6. Delete All Stream Definitions

Request Structure

Request Parameters

Example Request

Response Structure

42.6. Stream Validation

42.6.1. Request Structure

42.6.2. Path Parameters

42.6.3. Example Request

42.6.4. Response Structure

42.7. Stream Deployments

42.7.1. Deploying Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.7.2. Undeploy Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.7.3. Undeploy All Stream Definitions

Request Structure

Request Parameters

Example Request

Response Structure

42.7.4. Update Deployed Stream

Request Structure

Request Parameters

Example Request

Response Structure

42.7.5. Rollback Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.7.6. Get Manifest

Request Structure

Request Parameters

Example Request

Response Structure

42.7.7. Get Deployment History

Request Structure

Example Request

Response Structure

42.7.8. Get Deployment Platforms

Request Structure

Example Request

Response Structure

42.7.9. Scale Stream Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.8. Task Definitions

42.8.1. Creating a New Task Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.8.2. List All Task Definitions

Request Structure

Request Parameters

Example Request

Response Structure

42.8.3. Retrieve Task Definition Detail

Request Structure

Request Parameters

Example Request

Response Structure

42.8.4. Delete Task Definition

Request Structure

Request Parameters

Example Request

Response Structure

42.9. Task Scheduler

42.9.1. Creating a New Task Schedule

Request Structure

Request Parameters

Example Request

Response Structure

42.9.2. List All Schedules

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.9.3. List Filtered Schedules

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.9.4. Delete Task Schedule

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.10. Task Validation

42.10.1. Request Structure

42.10.2. Path Parameters

42.10.3. Example Request

42.10.4. Response Structure

42.11. Task Executions

42.11.1. Launching a Task

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.11.2. Stopping a Task

- Request Structure

- Path Parameters

- Request Parameters

- Example Request

- Response Structure

42.11.3. List All Task Executions

- Request Structure

- Request Parameters

- Example Request

- Response Structure

42.11.4. List All Task Executions With a Specified Task Name

- Request Structure

- Request Parameters

Example Request

Response Structure

42.11.5. Task Execution Detail

Request Structure

Request Parameters

Example Request

Response Structure

42.11.6. Delete Task Execution

Request Structure

Request Parameters

Example Request

Response Structure

42.11.7. Deleting Task Execution Data

42.11.8. Task Execution Current Count

Request Structure

Request Parameters

Example Request

Response Structure

42.12. Job Executions

42.12.1. List All Job Executions

Request Structure

Request Parameters

Example Request

Response Structure

42.12.2. List All Job Executions Without Step Executions Included

Request Structure

Request Parameters

Example Request

Response Structure

42.12.3. List All Job Executions With a Specified Job Name

Request Structure

Request Parameters

Example Request

Response Structure

42.12.4. List All Job Executions With a Specified Job Name Without Step Executions Included

Request Structure

Request Parameters

Example Request

Response Structure

42.12.5. Job Execution Detail

Request Structure

Request Parameters

Example Request

Response Structure

42.12.6. Stop Job Execution

Request structure

Request parameters

Example request

Response structure

42.12.7. Restart Job Execution

Request Structure

Request Parameters

Example Request

Response Structure

42.13. Job Instances

42.13.1. List All Job Instances

Request Structure

Request Parameters

Example Request

Response Structure

42.13.2. Job Instance Detail

Request Structure

Request Parameters

Example Request

Response Structure

42.14. Job Step Executions

42.14.1. List All Step Executions For a Job Execution

Request Structure

Request Parameters

Example Request

Response Structure

42.14.2. Job Step Execution Detail

Request Structure

Request Parameters

Example Request

Response Structure

42.14.3. Job Step Execution Progress

Request Structure

- Request Parameters

- Example Request

- Response Structure

42.15. Runtime Information about Applications

42.15.1. Listing All Applications at Runtime

- Request Structure

- Example Request

- Response Structure

42.15.2. Querying All Instances of a Single App

- Request Structure

- Example Request

- Response Structure

42.15.3. Querying a Single Instance of a Single App

- Request Structure

- Example Request

- Response Structure

42.16. Stream Logs

42.16.1. Get the applications' logs by the stream name

- Request Structure

- Example Request

- Response Structure

42.16.2. Get the logs of a specific application from the stream

- Request Structure

- Example Request

- Response Structure

42.17. Task Logs

42.17.1. Get the task execution log

- Request Structure

- Request Parameters

- Example Request

- Response Structure

Appendices

Appendix A: Data Flow Template

A.1. Using the Data Flow Template

A.2. Data Flow Template and Security

Appendix B: "How-to" guides

B.1. Configure Maven Properties

B.2. Troubleshooting

B.3. Frequently Asked Questions

Appendix C: Building

C.1. Documentation

C.2. Working with the Code

C.2.1. Importing into Eclipse with m2eclipse

C.2.2. Importing into Eclipse without m2eclipse

Appendix D: Contributing

D.1. Sign the Contributor License Agreement

D.2. Code Conventions and Housekeeping

Version 2.4.0.RELEASE

© 2012-2020 Pivotal Software, Inc.

Copies of this document may be made for your own use and for distribution to others, provided that you do not charge any fee for such copies and further provided that each copy contains this Copyright Notice, whether distributed in print or electronically.

Preface

1. About the documentation

The documentation for this release is available in [HTML](#).

The latest copy of the Spring Cloud Data Flow reference guide can be found [here](#).

Copies of this document may be made for your own use and for distribution to others, provided that you do not charge any fee for such copies and further provided that each copy contains this Copyright Notice, whether distributed in print or electronically.

2. Getting help

Having trouble with Spring Cloud Data Flow, We would like to help!

- Ask a question. We monitor [stackoverflow.com](#) for questions tagged with `spring-cloud-dataflow`.
- Report bugs with Spring Cloud Data Flow at [github.com/spring-cloud/spring-cloud-dataflow/issues](#).
- Chat with the community and developers on [Gitter](#).

All of Spring Cloud Data Flow is open source, including the documentation! If you find problems with the docs or if you just want to improve them, please [get involved](#).

Getting Started

3. Getting Started - Local

Please see the [Local Machine](#) section of the microsite for more information on setting up docker compose and manual installation.

Once you have the Data Flow server installed locally, you will want to get started with orchestrating the deployment of readily available pre-built applications into a coherent streaming or batch data pipelines. Below we have guides on how to get started with both [Stream](#) and [Batch](#) processing.

4. Getting Started - Cloud Foundry

This section covers how to get started with Spring Cloud Data Flow on Cloud Foundry. Please see the [Cloud Foundry](#) section of the microsite for more information on installing Spring Cloud Data Flow on Cloud Foundry.

Once you have the Data Flow server installed on Cloud Foundry, you will want to get started with orchestrating the deployment of readily available pre-built applications into a coherent streaming or batch data pipelines. Below we have guides on how to get started with both [Stream](#) and [Batch](#) processing.

5. Getting Started - Kubernetes

[Spring Cloud Data Flow](#) is a toolkit for building data integration and real-time data-processing pipelines.

Pipelines consist of Spring Boot apps, built with the Spring Cloud Stream or Spring Cloud Task microservice frameworks. This makes Spring Cloud Data Flow suitable for a range of data-processing use cases, from import-export to event-streaming and predictive analytics.

This project provides support for using Spring Cloud Data Flow with Kubernetes as the runtime for these pipelines, with applications packaged as Docker images.

Please see the [Kubernetes](#) section of the microsite for more information on installing Spring Cloud Data Flow on Kubernetes.

Once you have the Data Flow server installed on Kubernetes, you will want to get started with orchestrating the deployment of readily available pre-built applications into a coherent streaming or batch data pipelines. Below we have guides on how to get started with both [Stream](#) and [Batch](#) processing.

5.1. Application and Server Properties

This section covers how you can customize the deployment of your applications. You can use a number of properties to influence settings for the applications that are deployed. Properties can be applied on a per-application basis or in the appropriate server configuration for all deployed applications.

Properties set on a per-application basis always take precedence over properties set as the server configuration. This arrangement lets you override global server level properties on a per-application basis.

Properties to be applied for all deployed Tasks are defined in the `src/kubernetes/server/server-config-(binder).yaml` file and for Streams in `src/kubernetes/skipper/skipper-config-(binder).yaml`. Replace `(binder)` with the messaging middleware you are using – for example, `rabbit` or `kafka`.

5.1.1. Memory and CPU Settings

Applications are deployed with default memory and CPU settings. If needed, these values can be adjusted. The following example shows how to set `Limits` to `1000m` for CPU and `1024Mi` for memory and `Requests` to `800m` for CPU and `640Mi` for memory:

```

deployer.<app>.kubernetes.limits.cpu=1000m
deployer.<app>.kubernetes.limits.memory=1024Mi
deployer.<app>.kubernetes.requests.cpu=800m
deployer.<app>.kubernetes.requests.memory=640Mi

```

Those values results in the following container settings being used:

```

Limits:
  cpu: 1
  memory: 1Gi
Requests:
  cpu: 800m
  memory: 640Mi

```

You can also control the default values to which to set the `cpu` and `memory` globally.

The following example shows how to set the CPU and memory for streams:

```

data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    limits:
                      memory: 640mi
                      cpu: 500m

```

YAML

The following example shows how to set the CPU and memory for tasks:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    limits:
                      memory: 640mi
                      cpu: 500m
```

The settings we have used so far only affect the settings for the container. They do not affect the memory setting for the JVM process in the container. If you would like to set JVM memory settings, you can provide an environment variable to do so. See the next section for details.

5.1.2. Environment Variables

To influence the environment settings for a given application, you can use the `spring.cloud.deployer.kubernetes.environmentVariables` `deployer` property. For example, a common requirement in production settings is to influence the JVM memory arguments. You can do so by using the `JAVA_TOOL_OPTIONS` environment variable, as the following example shows:

```
deployer.<app>.kubernetes.environmentVariables=JAVA_TOOL_OPTIONS=-Xmx1024m
```

The `environmentVariables` property accepts a comma-delimited string. If an environment variable contains a value which is also a comma-delimited string, it must be enclosed in single quotation marks — for example,

```
spring.cloud.deployer.kubernetes.environmentVariables=spring.cloud.stream.kafka.binder.brokers='somehost:9092,anotherhost:9093'
```

This overrides the JVM memory setting for the desired `<app>` (replace `<app>` with the name of your application).

5.1.3. Liveness and Readiness Probes

The `liveness` and `readiness` probes use paths called `/health` and `/info`, respectively. They use a `delay` of 10 for both and a `period` of 60 and 10 respectively. You can change these defaults when you deploy the stream by using `deployer` properties. Liveness and readiness probes are only applied to streams.

The following example changes the `liveness` probe (replace `<app>` with the name of your application) by setting `deployer` properties:

```
deployer.<app>.kubernetes.livenessProbePath=/health
deployer.<app>.kubernetes.livenessProbeDelay=120
deployer.<app>.kubernetes.livenessProbePeriod=20
```


You can declare the same as part of the server global configuration for streams, as the following example shows:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    livenessProbePath: /health
                    livenessProbeDelay: 120
                    livenessProbePeriod: 20
```

YAML

Similarly, you can swap `liveness` for `readiness` to override the default `readiness` settings.

By default, port 8080 is used as the probe port. You can change the defaults for both `liveness` and `readiness` probe ports by using deployer properties, as the following example shows:

```
deployer.<app>.kubernetes.readinessProbePort=7000
deployer.<app>.kubernetes.livenessProbePort=7000
```

You can declare the same as part of the global configuration for streams, as the following example shows:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    readinessProbePort: 7000
                    livenessProbePort: 7000
```

YAML

By default, the `liveness` and `readiness` probe paths use Spring Boot 2.x+ actuator endpoints. To use Spring Boot 1.x actuator endpoint paths, you must adjust the `liveness` and `readiness` values, as the following example shows (replace `<app>` with the name of your application):

```
deployer.<app>.kubernetes.livenessProbePath=/health
deployer.<app>.kubernetes.readinessProbePath=/info
```

To automatically set both `liveness` and `readiness` endpoints on a per-application basis to the default Spring Boot 1.x paths, you can set the following property:

```
deployer.<app>.kubernetes.bootMajorVersion=1
```

You can access secured probe endpoints by using credentials stored in a [Kubernetes secret](#). You can use an existing secret, provided the credentials are contained under the `credentials` key name of the secret's `data` block. You can configure probe authentication on a per-application basis. When enabled, it is applied to both the `liveness` and `readiness` probe endpoints by using the same credentials and authentication type. Currently, only `Basic` authentication is supported.

To create a new secret:

1. Generate the base64 string with the credentials used to access the secured probe endpoints.

Basic authentication encodes a username and password as a base64 string in the format of `username:password`.

The following example (which includes output and in which you should replace `user` and `pass` with your values) shows how to generate a base64 string:

```
$ echo -n "user:pass" | base64
dXNlcjpwYXNz
```

SHELL

2. With the encoded credentials, create a file (for example, `myprobesecret.yml`) with the following contents:

```
apiVersion: v1
kind: Secret
metadata:
  name: myprobesecret
type: Opaque
data:
  credentials: GENERATED_BASE64_STRING
```

3. Replace `GENERATED_BASE64_STRING` with the base64-encoded value generated earlier.
4. Create the secret by using `kubectl`, as the following example shows:

```
$ kubectl create -f ./myprobesecret.yml
secret "myprobesecret" created
```

SHELL

5. Set the following deployer properties to use authentication when accessing probe endpoints, as the following example shows:

```
deployer.<app>.kubernetes.probeCredentialsSecret=myprobesecret
```

Replace `<app>` with the name of the application to which to apply authentication.

5.1.4. Using `SPRING_APPLICATION_JSON`

You can use a `SPRING_APPLICATION_JSON` environment variable to set Data Flow server properties (including the configuration of maven repository settings) that are common across all of the Data Flow server implementations. These settings go at the server level in the container `env` section of a deployment YAML. The following example shows how to do so:

```
env:
- name: SPRING_APPLICATION_JSON
  value: "{ \"maven\": { \"local-repository\": null, \"remote-repositories\": { \"repo1\": { \"url\": \"https://repo.:
```

5.1.5. Private Docker Registry

You can pull Docker images from a private registry on a per-application basis. First, you must create a secret in the cluster. Follow the [Pull an Image from a Private Registry](#) guide to create the secret.

Once you have created the secret, you can use the `imagePullSecret` property to set the secret to use, as the following example shows:

```
deployer.<app>.kubernetes.imagePullSecret=mysecret
```

Replace `<app>` with the name of your application and `mysecret` with the name of the secret you created earlier.

You can also configure the image pull secret at the global server level.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    imagePullSecret: mysecret
```

YAML

The following example shows how to do so for tasks:

```
data:
  application.yaml: |-
    spring:
```

YAML

```
cloud:
  dataflow:
    task:
      platform:
        kubernetes:
          accounts:
            default:
              imagePullSecret: mysecret
```

Replace `mysecret` with the name of the secret you created earlier.

5.1.6. Annotations

You can add annotations to Kubernetes objects on a per-application basis. The supported object types are `pod`, `Deployment`, `Service`, and `Job`. Annotations are defined in a `key:value` format, allowing for multiple annotations separated by a comma. For more information and use cases on annotations, see [Annotations](#).

The following example shows how you can configure applications to use annotations:

```
deployer.<app>.kubernetes.podAnnotations=annotationName:annotationValue
deployer.<app>.kubernetes.serviceAnnotations=annotationName:annotationValue,annotationName2:annotationValue2
deployer.<app>.kubernetes.jobAnnotations=annotationName:annotationValue
```

Replace `<app>` with the name of your application and the value of your annotations.

5.1.7. Entry Point Style

An entry point style affects how application properties are passed to the container to be deployed. Currently, three styles are supported:

- `exec` (default): Passes all application properties and command line arguments in the deployment request as container arguments. Application properties are transformed into the format of `--key=value`.
- `shell`: Passes all application properties as environment variables. Command line arguments from the deployment request are not converted into environment variables nor set on the container. Application properties are transformed into an uppercase string and `.` characters are replaced with `_`.
- `boot`: Creates an environment variable called `SPRING_APPLICATION_JSON` that contains a JSON representation of all application properties. Command line arguments from the deployment request are set as container args.

In all cases, environment variables defined at the server-level configuration and on a per-application basis are set onto the container as is.

You can configure applications as follows:

```
deployer.<app>.kubernetes.entryPointStyle=<Entry Point Style>
```

Replace `<app>` with the name of your application and `<Entry Point Style>` with your desired entry point style.

You can also configure the entry point style at the global server level.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    entryPointStyle: entryPointStyle
```

YAML

The following example shows how to do so for tasks:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    entryPointStyle: entryPointStyle
```

YAML

Replace `entryPointStyle` with the desired entry point style.

You should choose an Entry Point Style of either `exec` or `shell`, to correspond to how the `ENTRYPOINT` syntax is defined in the container's `Dockerfile`. For more information and uses cases on `exec` versus `shell`, see the [ENTRYPOINT](#) section of the Docker documentation.

Using the `boot` entry point style corresponds to using the `exec` style `ENTRYPOINT`. Command line arguments from the deployment request are passed to the container, with the addition of application properties being mapped into the `SPRING_APPLICATION_JSON` environment variable rather than command line arguments.

When you use the `boot` Entry Point Style, the `deployer.<app>.kubernetes.environmentVariables` property must not contain `SPRING_APPLICATION_JSON`.

5.1.8. Deployment Service Account

You can configure a custom service account for application deployments through properties. You can use an existing service account or create a new one. One way to create a service account is by using `kubectl`, as the following example shows:

```
$ kubectl create serviceaccount myserviceaccountname
serviceaccount "myserviceaccountname" created
```

SHELL

Then you can configure individual applications as follows:

```
deployer.<app>.kubernetes.deploymentServiceAccountName=myserviceaccountname
```

Replace `<app>` with the name of your application and `myserviceaccountname` with your service account name.

You can also configure the service account name at the global server level.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    deploymentServiceAccountName: myserviceaccountname
```

YAML

The following example shows how to do so for tasks:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    deploymentServiceAccountName: myserviceaccountname
```

YAML

Replace `myserviceaccountname` with the service account name to be applied to all deployments.

5.1.9. Image Pull Policy

An image pull policy defines when a Docker image should be pulled to the local registry. Currently, three policies are supported:

- `IfNotPresent` (default): Do not pull an image if it already exists.
- `Always` : Always pull the image regardless of whether it already exists.
- `Never` : Never pull an image. Use only an image that already exists.

The following example shows how you can individually configure applications:

```
deployer.<app>.kubernetes.imagePullPolicy=Always
```

Replace `<app>` with the name of your application and `Always` with your desired image pull policy.

You can configure an image pull policy at the global server level.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    imagePullPolicy: Always
```

YAML

The following example shows how to do so for tasks:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    imagePullPolicy: Always
```

YAML

Replace `Always` with your desired image pull policy.

5.1.10. Deployment Labels

You can set custom labels on objects related to [Deployment](#). See [Labels](#) for more information on labels. Labels are specified in `key:value` format.

The following example shows how you can individually configure applications:

```
deployer.<app>.kubernetes.deploymentLabels=myLabelName:myLabelValue
```

Replace `<app>` with the name of your application, `myLabelName` with your label name, and `myLabelValue` with the value of your label.

Additionally, you can apply multiple labels, as the following example shows:

```
deployer.<app>.kubernetes.deploymentLabels=myLabelName:myLabelValue,myLabelName2:myLabelValue2
```

5.1.11. Tolerations

Tolerations work with taints to ensure pods are not scheduled onto particular nodes. Tolerations are set into the pod configuration while taints are set onto nodes. Refer to the [Taints and Tolerations](#) section of the Kubernetes reference for more information.

The following example shows how you can individually configure applications:

```
deployer.<app>.kubernetes.tolerations=[{key: 'mykey' operator: 'Equal', value: 'myvalue', effect: 'NoSchedule'}]
```

Replace `<app>` with the name of your application and the key / value pairs according to your desired toleration configuration.

You can configure tolerations at the global server level as well.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    tolerations:
                      - key: mykey
                        operator: Equal
                        value: myvalue
                        effect: NoSchedule
```

YAML

The following example shows how to do so for tasks:


```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    tolerations:
                      - key: mykey
                        operator: Equal
                        value: myvalue
                        effect: NoSchedule
```

Replacing the `tolerations` key / value pairs according to your desired toleration configuration.

5.1.12. Secret Key References

Secrets can be referenced and their decoded value inserted into the pod(s) environment. Refer to the [Using Secrets as Environment Variables](#) section of the Kubernetes reference for more information.

The following example shows how you can individually configure applications:

```
deployer.<app>.kubernetes.secretKeyRefs=[{envVarName: 'MY_SECRET', secretName: 'testsecret', dataKey: 'password'}]
```

Replace `<app>` with the name of your application and the `envVarName`, `secretName` and `dataKey` attributes with the appropriate values for your application environment and secret.

You can configure secret key references at the global server level as well.

The following example shows how to do so for streams:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    secretKeyRefs:
                      - envVarName: MY_SECRET
                        secretName: testsecret
                        dataKey: password
```

The following example shows how to do so for tasks:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    secretKeyRefs:
                      - envVarName: MY_SECRET
                        secretName: testsecret
                        dataKey: password
```

Replacing the `envVarName`, `secretName` and `dataKey` attributes with the appropriate values for your secret.

5.1.13. Config Map Key References

ConfigMap's can be referenced and their associated key value inserted into the pod(s) environment. Refer to the [Define container environment variables using ConfigMap data](#) section of the Kubernetes reference for more information.

The following example shows how you can individually configure applications:

```
deployer.<app>.kubernetes.configMapKeyRefs=[{envVarName: 'MY_CM', configMapName: 'testcm', dataKey: 'platform'}]
```

Replace `<app>` with the name of your application and the `envVarName`, `configMapName` and `dataKey` attributes with the appropriate values for your application environment and ConfigMap.

You can configure ConfigMap references at the global server level as well.

The following example shows how to do so for streams. Edit the appropriate `skipper-config-(binder).yaml`, replacing `(binder)` with the corresponding binder in use:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    configMapKeyRefs:
                      - envVarName: MY_CM
                        configMapName: testcm
                        dataKey: platform
```

The following example shows how to do so for tasks by editing the `server-config.yaml` file:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    configMapKeyRefs:
                      - envVarName: MY_CM
                        configMapName: testcm
                        dataKey: platform
```

Replacing the `envVarName`, `configMapName` and `dataKey` attributes with the appropriate values for your ConfigMap.

5.1.14. Pod Security Context

The pod security context can be configured to run processes under the specified UID (user ID) or GID (group ID). This is useful when its preferred not to run processes under the default `root` UID and GID. Either the `runAsUser` (UID) or `fsGroup` (GID) can be defined, as well as configured together. Refer to the [Security Context](#) section of the Kubernetes reference for more information.

The following example shows how you can individually configure application pods:

```
deployer.<app>.kubernetes.podSecurityContext={runAsUser: 65534, fsGroup: 65534}
```

Replace `<app>` with the name of your application and the `runAsUser` and/or `fsGroup` attributes with the appropriate values for your container environment.

You can configure the pod security context at the global server level as well.

The following example shows how to do so for streams. Edit the appropriate `skipper-config-(binder).yaml`, replacing `(binder)` with the corresponding binder in use:

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    podSecurityContext:
                      runAsUser: 65534
                      fsGroup: 65534
```

The following example shows how to do so for tasks by editing the `server-config.yaml` file:

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    podSecurityContext:
                      runAsUser: 65534
                      fsGroup: 65534
```

Replacing the `runAsUser` and/or `fsGroup` attributes with the appropriate values for your container environment.

5.1.15. Service Ports

When deploying applications a kubernetes Service object is created with a default port of `8080` or, if set the application property value of `server.port`. Additional ports can be added to the Service object on a per-application basis. Multiple ports can be added when specified with a comma delimiter.

The following example shows how you can configure additional ports on a Service object for an application:

```
deployer.<app>.kubernetes.servicePorts=5000
deployer.<app>.kubernetes.servicePorts=5000,9000
```

Replace `<app>` with the name of your application and the value of your port(s).

5.1.16. StatefulSet Init Container

When deploying an application using a StatefulSet, an Init Container is used to set the instance index into the pod. By default the image used is `busybox`, which can be customized if needed.

The following example shows how you can individually configure application pods:

```
deployer.<app>.kubernetes.statefulSetInitContainerImageName=myimage:mylabel
```

Replace `<app>` with the name of your application and the `statefulSetInitContainerImageName` attribute with the appropriate value for your environment.

You can configure the StatefulSet Init Container at the global server level as well.

The following example shows how to do so for streams. Edit the appropriate `skipper-config-(binder).yaml`, replacing `(binder)` with the corresponding binder in use:

YAML

```
data:
  application.yaml: |-
    spring:
      cloud:
        skipper:
          server:
            platform:
              kubernetes:
                accounts:
                  default:
                    statefulSetInitContainerImageName: myimage:mylabel
```

The following example shows how to do so for tasks by editing the `server-config.yaml` file:

YAML

```
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    statefulSetInitContainerImageName: myimage:mylabel
```

Replacing the `statefulSetInitContainerImageName` attribute with the appropriate value for your environment.

5.1.17. Init Containers

When applications are deployed, a custom Init Container can be set on a per application basis. Refer to the [Init Containers](#) section of the Kubernetes reference for more information.

The following example shows how you can configure an Init Container for an application:

```
deployer.<app>.kubernetes.initContainer={containerName: 'test', imageName: 'busybox:latest', commands: ['sh', '-c', 'i
```

Replace `<app>` with the name of your application and set the values of the `initContainer` attributes appropriate for your Init Container.

Applications

A selection of pre-built [stream](#) and [task/batch](#) starter apps for various data integration and processing scenarios to facilitate learning and experimentation. The table below includes the pre-built applications at a glance. For more details, review how to [register supported applications](#).

6. Available Applications

Source	Processor	Sink	Task
sftp	tcp-client	mqtt	timestamp
jms	scriptable-transform	log	composed-task-runner
ftp	transform	throughput	timestamp-batch
time	header-enricher	mongodb	
load-generator	python-http	ftp	
syslog	twitter-sentiment	jdbc	
s3	splitter	cassandra	
loggregator	image-recognition	router	
triggertask (deprecated)	bridge	redis-pubsub	
twitterstream	pmml	file	
mongodb	python-jython	websocket	
gemfire-cq	groovy-transform	s3	
http	httpClient	rabbit	
rabbit	filter	counter	
tcp	pose-estimation	pgcopy	
trigger	grpc	gpfdist	
mqtt	groovy-filter	sftp	
tcp-client	aggregator	tasklauncher-dataflow	
mail	counter	hdfs	
jdbc	tensorflow	tcp	
gemfire	tasklaunchrequest-transform (deprecated)	gemfire	
file	object-detection		
sftp-dataflow			

Architecture

7. Introduction

Spring Cloud Data Flow simplifies the development and deployment of applications focused on data processing use cases.

The [Architecture](#) section of the microsite contains the description of Data Flow's architecture.

Configuration

8. Maven

If you want to override specific maven configuration properties (remote repositories, proxies, and others) or run the Data Flow Server behind a proxy, you need to specify those properties as command line arguments when starting the Data Flow Server, as shown in the following example:

```
$ java -jar spring-cloud-dataflow-server-2.4.0.RELEASE.jar --spring.config.additional-location=/home/joe/maven.yml
```

BASH

where `maven.yml` is

```
maven:
  localRepository: mylocal
  remote-repositories:
    repo1:
      url: https://repo1
      auth:
        username: user1
        password: pass1
      snapshot-policy:
        update-policy: daily
        checksum-policy: warn
      release-policy:
        update-policy: never
        checksum-policy: fail
    repo2:
      url: https://repo2
      policy:
        update-policy: always
        checksum-policy: fail
  proxy:
    host: proxy1
    port: "9010"
    auth:
      username: proxyuser1
      password: proxypass1
```

YML

By default, the protocol is set to `http`. You can omit the `auth` properties if the proxy does not need a username and password. Also, the maven `localRepository` is set to `${user.home}/.m2/repository/` by default. As shown in the preceding example, the remote repositories can be specified along with their authentication (if needed). If the remote repositories are behind a proxy, then the proxy properties can be specified as shown in the preceding example.

The repository policies can be specified for each remote repository configuration as shown in the preceding example. The key `policy` is applicable to both `snapshot` and the `release` repository policies.

You can refer to [Repository Policies](#) for the list of supported repository policies.


```

maven:
  use-wagon: true
  remote-repositories:
    springRepo:
      url: https://repo.example.org
      wagon:
        http:
          get:
            use-preemptive: true
          head:
            use-preemptive: true
            use-default-headers: true
            connection-timeout: 1000
            read-timeout: 1000
            headers:
              Foo: Bar
            params:
              http.socket.timeout: 1000
              http.connection.stalecheck: true
  auth:
    username: user
    password: password

```

There are settings for `use-default-headers`, `connection-timeout`, `read-timeout`, `request headers` and `HttpClient params`. More about params check [Wagon ConfigurationUtils](#).

9. Configuration - Local

9.1. Feature Toggles

Spring Cloud Data Flow Server offers specific set of features that can be enabled/disabled when launching. These features include all the lifecycle operations and REST endpoints (server and client implementations, including the shell and the UI) for:

- Streams (requires Skipper)
- Tasks
- Task Scheduler

One can enable and disable these features by setting the following boolean properties when launching the Data Flow server:

- `spring.cloud.dataflow.features.streams-enabled`
- `spring.cloud.dataflow.features.tasks-enabled`
- `spring.cloud.dataflow.features.schedules-enabled`

By default, stream (requires Skipper), and tasks are enabled and Task Scheduler is disabled by default.

The REST `/about` endpoint provides information on the features that have been enabled and disabled.

9.2. Database

A relational database is used to store stream and task definitions as well as the state of executed tasks. Spring Cloud Data Flow provides schemas for **H2**, **MySQL**, **Oracle**, **PostgreSQL**, **Db2**, and **SQL Server**. The schema is automatically created when the server starts.

By default, Spring Cloud Data Flow offers an embedded instance of the **H2** database. The **H2** database is good for development purposes but is not recommended for production use.

H2 database is not supported as an external mode.

The JDBC drivers for **MySQL** (through the MariaDB driver), **PostgreSQL**, **SQL Server**, and embedded **H2** are available without additional configuration. If you are using any other database, then you need to put the corresponding JDBC driver jar on the classpath of the server.

The database properties can be passed as environment variables or command-line arguments to the Data Flow Server.

9.2.1. MySQL

The following example shows how to define a MySQL database connection using MariaDB driver.

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
  --spring.datasource.url=jdbc:mysql://localhost:3306/mydb \
  --spring.datasource.username= \
  --spring.datasource.password= \
  --spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
```

BASH

MySQL versions up to 5.7 can be used with a MariaDB driver. Starting from version 8.0 MySQL's own driver has to be used.

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
  --spring.datasource.url=jdbc:mysql://localhost:3306/mydb \
  --spring.datasource.username= \
  --spring.datasource.password= \
  --spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

BASH

Due to licensing restrictions we're unable to bundle MySQL driver. You need to add it to server's classpath yourself.

9.2.2. MariaDB

The following example shows how to define a MariaDB database connection with command Line arguments

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
  --spring.datasource.url=jdbc:mariadb://localhost:3306/mydb?useMysqlMetadata=true \
  --spring.datasource.username= \
  --spring.datasource.password= \
  --spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
```

BASH

Starting with MariaDB v2.4.1 connector release, it is required to also add `useMySQLMetadata=true` to the JDBC URL. This is a required workaround until when MySQL and MariaDB entirely switch as two different databases.

MariaDB version *10.3* introduced a support for real database sequences which is yet another breaking change while toolings around these databases fully support MySQL and MariaDB as a separate database types. Workaround is to use older hibernate dialect which doesn't try to use sequences.

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
--spring.datasource.url=jdbc:mariadb://localhost:3306/mydb?useMySQLMetadata=true \
--spring.datasource.username= \
--spring.datasource.password= \
--spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MariaDB102Dialect \
--spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
```

BASH

9.2.3. PostgreSQL

The following example shows how to define a PostgreSQL database connection with command line arguments:

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
--spring.datasource.url=jdbc:postgresql://localhost:5432/mydb \
--spring.datasource.username= \
--spring.datasource.password= \
--spring.datasource.driver-class-name=org.postgresql.Driver
```

BASH

9.2.4. SQL Server

The following example shows how to define a SQL Server database connection with command line arguments:

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
--spring.datasource.url='jdbc:sqlserver://localhost:1433;databaseName=mydb' \
--spring.datasource.username= \
--spring.datasource.password= \
--spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
```

BASH

9.2.5. Db2

The following example shows how to define a Db2 database connection with command line arguments:

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
--spring.datasource.url=jdbc:db2://localhost:50000/mydb \
--spring.datasource.username= \
--spring.datasource.password= \
--spring.datasource.driver-class-name=com.ibm.db2.jcc.DB2Driver
```

BASH

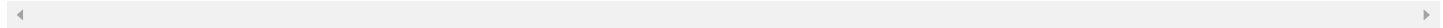
Due to licensing restrictions we're unable to bundle Db2 driver. You need to add it to server's classpath yourself.

9.2.6. Oracle

The following example shows how to define a Oracle database connection with command line arguments

```
java -jar spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.RELEASE.jar \
--spring.datasource.url=jdbc:oracle:thin:@localhost:1521/MYDB \
--spring.datasource.username= \
--spring.datasource.password= \
--spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
```

Due to licensing restrictions we're unable to bundle Oracle driver. You need to add it to server's classpath yourself.



9.2.7. Adding a Custom JDBC Driver

To add a custom driver for the database (for example, Oracle), you should rebuild the Data Flow Server and add the dependency to the Maven `pom.xml` file. You need to modify the maven `pom.xml` of `spring-cloud-dataflow-server` module. There are GA release tags in GitHub repository, so you can switch to desired GA tags to add the drivers on the production-ready codebase.

To add a custom JDBC driver dependency for the Spring Cloud Data Flow server:

1. Select the tag that corresponds to the version of the server you want to rebuild and clone the github repository.
2. Edit the `spring-cloud-dataflow-server/pom.xml` and, in the `dependencies` section, add the dependency for the database driver required. In the following example , an Oracle driver has been chosen:

```
<dependencies>
...
<dependency>
  <groupId>com.oracle.jdbc</groupId>
  <artifactId>ojdbc8</artifactId>
  <version>12.2.0.1</version>
</dependency>
...
</dependencies>
```

XML

3. Build the application as described in [Building Spring Cloud Data Flow](#)

You can also provide default values when rebuilding the server by adding the necessary properties to the `dataflow-server.yml` file, as shown in the following example for PostgreSQL:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: myuser
    password: mypass
    driver-class-name: org.postgresql.Driver
```

4. Alternatively, you can build a custom Spring Cloud Data Flow server with your build files. There are examples of a custom server builds in our [samples repo](#) if there is a need to add a driver jars.

9.3. Deployer Properties

You can use the following configuration properties of the [Local deployer](#) to customize how Streams and Tasks are deployed. When deploying using the Data Flow shell, you can use the syntax `deployer.<appName>.local.<deployerProp>` to set the property.

below for an example shell usage. These properties are also used when configuring [Local Task Platforms](#) in the Data Flow server and local platforms in Skipper for deploying Streams.

Deployer Property Name	Description	Default Value
<code>workingDirectoriesRoot</code>	Directory in which all created processes will run and create log files.	<code>java.io.tmpdir</code>
<code>envVarsToInherit</code>	Array of regular expression patterns for environment variables that are passed to launched applications.	<code><"TMP", "LANG", "LANGUAGE", "LC_.*", "PATH", "SPRING_APPLICATION_JSON"></code> on windows and <code><"TMP", "LANG", "LANGUAGE", "LC_.*", "PATH"></code> on Unix
<code>deleteFilesOnExit</code>	Whether to delete created files and directories on JVM exit.	<code>true</code>
<code>javaCmd</code>	Command to run java	<code>java</code>
<code>shutdownTimeout</code>	Max number of seconds to wait for app shutdown.	<code>30</code>
<code>javaOpts</code>	The Java Options to pass to the JVM, e.g <code>-Dtest=foo</code>	<code><none></code>
<code>inheritLogging</code>	allow logging to be redirected to the output stream of the process that triggered child process.	<code>false</code>
<code>debugPort</code>	Port for remote debugging	<code><none></code>

As an example, to set Java options for the time application in the `ticktock` stream, use the following stream deployment properties.

```
dataflow:> stream create --name ticktock --definition "time --server.port=9000 | log"
dataflow:> stream deploy --name ticktock --properties "deployer.time.local.javaOpts=-Xmx2048m -Dtest=foo"
```

BASH

As a convenience, you can set the `deployer.memory` property to set the Java option `-Xmx`, as shown in the following example:

```
dataflow:> stream deploy --name ticktock --properties "deployer.time.memory=2048m"
```

BASH

At deployment time, if you specify an `-Xmx` option in the `deployer.<app>.local.javaOpts` property in addition to a value of the `deployer.<app>.local.memory` option, the value in the `javaOpts` property has precedence. Also, the `javaOpts` property set when deploying the application has precedence over the Data Flow Server's `spring.cloud.deployer.local.javaOpts` property.

9.4. Logging

Spring Cloud Data Flow `local` server is automatically configured to use `RollingFileAppender` for logging. The logging configuration is located on the classpath contained in a file named `logback-spring.xml`.

By default, the log file is configured to use:

```
<property name="LOG_FILE" value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/spring-cloud-dataflow-server
```

with the logback configuration for the `RollingPolicy`:

```
<appender name="FILE"
    class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>${LOG_FILE}.log</file>
    <rollingPolicy
        class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
        <!-- daily rolling -->
        <fileNamePattern>${LOG_FILE}.${LOG_FILE_ROLLING_FILE_NAME_PATTERN:-%d{yyyy-MM-dd}}.%i.gz</fileNamePattern>
        <maxFileSize>${LOG_FILE_MAX_SIZE:-100MB}</maxFileSize>
        <maxHistory>${LOG_FILE_MAX_HISTORY:-30}</maxHistory>
        <totalSizeCap>${LOG_FILE_TOTAL_SIZE_CAP:-500MB}</totalSizeCap>
    </rollingPolicy>
    <encoder>
        <pattern>${FILE_LOG_PATTERN}</pattern>
    </encoder>
</appender>
```

To check the `java.io.tmpdir` for the current Spring Cloud Data Flow Server local server,

```
jinfo <pid> | grep "java.io.tmpdir"
```

If you want to change or override any of the properties `LOG_FILE`, `LOG_PATH`, `LOG_TEMP`, `LOG_FILE_MAX_SIZE`, `LOG_FILE_MAX_HISTORY` and `LOG_FILE_TOTAL_SIZE_CAP`, please set them as system properties.

9.5. Streams

Data Flow Server delegates to the Skipper server the management of the Stream's lifecycle. Set the configuration property `spring.cloud.skipper.client.serverUri` to the location of Skipper, e.g.

```
$ java -jar spring-cloud-dataflow-server-2.4.0.RELEASE.jar --spring.cloud.skipper.client.serverUri=https://192.51.100.1:757
```

The configuration of show streams are deployed and to which platforms, is done by configuration of `platform` accounts on the Skipper server. See the documentation on [platforms](#) for more information.

9.6. Tasks

The Data Flow server is responsible for deploying Tasks. Tasks that are launched by Data Flow write their state to the same database that is used by the Data Flow server. For Tasks which are Spring Batch Jobs, the job and step execution data is also stored in this database. As with streams launched by Skipper, Tasks can be launched to multiple platforms. If no platform is defined, a platform named `default` is created using the default values of the class [LocalDeployerProperties](#), which is summarized in the table [Local Deployer Properties](#)

To configure new platform accounts for the local platform, provide an entry under the `spring.cloud.dataflow.task.platform.local` section in your `application.yaml` file for via another Spring Boot supported mechanism. In the following example, two local platform accounts named `localDev` and `localDevDebug` are created. The keys such as `shutdownTimeout` and `javaOpts` are local deployer properties.

```
spring:
  cloud:
    dataflow:
      task:
        platform:
          local:
            accounts:
              localDev:
                shutdownTimeout: 60
                javaOpts: "-Dtest=foo -Xmx1024m"
              localDevDebug:
                javaOpts: "-Xdebug -Xmx2048m"
```

YAML

By defining one platform as `default` allows you to skip using `platformName` where its use would otherwise be required.

When launching a task, pass the value of the platform account name using the task launch option `--platformName`. If you do not pass a value for `platformName`, the value `default` will be used.

When deploying a task to multiple platforms, the configuration of the task needs to connect to the same database as the Data Flow Server.

You can configure the Data Flow server that is running locally to deploy tasks to Cloud Foundry or Kubernetes. See the sections on [Cloud Foundry Task Platform Configuration](#) and [Kubernetes Task Platform Configuration](#) for more information.

9.7. Security

By default, the Data Flow server is unsecured and runs on an unencrypted HTTP connection. You can secure your REST endpoints as well as the Data Flow Dashboard by enabling HTTPS and requiring clients to authenticate using [OAuth 2.0](#).

By default, the REST endpoints (administration, management, and health) as well as the Dashboard UI do not require authenticated access.

While you can theoretically choose any OAuth provider in conjunction with Spring Cloud Data Flow, we recommend using the [CloudFoundry User Account and Authentication \(UAA\) Server](#).

Not only is the UAA OpenID certified and is used by Cloud Foundry but it can also be used in local stand-alone deployment scenarios. Furthermore, the UAA not only provides its own user store, but also provides comprehensive LDAP integration.

9.7.1. Enabling HTTPS

By default, the dashboard, management, and health endpoints use HTTP as a transport. You can switch to HTTPS by adding a certificate to your configuration in `application.yml`, as shown in the following example:

```
server:
  port: 8443
  ssl:
    key-alias: yourKeyAlias
    key-store: path/to/keystore
    key-store-password: yourKeyStorePassword
    key-password: yourKeyPassword
    trust-store: path/to/trust-store
    trust-store-password: yourTrustStorePassword
```

YAML

- 1 As the default port is 9393, you may choose to change the port to a more common HTTPS-typical port.
- 2 The alias (or name) under which the key is stored in the keystore.
- 3 The path to the keystore file. Classpath resources may also be specified, by using the classpath prefix - for example: `classpath:path/to/keystore`.
- 4 The password of the keystore.
- 5 The password of the key.
- 6 The path to the truststore file. Classpath resources may also be specified, by using the classpath prefix - for example: `classpath:path/to/trust-store`
- 7 The password of the trust store.

If HTTPS is enabled, it completely replaces HTTP as the protocol over which the REST endpoints and the Data Flow Dashboard interact. Plain HTTP requests will fail. Therefore, make sure that you configure your Shell accordingly.

Using Self-Signed Certificates

For testing purposes or during development, it might be convenient to create self-signed certificates. To get started, execute the following command to create a certificate:

```
$ keytool -genkey -alias dataflow -keyalg RSA -keystore dataflow.keystore \
  -validity 3650 -storetype JKS \
  -dname "CN=localhost, OU=Spring, O=Pivotal, L=Kailua-Kona, ST=HI, C=US" \
  -keypass dataflow -storepass dataflow
```

BASH

- 1 CN is the important parameter here. It should match the domain you are trying to access - for example, `localhost`.

Then add the following lines to your `application.yml` file:

```
server:
  port: 8443
  ssl:
    enabled: true
    key-alias: dataflow
```

YAML


```
key-store: "/your/path/to/dataflow.keystore"
key-store-type: jks
key-store-password: dataflow
key-password: dataflow
```

This is all that is needed for the Data Flow Server. Once you start the server, you should be able to access it at localhost:8443/. As this is a self-signed certificate, you should hit a warning in your browser, which you need to ignore.

Self-Signed Certificates and the Shell

By default, self-signed certificates are an issue for the shell, and additional steps are necessary to make the shell work with self-signed certificates. Two options are available:

- Add the self-signed certificate to the JVM truststore.
- Skip certificate validation.

Adding the Self-signed Certificate to the JVM Truststore

In order to use the JVM truststore option, we need to export the previously created certificate from the keystore, as follows:

```
$ keytool -export -alias dataflow -keystore dataflow.keystore -file dataflow_cert -storepass dataflow
```

BASH

Next, we need to create a truststore which the shell can use, as follows:

```
$ keytool -importcert -keystore dataflow.truststore -alias dataflow -storepass dataflow -file dataflow_cert -noprompt
```

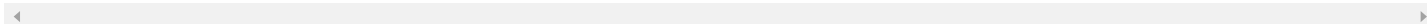
BASH

Now, you are ready to launch the Data Flow Shell by using the following JVM arguments:

```
$ java -Djavax.net.ssl.trustStorePassword=dataflow \
-Djavax.net.ssl.trustStore=/path/to/dataflow.truststore \
-Djavax.net.ssl.trustStoreType=jks \
-jar spring-cloud-dataflow-shell-2.4.0.RELEASE.jar
```

BASH

In case you run into trouble establishing a connection over SSL, you can enable additional logging by using and setting the `javax.net.debug` JVM argument to `ssl`.



Do not forget to target the Data Flow Server with the following:

```
dataflow:> dataflow config server https://localhost:8443/
```

BASH

Skipping Certificate Validation

Alternatively, you can also bypass the certification validation by providing the optional command-line parameter `--dataflow.skip-ssl-validation=true`.

If you set this command-line parameter, the shell accepts any (self-signed) SSL certificate.

If possible, you should avoid using this option. Disabling the trust manager defeats the purpose of SSL and makes you vulnerable to man-in-the-middle attacks.

9.7.2. Authentication using OAuth 2.0

In order to support authentication and authorization, Spring Cloud Data Flow is using [OAuth 2.0](#) and [OpenID Connect](#). It lets you integrate Spring Cloud Data Flow into Single Sign On (SSO) environments.

As of Spring Cloud Data Flow 2.0, OAuth2 is the only mechanism for providing authentication and authorization.

The following OAuth2 Grant Types are used:

- **Authorization Code:** Used for the GUI (browser) integration. Visitors are redirected to your OAuth Service for authentication
- **Password:** Used by the shell (and the REST integration), so visitors can log in with username and password
- **Client Credentials:** Retrieve an access token directly from your OAuth provider and pass it to the Data Flow server by using the Authorization HTTP header

Currently, Spring Cloud Data Flow uses opaque tokens and not transparent tokens (JWT).

The REST endpoints can be accessed in two ways:

- **Basic authentication**, which uses the *Password Grant Type* under the covers to authenticate with your OAuth2 service
- **Access token**, which uses the *Client Credentials Grant Type* under the covers

When authentication is set up, it is strongly recommended to enable HTTPS as well, especially in production environments.

You can turn on OAuth2 authentication by adding the following to `application.yml` or by setting environment variables. The following example shows the minimal setup needed for [CloudFoundry User Account and Authentication \(UAA\) Server](#):

```
spring:
  security:
    oauth2:
      client:
        registration:
          uaa:
            client-id: myclient
            client-secret: mysecret
```

1

2

YAML

```

redirect-uri: '{baseUrl}/login/oauth2/code/{registrationId}'
authorization-grant-type: authorization_code
scope:
  - openid
provider:
  uaa:
    jwk-set-uri: http://uaa.local:8080/uaa/token_keys
    token-uri: http://uaa.local:8080/uaa/oauth/token
    user-info-uri: http://uaa.local:8080/uaa/userinfo
    user-name-attribute: user_name
    authorization-uri: http://uaa.local:8080/uaa/oauth/authorize
resource-server:
  opaquetoken:
    introspection-uri: http://uaa.local:8080/uaa/introspect
    client-id: dataflow
    client-secret: dataflow

```

- 1 Providing this property activates OAuth2 security
- 2 The provider id. It is possible to specify more than 1 provider
- 3 As the UAA is an OpenID provider, you must at least specify the `openid` scope. If your provider also provides additional scopes to control the role assignments, you must specify those scopes here as well
- 4 OpenID endpoint. Used to retrieve user information such as the username. Mandatory.
- 5 The JSON property of the response that contains the username
- 6 Used to introspect and validate a directly passed-in token. Mandatory.

You can verify that basic authentication is working properly by using curl, as follows:

```
curl -u myusername:mypassword http://localhost:9393/ -H 'Accept: application/json'
```

BASH

As a result, you should see a list of available REST endpoints.

Please be aware that when accessing the Root URL with a web browser and enabled security, you are redirected to the Dashboard UI. In order to see the list of REST endpoints, specify the `application/json` Accept header. Also be sure to add the `Accept` header using tools such as [Postman](#) (Chrome) or [RESTClient](#) (Firefox).

Besides Basic Authentication, you can also provide an Access Token in order to access the REST Api. In order to make that happen, you would retrieve an OAuth2 Access Token from your OAuth2 provider first and then pass that Access Token to the REST Api using the **Authorization** Http header:

```
$ curl -H "Authorization: Bearer <ACCESS_TOKEN>" http://localhost:9393/ -H 'Accept: application/json'
```

9.7.3. Customizing Authorization

The preceding content deals with mostly with authentication - that is, how to assess the identity of the user. In this section we want to discuss the available **authorization** options - that is, who can do what.

The authorization rules are defined in `dataflow-server-defaults.yml` (part of the Spring Cloud Data Flow Core module).

Because the determination of security roles is environment-specific, Spring Cloud Data Flow, assigns all roles to authenticated OAuth2 users by default. The `DefaultDataflowAuthoritiesExtractor` class is used for that purpose.

Alternatively, Spring Cloud Data Flow can map OAuth2 scopes to Data Flow roles by setting the boolean property `map-oauth-scopes` for your provider to `true` (False is the default). For example, if your provider's id is `uaa`, the property would be `spring.cloud.dataflow.security.authorization.provider-role-mappings.uaa.map-oauth-scopes`.

For more details, please see the chapter on [Role Mappings](#).

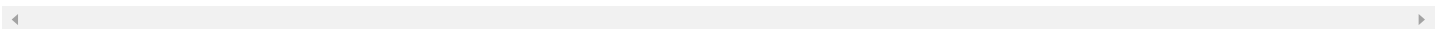
Lastly, you can customize the role mapping behavior by providing your own Spring bean definition that extends Spring Cloud Data Flow's `AuthorityMapper` interface. In that case, the custom bean definition takes precedence over the default one provided by Spring Cloud Data Flow.

The default scheme uses seven roles to protect the [REST endpoints](#) that Spring Cloud Data Flow exposes:

- **ROLE_CREATE** for anything that involves creating, e.g. creating streams or tasks
- **ROLE_DEPLOY** for deploying streams or launching tasks
- **ROLE_DESTROY** for anything that involves deleting streams, tasks etc.
- **ROLE_MANAGE** for boot management endpoints
- **ROLE_MODIFY** for anything that involves mutating the state of the system
- **ROLE_SCHEDULE** for scheduling related operation (e.g. schedule a task execution)
- **ROLE_VIEW** for anything that relates to retrieving state

As mentioned earlier, all authorization-related default settings are specified in `dataflow-server-defaults.yml`, which is part of the Spring Cloud Data Flow Core Module. Nonetheless, you can override those settings, if desired - for example, in `application.yml`. The configuration takes the form of a YAML list (as some rules may have precedence over others). Consequently, you need to copy and paste the whole list and tailor it to your needs (as there is no way to merge lists).

Always refer to your version of the `application.yml` file, as the following snippet may be outdated.



The default rules are as follows:

```
spring:
  cloud:
    dataflow:
      security:
        authorization:
          enabled: true
          loginUrl: "/"
          permit-all-paths: "/authenticate,/security/info,/assets/**,/dashboard/logout-success-oauth.html,/favicon.ico"
          rules:
            # About
            - GET    /about                                => hasRole('ROLE_VIEW')

            # Audit
            - GET    /audit-records                         => hasRole('ROLE_VIEW')
            - GET    /audit-records/**                      => hasRole('ROLE_VIEW')

            # Boot Endpoints
```

YAML

```

- GET /management/** => hasRole('ROLE_MANAGE')

# Apps

- GET /apps => hasRole('ROLE_VIEW')
- GET /apps/** => hasRole('ROLE_VIEW')
- DELETE /apps/** => hasRole('ROLE_DESTROY')
- POST /apps => hasRole('ROLE_CREATE')
- POST /apps/** => hasRole('ROLE_CREATE')
- PUT /apps/** => hasRole('ROLE_MODIFY')

# Completions

- GET /completions/** => hasRole('ROLE_VIEW')

# Job Executions & Batch Job Execution Steps && Job Step Execution Progress

- GET /jobs/executions => hasRole('ROLE_VIEW')
- PUT /jobs/executions/** => hasRole('ROLE_MODIFY')
- GET /jobs/executions/** => hasRole('ROLE_VIEW')
- GET /jobs/thinexecutions => hasRole('ROLE_VIEW')

# Batch Job Instances

- GET /jobs/instances => hasRole('ROLE_VIEW')
- GET /jobs/instances/* => hasRole('ROLE_VIEW')

# Running Applications

- GET /runtime/streams => hasRole('ROLE_VIEW')
- GET /runtime/streams/** => hasRole('ROLE_VIEW')
- GET /runtime/apps => hasRole('ROLE_VIEW')
- GET /runtime/apps/** => hasRole('ROLE_VIEW')

# Stream Definitions

- GET /streams/definitions => hasRole('ROLE_VIEW')
- GET /streams/definitions/* => hasRole('ROLE_VIEW')
- GET /streams/definitions/*/related => hasRole('ROLE_VIEW')
- POST /streams/definitions => hasRole('ROLE_CREATE')
- DELETE /streams/definitions/* => hasRole('ROLE_DESTROY')
- DELETE /streams/definitions => hasRole('ROLE_DESTROY')

# Stream Deployments

- DELETE /streams/deployments/* => hasRole('ROLE_DEPLOY')
- DELETE /streams/deployments => hasRole('ROLE_DEPLOY')
- POST /streams/deployments/** => hasRole('ROLE_MODIFY')
- GET /streams/deployments/** => hasRole('ROLE_VIEW')

# Stream Validations

- GET /streams/validation/ => hasRole('ROLE_VIEW')
- GET /streams/validation/* => hasRole('ROLE_VIEW')

# Stream Logs

- GET /streams/logs/* => hasRole('ROLE_VIEW')

# Task Definitions

- POST /tasks/definitions => hasRole('ROLE_CREATE')
- DELETE /tasks/definitions/* => hasRole('ROLE_DESTROY')
- GET /tasks/definitions => hasRole('ROLE_VIEW')
- GET /tasks/definitions/* => hasRole('ROLE_VIEW')

# Task Executions

- GET /tasks/executions => hasRole('ROLE_VIEW')
- GET /tasks/executions/* => hasRole('ROLE_VIEW')
- POST /tasks/executions => hasRole('ROLE_DEPLOY')
- POST /tasks/executions/* => hasRole('ROLE_DEPLOY')
- DELETE /tasks/executions/* => hasRole('ROLE_DESTROY')

# Task Schedules

- GET /tasks/schedules => hasRole('ROLE_VIEW')

```

```

- GET      /tasks/schedules/*           => hasRole('ROLE_VIEW')
- GET      /tasks/schedules/instances   => hasRole('ROLE_VIEW')
- GET      /tasks/schedules/instances/* => hasRole('ROLE_VIEW')
- POST     /tasks/schedules             => hasRole('ROLE_SCHEDULE')
- DELETE   /tasks/schedules/*           => hasRole('ROLE_SCHEDULE')

# Task Platform Account List */

- GET      /tasks/platforms             => hasRole('ROLE_VIEW')

# Task Validations

- GET      /tasks/validation/           => hasRole('ROLE_VIEW')
- GET      /tasks/validation/*          => hasRole('ROLE_VIEW')

# Task Logs
- GET      /tasks/logs/*                => hasRole('ROLE_VIEW')

# Tools

- POST     /tools/**                    => hasRole('ROLE_VIEW')
```

The format of each line is the following:

```
HTTP_METHOD URL_PATTERN '=>' SECURITY_ATTRIBUTE
```

where

- HTTP_METHOD is one http method, capital case
- URL_PATTERN is an Ant style URL pattern
- SECURITY_ATTRIBUTE is a SpEL expression. See [Expression-Based Access Control](#).
- Each of those separated by one or several blank characters (spaces, tabs, and so on)

Be mindful that the above is indeed a YAML list, not a map (thus the use of '-' dashes at the start of each line) that lives under the `spring.cloud.dataflow.security.authorization.rules` key.

Authorization - Shell and Dashboard Behavior

When security is enabled, the dashboard and the shell are role-aware, meaning that, depending on the assigned roles, not all functionality may be visible.

For instance, shell commands for which the user does not have the necessary roles are marked as unavailable.

Currently, the shell's `help` command lists commands that are unavailable. Please track the following issue: github.com/spring-projects/spring-shell/issues/115

Similarly, for the Dashboard, the UI does not show pages or page elements for which the user is not authorized.

Securing the Spring Boot Management Endpoints

When security is enabled, the [Spring Boot HTTP Management Endpoints](#) are secured the same way as the other REST endpoints. The management REST endpoints are available under `/management` and require the `MANAGEMENT` role.

The default configuration in `dataflow-server-defaults.yml` has the following configuration:

```
management:
  endpoints:
    web:
      base-path: /management
  security:
    roles: MANAGE
```

YAML

Currently, please refrain from customizing the default management path.

9.7.4. Setting up UAA Authentication

For local deployment scenarios, we recommend using the [CloudFoundry User Account and Authentication \(UAA\) Server](#), which is [OpenID certified](#). While the UAA is used by [Cloud Foundry](#), it is also a fully featured stand alone OAuth2 server with enterprise features such as [LDAP integration](#).

Requirements

Checkout, Build and Run UAA:

- Make sure you use Java 8
- [Git](#) installed
- You need the [CloudFoundry UAA Command Line Client](#) installed
- Use a different host name for UAA when running on the same machine, e.g. [uaa/](#)

In case you run into issues installing *uaac*, you may have to set the `GEM_HOME` environment variable:

```
export GEM_HOME="$HOME/.gem"
```

BASH

and add `~/.gem/gems/cf-uaac-4.2.0/bin` to your path.

Prepare UAA for JWT

As the UAA is an OpenID provider it uses JSON Web Tokens (JWT) it needs to have a private key for signing those JWTs:

```
openssl genrsa -out signingkey.pem 2048
openssl rsa -in signingkey.pem -pubout -out verificationkey.pem
export JWT_TOKEN_SIGNING_KEY=$(cat signingkey.pem)
export JWT_TOKEN_VERIFICATION_KEY=$(cat verificationkey.pem)
```

BASH

Later, once the UAA is started you can see the keys when accessing [uaa:8080/uaa/token_keys](#)

Here, the name `uaa` in the URL [uaa:8080/uaa/token_keys](#) is the hostname.

Download + Start UAA

```
git clone https://github.com/pivotal/uaa-bundled.git
cd uaa-bundled
./mvnw clean install
java -jar target/uaa-bundled-1.0.0.BUILD-SNAPSHOT.jar
```

BASH

The configuration of the UAA is driven by either a Yaml file `uaa.yml` or you can script the configuration using the UAA Command Line Client:

```
uaac target http://uaa:8080/uaa
uaac token client get admin -s adminsecret
uaac client add dataflow \
  --name dataflow \
  --secret dataflow \
  --scope cloud_controller.read,cloud_controller.write,openid,password.write,scim.userids,foo.create,foo.view,dataflow.crea
  --authorized_grant_types password,authorization_code,client_credentials,refresh_token \
  --authorities uaa.resource,dataflow.create,dataflow.deploy,dataflow.destroy,dataflow.manage,dataflow.modify,dataflow.sche
  --redirect_uri http://localhost:9393/login \
  --autoapprove openid

uaac group add "foo.view"
uaac group add "foo.create"
uaac group add "dataflow.view"
uaac group add "dataflow.create"

uaac user add springrocks -p mysecret --emails springrocks@someplace.com
uaac user add vieweronly -p mysecret --emails mrviewer@someplace.com

uaac member add "foo.view" springrocks
uaac member add "foo.create" springrocks
uaac member add "dataflow.view" springrocks
uaac member add "dataflow.create" springrocks
uaac member add "foo.view" vieweronly
```

BASH

This script will set up the dataflow client as well as 2 users:

- User *springrocks* will have both scopes `foo.view` and `foo.create`
- User *vieweronly* will only have one scope `foo.view`

Once added, you can quickly double-check that the UAA has the users created:

```
curl -v -d"username=springrocks&password=mysecret&client_id=dataflow&grant_type=password" -u "dataflow:dataflow" http://uaa
```

BASH

This should produce output similar to the following:

```
* Trying 127.0.0.1...
* TCP_NODELAY set
* Connected to uaa (127.0.0.1) port 8080 (#0)
* Server auth using Basic with user 'dataflow'
> POST /uaa/oauth/token HTTP/1.1
> Host: uaa:8080
> Authorization: Basic ZGF0YWZsb3c6ZGF0YWZsb3c=
> User-Agent: curl/7.54.0
> Accept: */*
> Content-Length: 97
> Content-Type: application/x-www-form-urlencoded
>
```

BASH


```
* upload completely sent off: 97 out of 97 bytes
< HTTP/1.1 200
< Cache-Control: no-store
< Pragma: no-cache
< X-XSS-Protection: 1; mode=block
< X-Frame-Options: DENY
< X-Content-Type-Options: nosniff
< Content-Type: application/json; charset=UTF-8
< Transfer-Encoding: chunked
< Date: Thu, 31 Oct 2019 21:22:59 GMT
<
* Connection #0 to host uaa left intact
{"access_token":"0329c8ecdf594ee78c271e022138be9d","token_type":"bearer","id_token":"eyJhbGciOiJIUzI1NiIsImprdiSI6Imh0dHBzOi"}
```

Using `token_format` parameter you can requested token to be either:

- opaque
- jwt

Start Skipper

```
git clone https://github.com/spring-cloud/spring-cloud-skipper.git
cd spring-cloud/spring-cloud-skipper
./mvnw clean package -DskipTests=true
java -jar spring-cloud-skipper-server/target/spring-cloud-skipper-server-2.2.0.BUILD-SNAPSHOT.jar
```

BASH

Start Spring Cloud Data Flow

```
git clone https://github.com/spring-cloud/spring-cloud-dataflow.git
cd spring-cloud-dataflow
./mvnw clean package -DskipTests=true
cd ..
```

BASH

Create a yaml file `scdf.yml` with the following contents:

```
spring:
  cloud:
    dataflow:
      security:
        authorization:
          provider-role-mappings:
            uaa:
              map-oauth-scopes: true
              role-mappings:
                ROLE_CREATE: foo.create
                ROLE_DEPLOY: foo.create
                ROLE_DESTROY: foo.create
                ROLE_MANAGE: foo.create
                ROLE_MODIFY: foo.create
                ROLE_SCHEDULE: foo.create
                ROLE_VIEW: foo.view
      security:
        oauth2:
          client:
            registration:
              uaa:
                redirect-uri: '{baseUrl}/login/oauth2/code/{registrationId}'
                authorization-grant-type: authorization_code
                client-id: dataflow
                client-secret: dataflow
                scope:
                  - openid
```

YAML

```

- foo.create
- foo.view
provider:
  uaa:
    jwk-set-uri: http://uaa:8080/uaa/token_keys
    token-uri: http://uaa:8080/uaa/oauth/token
    user-info-uri: http://uaa:8080/uaa/userinfo
    user-name-attribute: user_name
    authorization-uri: http://uaa:8080/uaa/oauth/authorize
  resourceserver:
    opaquetoken:
      introspection-uri: http://uaa:8080/uaa/introspect
      client-id: dataflow
      client-secret: dataflow

```

- 1 If you use scopes to identify roles, please make sure to also request the relevant scopes, e.g. `dataflow.view`, `dataflow.create` and don't forget to request the `openid` scope
- 2 Used to retrieve profile information, e.g. username for display purposes (mandatory)
- 3 Used for token introspection and validation (mandatory)

The `introspection-uri` property is especially important when passing an externally retrieved (opaque) OAuth Access Token to Spring Cloud Data Flow. In that case Spring Cloud Data Flow will take the OAuth Access, and use the UAA's [Introspect Token Endpoint](#) to not only check the validity of the token but also retrieve the associated OAuth scopes from the UAA

Finally startup Spring Cloud Data Flow:

```

java -jar spring-cloud-dataflow/spring-cloud-dataflow-server/target/spring-cloud-dataflow-server-2.4.0.BUILD-SNAPSHOT.jar -

```

Role Mappings

By default all roles are assigned to users that login to Spring Cloud Data Flow. However, you can set the property:

```
spring.cloud.dataflow.security.authorization.provider-role-mappings.uaa.map-oauth-scopes: true
```

This will instruct the underlying `DefaultAuthoritiesExtractor` to map OAuth scopes to the respective authorities. The following scopes are supported:

- Scope `dataflow.create` maps to the `CREATE` role
- Scope `dataflow.deploy` maps to the `DEPLOY` role
- Scope `dataflow.destroy` maps to the `DESTROY` role
- Scope `dataflow.manage` maps to the `MANAGE` role
- Scope `dataflow.modify` maps to the `MODIFY` role
- Scope `dataflow.schedule` maps to the `SCHEDULE` role
- Scope `dataflow.view` maps to the `VIEW` role

Additionally you can also map arbitrary scopes to each of the Data Flow roles:

```

spring:
  cloud:
    dataflow:
      security:

```

YAML

```

authorization:
  provider-role-mappings:
    uaa:
      map-oauth-scopes: true          1
      role-mappings:
        ROLE_CREATE: dataflow.create  2
        ROLE_DEPLOY: dataflow.deploy
        ROLE_DESTROY: dataflow.destroy
        ROLE_MANAGE: dataflow.manage
        ROLE_MODIFY: dataflow.modify
        ROLE_SCHEDULE: dataflow.schedule
        ROLE_VIEW: dataflow.view

```

- 1 Enables explicit mapping support from OAuth scopes to Data Flow roles
- 2 When role mapping support is enabled, you must provide a mapping for all 7 Spring Cloud Data Flow roles
ROLE_CREATE, ROLE_DEPLOY, ROLE_DESTROY, ROLE_MANAGE, ROLE_MODIFY, ROLE_SCHEDULE, ROLE_VIEW.

You can assign an OAuth scope to multiple Spring Cloud Data Flow roles, giving you flexible regarding the granularity of your authorization configuration.

9.7.5. LDAP Authentication

LDAP Authentication (Lightweight Directory Access Protocol) is indirectly provided by Spring Cloud Data Flow using the UAA. The UAA itself provides [comprehensive LDAP support](#).

While you may use your own OAuth2 authentication server, the LDAP support documented here requires using the UAA as authentication server. For any other provider, please consult the documentation for that particular provider.

The UAA supports authentication against an LDAP (Lightweight Directory Access Protocol) server using the following modes:

- [Direct bind](#)
- [Search and bind](#)
- [Search and Compare](#)

When integrating with an external identity provider such as LDAP, authentication within the UAA becomes **chained**. UAA first attempts to authenticate with a user's credentials against the UAA user store before the external provider, LDAP. For more information, see [Chained Authentication](#) in the *User Account and Authentication LDAP Integration* GitHub documentation.

LDAP Role Mapping

The OAuth2 authentication server (UAA), provides comprehensive support for [mapping LDAP groups to O](#).

The following options exist:

- `ldap/ldap-groups-null.xml` No groups will be mapped
- `ldap/ldap-groups-as-scopes.xml` Group names will be retrieved from an LDAP attribute. E.g. `CN`
- `ldap/ldap-groups-map-to-scopes.xml` Groups will be mapped to UAA groups using the `external_group_mapping` table

These values are specified via the configuration property `ldap.groups.file` controls. Under the covers these values reference a Spring XML configuration file.

During test and development it might be necessary to make frequent changes to LDAP groups and users and see those reflected in the UAA. However, user information is cached for the duration of the login. The following script helps to retrieve the updated information quickly:

```
#!/bin/bash
uaac token delete --all
uaac target http://localhost:8080/uaa
uaac token owner get cf <username> -s "" -p <password>
uaac token client get admin -s adminsecret
uaac user get <username>
```

BASH

LDAP Security and UAA Example Application

In order to get up and running quickly and to help you understand the security architecture, we provide the [LDAP Security and UAA Example](#) on GitHub.

This is solely a demo/example application and shall not be used in production.

The setup consists of:

- Spring Cloud Data Flow Server
- Skipper Server
- CloudFoundry User Account and Authentication (UAA) Server
- Lightweight Directory Access Protocol (LDAP) Server (provided by [Apache Directory Server](#) (ApacheDS))

Ultimately, as part of this example, you will learn how to configure and launch a Composed Task using this security setup.

9.7.6. Spring Security OAuth2 Resource/Authorization Server Sample

For local testing and development, you may also use the Resource and Authorization Server support provided by [Spring Security OAuth](#). It allows you to easily create your own (very basic) OAuth2 Server with the following simple annotations:

- `@EnableResourceServer`

- @EnableAuthorizationServer

In fact the UAA uses Spring Security OAuth2 under the covers, thus the basic endpoints are the same.

A working example application can be found at: <https://github.com/ghillert/oauth-test-server/>

Clone the project and configure Spring Cloud Data Flow with the respective Client ID and Client Secret:

```
security:
  oauth2:
    client:
      client-id: myclient
      client-secret: mysecret
      access-token-uri: http://127.0.0.1:9999/oauth/token
      user-authorization-uri: http://127.0.0.1:9999/oauth/authorize
    resource:
      user-info-uri: http://127.0.0.1:9999/me
      token-info-uri: http://127.0.0.1:9999/oauth/check_token
```

YAML

This sample application is not intended for production use

9.7.7. Data Flow Shell Authentication

When using the Shell, the credentials can either be provided via username and password or by specifying a *credentials-provider* command. If your OAuth2 provider supports the *Password* Grant Type you can start the *Data Flow Shell* with:

```
$ java -jar spring-cloud-dataflow-shell-2.4.0.RELEASE.jar \
--dataflow.uri=http://localhost:9393 \ 1
--dataflow.username=my_username \ 2
--dataflow.password=my_password \ 3
--skip-ssl-validation true \ 4
```

BASH

- 1 Optional, defaults to [localhost:9393](#).
- 2 Mandatory.
- 3 If the password is not provided, the user is prompted for it.
- 4 Optional, defaults to `false`, ignores certificate errors (when using self-signed certificates). Use cautiously!

Keep in mind that when authentication for Spring Cloud Data Flow is enabled, the underlying OAuth2 provider **must** support the *Password* OAuth2 Grant Type if you want to use the Shell via username/password authentication.

From within the Data Flow Shell you can also provide credentials by using the following command:

```
server-unknown:>dataflow config server \
--uri http://localhost:9393 \ 1
--username myuser \ 2
--password mysecret \ 3
--skip-ssl-validation true \ 4
```

- 1 Optional, defaults to `localhost:9393`.
- 2 Mandatory..
- 3 If security is enabled, and the password is not provided, the user is prompted for it.
- 4 Optional, ignores certificate errors (when using self-signed certificates). Use cautiously!

The following image shows a typical shell command to connect to and authenticate a Data Flow Server:

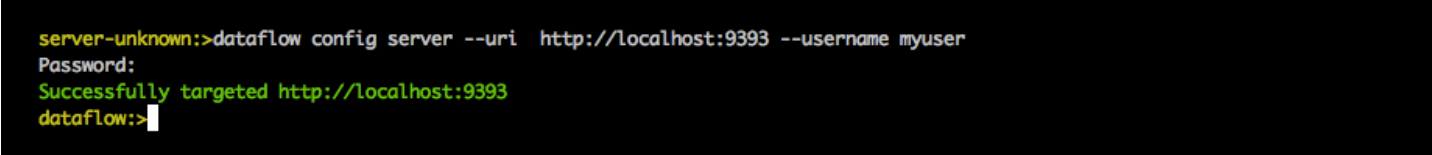


Figure 1. Target and Authenticate with the Data Flow Server from within the Shell

Once successfully targeted, you should see the following output:

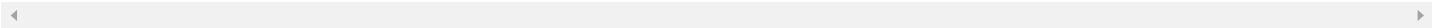
```
dataflow:>dataflow config info
dataflow config info
```

Credentials	[username='my_username, password=****']
Result	
Target	http://localhost:9393

Alternatively, you can specify the `credentials-provider` command in order to pass-in a bearer token directly, instead of providing a username and password. This works from within the shell or by providing the `--dataflow.credentials-provider-command` command-line argument when starting the Shell.

When using the `credentials-provider` command, please be aware that your specified command **must** return a *Bearer token* (Access Token prefixed with *Bearer*). For instance, in Unix environments the following simplistic command can be used:

```
$ java -jar spring-cloud-dataflow-shell-2.4.0.RELEASE.jar \
--dataflow.uri=http://localhost:9393 \
--dataflow.credentials-provider-command="echo Bearer 123456789"
```



9.8. About Configuration

The Spring Cloud Data Flow About Restful API result contains a display name, version, and, if specified, a URL for each of the major dependencies that comprise Spring Cloud Data Flow. The result (if enabled) also contains the sha1 and or sha256

checksum values for the shell dependency. The information that is returned for each of the dependencies is configurable by setting the following properties:

- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-core.name`: the name to be used for the core.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-core.version`: the version to be used for the core.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-dashboard.name`: the name to be used for the dashboard.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-dashboard.version`: the version to be used for the dashboard.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-implementation.name`: the name to be used for the implementation.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-implementation.version`: the version to be used for the implementation.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.name`: the name to be used for the shell.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.version`: the version to be used for the shell.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.url`: the URL to be used for downloading the shell dependency.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha1`: the sha1 checksum value that is returned with the shell dependency info.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha256`: the sha256 checksum value that is returned with the shell dependency info.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha1-url`: if the `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha1` is not specified, SCDF uses the contents of the file specified at this URL for the checksum.
- `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha256-url`: if the `spring.cloud.dataflow.version-info.spring-cloud-dataflow-shell.checksum-sha256` is not specified, SCDF uses the contents of the file specified at this URL for the checksum.

9.8.1. Enabling Shell Checksum values

By default, checksum values are not displayed for the shell dependency. If you need this feature enabled, set the `spring.cloud.dataflow.version-info.dependency-fetch.enabled` property to `true`.

9.8.2. Reserved Values for URLs

There are reserved values (surrounded by curly braces) that you can insert into the URL that will make sure that the links are up to date:

- `repository`: if using a build-snapshot, milestone, or release candidate of Data Flow, the repository refers to the repository-io repository. Otherwise, it refers to Maven Central.
- `version`: Inserts the version of the jar/pom.

For example, `myrepository/org/springframework/cloud/spring-cloud-dataflow-shell/{version}/spring-cloud-dataflow-shell-{version}.jar` produces `myrepository/org/springframework/cloud/spring-cloud-dataflow-shell/1.2.3.RELEASE/spring-cloud-dataflow-shell-1.2.3.RELEASE.jar` if you were using the 1.2.3.RELEASE version of the Spring Cloud Data Flow Shell

10. Configuration - Cloud Foundry

This section describes how to configure Spring Cloud Data Flow server's features, such as security and which relational database to use. It also describes how to configure Spring Cloud Data Flow shell's features.

10.1. Feature Toggles

Data Flow server offers a specific set of features that you can enable or disable when launching. These features include all the lifecycle operations and REST endpoints (server, client implementations including Shell and the UI) for:

- Streams
- Tasks

You can enable or disable these features by setting the following boolean properties when you launch the Data Flow server:

- `spring.cloud.dataflow.features.streams-enabled`
- `spring.cloud.dataflow.features.tasks-enabled`

By default, all features are enabled.

The REST endpoint (`/features`) provides information on the enabled and disabled features.

10.2. Deployer Properties

You can use the following configuration properties of the Data Flow server's [Cloud Foundry deployer](#) to customize how applications are deployed. When deploying with the Data Flow shell, you can use the syntax `deployer`.

`<appName>.cloudfoundry.<deployerPropertyName>`. See below for an example shell usage. These properties are also used when configuring the [Cloud Foundry Task platforms](#) in the Data Flow server and and Kubernetes platforms in Skipper for deploying Streams.

Deployer Property Name	Description	Default Value
services	The names of services to bind to the deployed application.	<none>
host	The host name to use as part of the route.	hostname derived by Cloud Foundry
domain	The domain to use when mapping routes for the application.	<none>
routes	The list of routes that the application should be bound to. Mutually exclusive with host and domain.	<none>
buildpack	The buildpack to use for deploying the application.	github.com/cloudfoundry/java-buildpack.git#v4.16.1

Deployer Property Name	Description	Default Value
memory	The amount of memory to allocate. Default unit is mebibytes, 'M' and 'G' suffixes supported	1024m
disk	The amount of disk space to allocate. Default unit is mebibytes, 'M' and 'G' suffixes supported.	1024m
healthCheck	The type of health check to perform on deployed application. Values can be HTTP, NONE, PROCESS, and PORT	PORT
healthCheckHttpEndpoint	The path that the http health check will use,	/health
healthCheckTimeout	The timeout value for health checks in seconds.	120
instances	The number of instances to run.	1
enableRandomAppNamePrefix	Flag to enable prefixing the app name with a random prefix.	true
apiTimeout	Timeout for blocking API calls, in seconds.	360
statusTimeout	Timeout for status API operations in milliseconds	5000
useSpringApplicationJson	Flag to indicate whether application properties are fed into SPRING_APPLICATION_JSON or as environment variables.	true
stagingTimeout	Timeout allocated for staging the application.	15 minutes
startupTimeout	Timeout allocated for starting the application.	5 minutes
appNamePrefix	String to use as prefix for name of deployed application	The Spring Boot property <code>spring.application.name</code> of the application that is using the deployer library.
deleteRoutes	Whether to also delete routes when un-deploying an application.	true
javaOpts	The Java Options to pass to the JVM, e.g -Dtest=foo	<none>

Deployer Property Name	Description	Default Value
pushTasksEnabled	Whether to push task applications or assume that the application already exists when launched.	true
autoDeleteMavenArtifacts	Whether to automatically delete Maven artifacts from the local repository when deployed.	true

Here are some examples using the Cloud Foundry deployment properties:

- You can set the buildpack that is used to deploy each application. For example, to use the Java offline buildback, set the following environment variable:

```
cf set-env dataflow-server SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_BUILDPACK java_bui
```

- You can customize the health check mechanism used by Cloud Foundry to assert whether apps are running by using the `SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_HEALTH_CHECK` environment variable. The current supported options are `http` (the default), `port`, and `none`.

You can also set environment variables that specify the HTTP-based health check endpoint and timeout:

`SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_HEALTH_CHECK_ENDPOINT` and `SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_HEALTH_CHECK_TIMEOUT`, respectively. These default to `/health` (the Spring Boot default location) and `120` seconds.

- You can also specify deployment properties by using the DSL. For instance, if you want to set the allocated memory for the `http` application to 512m and also bind a `mysql` service to the `jdbc` application, you can run the following commands:

```
dataflow:> stream create --name mysqlstream --definition "http | jdbc --tableName=names --columns=name"
dataflow:> stream deploy --name mysqlstream --properties "deployer.http.memory=512, deployer.jdbc.cloudfoundry.services=mys
```

You can configure these settings separately for stream and task apps. To alter settings for tasks, substitute `TASK` for `STREAM` in the property name, as the following example shows:

```
cf set-env dataflow-server SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_MEMORY 512
```

10.3. Tasks

The Data Flow server is responsible for deploying Tasks. Tasks that are launched by Data Flow write their state to the same database that is used by the Data Flow server. For Tasks which are Spring Batch Jobs, the job and step execution data is

also stored in this database. As with Skipper, Tasks can be launched to multiple platforms. When Data Flow is running on Cloud Foundry, a Task platform must be defined. To configure new platform accounts that target Cloud Foundry, provide an entry under the `spring.cloud.dataflow.task.platform.cloudfoundry` section in your `application.yaml` file for via another Spring Boot supported mechanism. In the following example, two Cloud Foundry platform accounts named `dev` and `qa` are created. The keys such as `memory` and `disk` are [Cloud Foundry Deployer Properties](#).

YAML

```
spring:
  cloud:
    dataflow:
      task:
        platform:
          cloudfoundry:
            accounts:
              dev:
                connection:
                  url: https://api.run.pivotal.io
                  org: myOrg
                  space: mySpace
                  domain: cfapps.io
                  username: user@example.com
                  password: drowssap
                  skipSslValidation: false
                deployment:
                  memory: 512m
                  disk: 2048m
                  instances: 4
                  services: rabbit,mysql
                  appNamePrefix: dev1
              qa:
                connection:
                  url: https://api.run.pivotal.io
                  org: myOrgQA
                  space: mySpaceQA
                  domain: cfapps.io
                  username: user@example.com
                  password: drowssap
                  skipSslValidation: true
                deployment:
                  memory: 756m
                  disk: 724m
                  instances: 2
                  services: rabbitQA,mysqlQA
                  appNamePrefix: qa1
```

By defining one platform as `default` allows you to skip using `platformName` where its use would otherwise be required.

When launching a task, pass the value of the platform account name using the task launch option `--platformName`. If you do not pass a value for `platformName`, the value `default` will be used.

When deploying a task to multiple platforms, the configuration of the task needs to connect to the same database as the Data Flow Server.

You can configure the Data Flow server that is on Cloud Foundry to deploy tasks to Cloud Foundry or Kubernetes. See the section on [Kubernetes Task Platform Configuration](#) for more information.

10.4. Application Names and Prefixes

To help avoid clashes with routes across spaces in Cloud Foundry, a naming strategy that provides a random prefix to a deployed application is available and is enabled by default. You can override the [default configurations](#) and set the respective properties by using `cf set-env` commands.

For instance, if you want to disable the randomization, you can override it by using the following command:

```
cf set-env dataflow-server SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_ENABLE_RANDOM_APP_
```

10.5. Custom Routes

As an alternative to a random name or to get even more control over the hostname used by the deployed apps, you can use custom deployment properties, as the following example shows:

```
dataflow:>stream create foo --definition "http | log"
sdataflow:>stream deploy foo --properties "deployer.http.cloudfoundry.domain=mydomain.com,
deployer.http.cloudfoundry.host=myhost,
deployer.http.cloudfoundry.route-path=my-path"
```

The preceding example binds the `http` app to the `myhost.mydomain.com/my-path` URL. Note that this example shows **all** of the available customization options. In practice, you can use only one or two out of the three.

10.6. Docker Applications

Starting with version 1.2, it is possible to register and deploy Docker based apps as part of streams and tasks by using Data Flow for Cloud Foundry.

If you use Spring Boot and RabbitMQ-based Docker images, you can provide a common deployment property to facilitate binding the apps to the RabbitMQ service. Assuming your RabbitMQ service is named `rabbit`, you can provide the following:

```
cf set-env dataflow-server SPRING_APPLICATION_JSON '{"spring.cloud.dataflow.applicationProperties.stream.spring.rabbitmq.ad
```

For Spring Cloud Task apps, you can use something similar to the following, if you use a database service instance named `mysql`:

```
cf set-env SPRING_DATASOURCE_URL '${vcap.services.mysql.credentials.jdbcUrl}'
cf set-env SPRING_DATASOURCE_USERNAME '${vcap.services.mysql.credentials.username}'
cf set-env SPRING_DATASOURCE_PASSWORD '${vcap.services.mysql.credentials.password}'
cf set-env SPRING_DATASOURCE_DRIVER_CLASS_NAME 'org.mariadb.jdbc.Driver'
```

For non-Java or non-Boot applications, your Docker app must parse the `VCAP_SERVICES` variable in order to bind to any available services.

Passing application properties

When using non-Boot applications, chances are that you want to pass the application properties by using traditional environment variables, as opposed to using the special `SPRING_APPLICATION_JSON` variable. To do so, set the following variables for streams and tasks, respectively:

```
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_USE_SPRING_APPLICATION_JSON=PROPERTIES=false
```

10.7. Application-level Service Bindings

When deploying streams in Cloud Foundry, you can take advantage of application-specific service bindings, so not all services are globally configured for all the apps orchestrated by Spring Cloud Data Flow.

For instance, if you want to provide a `mysql` service binding only for the `jdbc` application in the following stream definition, you can pass the service binding as a deployment property:

```
dataflow:>stream create --name httptojdbc --definition "http | jdbc"
dataflow:>stream deploy --name httptojdbc --properties "deployer.jdbc.cloudfoundry.services=mysqlService" BASH
```

where `mysqlService` is the name of the service specifically bound only to the `jdbc` application and the `http` application does not get the binding by this method.

If you have more than one service to bind, they can be passed as comma-separated items (for example: `deployer.jdbc.cloudfoundry.services=mysqlService,someService`).

10.8. Configuring Service binding parameters

The CloudFoundry API supports providing configuration parameters when binding a service instance. Some service brokers require or recommend binding configuration. For example, binding the [Google Cloud Platform service](#) using the CF CLI looks something like:

```
cf bind-service my-app my-google-bigquery-example -c '{"role":"bigquery.user"}'
```

Likewise the [NFS Volume Service](#) supports binding configuration such as:

```
cf bind-service my-app nfs_service_instance -c '{"uid":"1000","gid":"1000","mount":"/var/volume1","readonly":true}'
```

Starting with version 2.0, Data Flow for Cloud Foundry allows you to provide binding configuration parameters may be provided in the app level or server level `cloudfoundry.services` deployment property. For example, to bind to the `nfs` service, as above :

```
dataflow:> stream deploy --name mystream --properties "deployer.<app>.cloudfoundry.services='nfs_service_instance uid:1000,"
```

The format is intended to be compatible with the Data Flow DSL parser. Generally, the `cloudfoundry.services` deployment property accepts a comma delimited value. Since a comma is also used to separate configuration parameters, and to avoid white space issues, any item including configuration parameters must be enclosed in single quotes. Valid values include things like:

```
rabbitmq,'nfs_service_instance uid:1000,gid:1000,mount:/var/volume1,readonly:true',mysql,'my-google-bigquery-example role:b
```

Spaces are permitted within single quotes and `=` may be used instead of `:` to delimit key-value pairs.

10.9. User-provided Services

In addition to marketplace services, Cloud Foundry supports [User-provided Services](#) (UPS). Throughout this reference manual, regular services have been mentioned, but there is nothing precluding the use of User-provided Services as well, whether for use as the messaging middleware (for example, if you want to use an external Apache Kafka installation) or for use by some of the stream applications (for example, an Oracle Database).

Now we review an example of extracting and supplying the connection credentials from a UPS.

The following example shows a sample UPS setup for Apache Kafka:

```
cf create-user-provided-service kafkacups -p '{"brokers":"HOST:PORT","zkNodes":"HOST:PORT"}'
```

BASH

The UPS credentials are wrapped within `VCAP_SERVICES`, and they can be supplied directly in the stream definition, as the following example shows.

```
stream create fooz --definition "time | log"
stream deploy fooz --properties "app.time.spring.cloud.stream.kafka.binder.brokers=${vcap.services.kafkacups.credentials.br
```

BASH

10.10. Database Connection Pool

As of Data Flow 2.0, the Spring Cloud Connector library is no longer used to create the `DataSource`. The library [java-cfenv](#) is now used which allows you to set [Spring Boot properties](#) to configure the connection pool.

10.11. Maximum Disk Quota

By default, every application in Cloud Foundry starts with 1G disk quota and this can be adjusted to a default maximum of 2G. The default maximum can also be overridden up to 10G by using Pivotal Cloud Foundry's (PCF) Ops Manager GUI.

This configuration is relevant for Spring Cloud Data Flow because every task deployment is composed of applications (typically Spring Boot uber-jar's), and those applications are resolved from a remote maven repository. After resolution, the application artifacts are downloaded to the local Maven Repository for caching and reuse. With this happening in the background, the default disk quota (1G) can fill up rapidly, especially when we experiment with streams that are made up of unique applications. In order to overcome this disk limitation and depending on your scaling requiren

want to change the default maximum from 2G to 10G. Let's review the steps to change the default maximum disk quota allocation.

10.11.1. PCF's Operations Manager

From PCF's Ops Manager, select the "Pivotal Elastic Runtime" tile and navigate to the "Application Developer Controls" tab. Change the "Maximum Disk Quota per App (MB)" setting from 2048 (2G) to 10240 (10G). Save the disk quota update and click "Apply Changes" to complete the configuration override.

10.12. Scale Application

Once the disk quota change has been successfully applied and assuming you have a [running application](#), you can scale the application with a new `disk_limit` through the CF CLI, as the following example shows:

```
→ cf scale dataflow-server -k 10GB
```

BASH

```
Scaling app dataflow-server in org ORG / space SPACE as user...
OK
```

```
....
....
....
....
```

	state	since	cpu	memory	disk	details
#0	running	2016-10-31 03:07:23 PM	1.8%	497.9M of 1.1G	193.9M of 10G	

You can then list the applications and see the new maximum disk space, as the following example shows:

```
→ cf apps
```

BASH

```
Getting apps in org ORG / space SPACE as user...
OK
```

name	requested state	instances	memory	disk	urls
dataflow-server	started	1/1	1.1G	10G	dataflow-server.apps.io

10.13. Managing Disk Use

Even when configuring the Data Flow server to use 10G of space, there is the possibility of exhausting the available space on the local disk. To prevent this, `jar` artifacts downloaded from external sources, i.e., apps registered as `http` or `maven` resources, are automatically deleted whenever the application is deployed, whether or not the deployment request succeeds. This behavior is optimal for production environments in which container runtime stability is more critical than I/O latency incurred during deployment. In development environments deployment happens more frequently. Additionally, the `jar` artifact (or a lighter `metadata jar`) contains metadata describing application configuration properties which is used by various operations related to application configuration, more frequently performed during pre-production activities. To provide a more responsive interactive developer experience at the expense of more disk usage in pre-production environments, you can set the CloudFoundry deployer property `autoDeleteMavenArtifacts` to `false`.

If you deploy the Data Flow server by using the default `port` health check type, you must explicitly monitor the disk space on the server in order to avoid running out space. If you deploy the server by using the `http` health check type (see the next example), the Data Flow server is restarted if there is low disk space. This is due to Spring Boot's [Disk Space Health Indicator](#). You can [configure](#) the settings of the Disk Space Health Indicator by using the properties that have the `management.health.diskspace` prefix.

For version 1.7, we are investigating the use of [Volume Services](#) for the Data Flow server to store `.jar` artifacts before pushing them to Cloud Foundry.

The following example shows how to deploy the `http` health check type to an endpoint called `/management/health`:

```
---
...
health-check-type: http
health-check-http-endpoint: /management/health
```

10.14. Application Resolution Alternatives

Though we recommend using a Maven Artifactory for application [resolution and registration](#), there might be situations where one of the following alternative approaches would make sense.

- We have custom-built and maintain a [SCDF APP Tool](#) that can run as a regular Spring Boot application in Cloud Foundry, but it will in turn host and serve the application JARs for SCDF at runtime.
- With the help of Spring Boot, we can serve [static content](#) in Cloud Foundry. A simple Spring Boot application can bundle all the required stream and task applications. By having it run on Cloud Foundry, the static application can then serve the über-jar's. From the shell, you can, for example, register the application with the name `http-source.jar` by using `--uri=http://<Route-To-StaticApp>/http-source.jar`.
- The über-jar's can be hosted on any external server that's reachable over HTTP. They can be resolved from raw GitHub URIs as well. From the shell, you can, for example, register the app with the name `http-source.jar` by using `--uri=http://<Raw_GitHub_URI>/http-source.jar`.
- [Static Buildpack](#) support in Cloud Foundry is another option. A similar HTTP resolution works on this model, too.
- [Volume Services](#) is another great option. The required über-jars can be hosted in an external file system. With the help of volume-services, you can, for example, register the application with the name `http-source.jar` by using `--uri=file://<Path-To-FileSystem>/http-source.jar`.

10.15. Security

By default, the Data Flow server is unsecured and runs on an unencrypted HTTP connection. You can secure your REST endpoints (as well as the Data Flow Dashboard) by enabling HTTPS and requiring clients to authenticate. For more details about securing the REST endpoints and configuring to authenticate against an OAuth backend (UAA and SSO running on Cloud Foundry), see the security section from the core [reference guide](#). You can configure the security details in `dataflow-server.yml` or pass them as environment variables through `cf set-env` commands.

10.15.1. Authentication

Spring Cloud Data Flow can either integrate with Pivotal Single Sign-On Service (for example, on PWS) or Cloud Foundry User Account and Authentication (UAA) Server.

Pivotal Single Sign-On Service

When deploying Spring Cloud Data Flow to Cloud Foundry, you can bind the application to the Pivotal Single Sign-On Service. By doing so, Spring Cloud Data Flow takes advantage of the [Java CFEnv](#), which provides Cloud Foundry-specific auto-configuration support for OAuth 2.0.

To do so, bind the Pivotal Single Sign-On Service to your Data Flow Server application and provide the following properties:

```
SPRING_CLOUD_DATAFLOW_SECURITY_CFUSEUAA: false
SECURITY_OAUTH2_CLIENT_CLIENTID: "${security.oauth2.client.clientId}"
SECURITY_OAUTH2_CLIENT_CLIENTSECRET: "${security.oauth2.client.clientSecret}"
SECURITY_OAUTH2_CLIENT_ACESSTOKENURI: "${security.oauth2.client.accessTokenUri}"
SECURITY_OAUTH2_CLIENT_USERAUTHORIZATIONURI: "${security.oauth2.client.userAuthorizationUri}"
SECURITY_OAUTH2_RESOURCE_USERINFOURI: "${security.oauth2.resource.userInfoUri}"
```

1

YAML

- 1 It is important that the property `spring.cloud.dataflow.security.cf-use-uaa` is set to `false`

Authorization is similarly supported for non-Cloud Foundry security scenarios. See the security section from the core Data Flow [reference guide](#).

As the provisioning of roles can vary widely across environments, we by default assign all Spring Cloud Data Flow roles to users.

You can customize this behavior by providing your own `AuthoritiesExtractor`.

The following example shows one possible approach to set the custom `AuthoritiesExtractor` on the `UserInfoTokenServices`:

```
public class MyUserInfoTokenServicesPostProcessor
    implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) {
        if (bean instanceof UserInfoTokenServices) {
            final UserInfoTokenServices userInfoTokenServices == (UserInfoTokenServices) bean;
            userInfoTokenServices.setAuthoritiesExtractor(ctx.getBean(AuthoritiesExtractor.class));
        }
        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) {
        return bean;
    }
}
```

JAVA

Then you can declare it in your configuration class as follows:

```
@Bean
public BeanPostProcessor myUserInfoTokenServicesPostProcessor() {
    BeanPostProcessor postProcessor == new MyUserInfoTokenServicesPostProcessor();
    return postProcessor;
}
```

JAVA

Cloud Foundry UAA

The availability of Cloud Foundry User Account and Authentication (UAA) depends on the Cloud Foundry environment. In order to provide UAA integration, you have to provide the necessary OAuth2 configuration properties (for example, by setting the `SPRING_APPLICATION_JSON` property).

The following JSON example shows how to create a security configuration:

```
{
  "security.oauth2.client.client-id": "scdf",
```

JSON

```

"security.oauth2.client.client-secret": "scdf-secret",
"security.oauth2.client.access-token-uri": "https://login.cf.myhost.com/oauth/token",
"security.oauth2.client.user-authorization-uri": "https://login.cf.myhost.com/oauth/authorize",
"security.oauth2.resource.user-info-uri": "https://login.cf.myhost.com/userinfo"
}

```

By default, the `spring.cloud.dataflow.security.cf-use-uaa` property is set to `true`. This property activates a special `AuthoritiesExtractor` called `CloudFoundryDataflowAuthoritiesExtractor`.

If you do not use CloudFoundry UAA, you should set `spring.cloud.dataflow.security.cf-use-uaa` to `false`.

Under the covers, this `AuthoritiesExtractor` calls out to the [Cloud Foundry Apps API](#) and ensure that users are in fact Space Developers.

If the authenticated user is verified as a Space Developer, all roles are assigned.

10.16. Configuration Reference

You must provide several pieces of configuration. These are Spring Boot `@ConfigurationProperties`, so you can set them as environment variables or by any other means that Spring Boot supports. The following listing is in environment variable format, as that is an easy way to get started configuring Boot applications in Cloud Foundry. Note that in the future, you will be able to deploy tasks to multiple platforms, but for 2.0.0.M1 you can deploy only to a single platform and the name must be `default`.

```

# Default values appear after the equal signs.
# Example values, typical for Pivotal Web Services, are included as comments.

# URL of the CF API (used when using cf login -a for example) - for example, https://api.run.pivotal.io
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL=

# The name of the organization that owns the space above - for example, youruser-org
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG=

# The name of the space into which modules will be deployed - for example, development
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE=

# The root domain to use when mapping routes - for example, cfapps.io
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_DOMAIN=

# The user name and password of the user to use to create applications
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME=
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD=

# The identity provider to be used when accessing the Cloud Foundry API (optional).
# The passed string has to be a URL-Encoded JSON Object, containing the field origin with value as origin_key of an identit
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_LOGIN_HINT=

# Whether to allow self-signed certificates during SSL validation (you should NOT do so in production)
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SKIP_SSL_VALIDATION=

# A comma-separated set of service instance names to bind to every deployed task application.
# Among other things, this should include an RDBMS service that is used
# for Spring Cloud Task execution reporting, such as my_postgres
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES=
spring.cloud.deployer.cloudfoundry.task.services=

# Timeout, in seconds, to use when doing blocking API calls to Cloud Foundry
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_API_TIMEOUT=

# Timeout, in milliseconds, to use when querying the Cloud Foundry API to compute app status
SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_STATUS_TIMEOUT=

```

Note that you can set `spring.cloud.deployer.cloudfoundry.services`, `spring.cloud.deployer.cloudfoundry.buildpack`, or the Spring Cloud Deployer-standard `spring.cloud.deployer.memory` and `spring.cloud.deployer.disk` as part of an individual deployment request by using the `deployer.<app-name>` shortcut, as the following example shows:

```
stream create --name ticktock --definition "time | log"
stream deploy --name ticktock --properties "deployer.time.memory=2g"
```

BASH

The commands in the preceding example deploy the time source with 2048MB of memory, while the log sink uses the default 1024MB.

When you deploy a stream, you can also pass `JAVA_OPTS` as a deployment property, as the following example shows:

```
stream deploy --name ticktock --properties "deployer.time.cloudfoundry.javaOpts=-Duser.timezone=America/New_York"
```

BASH

10.17. Debugging

If you want to get better insights into what is happening when your streams and tasks are being deployed, you may want to turn on the following features:

- Reactor “stacktraces”, showing which operators were involved before an error occurred. This feature is helpful, as the deployer relies on project reactor and regular stacktraces may not always allow understanding the flow before an error happened. Note that this comes with a performance penalty, so it is disabled by default.

```
spring.cloud.dataflow.server.cloudfoundry.debugReactor == true
```

BASH

- Deployer and Cloud Foundry client library request and response logs. This feature allows seeing a detailed conversation between the Data Flow server and the Cloud Foundry Cloud Controller.

```
logging.level.cloudfoundry-client == DEBUG
```

DATA

10.18. Spring Cloud Config Server

You can use Spring Cloud Config Server to centralize configuration properties for Spring Boot applications. Likewise, both Spring Cloud Data Flow and the applications orchestrated by Spring Cloud Data Flow can be integrated with a configuration server to use the same capabilities.

10.18.1. Stream, Task, and Spring Cloud Config Server

Similar to Spring Cloud Data Flow server, you can configure both the stream and task applications to resolve the centralized properties from the configuration server. Setting the `spring.cloud.config.uri` property for the deployed applications is a common way to bind to the configuration server. See the [Spring Cloud Config Client](#) reference guide for more information. Since this property is likely to be used across all applications deployed by the Data Flow server, the Data Flow server’s `spring.cloud.dataflow.applicationProperties.stream` property for stream applications and `spring.cloud.dataflow.applicationProperties.task` property for task applications can be used to pass the `uri` of the Config Server to each deployed stream or task application. See the section on [common application properties](#) for more information.

Note that, if you use applications from the [App Starters project](#), these applications already embed the `spring-cloud-services-starter-config-client` dependency. If you build your application from scratch and want to add the client side support for config server, you can add a dependency reference to the config server client library. The following snippet shows a Maven example:

XML

```
...
<dependency>
  <groupId>io.pivotal.spring.cloud</groupId>
  <artifactId>spring-cloud-services-starter-config-client</artifactId>
  <version>CONFIG_CLIENT_VERSION</version>
</dependency>
...
```

where `CONFIG_CLIENT_VERSION` can be the latest release of the [Spring Cloud Config Server](#) client for Pivotal Cloud Foundry.

You may see a `WARN` logging message if the application that uses this library cannot connect to the configuration server when the application starts and whenever the `/health` endpoint is accessed. If you know that you are not using config server functionality, you can disable the client library by setting the `SPRING_CLOUD_CONFIG_ENABLED` environment variable to `false`.

10.18.2. Sample Manifest Template

The following SCDF and Skipper `manifest.yml` templates includes the required environment variables for the Skipper and Spring Cloud Data Flow server and deployed applications and tasks to successfully run on Cloud Foundry and automatically resolve centralized properties from `my-config-server` at runtime:

YML

```
---
applications:
- name: data-flow-server
  host: data-flow-server
  memory: 2G
  disk_quota: 2G
  instances: 1
  path: {PATH TO SERVER UBER-JAR}
  env:
    SPRING_APPLICATION_NAME: data-flow-server
    MAVEN_REMOTE_REPOSITORIES_REPO1_URL: https://repo.spring.io/libs-snapshot
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL: https://api.sys.huron.cf-app.com
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG: sabby20
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE: sabby20
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_DOMAIN: apps.huron.cf-app.com
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME: admin
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD: ***
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SKIP_SSL_VALIDATION: true
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES: mysql
    SPRING_CLOUD_SKIPPER_CLIENT_SERVER_URI: https://{skipper-host-name}/api
  services:
  - mysql
  - my-config-server

---
applications:
- name: skipper-server
  host: skipper-server
  memory: 1G
  disk_quota: 1G
  instances: 1
  timeout: 180
  buildpack: java_buildpack
```

```

path: <PATH TO THE DOWNLOADED SKIPPER SERVER UBER-JAR>
env:
  SPRING_APPLICATION_NAME: skipper-server
  SPRING_CLOUD_SKIPPER_SERVER_ENABLE_LOCAL_PLATFORM: false
  SPRING_CLOUD_SKIPPER_SERVER_STRATEGIES_HEALTHCHECK_TIMEOUTINMILLIS: 300000
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL: https://api.local.pcfdev.io
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG: pcfdev-org
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE: pcfdev-space
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_DOMAIN: cfapps.io
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME: admin
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD: admin
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SKIP_SSL_VALIDATION: false
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_DELETE_ROUTES: false
  SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES: rabbit, my-config-server
services:
- mysql
  my-config-server

```

where `my-config-server` is the name of the Spring Cloud Config Service instance running on Cloud Foundry.

By binding the service to Spring Cloud Data Flow server, Spring Cloud Task and via Skipper to all the Spring Cloud Stream applications respectively, we can now resolve centralized properties backed by this service.

10.18.3. Self-signed SSL Certificate and Spring Cloud Config Server

Often, in a development environment, we may not have a valid certificate to enable SSL communication between clients and the backend services. However, the configuration server for Pivotal Cloud Foundry uses HTTPS for all client-to-service communication, so we need to add a self-signed SSL certificate in environments with no valid certificates.

By using the same `manifest.yml` templates listed in the previous section for the server, we can provide the self-signed SSL certificate by setting `TRUST_CERTS: <API_ENDPOINT>`.

However, the deployed applications also require `TRUST_CERTS` as a flat environment variable (as opposed to being wrapped inside `SPRING_APPLICATION_JSON`), so we must instruct the server with yet another set of tokens

(`SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_USE_SPRING_APPLICATION_JSON: false`) for tasks. With this setup, the applications receive their application properties as regular environment variables.

The following listing shows the updated `manifest.yml` with the required changes. Both the Data Flow server and deployed applications get their configuration from the `my-config-server` Cloud Config server (deployed as a Cloud Foundry service).

```

---
applications:
- name: test-server
  host: test-server
  memory: 1G
  disk_quota: 1G
  instances: 1
  path: spring-cloud-dataflow-server-VERSION.jar
  env:
    SPRING_APPLICATION_NAME: test-server
    MAVEN_REMOTE_REPOSITORIES_REPO1_URL: https://repo.spring.io/libs-snapshot
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL: https://api.sys.huron.cf-app.com
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG: sabby20
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE: sabby20
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_DOMAIN: apps.huron.cf-app.com
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME: admin
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD: ***
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SKIP_SSL_VALIDATION: true
    SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES: mysql, config-server
    SPRING_CLOUD_SKIPPER_CLIENT_SERVER_URI: https://<skipper-host-name>/api
    TRUST_CERTS: <API_ENDPOINT> #this is for the server
    SPRING_CLOUD_DATAFLOW_APPLICATION_PROPERTIES_TASK_TRUST_CERTS: <API_ENDPOINT> #this propagates to all tasks
  services:

```

```
- mysql
- my-config-server #this is for the server
```

Also add the `my-config-server` service to the Skipper's manifest environment

```
---
applications:
- name: skipper-server
  host: skipper-server
  memory: 1G
  disk_quota: 1G
  instances: 1
  timeout: 180
  buildpack: java_buildpack
  path: <PATH TO THE DOWNLOADED SKIPPER SERVER UBER-JAR>
  env:
    SPRING_APPLICATION_NAME: skipper-server
    SPRING_CLOUD_SKIPPER_SERVER_ENABLE_LOCAL_PLATFORM: false
    SPRING_CLOUD_SKIPPER_SERVER_STRATEGIES_HEALTHCHECK_TIMEOUTINMILLIS: 300000
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL: <URL>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG: <ORG>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE: <SPACE>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_DOMAIN: <DOMAIN>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME: <USER>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD: <PASSWORD>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES: rabbit, my-config-server #this
services:
- mysql
  my-config-server
```

YML

10.19. Configure Scheduling

This section discusses how to configure Spring Cloud Data Flow to connect to the [PCF-Scheduler](#) as its agent to execute tasks.

Before following these instructions, be sure to have an instance of the PCF-Scheduler service running in your Cloud Foundry space. To create a PCF-Scheduler in your space (assuming it is in your Market Place) execute the following from the CF CLI: `cf create-service scheduler-for-pcf standard <name of service>`. Name of a service is later used to bound running application in *PCF*.

For scheduling, you must add (or update) the following environment variables in your environment:

- Enable scheduling for Spring Cloud Data Flow by setting `spring.cloud.dataflow.features.schedules-enabled` to `true`.
- Bind the task deployer to your instance of PCF-Scheduler by adding the PCF-Scheduler service name to the `SPRING_CLOUD_DATAFLOW_TASK_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES` environment variable.
- Establish the URL to the PCF-Scheduler by setting the `SPRING_CLOUD_SCHEDULER_CLOUDFOUNDRY_SCHEDULER_URL` environment variable.

After creating the preceding configurations, you must create any task definitions that need to be scheduled.

The following sample manifest shows both environment properties configured (assuming you have a PCF-Scheduler service available with the name `myscheduler`):

YML

```
---
applications:
- name: data-flow-server
  host: data-flow-server
  memory: 2G
  disk_quota: 2G
  instances: 1
  path: {PATH TO SERVER UBER-JAR}
  env:
    SPRING_APPLICATION_NAME: data-flow-server
    SPRING_CLOUD_SKIPPER_SERVER_ENABLE_LOCAL_PLATFORM: false
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_URL: <URL>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_ORG: <ORG>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_SPACE: <SPACE>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_DOMAIN: <DOMAIN>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_USERNAME: <USER>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_CONNECTION_PASSWORD: <PASSWORD>
    SPRING_CLOUD_SKIPPER_SERVER_PLATFORM_CLOUDFOUNDRY_ACCOUNTS[default]_DEPLOYMENT_SERVICES: rabbit, myscheduler
    SPRING_CLOUD_DATAFLOW_FEATURES_SCHEDULES_ENABLED: true
    SPRING_CLOUD_SKIPPER_CLIENT_SERVER_URI: https://<skipper-host-name>/api
    SPRING_CLOUD_SCHEDULER_CLOUDFOUNDRY_SCHEDULER_URL: https://scheduler.local.pcfdev.io
  services:
  - mysql
```

Where the `SPRING_CLOUD_SCHEDULER_CLOUDFOUNDRY_SCHEDULER_URL` has the following format: `scheduler.<Domain-Name>` (for example, `scheduler.local.pcfdev.io`). Check the actual address from your *PCF* environment.

11. Configuration - Kubernetes

This section describes how to configure Spring Cloud Data Flow features, such as deployer properties, tasks, and which relational database to use.

11.1. Feature Toggles

Data Flow server offers specific set of features that can be enabled or disabled when launching. These features include all the lifecycle operations, REST endpoints (server and client implementations including Shell and the UI) for:

- Streams
- Tasks
- Schedules

You can enable or disable these features by setting the following boolean environment variables when launching the Data Flow server:

- `SPRING_CLOUD_DATAFLOW_FEATURES_STREAMS_ENABLED`
- `SPRING_CLOUD_DATAFLOW_FEATURES_TASKS_ENABLED`
- `SPRING_CLOUD_DATAFLOW_FEATURES_SCHEDULES_ENABLED`

By default, all the features are enabled.

The `/features` REST endpoint provides information on the features that have been enabled and disabled

11.2. Deployer Properties

You can use the following configuration properties the [Kubernetes deployer](#) to customize how Streams and Tasks are deployed. When deploying with the Data Flow shell, you can use the syntax `deployer.<appName>.kubernetes.<deployerPropertyName>`. These properties are also used when configuring the [Kubernetes task platforms](#) in the Data Flow server and Kubernetes platforms in Skipper for deploying Streams.

Deployer Property Name	Description	Default Value
namespace	Namespace to use	environment variable <code>KUBERNETES_NAMESPACE</code> , otherwise default
deployment.nodeSelector	The node selectors to apply to the deployment in <code>key:value</code> format. Multiple node selectors are comma separated.	<none>
imagePullSecret	Secrets for a access a private registry to pull images.	<none>
imagePullPolicy	The Image Pull Policy to apply when pulling images. Valid options are <code>Always</code> , <code>IfNotPresent</code> , and <code>Never</code> .	IfNotPresent
livenessProbeDelay	Delay in seconds when the Kubernetes liveness check of the app container should start checking its health status.	10
livenessProbePeriod	Period in seconds for performing the Kubernetes liveness check of the app container.	60
livenessProbeTimeout	Timeout in seconds for the Kubernetes liveness check of the app container. If the health check takes longer than this value to return it is assumed as 'unavailable'.	2
livenessProbePath	Path that app container has to respond to for liveness check.	<none>
livenessProbePort	Port that app container has to respond on for liveness check.	<none>
readinessProbeDelay	Delay in seconds when the readiness check of the app container should start checking if the module is fully up and running.	10

Deployer Property Name	Description	Default Value
readinessProbePeriod	Period in seconds to perform the readiness check of the app container.	10
readinessProbeTimeout	Timeout in seconds that the app container has to respond to its health status during the readiness check.	2
readinessProbePath	Path that app container has to respond to for readiness check.	<none>
readinessProbePort	Port that app container has to respond on for readiness check.	<none>
probeCredentialsSecret	The secret name containing the credentials to use when accessing secured probe endpoints.	<none>
limits.memory	The memory limit, maximum needed value to allocate a pod, Default unit is mebibytes, 'M' and 'G' suffixes supported	<none>
limits.cpu	The CPU limit, maximum needed value to allocate a pod	<none>
requests.memory	The memory request, guaranteed needed value to allocate a pod.	<none>
requests.cpu	The CPU request, guaranteed needed value to allocate a pod.	<none>
statefulSet.volumeClaimTemplate.storageClassName	Name of the storage class for a stateful set	<none>
statefulSet.volumeClaimTemplate.storage	The storage amount. Default unit is mebibytes, 'M' and 'G' suffixes supported	<none>
environmentVariables	List of environment variables to set for any deployed app container	<none>
entryPointStyle	Entry point style used for the Docker image. Used to determine how to pass in properties. Can be <code>exec</code> , <code>shell</code> , and <code>boot</code>	<code>exec</code>
createLoadBalancer	Create a "LoadBalancer" for the service created for each app. This facilitates assignment of external IP to app.	false
serviceAnnotations	Service annotations to set for the service created for each application. String of the format <code>annotation1:value1,annotation2:value2</code>	<none>

Deployer Property Name	Description	Default Value
podAnnotations	Pod annotations to set for the pod created for each deployment. String of the format <code>annotation1:value1,annotation2:value2</code>	<none>
jobAnnotations	Job annotations to set for the pod or job created for a job. String of the format <code>annotation1:value1,annotation2:value2</code>	<none>
minutesToWaitForLoadBalancer	Time to wait for load balancer to be available before attempting delete of service (in minutes).	5
maxTerminatedErrorRestarts	Maximum allowed restarts for app that fails due to an error or excessive resource use.	2
maxCrashLoopBackOffRestarts	Maximum allowed restarts for app that is in a CrashLoopBackOff. Values are <code>Always</code> , <code>IfNotPresent</code> , <code>Never</code>	<code>IfNotPresent</code>
volumeMounts	volume mounts expressed in YAML format. e.g. <pre>[{name: 'testhostpath', mountPath: '/test/hostPath'}, {name: 'testpvc', mountPath: '/test/pvc'}, {name: 'testnfs', mountPath: '/test/nfs'}]</pre>	<none>
volumes	The volumes that a Kubernetes instance supports specified in YAML format. e.g. <pre>[{name: testhostpath, hostPath: { path: '/test/override/hostPath' }},{name: 'testpvc', persistentVolumeClaim: { claimName: 'testClaim', readOnly: 'true' }}, {name: 'testnfs', nfs: { server: '10.0.0.1:111', path: '/test/nfs' } }]</pre>	<none>
hostNetwork	The hostNetwork setting for the deployments, see kubernetes.io/docs/api-reference/v1/definitions/#_v1_podspec	false
createDeployment	Create a "Deployment" with a "Replica Set" instead of a "Replication Controller".	true
createJob	Create a "Job" instead of just a "Pod" when launching tasks.	false
containerCommand	Overrides the default entry point command with the provided command and arguments.	<none>
containerPorts	Adds additional ports to expose on the container.	<none>
createNodePort	The explicit port to use when <code>NodePort</code> is the <code>Service</code> type.	<none>

Deployer Property Name	Description	Default Value
deploymentServiceAccountName	Service account name to use for app deployments	<none>
deploymentLabels	Additional labels to add to the deployment in <code>key:value</code> format. Multiple labels are comma separated.	<none>
bootMajorVersion	The Spring Boot major version to use. Currently only used to configure Spring Boot version specific probe paths automatically. Valid options are 1 or 2.	2
tolerations.key	The key to use for the toleration.	<none>
tolerations.effect	The toleration effect. See kubernetes.io/docs/concepts/configuration/taint-and-toleration for valid options.	<none>
tolerations.operator	The toleration operator. See kubernetes.io/docs/concepts/configuration/taint-and-toleration/ for valid options.	<none>
tolerations.tolerationSeconds	The number of seconds defining how long the pod will stay bound to the node after a taint is added.	<none>
tolerations.value	The toleration value to apply, used in conjunction with <code>operator</code> to select to appropriate <code>effect</code> .	<none>
secretKeyRefs.envVarName	The environment variable name to hold the secret data	<none>
secretKeyRefs.secretName	The secret name to access	<none>
secretKeyRefs.dataKey	The key name to obtain secret data from	<none>
configMapKeyRefs.envVarName	The environment variable name to hold the ConfigMap data	<none>
configMapKeyRefs.configMapName	The ConfigMap name to access	<none>
configMapKeyRefs.dataKey	The key name to obtain ConfigMap data from	<none>
maximumConcurrentTasks	The maximum concurrent tasks allowed for this platform instance.	20
podSecurityContext.runAsUser	The numeric user ID to run pod container processes under	<none>
podSecurityContext.fsGroup	The numeric group ID to run pod container processes under	<none>

Deployer Property Name	Description	Default Value
affinity.nodeAffinity	The node affinity expressed in YAML format. e.g. <pre>{ requiredDuringSchedulingIgnoredDuringExecution: { nodeSelectorTerms: [{ matchExpressions: [{ key: 'kubernetes.io/e2e-az-name', operator: 'In', values: ['e2e-az1', 'e2e-az2'] }] }], preferredDuringSchedulingIgnoredDuringExecution: [{ weight: 1, preference: { matchExpressions: [{ key: 'another-node-label-key', operator: 'In', values: ['another-node-label-value'] }] }] }] }</pre>	<none>
affinity.podAffinity	The pod affinity expressed in YAML format. e.g. <pre>{ requiredDuringSchedulingIgnoredDuringExecution: { labelSelector: [{ matchExpressions: [{ key: 'app', operator: 'In', values: ['store'] }] }], topologyKey: 'kubernetes.io/hostnam'}, preferredDuringSchedulingIgnoredDuringExecution: [{ weight: 1, podAffinityTerm: { labelSelector: { matchExpressions: [{ key: 'security', operator: 'In', values: ['S2'] }] }, topologyKey: 'failure- domain.beta.kubernetes.io/zone' } }] }</pre>	<none>
affinity.podAntiAffinity	The pod anti-affinity expressed in YAML format. e.g. <pre>{ requiredDuringSchedulingIgnoredDuringExecution: { labelSelector: { matchExpressions: [{ key: 'app', operator: 'In', values: ['store'] }] } }, topologyKey: 'kubernetes.io/hostname'}, preferredDuringSchedulingIgnoredDuringExecution: [{ weight: 1, podAffinityTerm: { labelSelector: { matchExpressions: [{ key: 'security', operator: 'In', values: ['S2'] }] }, topologyKey: 'failure- domain.beta.kubernetes.io/zone' } }] }</pre>	<none>
statefulSetInitContainerImageName	A custom image name to use for the StatefulSet Init Container	<none>
initContainer	An Init Container expressed in YAML format to be applied to a pod. e.g. <pre>{containerName: 'test', imageName: 'busybox:latest', commands: ['sh', '-c', 'echo hello']}</pre>	<none>

11.3. Tasks

The Data Flow server is responsible for deploying Tasks. Tasks that are launched by Data Flow write their state to the same database that is used by the Data Flow server. For Tasks which are Spring Batch Jobs, the job and step execution data is also stored in this database. As with Skipper, Tasks can be launched to multiple platforms. When Data Flow is running on Kubernetes, a Task platform must be defined. To configure new platform accounts that target Kubernetes, provide an entry

under the `spring.cloud.dataflow.task.platform.kubernetes` section in your `application.yaml` file for via another Spring Boot supported mechanism. In the following example, two Kubernetes platform accounts named `dev` and `qa` are created. The keys such as `memory` and `disk` are [Cloud Foundry Deployer Properties](#).

```
spring:
  cloud:
    dataflow:
      task:
        platform:
          kubernetes:
            accounts:
              dev:
                namespace: devNamespace
                imagePullPolicy: Always
                entryPointStyle: exec
                limits:
                  cpu: 4
              qa:
                namespace: qaNamespace
                imagePullPolicy: IfNotPresent
                entryPointStyle: boot
                limits:
                  memory: 2048m
```

YAML

By defining one platform as `default` allows you to skip using `platformName` where its use would otherwise be required.

When launching a task, pass the value of the platform account name using the task launch option `--platformName`. If you do not pass a value for `platformName`, the value `default` will be used.

When deploying a task to multiple platforms, the configuration of the task needs to connect to the same database as the Data Flow Server.

You can configure the Data Flow server that is on Kubernetes to deploy tasks to Cloud Foundry and Kubernetes. See the section on [Cloud Foundry Task Platform Configuration](#) for more information.

11.4. General Configuration

The Spring Cloud Data Flow server for Kubernetes uses the `spring-cloud-kubernetes` module process both the ConfigMap and the secrets settings. To enable the ConfigMap support, pass in an environment variable of `SPRING_CLOUD_KUBERNETES_CONFIG_NAME` and set it to the name of the ConfigMap. The same is true for the secrets, where the environment variable is `SPRING_CLOUD_KUBERNETES_SECRETS_NAME`. To use the secrets, you also need to set `SPRING_CLOUD_KUBERNETES_SECRETS_ENABLE_API` to `true`.

The following example shows a snippet from a deployment script that sets these environment variables:

```
env:
- name: SPRING_CLOUD_KUBERNETES_SECRETS_ENABLE_API
  value: 'true'
- name: SPRING_CLOUD_KUBERNETES_SECRETS_NAME
```

YAML

```
value: mysql
- name: SPRING_CLOUD_KUBERNETES_CONFIG_NAME
value: scdf-server
```

11.4.1. Using ConfigMap and Secrets

You can pass configuration properties to the Data Flow Server by using Kubernetes [ConfigMap](#) and [secrets](#).

The following example shows one possible configuration, which enables MySQL and sets a memory limit:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: scdf-server
  labels:
    app: scdf-server
data:
  application.yaml: |-
    spring:
      cloud:
        dataflow:
          task:
            platform:
              kubernetes:
                accounts:
                  default:
                    limits:
                      memory: 1024Mi
  datasource:
    url: jdbc:mysql://${MYSQL_SERVICE_HOST}:${MYSQL_SERVICE_PORT}/mysql
    username: root
    password: ${mysql-root-password}
    driverClassName: org.mariadb.jdbc.Driver
    testOnBorrow: true
    validationQuery: "SELECT 1"
```

YAML

The preceding example assumes that MySQL is deployed with `mysql` as the service name. Kubernetes publishes the host and port values of these services as environment variables that we can use when configuring the apps we deploy.

We prefer to provide the MySQL connection password in a Secrets file, as the following example shows:

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql
  labels:
    app: mysql
data:
  mysql-root-password: eW91cnBhc3N3b3Jk
```

YAML

The password is a base64-encoded value.

11.5. Database Configuration

Spring Cloud Data Flow provides schemas for H2, HSQLDB, MySQL, Oracle, PostgreSQL, DB2, and SQL Server. The appropriate schema is automatically created when the server starts, provided the right database driver and appropriate credentials are in the classpath.

The JDBC drivers for MySQL (via MariaDB driver), HSQLDB, PostgreSQL, and embedded H2 are available out of the box. If you use any other database, you need to put the corresponding JDBC driver jar on the classpath of the server.

For instance, if you use MySQL in addition to a password in the secrets file, you could provide the following properties in the ConfigMap:

```
data:
  application.yaml: |-
    spring:
      datasource:
        url: jdbc:mysql://${MYSQL_SERVICE_HOST}:${MYSQL_SERVICE_PORT}/mysql
        username: root
        password: ${mysql-root-password}
        driverClassName: org.mariadb.jdbc.Driver
        url: jdbc:mysql://${MYSQL_SERVICE_HOST}:${MYSQL_SERVICE_PORT}/test
        driverClassName: org.mariadb.jdbc.Driver
```

YAML

For PostgreSQL, you could use the following configuration:

```
data:
  application.yaml: |-
    spring:
      datasource:
        url: jdbc:postgresql://${PGSQL_SERVICE_HOST}:${PGSQL_SERVICE_PORT}/database
        username: root
        password: ${postgres-password}
        driverClassName: org.postgresql.Driver
```

YAML

For HSQLDB, you could use the following configuration:

```
data:
  application.yaml: |-
    spring:
      datasource:
        url: jdbc:hsqldb:hsqldb://${HSQLDB_SERVICE_HOST}:${HSQLDB_SERVICE_PORT}/database
        username: sa
        driverClassName: org.hsqldb.jdbc.JDBCdriver
```

YAML

You can find migration scripts for specific database types in the [spring-cloud-task](#) repo.

11.6. Monitoring and Management

We recommend using the `kubectl` command for troubleshooting streams and tasks.

You can list all artifacts and resources used by using the following command:

```
kubectl get all,cm,secrets,pvc
```

SHELL

You can list all resources used by a specific application or service by using a label to select resources. The following command lists all resources used by the `mysql` service:

```
kubectl get all -l app=mysql
```

SHELL

You can get the logs for a specific pod by issuing the following command:

```
kubectl logs pod <pod-name>
```

SHELL

If the pod is continuously getting restarted, you can add `-p` as an option to see the previous log, as follows:

```
kubectl logs -p <pod-name>
```

SHELL

You can also tail or follow a log by adding an `-f` option, as follows:

```
kubectl logs -f <pod-name>
```

SHELL

A useful command to help in troubleshooting issues, such as a container that has a fatal error when starting up, is to use the `describe` command, as the following example shows:

```
kubectl describe pod ticktock-log-0-qnk72
```

SHELL

11.6.1. Inspecting Server Logs

You can access the server logs by using the following command:

```
kubectl get pod -l app=scdf=server  
kubectl logs <scdf-server-pod-name>
```

SHELL

11.6.2. Streams

Stream applications are deployed with the stream name followed by the name of the application. For processors and sinks, an instance index is also appended.

To see all the pods that are deployed by the Spring Cloud Data Flow server, you can specify the `role=spring-app` label, as follows:

```
kubectl get pod -l role=spring-app
```

SHELL

To see details for a specific application deployment you can use the following command:

```
kubectl describe pod <app-pod-name>
```

SHELL

To view the application logs, you can use the following command:

```
kubectl logs <app-pod-name>
```

SHELL

If you would like to tail a log you can use the following command:

```
kubectl logs -f <app-pod-name>
```

SHELL

11.6.3. Tasks

Tasks are launched as bare pods without a replication controller. The pods remain after the tasks complete, which gives you an opportunity to review the logs.

To see all pods for a specific task, use the following command:

```
kubectl get pod -l task-name=<task-name>
```

SHELL

To review the task logs, use the following command:

```
kubectl logs <task-pod-name>
```

SHELL

You have two options to delete completed pods. You can delete them manually once they are no longer needed or you can use the Data Flow shell `task execution cleanup` command to remove the completed pod for a task execution.

To delete the task pod manually, use the following command:

```
kubectl delete pod <task-pod-name>
```

SHELL

To use the `task execution cleanup` command, you must first determine the `ID` for the task execution. To do so, use the `task execution list` command, as the following example (with output) shows:

```
dataflow:>task execution list
```

SHELL

Task Name	ID	Start Time	End Time	Exit Code
task1	1	Fri May 05 18:12:05 EDT 2017	Fri May 05 18:12:05 EDT 2017	0

Once you have the ID, you can issue the command to cleanup the execution artifacts (the completed pod), as the following example shows:

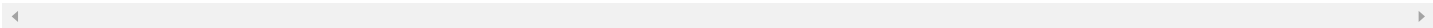
```
dataflow:>task execution cleanup --id 1
Request to clean up resources for task execution 1 has been submitted
```

SHELL

11.7. Scheduling

This section covers customization of how scheduled tasks are configured. Scheduling of tasks is enabled by default in the Spring Cloud Data Flow Kubernetes Server. Properties are used to influence settings for scheduled tasks and can be configured on a global or per-schedule basis.

Unless noted, properties set on a per-schedule basis always take precedence over properties set as the server configuration. This arrangement allows for the ability to override global server level properties for a specific schedule.



See `KubernetesSchedulerProperties` for more on the supported options.

11.7.1. Entry Point Style

An Entry Point Style affects how application properties are passed to the task container to be deployed. Currently, three styles are supported:

- `exec` : (default) Passes all application properties as command line arguments.
- `shell` : Passes all application properties as environment variables.
- `boot` : Creates an environment variable called `SPRING_APPLICATION_JSON` that contains a JSON representation of all application properties.

You can configure the entry point style as follows:

```
scheduler.kubernetes.entryPointStyle=<Entry Point Style>
```

Replace `<Entry Point Style>` with your desired Entry Point Style.

You can also configure the Entry Point Style at the server level in the container `env` section of a deployment YAML, as the following example shows:

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_ENTRY_POINT_STYLE
  value: entryPointStyle
```

Replace `entryPointStyle` with the desired Entry Point Style.

You should choose an Entry Point Style of either `exec` or `shell`, to correspond to how the `ENTRYPOINT` syntax is defined in the container's `Dockerfile`. For more information and uses cases on `exec` vs `shell`, see the [ENTRYPOINT](#) section of the Docker documentation.

Using the `boot` Entry Point Style corresponds to using the `exec` style `ENTRYPOINT`. Command line arguments from the deployment request are passed to the container, with the addition of application properties mapped into the `SPRING_APPLICATION_JSON` environment variable rather than command line arguments.

11.7.2. Environment Variables

To influence the environment settings for a given application, you can take advantage of the `spring.cloud.scheduler.kubernetes.environmentVariables` property. For example, a common requirement in production settings is to influence the JVM memory arguments. You can achieve this by using the `JAVA_TOOL_OPTIONS` environment variable, as the following example shows:

```
scheduler.kubernetes.environmentVariables=JAVA_TOOL_OPTIONS=-Xmx1024m
```

When deploying stream applications or launching task applications where some of the properties may contain sensitive information, use the `shell` or `boot` as the `entryPointStyle`. This is because the `exec` (default) converts all properties to command line arguments and thus may not be secure in some environments.

Additionally you can configure environment variables at the server level in the container `env` section of a deployment YAML, as the following example shows:

When specifying environment variables in the server configuration and on a per-schedule basis, environment variables will be merged. This allows for the ability to set common environment variables in the server configuration and more specific at the specific schedule level.

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_ENVIRONMENT_VARIABLES
  value: myVar=myVal
```

Replace `myVar=myVal` with your desired environment variables.

11.7.3. Image Pull Policy

An image pull policy defines when a Docker image should be pulled to the local registry. Currently, three policies are supported:

- `IfNotPresent` : (default) Do not pull an image if it already exists.
- `Always` : Always pull the image regardless of whether it already exists.
- `Never` : Never pull an image. Use only an image that already exists.

The following example shows how you can individually configure containers:

```
scheduler.kubernetes.imagePullPolicy=Always
```

Replace `Always` with your desired image pull policy.

You can configure an image pull policy at the server level in the container `env` section of a deployment YAML, as the following example shows:

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_IMAGE_PULL_POLICY
  value: Always
```

Replace `Always` with your desired image pull policy.

11.7.4. Private Docker Registry

Docker images that are private and require authentication can be pulled by configuring a Secret. First, you must create a Secret in the cluster. Follow the [Pull an Image from a Private Registry](#) guide to create the Secret.

Once you have created the secret, use the `imagePullSecret` property to set the secret to use, as the following example shows:

```
scheduler.kubernetes.imagePullSecret=mysecret
```

Replace `mysecret` with the name of the secret you created earlier.

You can also configure the image pull secret at the server level in the container `env` section of a deployment YAML, as the following example shows:

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_IMAGE_PULL_SECRET
  value: mysecret
```

Replace `mysecret` with the name of the secret you created earlier.

11.7.5. Namespace

By default the namespace used for scheduled tasks is `default`. This value can be set at the server level configuration in the container `env` section of a deployment YAML, as the following example shows:

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_NAMESPACE
  value: mynamespace
```

11.7.6. Service Account

You can configure a custom service account for scheduled tasks through properties. An existing service account can be used or a new one created. One way to create a service account is by using `kubectl`, as the following example shows:

```
$ kubectl create serviceaccount myserviceaccountname
serviceaccount "myserviceaccountname" created
```

SHELL

Then you can configure the service account to use on a per-schedule basis as follows:

```
scheduler.kubernetes.taskServiceAccountName=myserviceaccountname
```

Replace `myserviceaccountname` with your service account name.

You can also configure the service account name at the server level in the container `env` section of a deployment YAML, as the following example shows:

```
env:
- name: SPRING_CLOUD_SCHEDULER_KUBERNETES_TASK_SERVICE_ACCOUNT_NAME
  value: myserviceaccountname
```

Replace `myserviceaccountname` with the service account name to be applied to all deployments.

For more information on scheduling tasks see [Scheduling Tasks](#).

11.8. Debug Support

Debugging the Spring Cloud Data Flow Kubernetes Server and included components (such as the [Spring Cloud Kubernetes Deployer](#)) is supported through the [Java Debug Wire Protocol \(JDWP\)](#). This section outlines an approach to manually enable debugging and another approach that uses configuration files provided with Spring Cloud Data Flow Server Kubernetes to “patch” a running deployment.

JDWP itself does not use any authentication. This section assumes debugging is being done on a local development environment (such as Minikube), so guidance on securing the debug port is not provided.

11.8.1. Enabling Debugging Manually

To manually enable JDWP, first edit `src/kubernetes/server/server-deployment.yaml` and add an additional `containerPort` entry under `spec.template.spec.containers.ports` with a value of `5005`. Additionally, add the `JAVA_TOOL_OPTIONS` environment variable under `spec.template.spec.containers.env` as the following example shows:

```
spec:
  ...
  template:
    ...
    spec:
      containers:
        - name: scdf-server
          ...
          ports:
            ...
            - containerPort: 5005
          env:
            - name: JAVA_TOOL_OPTIONS
              value: '-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=5005'
```

The preceding example uses port 5005, but it can be any number that does not conflict with another port. The chosen port number must also be the same for the added `containerPort` value and the `address` parameter of the `JAVA_TOOL_OPTIONS -agentlib` flag, as shown in the preceding example.

You can now start the Spring Cloud Data Flow Kubernetes Server. Once the server is up, you can verify the configuration changes on the `scdf-server` deployment, as the following example (with output) shows:

```
kubectl describe deployment/scdf-server
...
...
Pod Template:
...
Containers:
  scdf-server:
    ...
    Ports:      80/TCP, 5005/TCP
    ...
    Environment:
      JAVA_TOOL_OPTIONS: -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=5005
    ...
```

SHELL

With the server started and JDWP enabled, you need to configure access to the port. In this example, we use the `port-forward` subcommand of `kubectl`. The following example (with output) shows how to expose a local port to your debug target by using `port-forward`:

```
$ kubectl get pod -l app=scdf-server
NAME                                READY    STATUS    RESTARTS   AGE
scdf-server-5b7cfd86f7-d8mj4        1/1      Running   0           10m
$ kubectl port-forward scdf-server-5b7cfd86f7-d8mj4 5005:5005
Forwarding from 127.0.0.1:5005 -> 5005
Forwarding from [::1]:5005 -> 5005
```

SHELL

You can now attach a debugger by pointing it to `127.0.0.1` as the host and `5005` as the port. The `port-forward` subcommand runs until stopped (by pressing `CTRL+C`, for example).

You can remove debugging support by reverting the changes to `src/kubernetes/server/server-deployment.yaml`. The reverted changes are picked up on the next deployment of the Spring Cloud Data Flow Kubernetes Server. Manually adding debug support to the configuration is useful when debugging should be enabled by default each time the server is deployed.

11.8.2. Enabling Debugging with Patching

Rather than manually changing the `server-deployment.yaml`, Kubernetes objects can be “patched” in place. For convenience, patch files that provide the same configuration as the manual approach are included. To enable debugging by patching, use the following command:

```
kubectl patch deployment scdf-server -p "$(cat src/kubernetes/server/server-deployment-debug.yaml)"
```

SHELL

Running the preceding command automatically adds the `containerPort` attribute and the `JAVA_TOOL_OPTIONS` environment variable. The following example (with output) shows how to verify changes to the `scdf-server` deployment:

```
$ kubectl describe deployment/scdf-server
...
...
Pod Template:
...
Containers:
  scdf-server:
    ...
    Ports:      5005/TCP, 80/TCP
    ...
    Environment:
      JAVA_TOOL_OPTIONS:  -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=5005
    ...
```

SHELL

To enable access to the debug port, rather than using the `port-forward` subcommand of `kubectl`, you can patch the `scdf-server` Kubernetes service object. You must first ensure that the `scdf-server` Kubernetes service object has the proper configuration. The following example (with output) shows how to do so:

```
kubectl describe service/scdf-server
Port:          <unset>  80/TCP
TargetPort:    80/TCP
NodePort:      <unset>  30784/TCP
```

SHELL

If the output contains `<unset>`, you must patch the service to add a name for this port. The following example shows how to do so:

```
$ kubectl patch service scdf-server -p "$(cat src/kubernetes/server/server-svc.yaml)"
```

SHELL

A port name should only be missing if the target cluster had been created prior to debug functionality being added. Since multiple ports are being added to the `scdf-server` Kubernetes Service Object, each needs to have its own name.

Now you can add the debug port, as the following example shows:

```
kubectl patch service scdf-server -p "$(cat src/kubernetes/server/server-svc-debug.yaml)"
```

SHELL

The following example (with output) shows how to verify the mapping:

```
$ kubectl describe service scdf-server
Name:                scdf-server
...
...
Port:                scdf-server-jdwp  5005/TCP
TargetPort:          5005/TCP
NodePort:            scdf-server-jdwp  31339/TCP
...
...
Port:                scdf-server  80/TCP
TargetPort:          80/TCP
NodePort:            scdf-server  30883/TCP
...
...
```

SHELL

The output shows that container port 5005 has been mapped to the NodePort of 31339. The following example (with output) shows how to get the IP address of the Minikube node:

```
$ minikube ip
192.168.99.100
```

SHELL

With this information, you can create a debug connection by using a host of 192.168.99.100 and a port of 31339.

The following example shows how to disable JDWP:

```
$ kubectl rollout undo deployment/scdf-server
$ kubectl patch service scdf-server --type json -p='[{"op": "remove", "path": "/spec/ports/0"}]'
```

SHELL

The Kubernetes deployment object is rolled back to its state before being patched. The Kubernetes service object is then patched with a `remove` operation to remove port 5005 from the `containerPorts` list.

`kubectl rollout undo` forces the pod to restart. Patching the Kubernetes Service Object does not re-create the service, and the port mapping to the `scdf-server` deployment remains the same.

See [Rolling Back a Deployment](#) for more information on deployment rollbacks, including managing history and [Updating API Objects in Place Using kubectl Patch](#).

Shell

This section covers the options for starting the shell and more advanced functionality relating to how the shell handles white spaces, quotes, and interpretation of SpEL expressions. The introductory chapters to the [Stream DSL](#) and [Composed Task DSL](#) are good places to start for the most common usage of shell commands.

12. Shell Options

The shell is built upon the [Spring Shell](#) project. There are command line options generic to Spring Shell and some specific to Data Flow. The shell takes the following command line options

```
unix:>java -jar spring-cloud-dataflow-shell-2.4.0.RELEASE.jar --help
Data Flow Options:
--dataflow.uri=                        Address of the Data Flow Server [default: http://localhost:9393].
--dataflow.username=                  Username of the Data Flow Server [no default].
--dataflow.password=                  Password of the Data Flow Server [no default].
--dataflow.credentials-provider-command= Executes an external command which must return an
                                       OAuth Bearer Token (Access Token prefixed with 'Bearer '),
                                       e.g. 'Bearer 12345'), [no default].
--dataflow.skip-ssl-validation=        Accept any SSL certificate (even self-signed) [default: no].
--dataflow.proxy.uri=                  Address of an optional proxy server to use [no default].
--dataflow.proxy.username=              Username of the proxy server (if required by proxy server) [no default].
--dataflow.proxy.password=              Password of the proxy server (if required by proxy server) [no default].
--spring.shell.historySize=              Default size of the shell log file [default: 3000].
--spring.shell.commandFile=              Data Flow Shell executes commands read from the file(s) and then exits.
--help                                  This message.
```

BASH

The `spring.shell.commandFile` option can be used to point to an existing file that contains all the shell commands to deploy one or many related streams and tasks.

Multiple files execution is also supported, they should be passed as comma delimited string :

```
--spring.shell.commandFile=file1.txt,file2.txt
```

This is useful when creating some scripts to help automate deployment.

Also, the following shell command helps to modularize a complex script into multiple independent files:

```
dataflow:>script --file <YOUR_AWESOME_SCRIPT>
```

13. Listing Available Commands

Typing `help` at the command prompt gives a listing of all available commands. Most of the commands are for Data Flow functionality, but a few are general purpose.

```
! - Allows execution of operating system (OS) commands
clear - Clears the console
cls - Clears the console
```

BASH


```

date - Displays the local date and time
exit - Exits the shell
http get - Make GET request to http endpoint
http post - POST data to http endpoint
quit - Exits the shell
system properties - Shows the shell's properties
version - Displays shell version

```

Adding the name of the command to `help` shows additional information on how to invoke the command.

```

dataflow:>help stream create
Keyword:                stream create
Description:            Create a new stream definition
Keyword:                ** default **
Keyword:                name
Help:                   the name to give to the stream
Mandatory:              true
Default if specified:   '__NULL__'
Default if unspecified: '__NULL__'

Keyword:                definition
Help:                   a stream definition, using the DSL (e.g. "http --port=9000 | hdfs")
Mandatory:              true
Default if specified:   '__NULL__'
Default if unspecified: '__NULL__'

Keyword:                deploy
Help:                   whether to deploy the stream immediately
Mandatory:              false
Default if specified:   'true'
Default if unspecified: 'false'

```

BASH

14. Tab Completion

The shell command options can be completed in the shell by pressing the `TAB` key after the leading `--`. For example, pressing `TAB` after `stream create --` results in

```

dataflow:>stream create --
stream create --definition  stream create --name

```

If you type `--de` and then hit `tab`, `--definition` will be expanded.

Tab completion is also available inside the stream or composed task DSL expression for application or task properties. You can also use `TAB` to get hints in a stream DSL expression for what available sources, processors, or sinks can be used.

15. White Space and Quoting Rules

It is only necessary to quote parameter values if they contain spaces or the `|` character. The following example passes a SpEL expression (which is applied to any data it encounters) to a transform processor:

```
transform --expression='new StringBuilder(payload).reverse()'
```

If the parameter value needs to embed a single quote, use two single quotes, as follows:

```
scan --query='Select * from /Customers where name='Smith'''
```

15.1. Quotes and Escaping

There is a Spring Shell-based client that talks to the Data Flow Server and is responsible for **parsing** the DSL. In turn, applications may have application properties that rely on embedded languages, such as the **Spring Expression Language**.

The shell, Data Flow DSL parser, and SpEL have rules about how they handle quotes and how syntax escaping works. When combined together, confusion may arise. This section explains the rules that apply and provides examples of the most complicated situations you may encounter when all three components are involved.

It's not always that complicated

If you do not use the Data Flow shell (for example, you use the REST API directly) or if application properties are not SpEL expressions, then the escaping rules are simpler.

15.1.1. Shell rules

Arguably, the most complex component when it comes to quotes is the shell. The rules can be laid out quite simply, though:

- A shell command is made of keys (`--something`) and corresponding values. There is a special, keyless mapping, though, which is described later.
- A value cannot normally contain spaces, as space is the default delimiter for commands.
- Spaces can be added though, by surrounding the value with quotes (either single (`'`) or double (`"`) quotes).
- Values passed inside deployment properties (e.g. `deployment <stream-name> --properties " ..."`) should not be quoted again.
- If surrounded with quotes, a value can embed a literal quote of the same kind by prefixing it with a backslash (`\`).
- Other escapes are available, such as `\t` , `\n` , `\r` , `\f` and unicode escapes of the form `\uxxxx` .
- The keyless mapping is handled in a special way such that it does not need quoting to contain spaces.

For example, the shell supports the `!` command to execute native shell commands. The `!` accepts a single keyless argument. This is why the following works:

```
dataflow:>! rm something
```

The argument here is the whole `rm something` string, which is passed as is to the underlying shell.

As another example, the following commands are strictly equivalent, and the argument value is `something` (without the quotes):

```
dataflow:>stream destroy something
dataflow:>stream destroy --name something
dataflow:>stream destroy "something"
dataflow:>stream destroy --name "something"
```

15.1.2. Property files rules

Rules are relaxed when loading the properties from files. * The special characters used in property files (both Java and YAML) needs to be escaped. For example `\` should be replaced by `\\`, `\t`` by `\\t` and so forth. * For Java property files (`--propertiesFile <FILE_PATH>.properties`) the property values should not be surrounded by quotes! It is not needed even if they contain spaces.

```
filter.expression=payload > 5
```

- For YAML property files (`--propertiesFile <FILE_PATH>.yaml`), though, the values need to be surrounded by double quotes.

```
app:
  filter:
    filter:
      expression: "payload > 5"
```

15.1.3. DSL Parsing Rules

At the parser level (that is, inside the body of a stream or task definition) the rules are as follows:

- Option values are normally parsed until the first space character.
- They can be made of literal strings, though, surrounded by single or double quotes.
- To embed such a quote, use two consecutive quotes of the desired kind.

As such, the values of the `--expression` option to the filter application are semantically equivalent in the following examples:

```
filter --expression=payload>5
filter --expression="payload>5"
filter --expression='payload>5'
filter --expression='payload > 5'
```

Arguably, the last one is more readable. It is made possible thanks to the surrounding quotes. The actual expression is `payload > 5` (without quotes).

Now, imagine that we want to test against string messages. If we want to compare the payload to the SpEL literal string, `"something"`, we could use the following:

<code>filter --expression=payload=='something'</code>	1
<code>filter --expression='payload == ''something'''</code>	2
<code>filter --expression='payload == "something"'</code>	3

- 1** This works because there are no spaces. It is not very legible, though.
- 2** This uses single quotes to protect the whole argument. Hence, the actual single quotes need to be doubled.
- 3** SpEL recognizes String literals with either single or double quotes, so this last method is arguably the most readable.

Please note that the preceding examples are to be considered outside of the shell (for example, when calling the REST API directly). When entered inside the shell, chances are that the whole stream definition is itself inside double quotes, which would need to be escaped. The whole example then becomes the following:

```
dataflow:>stream create something --definition "http | filter --expression=payload='something' | log"

dataflow:>stream create something --definition "http | filter --expression='payload == ''something''' | log"

dataflow:>stream create something --definition "http | filter --expression='payload == \"something\"' | log"
```

15.1.4. SpEL Syntax and SpEL Literals

The last piece of the puzzle is about SpEL expressions. Many applications accept options that are to be interpreted as SpEL expressions, and, as seen above, String literals are handled in a special way there, too. The rules are as follows:

- Literals can be enclosed in either single or double quotes.
- Quotes need to be doubled to embed a literal quote. Single quotes inside double quotes need no special treatment, and the reverse is also true.

As a last example, assume you want to use the [transform processor](#). This processor accepts an `expression` option which is a SpEL expression. It is to be evaluated against the incoming message, with a default of `payload` (which forwards the message payload untouched).

It is important to understand that the following statements are equivalent:

```
transform --expression=payload
transform --expression='payload'
```

However, they are different from the following (and variations upon them):

```
transform --expression="'payload'"
transform --expression='''payload'''
```

The first series evaluates to the message payload, while the latter examples evaluate to the literal string, `payload`, (again, without quotes).

15.1.5. Putting It All Together

As a last, complete example, consider how one could force the transformation of all messages to the string literal, `hello world`, by creating a stream in the context of the Data Flow shell:

```
dataflow:>stream create something --definition "http | transform --expression='''hello world''' | log" 1
```

```
dataflow:>stream create something --definition "http | transform --expression=\"'hello world'\" | log" 2
```

```
dataflow:>stream create something --definition "http | transform --expression=\"'hello world'\" | log" 2
```

- 1 In the first line, there are single quotes around the string (at the Data Flow parser level), but they need to be doubled because they are inside a string literal (started by the first single quote after the equals sign).
- 2 The second and third lines, use single and double quotes respectively to encompass the whole string at the Data Flow parser level. Consequently, the other kind of quote can be used inside the string. The whole thing is inside the `--definition` argument to the shell, though, which uses double quotes. Consequently, double quotes are escaped (at the shell level)

Streams

This section goes into more detail about how you can create Streams, which are collections of [Spring Cloud Stream](#) applications. It covers topics such as creating and deploying Streams.

If you are just starting out with Spring Cloud Data Flow, you should probably read the [Getting Started](#) guide before diving into this section.

16. Introduction

A Stream is are a collection of long-lived [Spring Cloud Stream](#) applications that communicate with each other over messaging middleware. A text-based DSL defines the configuration and data flow between the applications. While many applications are provided for you to implement common use-cases, you typically create a custom Spring Cloud Stream application to implement custom business logic.

The general lifecycle of a Stream is:

1. Register applications.
2. Create a Stream Definition.
3. Deploy the Stream.
4. Undeploy or Destroy the Stream.
5. Upgrade or Rollback applications in the Stream.

For deploying streams, the Data Flow Server has to be configured to delegate the deployment to a new server in the Spring Cloud ecosystem named [Skipper](#).

Furthermore you can configure Skipper to deploy applications to one or more Cloud Foundry orgs and spaces, one or more namespaces on a Kubernetes cluster, or to the local machine. When deploying a stream in Data Flow, you can specify which platform to use at deployment time. Skipper also provides Data Flow with the ability to perform updates to deployed streams. There are many ways the applications in a stream can be updated, but one of the most common examples is to

upgrade a processor application with new custom business logic while leaving the existing source and sink applications alone.

16.1. Stream Pipeline DSL

A stream is defined by using a unix-inspired [Pipeline syntax](#). The syntax uses vertical bars, also known as “pipes” to connect multiple commands. The command `ls -l | grep key | less` in Unix takes the output of the `ls -l` process and pipes it to the input of the `grep key` process. The output of `grep` in turn is sent to the input of the `less` process. Each `|` symbol connects the standard output of the command on the left to the standard input of the command on the right. Data flows through the pipeline from left to right.

In Data Flow, the Unix command is replaced by a [Spring Cloud Stream](#) application and each pipe symbol represents connecting the input and output of applications over messaging middleware, such as RabbitMQ or Apache Kafka.

Each Spring Cloud Stream application is registered under a simple name. The registration process specifies where the application can be obtained (for example, in a Maven Repository or a Docker registry). You can find out more information on how to register Spring Cloud Stream applications in this [section](#). In Data Flow, we classify the Spring Cloud Stream applications as Sources, Processors, or Sinks.

As a simple example, consider the collection of data from an HTTP Source writing to a File Sink. Using the DSL, the stream description is:

```
http | file
```

A stream that involves some processing would be expressed as:

```
http | filter | transform | file
```

Stream definitions can be created by using the shell’s `stream create` command, as shown in the following example:

```
dataflow:> stream create --name httpIngest --definition "http | file"
```

The Stream DSL is passed in to the `--definition` command option.

The deployment of stream definitions is done through the shell’s `stream deploy` command.

```
dataflow:> stream deploy --name ticktock
```

The [Getting Started](#) section shows you how to start the server and how to start and use the Spring Cloud Data Flow shell.

Note that the shell calls the Data Flow Servers’ REST API. For more information on making HTTP requests directly to the server, consult the [REST API Guide](#).

16.2. Stream Application DSL

The Stream Application DSL can be used to define custom binding properties for each of the Spring Cloud Stream applications. Please see the [Stream Application DSL](#) section of the microsite for more information.

```
public interface Barista {

    @Input
    SubscribableChannel orders();
}
```

JAVA

```

@Output
MessageChannel hotDrinks();

@Output
MessageChannel coldDrinks();
}

```

or as is common when creating a Kafka Streams application,

```

interface KStreamKTableBinding {

    @Input
    KStream<?, ?> inputStream();

    @Input
    KTable<?, ?> inputTable();
}

```

JAVA

In these cases with multiple input and output bindings, Data Flow cannot make any assumptions about the flow of data from one application to another. Therefore the developer needs to set the binding properties to 'wire up' the application. The **Stream Application DSL** uses a double pipe, instead of the pipe symbol, to indicate that Data Flow should not configure the binding properties of the application. Think of `||` as meaning 'in parallel'. For example:

```

dataflow:> stream create --definition "orderGeneratorApp || baristaApp || hotDrinkDeliveryApp || coldDrinkDeliveryApp"
--name myCafeStream

```

Breaking Change! Versions of SCDF Local, Cloud Foundry 1.7.0 to 1.7.2 and SCDF Kubernetes 1.7.0 to 1.7.1 used the `comma` character as the separator between applications. This caused breaking changes in the traditional Stream DSL. While not ideal, changing the separator character was felt to be the best solution with the least impact on existing users.

There are four applications in this stream. The `baristaApp` has two output destinations, `hotDrinks` and `coldDrinks` intended to be consumed by the `hotDrinkDeliveryApp` and `coldDrinkDeliveryApp` respectively. When deploying this stream, you need to set the binding properties so that the `baristaApp` sends hot drink messages to the `hotDrinkDeliveryApp` destination and cold drink messages to the `coldDrinkDeliveryApp` destination. For example

```

app.baristaApp.spring.cloud.stream.bindings.hotDrinks.destination=hotDrinksDest
app.baristaApp.spring.cloud.stream.bindings.coldDrinks.destination=coldDrinksDest
app.hotDrinkDeliveryApp.spring.cloud.stream.bindings.input.destination=hotDrinksDest
app.coldDrinkDeliveryApp.spring.cloud.stream.bindings.input.destination=coldDrinksDest

```

BASH

If you want to use consumer groups, you will need to set the Spring Cloud Stream application property `spring.cloud.stream.bindings.<channelName>.producer.requiredGroups` and `spring.cloud.stream.bindings.<channelName>.group` on the producer and consumer applications respectively.

Another common use case for the Stream Application DSL is to deploy a http gateway application that sends a synchronous request/reply message to a Kafka or RabbitMQ application. In this case both the http gateway application and the Kafka or RabbitMQ application can be a Spring Integration application that does not make use of the Spring Cloud Stream library.

It is also possible to deploy just a single application using the Stream application DSL.

16.3. Application properties

Each application takes properties to customize its behavior. As an example, the `http` source module exposes a `port` setting that allows the data ingestion port to be changed from the default value.

```
dataflow:> stream create --definition "http --port=8090 | log" --name myhttpstream
```

This `port` property is actually the same as the standard Spring Boot `server.port` property. Data Flow adds the ability to use the shorthand form `port` instead of `server.port`. One may also specify the longhand version as well, as shown in the following example:

```
dataflow:> stream create --definition "http --server.port=8000 | log" --name myhttpstream
```

This shorthand behavior is discussed more in the section on [Whitelisting application properties](#). If you have [registered application property metadata](#) you can use tab completion in the shell after typing `--` to get a list of candidate property names.

The shell provides tab completion for application properties. The shell command `app info --name <appName> --type <appType>` provides additional documentation for all the supported properties.

Supported Stream `<appType>` possibilities are: source, processor, and sink.

17. Stream Lifecycle

The lifecycle of a stream, goes through the following stages:

1. [Register a Stream App](#)
2. [Creating a Stream](#)
3. [Deploying a Stream](#)
4. [Destroying a Stream](#) or [Undeploying a Stream](#)
5. [Upgrade](#) or [Rollback](#) applications in the Stream.

[Skipper](#) is a server that you discover Spring Boot applications and manage their lifecycle on multiple Cloud Platforms.

Applications in Skipper are bundled as packages that contain the application's resource location, application properties and deployment properties. You can think Skipper packages as analogous to packages found in tools such as `apt-get` or `brew`.

When Data Flow deploys a Stream, it will generate and upload a package to Skipper that represents the applications in the Stream. Subsequent commands to upgrade or rollback the applications within the Stream are passed through to Skipper. In addition, the Stream definition is reverse engineered from the package and the status of the Stream is also delegated to Skipper.

17.1. Register a Stream App

Register a versioned stream application using the `app register` command. You must provide a unique name, application type, and a URI that can be resolved to the app artifact. For the type, specify "source", "processor", or "sink". The version is resolved from the URI. Here are a few examples:

BASH

```
dataflow:>app register --name mysource --type source --uri maven://com.example:mysource:0.0.1
dataflow:>app register --name mysource --type source --uri maven://com.example:mysource:0.0.2
dataflow:>app register --name mysource --type source --uri maven://com.example:mysource:0.0.3

dataflow:>app list --id source:mysource
```

app	source	processor	sink	task
	> mysource-0.0.1 < mysource-0.0.2 mysource-0.0.3			

```
dataflow:>app register --name myprocessor --type processor --uri file:///Users/example/myprocessor-1.2.3.jar

dataflow:>app register --name mysink --type sink --uri https://example.com/mysink-2.0.1.jar
```

The application URI should conform to one the following schema formats:

- maven schema

```
maven://<groupId>:<artifactId>[:<extension>[:<classifier>]]:<version>
```

- http schema

```
http://<web-path>/<artifactName>-<version>.jar
```

- file schema

```
file:///<local-path>/<artifactName>-<version>.jar
```

- docker schema

```
docker:<docker-image-path>/<imageName>:<version>
```

The URI `<version>` part is compulsory for the versioned stream applications. Skipper leverages the multi-versioned stream applications to allow upgrade or rollback of those applications at runtime using the deployment properties.

If you would like to register the snapshot versions of the `http` and `log` applications built with the RabbitMQ binder, you could do the following:

BASH

```
dataflow:>app register --name http --type source --uri maven://org.springframework.cloud.stream.app:http-source-rabbit:1.2.
dataflow:>app register --name log --type sink --uri maven://org.springframework.cloud.stream.app:log-sink-rabbit:1.2.1.BUIL
```

If you would like to register multiple apps at one time, you can store them in a properties file where the keys are formatted as `<type>.<name>` and the values are the URIs.

For example, if you would like to register the snapshot versions of the `http` and `log` applications built with the RabbitMQ binder, you could have the following in a properties file (for example, `stream-apps.properties`):

BASH

```
source.http=maven://org.springframework.cloud.stream.app:http-source-rabbit:1.2.1.BUILD-SNAPSHOT
sink.log=maven://org.springframework.cloud.stream.app:log-sink-rabbit:1.2.1.BUILD-SNAPSHOT
```

Then to import the apps in bulk, use the `app import` command and provide the location of the properties file with the `--uri` switch, as follows:

BASH

```
dataflow:>app import --uri file:///<YOUR_FILE_LOCATION>/stream-apps.properties
```

Registering an application using `--type app` is the same as registering a `source`, `processor` or `sink`. Applications of the type `app` are only allowed to be used in the Stream Application DSL, which uses a comma instead of the pipe symbol in the DSL, and instructs Data Flow not to configure the Spring Cloud Stream binding properties of the application. The application that is registered using `--type app` does not have to be a Spring Cloud Stream app, it can be any Spring Boot application. See the [Stream Application DSL introduction](#) for more information on using this application type.

Multiple versions can be registered for the same applications (e.g. same name and type) but only one can be set as default. The default version is used for deploying Streams.

The first time an application is registered it will be marked as default. The default application version can be altered with the `app default` command:

BASH

```
dataflow:>app default --id source:mysource --version 0.0.2
dataflow:>app list --id source:mysource
```

app	source	processor	sink	task
	mysource-0.0.1 > mysource-0.0.2 < mysource-0.0.3			

The `app list --id <type:name>` command lists all versions for a given stream application.

The `app unregister` command has an optional `--version` parameter to specify the app version to unregister.

BASH

```
dataflow:>app unregister --name mysource --type source --version 0.0.1
dataflow:>app list --id source:mysource
```

app	source	processor	sink	task
	> mysource-0.0.2 < mysource-0.0.3			

If a `--version` is not specified, the default version is unregistered.

All applications in a stream should have a default version set for the stream to be deployed. Otherwise they will be treated as unregistered application during the deployment. Use the `app default` to set the defaults.

```
app default --id source:mysource --version 0.0.3
dataflow:>app list --id source:mysource
```

app	source	processor	sink	task
	mysource-0.0.2 > mysource-0.0.3 <			

BASH

The `stream deploy` necessitates default app versions to be set. The `stream update` and `stream rollback` commands though can use all (default and non-default) registered app versions.

```
dataflow:>stream create foo --definition "mysource | log"
```

BASH

This will create stream using the default mysource version (0.0.3). Then we can update the version to 0.0.2 like this:

```
dataflow:>stream update foo --properties version.mysource=0.0.2
```

BASH

Only pre-registered applications can be used to `deploy` , `update` or `rollback` a Stream.

An attempt to update the `mysource` to version `0.0.1` (not registered) will fail!

17.1.1. Register Supported Applications and Tasks

For convenience, we have the static files with application-URIs (for both maven and docker) available for all the out-of-the-box stream and task/batch app-starters. You can point to this file and import all the application-URIs in bulk. Otherwise, as explained previously, you can register them individually or have your own custom property file with only the required application-URIs in it. It is recommended, however, to have a “focused” list of desired application-URIs in a custom property file.

Spring Cloud Stream App Starters

The following table includes the `dataflow.spring.io` links to the available Stream Application Starters based on Spring Cloud Stream 2.1.x and Spring Boot 2.1.x:

Artifact Type	Stable Release	SNAPSHOT Release
---------------	----------------	------------------

Artifact Type	Stable Release	SNAPSHOT Release
RabbitMQ + Maven	dataflow.spring.io/rabbitmq-maven-latest	dataflow.spring.io/Einstein-BUILD-SNAPSHOT-stream-applications-rabbit-maven
RabbitMQ + Docker	dataflow.spring.io/rabbitmq-docker-latest	dataflow.spring.io/Einstein-BUILD-SNAPSHOT-stream-applications-rabbit-docker
Apache Kafka + Maven	dataflow.spring.io/kafka-maven-latest	dataflow.spring.io/Einstein-BUILD-SNAPSHOT-stream-applications-kafka-maven
Apache Kafka + Docker	dataflow.spring.io/kafka-docker-latest	dataflow.spring.io/Einstein-BUILD-SNAPSHOT-stream-applications-kafka-docker

App Starter actuator endpoints are secured by default. You can disable security by deploying streams with the property

```
app.*.spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

On Kubernetes refer to the section [Liveness and readiness probes](#) to configure security for actuator endpoints.

Starting with Spring Cloud Stream 2.1 GA release, we now have robust interoperability with Spring Cloud Function programming model. Building on that, with Einstein release-train, it is now possible to pick a few Stream App Starters, and compose them into a single application using the functional-style programming model. Check out the ["Composed Function Support in Spring Cloud Data Flow"](#) blog to learn more about the developer and orchestration-experience with an example.

Spring Cloud Task App Starters

The following table includes the available Task Application Starters based on Spring Cloud Task 2.1.x and Spring Boot 2.1.x:

Artifact Type	Stable Release	SNAPSHOT Release
Maven	dataflow.spring.io/task-maven-latest	dataflow.spring.io/Elston-BUILD-SNAPSHOT-task-applications-maven
Docker	dataflow.spring.io/task-docker-latest	dataflow.spring.io/Elston-BUILD-SNAPSHOT-task-applications-docker

You can find more information about the available task starters in the [Task App Starters Project Page](#) and related reference documentation. For more information about the available stream starters, look at the [Stream App Starters Project Page](#) and related reference documentation.

As an example, if you would like to register all out-of-the-box stream applications built with the Kafka binder in bulk, you can use the following command:

```
$ dataflow:>app import --uri https://dataflow.spring.io/kafka-maven-latest
```

BASH

Alternatively you can register all the stream applications with the Rabbit binder, as follows:

```
$ dataflow:>app import --uri https://dataflow.spring.io/rabbitmq-maven-latest
```

BASH

You can also pass the `--local` option (which is `true` by default) to indicate whether the properties file location should be resolved within the shell process itself. If the location should be resolved from the Data Flow Server process, specify `--local false`.

When using either `app register` or `app import`, if an app is already registered with the provided name and type and version, it is not overridden by default. If you would like to override the pre-existing app uri or metadata-uri coordinates, then include the `--force` option.

Note, however, that, once downloaded, applications may be cached locally on the Data Flow server, based on the resource location. If the resource location does not change (even though the actual resource *bytes* may be different), then it is not re-downloaded. When using `maven://` resources on the other hand, using a constant location may still circumvent caching (if using `-SNAPSHOT` versions).

Moreover, if a stream is already deployed and using some version of a registered app, then (forcibly) re-registering a different app has no effect until the stream is deployed again.

In some cases, the Resource is resolved on the server side. In others, the URI is passed to a runtime container instance where it is resolved. Consult the specific documentation of each Data Flow Server for more detail.

17.1.2. Whitelisting application properties

Stream and Task applications are Spring Boot applications that are aware of many [Common Application Properties](#), such as `server.port` but also families of properties such as those with the prefix `spring.jmx` and `logging`. When creating your own application, you should whitelist properties so that the shell and the UI can display them first as primary properties when presenting options through TAB completion or in drop-down boxes.

To whitelist application properties, create a file named `spring-configuration-metadata-whitelist.properties` in the `META-INF` resource directory. There are two property keys that can be used inside this file. The first key is named `configuration-properties.classes`. The value is a comma separated list of fully qualified `@ConfigurationProperty` class names. The second key is `configuration-properties.names`, whose value is a comma-separated list of property names. This can contain the full name of the property, such as `server.port`, or a partial name to whitelist a category of property names, such as `spring.jmx`.

The [Spring Cloud Stream application starters](#) are a good place to look for examples of usage. The following example comes from the file sink's `spring-configuration-metadata-whitelist.properties` file:

```
configuration-properties.classes=org.springframework.cloud.stream.app.file.sink.FileSinkProperties
```

If we also want to add `server.port` to be white listed, it would become the following line:

```
configuration-properties.classes=org.springframework.cloud.stream.app.file.sink.FileSinkProperties
configuration-properties.names=server.port
```

Make sure to add 'spring-boot-configuration-processor' as an optional dependency to generate configuration metadata file for the properties.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

XML

17.1.3. Creating and Using a Dedicated Metadata Artifact

You can go a step further in the process of describing the main properties that your stream or task app supports by creating a metadata companion artifact. This jar file contains only the Spring boot JSON file about configuration properties metadata and the whitelisting file described in the previous section.

The following example shows the contents of such an artifact, for the canonical `log` sink:

```
$ jar tvf log-sink-rabbit-1.2.1.BUILD-SNAPSHOT-metadata.jar
373848 META-INF/spring-configuration-metadata.json
174 META-INF/spring-configuration-metadata-whitelist.properties
```

BASH

Note that the `spring-configuration-metadata.json` file is quite large. This is because it contains the concatenation of *all* the properties that are available at runtime to the `log` sink (some of them come from `spring-boot-actuator.jar`, some of them come from `spring-boot-autoconfigure.jar`, some more from `spring-cloud-starter-stream-sink-log.jar`, and so on). Data Flow always relies on all those properties, even when a companion artifact is not available, but here all have been merged into a single file.

To help with that (you do not want to try to craft this giant JSON file by hand), you can use the following plugin in your build:

```
<plugin>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-app-starter-metadata-maven-plugin</artifactId>
  <executions>
    <execution>
      <id>aggregate-metadata</id>
      <phase>compile</phase>
      <goals>
        <goal>aggregate-metadata</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

XML

```

    </execution>
  </executions>
</plugin>

```

This plugin comes in addition to the `spring-boot-configuration-processor` that creates the individual JSON files. Be sure to configure both.



The benefits of a companion artifact include:

- Being much lighter. (The companion artifact is usually a few kilobytes, as opposed to megabytes for the actual app.) Consequently, they are quicker to download, allowing quicker feedback when using, for example, `app info` or the Dashboard UI.
- As a consequence of being lighter, they can be used in resource constrained environments (such as PaaS) when metadata is the only piece of information needed.
- For environments that do not deal with Spring Boot uber jars directly (for example, Docker-based runtimes such as Kubernetes or Cloud Foundry), this is the only way to provide metadata about the properties supported by the app.

Remember, though, that this is entirely optional when dealing with uber jars. The uber jar itself also includes the metadata in it already.

17.1.4. Using the Companion Artifact

Once you have a companion artifact at hand, you need to make the system aware of it so that it can be used.

When registering a single app with `app register`, you can use the optional `--metadata-uri` option in the shell, as follows:

```

dataflow:>app register --name log --type sink
--uri maven://org.springframework.cloud.stream.app:log-sink:2.1.0.RELEASE
--metadata-uri maven://org.springframework.cloud.stream.app:log-sink:jar:metadata:2.1.0.RELEASE

```

BASH

When registering several files by using the `app import` command, the file should contain a `<type>.<name>.metadata` line in addition to each `<type>.<name>` line. Strictly speaking, doing so is optional (if some apps have it but some others do not, it works), but it is best practice.

The following example shows a Dockerized app, where the metadata artifact is being hosted in a Maven repository (retrieving it through `http://` or `file://` would be equally possible).

```

...
source.http=docker:springcloudstream/http-source-rabbit:latest
source.http.metadata=maven://org.springframework.cloud.stream.app:http-source-rabbit:jar:metadata:2.1.0.RELEASE
...

```

PROPERTIES

17.1.5. Creating Custom Applications

While there are out-of-the-box source, processor, sink applications available, you can extend these applications or write a custom [Spring Cloud Stream](#) application.

The process of creating Spring Cloud Stream applications with [Spring Initializr](#) is detailed in the [Spring Cloud Stream documentation](#). It is possible to include multiple binders to an application. If doing so, see the instructions in [\[passing_producer_consumer_properties\]](#) for how to configure them.

For supporting property whitelisting, Spring Cloud Stream applications running in Spring Cloud Data Flow may include the `Spring Boot configuration-processor` as an optional dependency, as shown in the following example:

```
<dependencies>
  <!-- other dependencies -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

XML

Make sure that the `spring-boot-maven-plugin` is included in the POM. The plugin is necessary for creating the executable jar that is registered with Spring Cloud Data Flow. Spring Initializr includes the plugin in the generated POM.

Once a custom application has been created, it can be registered as described in [Register a Stream App](#).

17.2. Creating a Stream

The Spring Cloud Data Flow Server exposes a full RESTful API for managing the lifecycle of stream definitions, but the easiest way to use it is through the Spring Cloud Data Flow shell. Start the shell as described in the [Getting Started](#) section.

New streams are created with the help of stream definitions. The definitions are built from a simple DSL. For example, consider what happens if we execute the following shell command:

```
dataflow:> stream create --definition "time | log" --name ticktock
```

BASH

This defines a stream named `ticktock` that is based off the DSL expression `time | log`. The DSL uses the "pipe" symbol (`|`), to connect a source to a sink.

The `stream info` command shows useful information about the stream, as shown (with its output) in the following example:

```
dataflow:>stream info ticktock
```

BASH

Stream Name	Stream Definition	Status
ticktock	time log	undeployed

17.2.1. Application Properties

Application properties are the properties associated with each application in the stream. When the application is deployed, the application properties are applied to the application through command line arguments or environment variables, depending on the underlying deployment implementation.

The following stream can have application properties defined at the time of stream creation:

```
dataflow:> stream create --definition "time | log" --name ticktock
```

BASH

The shell command `app info --name <appName> --type <appType>` displays the white-listed application properties for the application. For more info on the property white listing, refer to [Whitelisting application properties](#)

The following listing shows the white_listed properties for the `time` app:

```
dataflow:> app info --name time --type source
```

Option Name	Description	Default	Type
trigger.time-unit	The TimeUnit to apply to delay values.	<none>	java.util.concurrent.TimeUnit
trigger.fixed-delay	Fixed delay for periodic triggers.	1	java.lang.Integer
trigger.cron	Cron expression value for the Cron Trigger.	<none>	java.lang.String
trigger.initial-delay	Initial delay for periodic triggers.	0	java.lang.Integer
trigger.max-messages	Maximum messages per poll, -1 means infinity.	1	java.lang.Long
trigger.date-format	Format for the date value.	<none>	java.lang.String

The following listing shows the white-listed properties for the `log` app:

```
dataflow:> app info --name log --type sink
```

Option Name	Description	Default	Type
log.name	The name of the logger to use.	<none>	java.lang.String
log.level	The level at which to log messages.	<none>	org.springframework.integration.handler.LoggingHandler\$Level
log.expression	A SpEL expression (against the incoming message) to evaluate as the logged message.	payload	java.lang.String

The application properties for the `time` and `log` apps can be specified at the time of `stream` creation as follows:

```
dataflow:> stream create --definition "time --fixed-delay=5 | log --level=WARN" --name ticktock
```

BASH

Note that, in the preceding example, the `fixed-delay` and `level` properties defined for the apps `time` and `log` are the "short-form" property names provided by the shell completion. These "short-form" property names are applicable only for the white-listed properties. In all other cases, only fully qualified property names should be used.

17.2.2. Common Application Properties

In addition to configuration through DSL, Spring Cloud Data Flow provides a mechanism for setting common properties to all the streaming applications that are launched by it. This can be done by adding properties prefixed with `spring.cloud.dataflow.applicationProperties.stream` when starting the server. When doing so, the server passes all the properties, without the prefix, to the instances it launches.

For example, all the launched applications can be configured to use a specific Kafka broker by launching the Data Flow server with the following options:

```
--spring.cloud.dataflow.applicationProperties.stream.spring.cloud.stream.kafka.binder.brokers=192.168.1.100:9092
--spring.cloud.dataflow.applicationProperties.stream.spring.cloud.stream.kafka.binder.zkNodes=192.168.1.100:2181
```

Doing so causes the properties `spring.cloud.stream.kafka.binder.brokers` and `spring.cloud.stream.kafka.binder.zkNodes` to be passed to all the launched applications.

Properties configured with this mechanism have lower precedence than stream deployment properties. They are overridden if a property with the same key is specified at stream deployment time (for example, `app.http.spring.cloud.stream.kafka.binder.brokers` overrides the common property).

17.3. Deploying a Stream

This section describes how to deploy a Stream when the Spring Cloud Data Flow server is responsible for deploying the stream. It covers the deployment and upgrade of Streams leveraging the Skipper service. The description of how deployment properties applies to both approaches of Stream deployment.

Give the `ticktock` stream definition:

```
dataflow:> stream create --definition "time | log" --name ticktock
```

To deploy the stream, use the following shell command:

```
dataflow:> stream deploy --name ticktock
```

The Data Flow Server delegates to Skipper the resolution and deployment of the `time` and `log` applications.

The `stream info` command shows useful information about the stream, including the deployment properties:

```
dataflow:>stream info --name ticktock
```

Stream Name	Stream Definition	Status
ticktock	time log	deploying

```
Stream Deployment properties: {
  "log" : {
    "resource" : "maven://org.springframework.cloud.stream.app:log-sink-rabbit",
    "spring.cloud.deployer.group" : "ticktock",
    "version" : "2.0.1.RELEASE"
  },
  "time" : {
    "resource" : "maven://org.springframework.cloud.stream.app:time-source-rabbit",
    "spring.cloud.deployer.group" : "ticktock",
    "version" : "2.0.1.RELEASE"
  }
}
```

BASH

```
}
}
```

There is an important optional command argument (called `--platformName`) to the `stream deploy` command. Skipper can be configured to deploy to multiple platforms. Skipper is pre-configured with a platform named `default`, which deploys applications to the local machine where Skipper is running. The default value of the command line argument `--platformName` is `default`. If you commonly deploy to one platform, when installing Skipper, you can override the configuration of the `default` platform. Otherwise, specify the `platformName` to one of the values returned by the `stream platform-list` command.

In the preceding example, the time source sends the current time as a message each second, and the log sink outputs it by using the logging framework. You can tail the `stdout` log (which has an `<instance>` suffix). The log files are located within the directory displayed in the Data Flow Server's log output, as shown in the following listing:

```
$ tail -f /var/folders/wn/8jxm_tbd1vj28c8vj37n900m0000gn/T/spring-cloud-dataflow-912434582726479179/ticktock-1464788481708/
2016-06-01 09:45:11.250 INFO 79194 --- [ kafka-binder-] log.sink : 06/01/16 09:45:11
2016-06-01 09:45:12.250 INFO 79194 --- [ kafka-binder-] log.sink : 06/01/16 09:45:12
2016-06-01 09:45:13.251 INFO 79194 --- [ kafka-binder-] log.sink : 06/01/16 09:45:13
```

You can also create and deploy the stream in one step by passing the `--deploy` flag when creating the stream, as follows:

```
dataflow:> stream create --definition "time | log" --name ticktock --deploy
```

BASH

However, it is not very common in real-world use cases to create and deploy the stream in one step. The reason is that when you use the `stream deploy` command, you can pass in properties that define how to map the applications onto the platform (for example, what is the memory size of the container to use, the number of each application to run, and whether to enable data partitioning features). Properties can also override application properties that were set when creating the stream. The next sections cover this feature in detail.

17.3.1. Deployment Properties

When deploying a stream, you can specify properties that can control how apps are deployed and configured. Please see the [Deployment Properties](#) section of the microsite for more information.

17.4. Destroying a Stream

You can delete a stream by issuing the `stream destroy` command from the shell, as follows:

```
dataflow:> stream destroy --name ticktock
```

If the stream was deployed, it is undeployed before the stream definition is deleted.

17.5. Undeploying a Stream

Often you want to stop a stream but retain the name and definition for future use. In that case, you can `undeploy` the stream by name.

```
dataflow:> stream undeploy --name ticktock
dataflow:> stream deploy --name ticktock
```

You can issue the `deploy` command at a later time to restart it.

```
dataflow:> stream deploy --name ticktock
```

17.6. Validating a Stream

Sometimes the one or more of the apps contained within a stream definition contain an invalid URI in its registration. This can be caused by an invalid URI entered at app registration time or the app was removed from the repository from which it was to be drawn. To verify that all the apps contained in a stream are resolve-able, a user can use the `validate` command. For example:

```
dataflow:>stream validate ticktock
```

BASH

Stream Name	Stream Definition
ticktock	time log

ticktock is a valid stream.

App Name	Validation Status
source:time	valid
sink:log	valid

In the example above the user validated their ticktock stream. As we see that both the `source:time` and `sink:log` are valid. Now let's see what happens if we have a stream definition with a registered app with an invalid URI.

```
dataflow:>stream validate bad-ticktock
```

BASH

Stream Name	Stream Definition
bad-ticktock	bad-time log

bad-ticktock is an invalid stream.

App Name	Validation Status
source:bad-time	invalid
sink:log	valid

In this case Spring Cloud Data Flow states that the stream is invalid because `source:bad-time` has an invalid URI.

17.7. Updating a Stream

To update the stream, use the command `stream update` which takes as a command argument either `--properties` or `--propertiesFile`. There is an important new top level prefix available when using Skipper, which is `version`. If the Stream `http | log` was deployed, and the version of `log` which registered at the time of deployment was `1.1.0.RELEASE`:

```
dataflow:> stream create --name httptest --definition "http --server.port=9000 | log"
dataflow:> stream deploy --name httptest
dataflow:>stream info httptest
```

BASH

Name	DSL	Status
httpstest	http --server.port=9000 log	deploying

```
Stream Deployment properties: {
  "log" : {
    "spring.cloud.deployer.indexed" : "true",
    "spring.cloud.deployer.group" : "httpstest",
    "maven://org.springframework.cloud.stream.app:log-sink-rabbit" : "1.1.0.RELEASE"
  },
  "http" : {
    "spring.cloud.deployer.group" : "httpstest",
    "maven://org.springframework.cloud.stream.app:http-source-rabbit" : "1.1.0.RELEASE"
  }
}
```

Then the following command will update the Stream to use the 1.2.0.RELEASE of the log application. Before updating the stream with the specific version of the app, we need to make sure that the app is registered with that version.

```
dataflow:>app register --name log --type sink --uri maven://org.springframework.cloud.stream.app:log-sink-rabbit:1.2.0.RELEASE
Successfully registered application 'sink:log'
```

BASH

```
dataflow:>stream update --name httpstest --properties version.log=1.2.0.RELEASE
```

BASH

Only pre-registered application versions can be used to `deploy`, `update`, or `rollback` a stream.

To verify the deployment properties and the updated version, we can use `stream info`, as shown (with its output) in the following example:

```
dataflow:>stream info httpstest
```

BASH

Name	DSL	Status
httpstest	http --server.port=9000 log	deploying

```
Stream Deployment properties: {
  "log" : {
    "spring.cloud.deployer.indexed" : "true",
    "spring.cloud.deployer.count" : "1",
    "spring.cloud.deployer.group" : "httpstest",
    "maven://org.springframework.cloud.stream.app:log-sink-rabbit" : "1.2.0.RELEASE"
  },
  "http" : {
    "spring.cloud.deployer.group" : "httpstest",
    "maven://org.springframework.cloud.stream.app:http-source-rabbit" : "1.1.0.RELEASE"
  }
}
```

17.8. Force update of a Stream

When upgrading a stream, the `--force` option can be used to deploy new instances of currently deployed applications even if no application or deployment properties have changed. This behavior is needed in the case when configuration

2/18/2020

Spring Cloud Data Flow Reference Guide

information is obtained by the application itself at startup time, for example from Spring Cloud Config Server. You can specify which applications to force upgrade by using the option `--app-names`. If you do not specify any application names, all the applications will be force upgraded. You can specify `--force` and `--app-names` options together with `--properties` or `--propertiesFile` options.

17.9. Stream versions

Skipper keeps a history of the streams that were deployed. After updating a Stream, there will be a second version of the stream. You can query for the history of the versions using the command `stream history --name <name-of-stream>`.

dataflow:>stream history --name httptest

BASH

Version	Last updated	Status	Package Name	Package Version	Description
2	Mon Nov 27 22:41:16 EST 2017	DEPLOYED	httptest	1.0.0	Upgrade complete
1	Mon Nov 27 22:40:41 EST 2017	DELETED	httptest	1.0.0	Delete complete

17.10. Stream Manifests

Skipper keeps a “manifest” of the all the applications, their application properties, and their deployment properties after all values have been substituted. This represents the final state of what was deployed to the platform. You can view the manifest for any of the versions of a Stream by using the following command:

```
stream manifest --name <name-of-stream> --releaseVersion <optional-version>
```

If the `--releaseVersion` is not specified, the manifest for the last version is returned.

The following example shows the use of the manifest:

dataflow:>stream manifest --name httptest

BASH

Using the command results in the following output:

Source: log.yml
apiVersion: skipper.spring.io/v1
kind: SpringCloudDeployerApplication
metadata:
 name: log
spec:
 resource: maven://org.springframework.cloud.stream.app:log-sink-rabbit
 version: 1.2.0.RELEASE
 applicationProperties:
 spring.metrics.export.triggers.application.includes: integration**
 spring.cloud.dataflow.stream.app.label: log
 spring.cloud.stream.metrics.key: httptest.log.\${spring.cloud.application.guid}
 spring.cloud.stream.bindings.input.group: httptest
 spring.cloud.stream.metrics.properties: spring.application.name,spring.application.index,spring.cloud.application.*,spr
 spring.cloud.dataflow.stream.name: httptest
 spring.cloud.dataflow.stream.app.type: sink
 spring.cloud.stream.bindings.input.destination: httptest.http
 deploymentProperties:
 spring.cloud.deployer.indexed: true
 spring.cloud.deployer.group: httptest
 spring.cloud.deployer.count: 1

Source: http.yml

YAML

```

apiVersion: skipper.spring.io/v1
kind: SpringCloudDeployerApplication
metadata:
  name: http
spec:
  resource: maven://org.springframework.cloud.stream.app:http-source-rabbit
  version: 1.2.0.RELEASE
  applicationProperties:
    spring.metrics.export.triggers.application.includes: integration**
    spring.cloud.dataflow.stream.app.label: http
    spring.cloud.stream.metrics.key: httptest.http.${spring.cloud.application.guid}
    spring.cloud.stream.bindings.output.producer.requiredGroups: httptest
    spring.cloud.stream.metrics.properties: spring.application.name,spring.application.index,spring.cloud.application.*,spr
    server.port: 9000
    spring.cloud.stream.bindings.output.destination: httptest.http
    spring.cloud.dataflow.stream.name: httptest
    spring.cloud.dataflow.stream.app.type: source
  deploymentProperties:
    spring.cloud.deployer.group: httptest

```

The majority of the deployment and application properties were set by Data Flow to enable the applications to talk to each other and to send application metrics with identifying labels.

17.11. Rollback a Stream

You can rollback to a previous version of the stream using the command `stream rollback`.

```
dataflow:>stream rollback --name httptest
```

BASH

The optional `--releaseVersion` command argument adds the version of the stream. If not specified, the rollback goes to the previous stream version.

17.12. Application Count

The application count is a dynamic property of the system used to specify the number of instances of applications. Please see the [Application Count](#) section of the microsite for more information.

17.13. Skipper's Upgrade Strategy

Skipper has a simple 'red/black' upgrade strategy. It deploys the new version of the applications, using as many instances as the currently running version, and checks the `/health` endpoint of the application. If the health of the new application is good, then the previous application is undeployed. If the health of the new application is bad, then all new applications are undeployed and the upgrade is considered to be not successful.

The upgrade strategy is not a rolling upgrade, so if five applications of the application are running, then in a sunny-day scenario, five of the new applications are also running before the older version is undeployed.

18. Stream DSL

This section covers additional features of the Stream DSL not covered in the [Stream DSL introduction](#).

18.1. Tap a Stream

Taps can be created at various producer endpoints in a stream. Please see the [Tapping a Stream](#) section of the microsite for more information.

18.2. Using Labels in a Stream

When a stream is made up of multiple apps with the same name, they must be qualified with labels. Please see the [Labeling Applications](#) section of the microsite for more information.

18.3. Named Destinations

Instead of referencing a source or sink application, you can use a named destination. Please see the [Named Destinations](#) section of the microsite for more information.

18.4. Fan-in and Fan-out

By using named destinations, you can support fan-in and fan-out use cases. Please see the [Fan-in and Fan-out](#) section of the microsite for more information.

19. Stream Java DSL

Instead of using the shell to create and deploy streams, you can use the Java-based DSL provided by the `spring-cloud-dataflow-rest-client` module. Please see the [Java DSL](#) section of the microsite for more information.

20. Stream Applications with Multiple Binder Configurations

In some cases, a stream can have its applications bound to multiple spring cloud stream binders when they are required to connect to different messaging middleware configurations. In those cases, it is important to make sure the applications are configured appropriately with their binder configurations. For example, a multi-binder transformer that supports both Kafka and Rabbit binders is the processor in the following stream:

```
http | multibindertransform --expression=payload.toUpperCase() | log
```

BASH

In the example above you would write your own `multibindertransform` application.

In this stream, each application connects to messaging middleware in the following way:

1. The HTTP source sends events to RabbitMQ (`rabbit1`).

2. The Multi-Binder Transform processor receives events from RabbitMQ (`rabbit1`) and sends the processed events into Kafka (`kafka1`).
3. The log sink receives events from Kafka (`kafka1`).

Here, `rabbit1` and `kafka1` are the binder names given in the spring cloud stream application properties. Based on this setup, the applications have the following binder(s) in their classpath with the appropriate configuration:

- HTTP: Rabbit binder
- Transform: Both Kafka and Rabbit binders
- Log: Kafka binder

The spring-cloud-stream binder configuration properties can be set within the applications themselves. If not, they can be passed through deployment properties when the stream is deployed as shown in the following example:

```
dataflow:>stream create --definition "http | multibindertransform --expression=payload.toUpperCase() | log" --name mystream
dataflow:>stream deploy mystream --properties "app.http.spring.cloud.stream.bindings.output.binder=rabbit1,app.multibindertransform.spring.cloud.stream.bindings.output.binder=kafka1,app.log.spring.cloud.stream.bindings.input.binder=kafka1"
```

One can override any of the binder configuration properties by specifying them through deployment properties.

21. Function Composition

Function composition allows you can attach a functional logic dynamically to an existing event streaming application. Please see the [Function Composition](#) section of the microsite for more details.

22. Examples

This chapter includes the following examples:

- [Simple Stream Processing](#)
- [Stateful Stream Processing](#)
- [Other Source and Sink Application Types](#)

You can find links to more samples in the “[Samples](#)” chapter.

22.1. Simple Stream Processing

As an example of a simple processing step, we can transform the payload of the HTTP posted data to upper case by using the following stream definition: `http | transform --expression=payload.toUpperCase() | log`

To create this stream enter the following command in the shell

```
dataflow:> stream create --definition "http --server.port=9000 | transform --expression=payload.toUpperCase() | log" --name mystream
```

The following example uses a shell command to post some data:

```
dataflow:> http post --target http://localhost:9000 --data "hello"
```

The preceding example results in an upper-case 'HELLO' in the log, as follows:

```
2016-06-01 09:54:37.749 INFO 80083 --- [ kafka-binder-] log.sink : HELLO
```

22.2. Stateful Stream Processing

To demonstrate the data partitioning functionality, the following listing deploys a stream with Kafka as the binder:

```
dataflow:>stream create --name words --definition "http --server.port=9900 | splitter --expression=payload.split(' ') | log
Created new stream 'words'
```

```
dataflow:>stream deploy words --properties "app.splitter.producer.partitionKeyExpression=payload,deployer.log.count=2"
Deployed stream 'words'
```

```
dataflow:>http post --target http://localhost:9900 --data "How much wood would a woodchuck chuck if a woodchuck could chuck
> POST (text/plain;Charset=UTF-8) http://localhost:9900 How much wood would a woodchuck chuck if a woodchuck could chuck wo
> 202 ACCEPTED
```

```
dataflow:>runtime apps
```

App Id / Instance Id	Unit Status	No. of Instances / Attribute
words.log-v1	deployed	2
words.log-v1-0	deployed	guid = 24166 pid = 33097 port = 24166 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 url = https://192.168.0.102:24166 working.dir = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467
words.log-v1-1	deployed	guid = 41269 pid = 33098 port = 41269 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 url = https://192.168.0.102:41269 working.dir = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467
words.http-v1	deployed	1
words.http-v1-0	deployed	guid = 9900 pid = 33094 port = 9900 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 url = https://192.168.0.102:9900 working.dir = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467
words.splitter-v1	deployed	1
words.splitter-v1-0	deployed	guid = 33963 pid = 33093 port = 33963 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467 url = https://192.168.0.102:33963 working.dir = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-6467

When you review the `words.log-v1-0` logs, you should see the following:

```
2016-06-05 18:35:47.047 INFO 58638 --- [ kafka-binder-] log.sink : How
2016-06-05 18:35:47.066 INFO 58638 --- [ kafka-binder-] log.sink : chuck
2016-06-05 18:35:47.066 INFO 58638 --- [ kafka-binder-] log.sink : chuck
```

When you review the `words.log-v1-1` logs, you should see the following:

```
2016-06-05 18:35:47.047 INFO 58639 --- [ kafka-binder-] log.sink : much
2016-06-05 18:35:47.066 INFO 58639 --- [ kafka-binder-] log.sink : wood
2016-06-05 18:35:47.066 INFO 58639 --- [ kafka-binder-] log.sink : would
2016-06-05 18:35:47.066 INFO 58639 --- [ kafka-binder-] log.sink : a
2016-06-05 18:35:47.066 INFO 58639 --- [ kafka-binder-] log.sink : woodchuck
2016-06-05 18:35:47.067 INFO 58639 --- [ kafka-binder-] log.sink : if
2016-06-05 18:35:47.067 INFO 58639 --- [ kafka-binder-] log.sink : a
2016-06-05 18:35:47.067 INFO 58639 --- [ kafka-binder-] log.sink : woodchuck
2016-06-05 18:35:47.067 INFO 58639 --- [ kafka-binder-] log.sink : could
2016-06-05 18:35:47.067 INFO 58639 --- [ kafka-binder-] log.sink : wood
```

This example has shown that payload splits that contain the same word are routed to the same application instance.

22.3. Other Source and Sink Application Types

This example shows something a bit more complicated: swapping out the `time` source for something else. Another supported source type is `http`, which accepts data for ingestion over HTTP POSTs. Note that the `http` source accepts data on a different port from the Data Flow Server (default 8080). By default, the port is randomly assigned.

To create a stream using an `http` source but still using the same `log` sink, we would change the original command in the [Simple Stream Processing](#) example to the following:

```
dataflow:> stream create --definition "http | log" --name myhttpstream --deploy
```

BASH

Note that we do not see any other output this time until we actually post some data (by using a shell command). In order to see the randomly assigned port on which the `http` source is listening, run the following command:

```
dataflow:>runtime apps
```

BASH

App Id / Instance Id	Unit Status	No. of Instances /
myhttpstream.log-v1	deploying	1
myhttpstream.log-v1-0	deploying	guid = 39628 pid = 34403 port = 39628 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-64 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-64 url = https://192.168.0.102:39628 working.dir = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-64
myhttpstream.http-v1	deploying	1
myhttpstream.http-v1-0	deploying	guid = 52143 pid = 34401 port = 52143 stderr = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-64 stdout = /var/folders/js/7b_pn0t575l790x7j61slyxc0000gn/T/spring-cloud-deployer-64 url = https://192.168.0.102:52143

You should see that the corresponding `http` source has a `url` property containing the host and port information on which it is listening. You are now ready to post to that url, as shown in the following example:

```
dataflow:> http post --target http://localhost:1234 --data "hello"
dataflow:> http post --target http://localhost:1234 --data "goodbye"
```

BASH

The stream then funnels the data from the `http` source to the output log implemented by the log sink, yielding output similar to the following:

```
2016-06-01 09:50:22.121 INFO 79654 --- [ kafka-binder-] log.sink : hello
2016-06-01 09:50:26.810 INFO 79654 --- [ kafka-binder-] log.sink : goodbye
```

We could also change the sink implementation. You could pipe the output to a file (`file`), to `hadoop` (`hdfs`), or to any of the other sink applications that are available. You can also define your own applications.

Stream Developer Guide

Please see the [Stream Developer Guides](#) of the microsite for more information on how to create, test, and run Spring Cloud Stream applications on your local machine.

Stream Monitoring

Please see the [Stream Monitoring](#) Guide of the microsite for more information on how to monitor the applications that were deployed as part of a Stream.

Tasks

This section goes into more detail about how you can orchestrate [Spring Cloud Task](#) applications on Spring Cloud Data Flow.

If you are just starting out with Spring Cloud Data Flow, you should probably read the Getting Started guide for [“Local”](#), [“Cloud Foundry”](#), [“Kubernetes”](#) before diving into this section.

23. Introduction

A task application is short lived, meaning it stops running on purpose, and can be executed on demand or scheduled for execution. A use case might be to scrape a web page and write to the database. The [Spring Cloud Task](#) framework is based on Spring Boot and adds the capability for Boot applications to record the lifecycle events of a short lived application such as when it starts, when it ends and the exit status. The [TaskExecution](#) documentation shows which information is stored in the database. The entry point for code execution in a Spring Cloud Task application is most often an implementation of Boot's `CommandLineRunner` interface, as shown in this [example](#).

The Spring Batch project is probably what comes to mind for Spring developers writing short lived applications. Spring Batch provides a much richer set of functionality than Spring Cloud Task and is recommended when processing large

volumes of data. A use case might be to read many CSV files, transform each row of data, and write each transformed row to a database. Spring Batch provides its own database schema with a much more rich [set of information](#) about the execution of a Spring Batch job. Spring Cloud Task is integrated with Spring Batch so that if a Spring Cloud Task application defined a Spring Batch `Job`, a link between the Spring Cloud Task and Spring Cloud Batch execution tables is created.

When running Data Flow on your local machine, Tasks are launched in a separate JVM. When running on Cloud Foundry, tasks are launched using [Cloud Foundry's Task](#) functionality and when running on Kubernetes, tasks are launched using either a `Pod` or a `Job` resource.

24. The Lifecycle of a Task

Before we dive deeper into the details of creating Tasks, we need to understand the typical lifecycle for tasks in the context of Spring Cloud Data Flow:

1. [Creating a Task Application](#)
2. [Registering a Task Application](#)
3. [Creating a Task Definition](#)
4. [Launching a Task](#)
5. [Reviewing Task Executions](#)
6. [Destroying a Task Definition](#)
7. [Continuous Deployment](#)

24.1. Creating a Task Application

While Spring Cloud Task does provide a number of out-of-the-box applications (at [spring-cloud-task-app-starters](#)), most task applications require custom development. To create a custom task application:

1. Use the [Spring Initializer](#) to create a new project, making sure to select the following starters:
 - a. `Cloud Task`: This dependency is the `spring-cloud-starter-task`.
 - b. `JDBC`: This dependency is the `spring-jdbc` starter.
 - c. Select your database dependency: Enter the database dependency that Data Flow is currently using. For example: `H2`.
2. Within your new project, create a new class to serve as your main class, as follows:

```
@EnableTask
@SpringBootApplication
public class MyTask {

    public static void main(String[] args) {
        SpringApplication.run(MyTask.class, args);
    }
}
```

JAVA

3. With this class, you need one or more `CommandLineRunner` or `ApplicationRunner` implementations within your application. You can either implement your own or use the ones provided by Spring Boot (there is one for running a batch

jobs, for example).

4. Packaging your application with Spring Boot into an über jar is done through the standard [Spring Boot conventions](#). The packaged application can be registered and deployed as noted below.

24.1.1. Task Database Configuration

When launching a task application, be sure that the database driver that is being used by Spring Cloud Data Flow is also a dependency on the task application. For example, if your Spring Cloud Data Flow is set to use PostgreSQL, be sure that the task application also has PostgreSQL as a dependency.

When you run tasks externally (that is, from the command line) and you want Spring Cloud Data Flow to show the TaskExecutions in its UI, be sure that common datasource settings are shared among the both. By default, Spring Cloud Task uses a local H2 instance, and the execution is recorded to the database used by Spring Cloud Data Flow.

24.2. Registering a Task Application

You can register a Task App with the App Registry by using the Spring Cloud Data Flow Shell `app register` command. You must provide a unique name and a URI that can be resolved to the app artifact. For the type, specify "task". The following listing shows three examples:

```
dataflow:>app register --name task1 --type task --uri maven://com.example:mytask:1.0.2
dataflow:>app register --name task2 --type task --uri file:///Users/example/mytask-1.0.2.jar
dataflow:>app register --name task3 --type task --uri https://example.com/mytask-1.0.2.jar
```

BASH

When providing a URI with the `maven` scheme, the format should conform to the following:

```
maven://<groupId>:<artifactId>[:<extension>[:<classifier>]]:<version>
```

If you would like to register multiple apps at one time, you can store them in a properties file where the keys are formatted as `<type>.<name>` and the values are the URIs. For example, the following listing would be a valid properties file:

```
task.foo=file:///tmp/foo-1.2.1.BUILD-SNAPSHOT.jar
task.bar=file:///tmp/bar-1.2.1.BUILD-SNAPSHOT.jar
```

Then you can use the `app import` command and provide the location of the properties file by using the `--uri` option, as follows:

```
app import --uri file:///tmp/task-apps.properties
```

For example, if you would like to register all out-of-the-box task applications in bulk, you can do so with the following command:

```
dataflow:>app import --uri https://dataflow.spring.io/task-maven-latest
```

You can also pass the `--local` option (which is `TRUE` by default) to indicate whether the properties file location should be resolved within the shell process itself. If the location should be resolved from the Data Flow Server process, specify `--local false`.

When using either `app register` or `app import`, if a task app is already registered with the provided name and version, it is not overridden by default. If you would like to override the pre-existing task app with a different uri or uri-metadata location, then include the `--force` option.

In some cases, the Resource is resolved on the server side. In other cases, the URI is passed to a runtime container instance where it is resolved. Consult the specific documentation of each Data Flow Server for more detail.

24.3. Creating a Task Definition

You can create a task Definition from a task app by providing a definition name as well as properties that apply to the task execution. Creating a task definition can be done through the RESTful API or the shell. To create a task definition by using the shell, use the `task create` command to create the task definition, as shown in the following example:

```
dataflow:>task create mytask --definition "timestamp --format=\"%yyy\""
Created new task 'mytask'
```

BASH

A listing of the current task definitions can be obtained through the RESTful API or the shell. To get the task definition list by using the shell, use the `task list` command.

24.3.1. Automating the Creation of Task Definitions

As of version 2.3.0, the Data Flow server can be configured to auto create task definitions by setting `spring.cloud.dataflow.task.autocreate-task-definitions` to `true`. This is not the default behavior but provided as a convenience. When this property is enabled, a task launch request can specify the registered task application name as the task name. If the task application is registered, the server will create a basic task definition that specifies only the app name, as required. This eliminates a manual step equivalent to something like:

```
dataflow:>task create mytask --definition "mytask"
```

BASH

Command line arguments and deployment properties can still be specified for each task launch request.

24.4. Launching a Task

An adhoc task can be launched through the RESTful API or the shell. To launch an ad-hoc task through the shell, use the `task launch` command, as shown in the following example:

```
dataflow:>task launch mytask
Launched task 'mytask'
```

BASH

When a task is launched, any properties that need to be passed as command line arguments to the task application can be set when launching the task, as follows:

```
dataflow:>task launch mytask --arguments "--server.port=8080 --custom=value"
```

BASH

The arguments need to be passed as `space` delimited values.

Additional properties meant for a `TaskLauncher` itself can be passed in by using a `--properties` option. The format of this option is a comma-separated string of properties prefixed with `app.<task definition name>.<property>`. Properties are passed to `TaskLauncher` as application properties. It is up to an implementation to choose how those are passed into an actual task application. If the property is prefixed with `deployer` instead of `app`, it is passed to `TaskLauncher` as a deployment property and its meaning may be `TaskLauncher` implementation specific.

```
dataflow:>task launch mytask --properties "deployer.timestamp.custom1=value1,app.timestamp.custom2=value2"
```

24.4.1. Application properties

Each application takes properties to customize its behavior. As an example, the `timestamp` task format setting establishes a output format that is different from the default value.

```
dataflow:> task create --definition "timestamp --format=\"yyyy\"" --name printTimeStamp
```

This `timestamp` property is actually the same as the `timestamp.format` property specified by the `timestamp` application. Data Flow adds the ability to use the shorthand form `format` instead of `timestamp.format`. One may also specify the longhand version as well, as shown in the following example:

```
dataflow:> task create --definition "timestamp --timestamp.format=\"yyyy\"" --name printTimeStamp
```

This shorthand behavior is discussed more in the section on [Whitelisting application properties](#). If you have [registered application property metadata](#) you can use tab completion in the shell after typing `--` to get a list of candidate property names.

The shell provides tab completion for application properties. The shell command `app info --name <appName> --type <appType>` provides additional documentation for all the supported properties. The supported Task `<appType>` is `task`.

When restarting Spring Batch Jobs on Kubernetes, you must use the entry point of `shell` or `boot`.

Application Properties With Sensitive Information on Kubernetes

When launching task applications where some of the properties may contain sensitive information, use the `shell` or `boot` as the `entryPointStyle`. This is because the `exec` (default) converts all properties to command line arguments and thus may not be secure in some environments.

24.4.2. Common application properties

In addition to configuration through DSL, Spring Cloud Data Flow provides a mechanism for setting common properties to all the task applications that are launched by it. This can be done by adding properties prefixed with `spring.cloud.dataflow.applicationProperties.task` when starting the server. When doing so, the server passes all the properties, without the prefix, to the instances it launches.

For example, all the launched applications can be configured to use the properties `prop1` and `prop2` by launching the Data Flow server with the following options:

```
--spring.cloud.dataflow.applicationProperties.task.prop1=value1
--spring.cloud.dataflow.applicationProperties.task.prop2=value2
```

BASH

This causes the properties, `prop1=value1` and `prop2=value2`, to be passed to all the launched applications.

Properties configured by using this mechanism have lower precedence than task deployment properties. They are overridden if a property with the same key is specified at task launch time (for example, `app.trigger.prop2` overrides the common property).

24.5. Limit the number concurrent task launches

Spring Cloud Data Flow allows a user to limit the maximum number of concurrently running tasks for each configured platform to prevent the saturation of IaaS/hardware resources. The limit is set to `20` for all supported platforms by default. If the number of concurrently running tasks on a platform instance is greater or equal to the limit, the next task launch request will fail and an error message will be returned via the RESTful API, Shell or UI. This limit can be configured for a platform instance by setting the corresponding deployer property, `spring.cloud.dataflow.task.platform.<platform-type>.accounts[<account-name>].deployment.maximumConcurrentTasks` property, where `<account-name>` is the name of a configured platform account (`default` if no accounts are explicitly configured). The `<platform-type>` refers to one of the currently supported deployers: `local`, `cloudfoundry`, or `kubernetes`.

The TaskLauncher implementation for each supported platform determines the number of currently executing tasks by querying the underlying platform's runtime state if possible. The method for identifying a task varies by platform. For example, launching a task on the local host uses the `LocalTaskLauncher`. The `LocalTaskLauncher` executes a process for each launch request and keeps track of these processes in memory. In this case, we don't query the underlying OS, as it is impractical to identify tasks this way. For Cloud Foundry, tasks are a core concept supported by its deployment model. The state of all tasks, running, completed, or failed, is available directly via the API. This means that every running task container in the account's org and space is included in the running execution count, whether or not it was launched using Spring Cloud Data Flow, or invoking the `CloudFoundryTaskLauncher` directly. For Kubernetes, launching a task via the `KubernetesTaskLauncher`, if successful, results in a running pod which we expect to eventually complete or fail. In this environment there is generally no easy way to identify pods that correspond to a task. For this reason, we only count pods that were launched by the `KubernetesTaskLauncher`. Since the task launcher provides `task-name` label in the pod's metadata, we filter all running pods by the presence of this label.

24.6. Reviewing Task Executions

Once the task is launched, the state of the task is stored in a relational DB. The state includes:

- Task Name
- Start Time
- End Time
- Exit Code
- Exit Message
- Last Updated Time
- Parameters

A user can check the status of their task executions through the RESTful API or the shell. To display the latest task executions through the shell, use the `task execution list` command.

To get a list of task executions for just one task definition, add `--name` and the task definition name, for example `task execution list --name foo`. To retrieve full details for a task execution use the `task execution status` command with the id of the task execution, for example `task execution status --id 549`.

24.7. Destroying a Task Definition

Destroying a Task Definition removes the definition from the definition repository. This can be done through the RESTful API or the shell. To destroy a task through the shell, use the `task destroy` command, as shown in the following example:

```
dataflow:>task destroy mytask
Destroyed task 'mytask'
```

BASH

To destroy all tasks through the shell, use the `task all destroy` command as shown in the following example:

```
dataflow:>task all destroy
Really destroy all tasks? [y, n]: y
All tasks destroyed
```

BASH

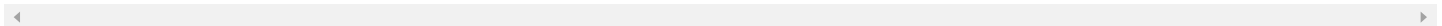
Or use the force command:

```
dataflow:>task all destroy --force
All tasks destroyed
```

BASH

The task execution information for previously launched tasks for the definition remains in the task repository.

This does not stop any currently executing tasks for this definition. Instead, it removes the task definition from the database.



The `task destroy <task-name>` deletes only the definition and not the task deployed on Cloud Foundry. The only way to do this now is through the CLI in two steps. First, obtain a list of the apps by using the `cf apps` command. Identify

the task application to be deleted and run the `cf delete <task-name>` command.

24.8. Validating a Task

Sometimes the one or more of the apps contained within a task definition contain an invalid URI in its registration. This can be caused by an invalid URI entered at app registration time or the app was removed from the repository from which it was to be drawn. To verify that all the apps contained in a task are resolve-able, a user can use the `validate` command. For example:

```
dataflow:>task validate time-stamp
```

BASH

Task Name	Task Definition
time-stamp	timestamp

```
time-stamp is a valid task.
```

App Name	Validation Status
task:timestamp	valid

In the example above the user validated their time-stamp task. As we see `task:timestamp` app is valid. Now let's see what happens if we have a stream definition with a registered app with an invalid URI.

```
dataflow:>task validate bad-timestamp
```

BASH

Task Name	Task Definition
bad-timestamp	badtimestamp

```
bad-timestamp is an invalid task.
```

App Name	Validation Status
task:badtimestamp	invalid

In this case Spring Cloud Data Flow states that the task is invalid because `task:badtimestamp` has an invalid URI.

24.9. Stopping a Task Execution

In some cases a task that is executing on a platform may not stop because of a problem on the platform or the application business logic itself. For such cases Spring Cloud Data Flow offers the user the ability to send a request to the platform to terminate the task execution. To do this a user can submit a `task execution stop` for a given set of task executions. For example:

```
task execution stop --ids 5
```

BASH

```
Request to stop the task execution with id(s): 5 has been submitted
```

With the above command, the trigger to stop the execution of id=5 will be submitted to the underlying deployer implementation, and as a result, the operation will stop the execution of that task. When we view the result for the task execution, we will see that the task execution completed with a 0 exit code.

```
dataflow:>task execution list
```

BASH

Task Name	ID	Start Time	End Time	Exit Code
batch-demo	5	Mon Jul 15 13:58:41 EDT 2019	Mon Jul 15 13:58:55 EDT 2019	0
timestamp	1	Mon Jul 15 09:26:41 EDT 2019	Mon Jul 15 09:26:41 EDT 2019	0

If a user submits a stop for a task execution that has child task executions associated with it, like a composed task, a stop request will be sent for each of the child task executions.

When stopping a task execution that has a running Spring Batch job, the job will be left with the batch status of `STARTED`. Each of the supported platforms send a SIG-INT to the task application when a stop is requested that allows Spring Cloud Task to capture the state of the app, however Spring Batch does not handle a SIG-INT and thus the job stops in the `STARTED` status.

24.9.1. Stopping a Task Execution that was started outside of Spring Cloud Data Flow

You may wish to stop a task that has been launched outside of Spring Cloud Data Flow. An example of this is the worker applications launched by a Remote Batch Partitioned application. In such cases the Remote Batch Partitioned Application stores the external-execution-id for each of the worker applications, however no platform information is stored. So when Spring Cloud Data Flow has to stop a Remote Batch Partitioned application and its worker applications, you will need to specify the platform name as shown below:

```
dataflow:>task execution stop --ids 1 --platform myplatform
Request to stop the task execution with id(s): 1 for platform myplatform has been submitted
```

BASH

25. Subscribing to Task/Batch Events

You can also tap into various task and batch events when the task is launched. If the task is enabled to generate task or batch events (with the additional dependencies `spring-cloud-task-stream` and, in the case of Kafka as the binder, `spring-cloud-stream-binder-kafka`), those events are published during the task lifecycle. By default, the destination names for those published events on the broker (Rabbit, Kafka, and others) are the event names themselves (for instance: `task-events`, `job-execution-events`, and so on).

```
dataflow:>task create myTask --definition "myBatchJob"
dataflow:>stream create task-event-subscriber1 --definition ":task-events > log" --deploy
dataflow:>task launch myTask
```

BASH

You can control the destination name for those events by specifying explicit names when launching the task, as follows:

The following table lists the default task and batch event and destination names on the broker:

Table 1. Task and Batch Event Destinations

Event	Destination
Task events	task-events
Job Execution events	job-execution-events
Step Execution events	step-execution-events
Item Read events	item-read-events
Item Process events	item-process-events
Item Write events	item-write-events
Skip events	skip-events

26. Composed Tasks

Spring Cloud Data Flow lets a user create a directed graph where each node of the graph is a task application. This is done by using the DSL for composed tasks. A composed task can be created through the RESTful API, the Spring Cloud Data Flow Shell, or the Spring Cloud Data Flow UI.

26.1. Configuring the Composed Task Runner

Composed tasks are executed through a task application called the [Composed Task Runner](#).

26.1.1. Registering the Composed Task Runner

By default, the Composed Task Runner application is not registered with Spring Cloud Data Flow. Consequently, to launch composed tasks, we must first register the Composed Task Runner as an application with Spring Cloud Data Flow, as follows:

```
app register --name composed-task-runner --type task --uri
maven://org.springframework.cloud.task.app:composedtaskrunner-task:2.1.0.RELEASE
```

You can also configure Spring Cloud Data Flow to use a different task definition name for the composed task runner. This can be done by setting the `spring.cloud.dataflow.task.composedTaskRunnerName` property to the name of your choice. You can then register the composed task runner application with the name you set by using that property.

26.1.2. Configuring the Composed Task Runner

The Composed Task Runner application has a `dataflow.server.uri` property that is used for validation and for launching child tasks. This defaults to `localhost:9393`. If you run a distributed Spring Cloud Data Flow server, as you would if you deploy the server on Cloud Foundry, YARN, or Kubernetes, you need to provide the URI that can be used to access the server. You can either provide this `dataflow.server.uri` property for the Composed Task Runner application when launching a composed task or you can provide a `spring.cloud.dataflow.server.uri` property for the Spring Cloud Data Flow server when it is started. For the latter case, the `dataflow.server.uri` Composed Task Runner application property is automatically set when a composed task is launched.

Configuration Options

The `ComposedTaskRunner` task has the following options:

- **composed-task-arguments** The command line arguments to be used for each of the tasks. (String, default: <none>).
- **increment-instance-enabled** Allows a single `ComposedTaskRunner` instance to be re-executed without changing the parameters. Default is false which means a `ComposedTaskRunner` instance can only be executed once with a given set of parameters, if true it can be re-executed. (Boolean, default: false). `ComposedTaskRunner` is built using [Spring Batch](#) and thus upon a successful execution the batch job is considered complete. To launch the same `ComposedTaskRunner` definition multiple times you must set the `increment-instance-enabled` property to true or change the parameters for the definition for each launch. When using this option it must be applied for all task launches for the desired application including the first launch.
- **interval-time-between-checks** The amount of time in millis that the `ComposedTaskRunner` will wait between checks of the database to see if a task has completed. (Integer, default: 10000). `ComposedTaskRunner` uses the datastore to determine the status of each child tasks. This interval indicates to `ComposedTaskRunner` how often it should check the status its child tasks.
- **max-wait-time** The maximum amount of time in millis that a individual step can run before the execution of the Composed task is failed (Integer, default: 0). Determines the maximum time each child task is allowed to run before the CTR will terminate with a failure. The default of 0 indicates no timeout.
- **split-thread-allow-core-thread-timeout** Specifies whether to allow split core threads to timeout. Default is false; (Boolean, default: false) Sets the policy governing whether core threads may timeout and terminate if no tasks arrive within the keep-alive time, being replaced if needed when new tasks arrive.
- **split-thread-core-pool-size** Split's core pool size. Default is 1; (Integer, default: 1) Each child task contained in a split requires a thread in order to execute. So for example a definition like: `<AAA || BBB || CCC> && <DDD || EEE>` would require a `split-thread-core-pool-size` of 3. This is because the largest split contains 3 child tasks. A count of 2 would mean that `AAA` and `BBB` would run in parallel but `CCC` would wait until either `AAA` or `BBB` to finish in order to run. Then `DDD` and `EEE` would run in parallel.
- **split-thread-keep-alive-seconds** Split's thread keep alive seconds. Default is 60. (Integer, default: 60) If the pool currently has more than `corePoolSize` threads, excess threads will be terminated if they have been idle for more than the `keepAliveTime`.
- **split-thread-max-pool-size** Split's maximum pool size. Default is `{@code Integer.MAX_VALUE}` (Integer, default: <none>). Establish the maximum number of threads allowed for the thread pool.
- **split-thread-queue-capacity** Capacity for Split's `BlockingQueue`. Default is `{@code Integer.MAX_VALUE}`. (Integer, default: <none>)
 - If fewer than `corePoolSize` threads are running, the `Executor` always prefers adding a new thread rather than queuing.

- If `corePoolSize` or more threads are running, the `Executor` always prefers queuing a request rather than adding a new thread.
- If a request cannot be queued, a new thread is created unless this would exceed `maximumPoolSize`, in which case, the task will be rejected.
- **split-thread-wait-for-tasks-to-complete-on-shutdown** Whether to wait for scheduled tasks to complete on shutdown, not interrupting running tasks and executing all tasks in the queue. Default is `false`; (Boolean, default: `false`)
- **dataflow-server-uri** The URI for the dataflow server that will receive task launch requests. (String, default: `localhost:9393`)
- **dataflow-server-username** The optional username for the dataflow server that will receive task launch requests. Used to access the the dataflow server using Basic Authentication. Not used if `dataflow-server-access-token` is set.
- **dataflow-server-password** The optional password for the dataflow server that will receive task launch requests. Used to access the the dataflow server using Basic Authentication. Not used if `dataflow-server-access-token` is set.
- **dataflow-server-access-token** This property sets optional OAuth2 Access Token. Typically the value is automatically set using the token from the currently logged in user, if available. However, for special use-cases this value can also be set explicitly.

A special boolean property `dataflow-server-use-user-access-token` exists for the case where you want to use the access token of the currently logged user and propagate it to the Composed Task Runner. This property is only used by Spring Cloud Data Flow and if set to `true` will auto-populate the property `dataflow-server-access-token`.

Note when using the options above as environment variables, convert to uppercase, remove the dash character and replace with the underscore character. For example: `increment-instance-enabled` would be `INCREMENT_INSTANCE_ENABLED`.

For the latest configuration options for Composed Task Runner click the link [here](#).

26.2. The Lifecycle of a Composed Task

The lifecycle of a composed task has three parts:

- [Creating a Composed Task](#)
- [Stopping a Composed Task](#)
- [Restarting a Composed Task](#)

26.2.1. Creating a Composed Task

The DSL for the composed tasks is used when creating a task definition through the `task create` command, as shown in the following example:

```
dataflow:> app register --name timestamp --type task --uri maven://org.springframework.cloud.task.app:timestamp-task:
dataflow:> app register --name mytaskapp --type task --uri file:///home/tasks/mytask.jar
dataflow:> task create my-composed-task --definition "mytaskapp && timestamp"
dataflow:> task launch my-composed-task
```

BASH

In the preceding example, we assume that the applications to be used by our composed task have not been registered yet. Consequently, in the first two steps, we register two task applications. We then create our composed task definition by using the `task create` command. The composed task DSL in the preceding example, when launched, runs `mytaskapp` and then runs the `timestamp` application.

But before we launch the `my-composed-task` definition, we can view what Spring Cloud Data Flow generated for us. This can be done by executing the `task list` command, as shown (including its output) in the following example:

dataflow:>task list

BASH

Task Name	Task Definition	Task Status
my-composed-task	mytaskapp && timestamp	unknown
my-composed-task-mytaskapp	mytaskapp	unknown
my-composed-task-timestamp	timestamp	unknown

In the example, Spring Cloud Data Flow created three task definitions, one for each of the applications that makes up our composed task (`my-composed-task-mytaskapp` and `my-composed-task-timestamp`) as well as the composed task (`my-composed-task`) definition. We also see that each of the generated names for the child tasks is made up of the name of the composed task and the name of the application, separated by a dash - (as in `my-composed-task - mytaskapp`).

Task Application Parameters

The task applications that make up the composed task definition can also contain parameters, as shown in the following example:

dataflow:> task create my-composed-task --definition "mytaskapp --displayMessage=hello && timestamp --format=YYYY"

26.2.2. Launching a Composed Task

Launching a composed task is done the same way as launching a stand-alone task, as follows:

task launch my-composed-task

Once the task is launched, and assuming all the tasks complete successfully, you can see three task executions when executing a `task execution list` , as shown in the following example:

dataflow:>task execution list

BASH

Task Name	ID	Start Time	End Time	Exit Code
my-composed-task-timestamp	713	Wed Apr 12 16:43:07 EDT 2017	Wed Apr 12 16:43:07 EDT 2017	0
my-composed-task-mytaskapp	712	Wed Apr 12 16:42:57 EDT 2017	Wed Apr 12 16:42:57 EDT 2017	0
my-composed-task	711	Wed Apr 12 16:42:55 EDT 2017	Wed Apr 12 16:43:15 EDT 2017	0

In the preceding example, we see that `my-compose-task` launched and that it also launched the other tasks in sequential order. All of them executed successfully with `Exit Code` as `0` .

Passing properties to the child tasks

To set the properties for child tasks in a composed task graph at task launch time, you would use the following format of `app.<composed task definition name>.<child task app name>.<property>` . Using the following Composed

an example:

```
dataflow:> task create my-composed-task --definition "mytaskapp && mytimestamp"
```

BASH

To have mytaskapp display 'HELLO' and set the mytimestamp timestamp format to 'YYYY' for the Composed Task definition, you would use the following task launch format:

```
task launch my-composed-task --properties "app.my-composed-task.mytaskapp.displayMessage=HELLO,app.my-composed-task.mytimes
```

BASH

Similar to application properties, the `deployer` properties can also be set for child tasks using the format `deployer.<composed task definition name>.<child task app name>.<deployer-property>`.

```
task launch my-composed-task --properties "deployer.my-composed-task.mytaskapp.memory=2048m,app.my-composed-task.mytimestamp
Launched task 'a1'
```

BASH

Passing arguments to the composed task runner

Command line arguments for the composed task runner can be passed using `--arguments` option.

For example:

```
dataflow:>task create my-composed-task --definition "<aaa: timestamp || bbb: timestamp>"
Created new task 'my-composed-task'
```

BASH

```
dataflow:>task launch my-composed-task --arguments "--increment-instance-enabled=true --max-wait-time=50000 --split-thread-
Launched task 'my-composed-task'
```

Launching a Composed Task using Custom Composed Task Runner

In some cases a user will need to launch a composed task using a custom version of a Composed Task Runner other than default application that is shipped out-of-the-box. To do this, a user will need to register the custom version of the Composed Task Runner and then specify the `--composedTaskRunnerName` property pointing to the custom application at task launch as shown below:

```
dataflow:>app register --name best-ctr --type task --uri maven://the.best.ctr.composed-task-runner:1.0.0.RELEASE
```

BASH

```
dataflow:>task create mycomposedtask --definition "te:timestamp && tr:timestamp"
Created new task 'mycomposedtask'
```

```
dataflow:>task launch --name mycomposedtask --composedTaskRunnerName best-ctr
```

The app specified by the `composedTaskRunnerName` needs to be a task registered in the Application Registry.

Exit Statuses

The following list shows how the Exit Status is set for each step (task) contained in the composed task following each step execution:

- If the `TaskExecution` has an `ExitMessage`, that is used as the `ExitStatus`.
- If no `ExitMessage` is present and the `ExitCode` is set to zero, then the `ExitStatus` for the step is `COMPLETED`.
- If no `ExitMessage` is present and the `ExitCode` is set to any non-zero number, the `ExitStatus` for the step is `FAILED`.

26.2.3. Destroying a Composed Task

The command used to destroy a stand-alone task is the same as the command used to destroy a composed task. The only difference is that destroying a composed task also destroys the child tasks associated with it. The following example shows the task list before and after using the `destroy` command:

```
dataflow:>task list
```

BASH

Task Name	Task Definition	Task Status
my-composed-task	mytaskapp && timestamp	COMPLETED
my-composed-task-mytaskapp	mytaskapp	COMPLETED
my-composed-task-timestamp	timestamp	COMPLETED

```
...
```

```
dataflow:>task destroy my-composed-task
dataflow:>task list
```

Task Name	Task Definition	Task Status
-----------	-----------------	-------------

26.2.4. Stopping a Composed Task

In cases where a composed task execution needs to be stopped, you can do so through the:

- RESTful API
- Spring Cloud Data Flow Dashboard

To stop a composed task through the dashboard, select the Jobs tab and click the Stop button next to the job execution that you want to stop.

The composed task run is stopped when the currently running child task completes. The step associated with the child task that was running at the time that the composed task was stopped is marked as `STOPPED` as well as the composed task job execution.

26.2.5. Restarting a Composed Task

In cases where a composed task fails during execution and the status of the composed task is `FAILED`, the task can be restarted. You can do so through the:

- RESTful API
- The shell
- Spring Cloud Data Flow Dashboard

To restart a composed task through the shell, launch the task with the same parameters. To restart a composed task through the dashboard, select the Jobs tab and click the Restart button next to the job execution that you want to restart.

Restarting a Composed Task job that has been stopped (through the Spring Cloud Data Flow Dashboard or RESTful API) relaunches the `STOPPED` child task and then launches the remaining (unlaunched) child tasks in the specified order.

27. Composed Tasks DSL

Composed tasks can be run in three ways:

- [Conditional Execution](#)
- [Transitional Execution](#)
- [Split Execution](#)

27.1. Conditional Execution

Conditional execution is expressed by using a double ampersand symbol (`&&`). This lets each task in the sequence be launched only if the previous task successfully completed, as shown in the following example:

```
task create my-composed-task --definition "task1 && task2"
```

When the composed task called `my-composed-task` is launched, it launches the task called `task1` and, if it completes successfully, then the task called `task2` is launched. If `task1` fails, then `task2` does not launch.

You can also use the Spring Cloud Data Flow Dashboard to create your conditional execution, by using the designer to drag and drop applications that are required and connecting them together to create your directed graph, as shown in the following image:

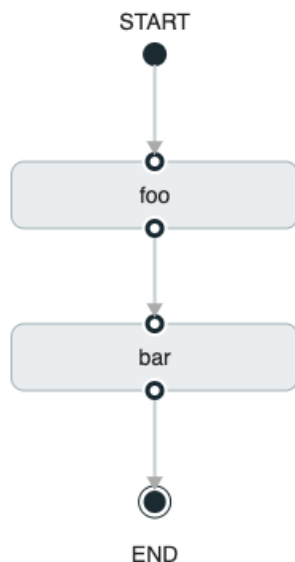


Figure 2. Conditional Execution

The preceding diagram is a screen capture of the directed graph as it being created by using the Spring Cloud Data Flow Dashboard. You can see that there are four components in the diagram that comprise a conditional execution:

- Start icon: All directed graphs start from this symbol. There is only one.
- Task icon: Represents each task in the directed graph.
- End icon: Represents the termination of a directed graph.
- Solid line arrow: Represents the flow conditional execution flow between:
 - Two applications.
 - The start control node and an application.
 - An application and the end control node.
- End icon: All directed graphs end at this symbol.

You can view a diagram of your directed graph by clicking the Detail button next to the composed task definition on the Definitions tab.

27.2. Transitional Execution

The DSL supports fine-grained control over the transitions taken during the execution of the directed graph. Transitions are specified by providing a condition for equality based on the exit status of the previous task. A task transition is represented by the following symbol `->`.

27.2.1. Basic Transition

A basic transition would look like the following:

```
task create my-transition-composed-task --definition "foo 'FAILED' -> bar 'COMPLETED' -> baz"
```

In the preceding example, `foo` would launch, and, if it had an exit status of `FAILED`, the `bar` task would launch. If the exit status of `foo` was `COMPLETED`, `baz` would launch. All other statuses returned by `foo` have no effect, and the task would terminate normally.

Using the Spring Cloud Data Flow Dashboard to create the same "basic transition" would resemble the following image:

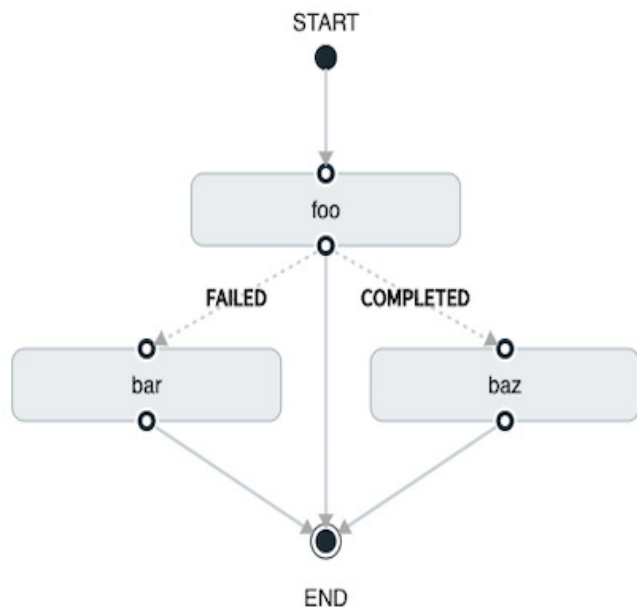


Figure 3. Basic Transition

The preceding diagram is a screen capture of the directed graph as it being created in the Spring Cloud Data Flow Dashboard. Notice that there are two different types of connectors:

- Dashed line: Represents transitions from the application to one of the possible destination applications.
- Solid line: Connects applications in a conditional execution or a connection between the application and a control node (start or end).

To create a transitional connector:

1. When creating a transition, link the application to each possible destination by using the connector.
2. Once complete, go to each connection and select it by clicking it.
3. A bolt icon appears.
4. Click that icon.
5. Enter the exit status required for that connector.
6. The solid line for that connector turns to a dashed line.

27.2.2. Transition With a Wildcard

Wildcards are supported for transitions by the DSL, as shown in the following:

```
task create my-transition-composed-task --definition "foo 'FAILED' -> bar '*' -> baz"
```

In the preceding example, `foo` would launch, and, if it had an exit status of `FAILED`, the `bar` task would launch. For any exit status of `foo` other than `FAILED`, `baz` would launch.

Using the Spring Cloud Data Flow Dashboard to create the same “transition with wildcard” would resemble the following image:

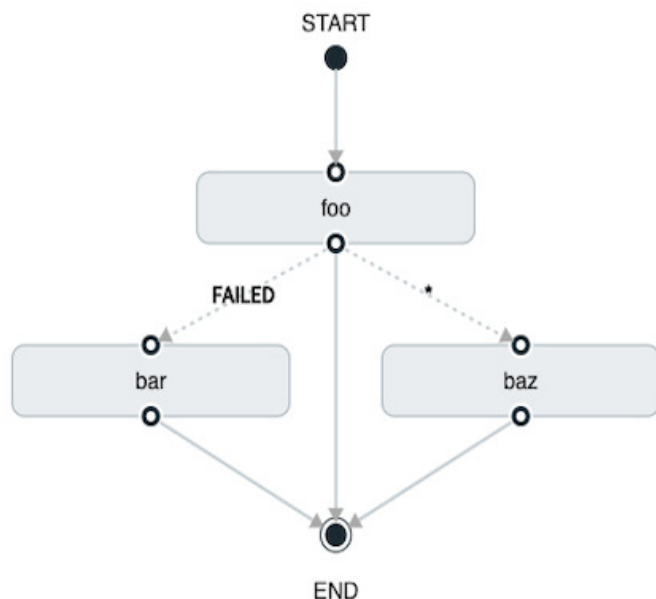


Figure 4. Basic Transition With Wildcard

27.2.3. Transition With a Following Conditional Execution

A transition can be followed by a conditional execution so long as the wildcard is not used, as shown in the following example:

```
task create my-transition-conditional-execution-task --definition "foo 'FAILED' -> bar 'UNKNOWN' -> baz && qux && quux"
```

In the preceding example, `foo` would launch, and, if it had an exit status of `FAILED`, the `bar` task would launch. If `foo` had an exit status of `UNKNOWN`, `baz` would launch. For any exit status of `foo` other than `FAILED` or `UNKNOWN`, `qux` would launch and, upon successful completion, `quux` would launch.

Using the Spring Cloud Data Flow Dashboard to create the same “transition with conditional execution” would resemble the following image:

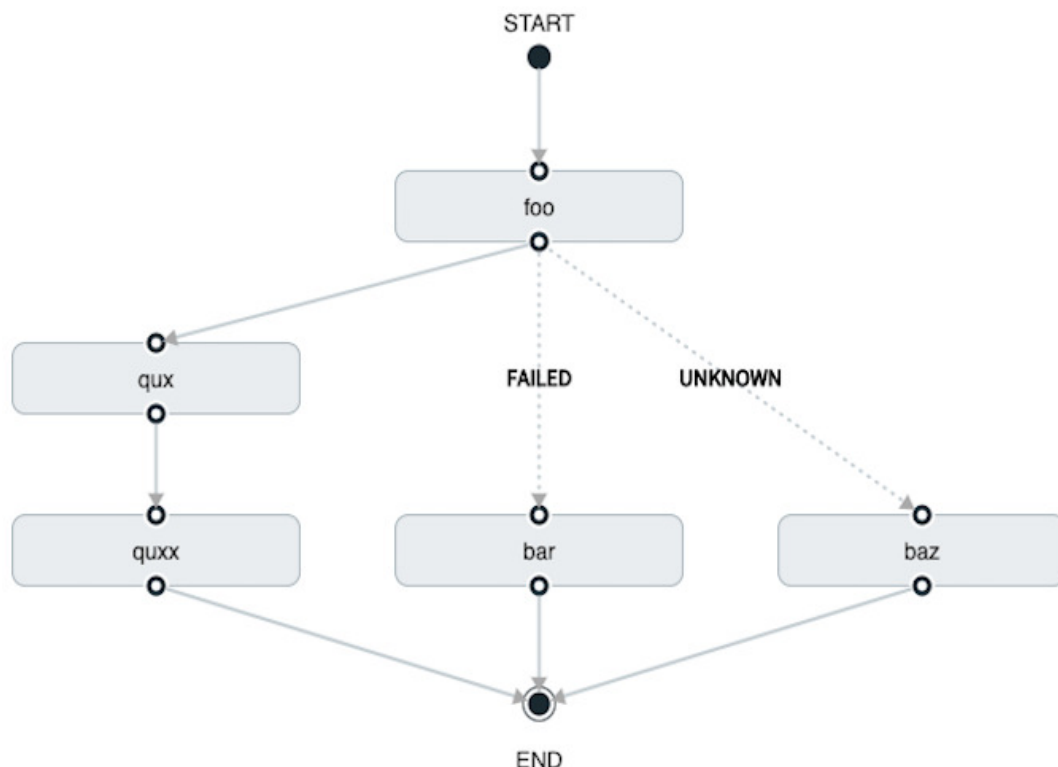


Figure 5. Transition With Conditional Execution

In this diagram we see the dashed line (transition) connecting the `foo` application to the target applications, but a solid line connecting the conditional executions between `foo`, `qux`, and `quux`.

27.3. Split Execution

Splits allow multiple tasks within a composed task to be run in parallel. It is denoted by using angle brackets (`<>`) to group tasks and flows that are to be run in parallel. These tasks and flows are separated by the double pipe `||` symbol, as shown in the following example:

```
task create my-split-task --definition "<foo || bar || baz>"
```

The preceding example above launches tasks `foo`, `bar` and `baz` in parallel.

Using the Spring Cloud Data Flow Dashboard to create the same “split execution” would resemble the following image:

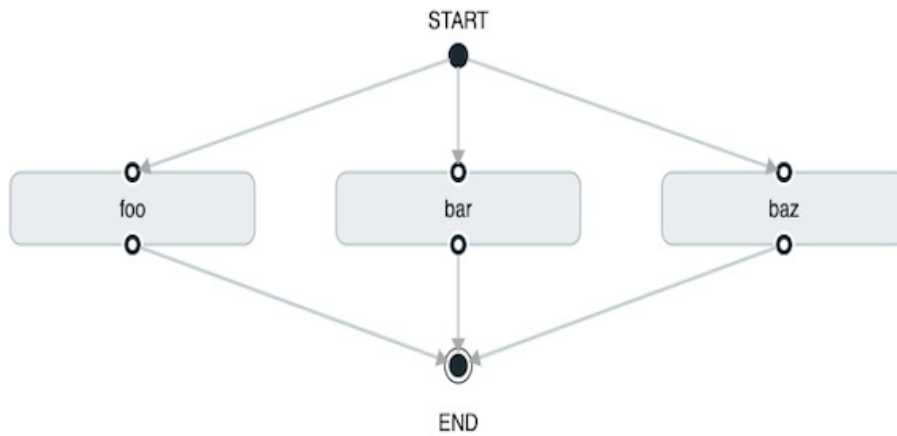


Figure 6. Split

With the task DSL, a user may also execute multiple split groups in succession, as shown in the following example:

```
`task create my-split-task --definition "<foo || bar || baz> && <qux || quux>"
```

In the preceding example, tasks `foo`, `bar`, and `baz` are launched in parallel. Once they all complete, then tasks `qux` and `quux` are launched in parallel. Once they complete, the composed task ends. However, if `foo`, `bar`, or `baz` fails, the split containing `qux` and `quux` does not launch.

Using the Spring Cloud Data Flow Dashboard to create the same “split with multiple groups” would resemble the following image:

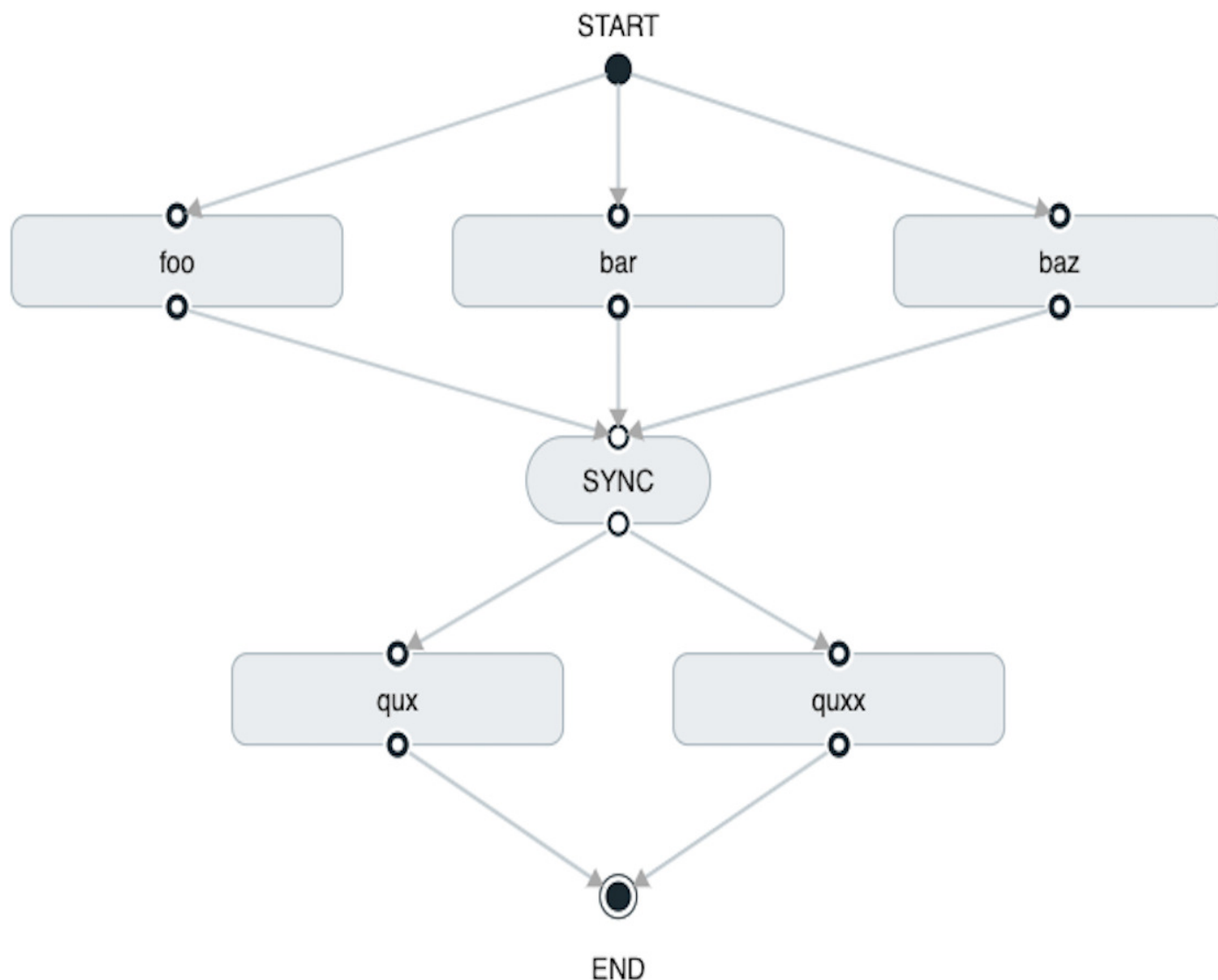


Figure 7. Split as a part of a conditional execution

Notice that there is a `SYNC` control node that is inserted by the designer when connecting two consecutive splits.

Tasks that are used in a split should not set their `ExitMessage`. Setting the `ExitMessage` is only to be used with [transitions](#).

27.3.1. Split Containing Conditional Execution

A split can also have a conditional execution within the angle brackets, as shown in the following example:

```
task create my-split-task --definition "<foo && bar || baz>"
```

In the preceding example, we see that `foo` and `baz` are launched in parallel. However, `bar` does not launch until `foo` completes successfully.

Using the Spring Cloud Data Flow Dashboard to create the same "split containing conditional execution" resembles the following image:

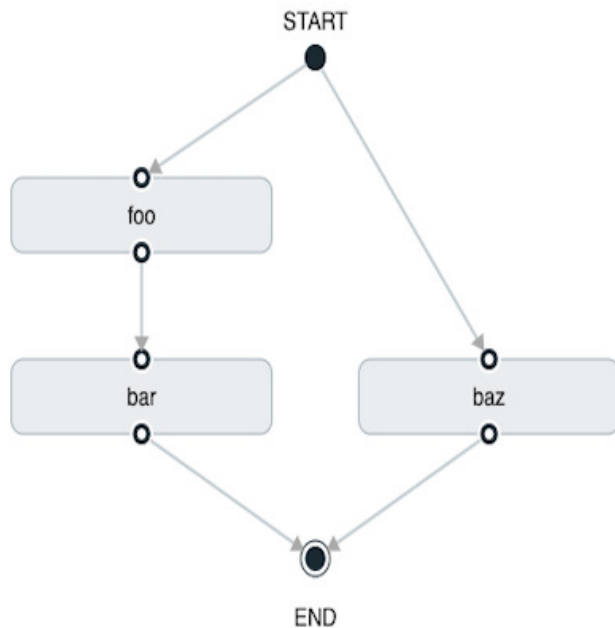


Figure 8. Split with conditional execution

27.3.2. Establishing the proper thread count for splits

Each child task contained in a split requires a thread in order to execute. To set this properly you want to look at your graph and count the split that has the largest number of child tasks, this will be the number of threads you will need to utilize. To set the thread count use the `split-thread-core-pool-size` property (defaults to 1). So for example a definition like: `<AAA || BBB || CCC> && <DDD || EEE>` would require a `split-thread-core-pool-size` of 3. This is because the largest split contains 3 child tasks. A count of 2 would mean that `AAA` and `BBB` would run in parallel but `CCC` would wait until either `AAA` or `BBB` to finish in order to run. Then `DDD` and `EEE` would run in parallel.

28. Launching Tasks from a Stream

You can launch a task from a stream by using the [tasklauncher-dataflow](#) sink. The sink connects to a Data Flow server and uses its REST API to launch any defined task. The sink accepts a [JSON payload](#) representing a task launch request which provides the name of the task to launch, and may include command line arguments and deployment properties.

The [app-starters-task-launch-request-common](#) component, in conjunction with Spring Cloud Stream [functional composition](#), can transform the output of any source or processor to a task launch request.

Adding a dependency to `app-starters-task-launch-request-common`, auto-configures a `java.util.function.Function` implementation, registered via [Spring Cloud Function](#) as `taskLaunchRequest`.

For example, you can start with the [time](#) source, add the following dependency, build it, and register it as a custom source. We'll call it `time-tlr` in this example.

```
<dependency>
  <groupId>org.springframework.cloud.stream.app</groupId>
  <artifactId>app-starters-task-launch-request-common</artifactId>
</dependency>
```

XML

[Spring Cloud Stream Initializer](#) provides a great starting point for creating stream applications.

Next, [register](#) the `tasklauncher-dataflow` sink, and create a task (we will use the provided timestamp task).

```
stream create --name task-every-minute --definition "time-tlr --trigger.fixed-delay=60 --spring.cloud.stream.function.definition=task-launcher"
```

The preceding stream will produce a task launch request every minute. The request provides the name of the task to launch: `{"name":"timestamp-task"}`.

The following stream definition illustrates the use of command line arguments. It will produce messages like `{"args":["foo=bar","time=12/03/18 17:44:12"],"deploymentProps":{"name":"timestamp-task"}}` to provide command line arguments to the task:

```
stream create --name task-every-second --definition "time-tlr --spring.cloud.stream.function.definition=task-launcher --args=foo=bar,time=12/03/18 17:44:12"
```

Note the use of SpEL expressions to map each message payload to the `time` command line argument, along with a static argument `foo=bar`.

You can then see the list of task executions by using the shell command `task execution list`, as shown (with its output) in the following example:

```
dataflow:>task execution list
```

BASH

Task Name	ID	Start Time	End Time	Exit Code
timestamp-task_26176	4	Tue May 02 12:13:49 EDT 2017	Tue May 02 12:13:49 EDT 2017	0
timestamp-task_32996	3	Tue May 02 12:12:49 EDT 2017	Tue May 02 12:12:49 EDT 2017	0
timestamp-task_58971	2	Tue May 02 12:11:50 EDT 2017	Tue May 02 12:11:50 EDT 2017	0
timestamp-task_13467	1	Tue May 02 12:10:50 EDT 2017	Tue May 02 12:10:50 EDT 2017	0

In this example, we have shown how to use the `time` source to launch a task at a fixed rate. This pattern may be applied to any source to launch a task in response to any event.

28.1. Launching a Composed Task From a Stream

A composed task can be launched with the `tasklauncher-dataflow` sink, as discussed [here](#). Since we use the `ComposedTaskRunner` directly, we need to set up the task definitions for the composed task runner itself, along with the composed tasks, prior to the creation of the composed task launching stream. Suppose we wanted to create the following composed task definition: `AAA && BBB`. The first step would be to create the task definitions, as shown in the following example:

```
task create composed-task-runner --definition "composed-task-runner"
task create AAA --definition "timestamp"
task create BBB --definition "timestamp"
```

Releases of `ComposedTaskRunner` can be found [here](#).

Now that the task definitions we need for composed task definition are ready, we need to create a stream that launches `ComposedTaskRunner`. So, in this case, we create a stream with

- The `time` source customized to emit task launch requests, as shown [above](#).
- The `tasklauncher-dataflow` sink that launches the `ComposedTaskRunner`

The stream should resemble the following:

```
stream create ctr-stream --definition "time --fixed-delay=30 --task.launch.request.task-name=composed-task-launcher --task."
```

For now, we focus on the configuration that is required to launch the `ComposedTaskRunner`:

- **graph:** this is the graph that is to be executed by the `ComposedTaskRunner`. In this case it is `AAA&&BBB`.
- **increment-instance-enabled:** This lets each execution of `ComposedTaskRunner` be unique. `ComposedTaskRunner` is built by using [Spring Batch](#). Thus, we want a new Job Instance for each launch of the `ComposedTaskRunner`. To do this, we set `increment-instance-enabled` to be `true`.

29. Sharing Spring Cloud Data Flow's Datastore with Tasks

As discussed in the [Tasks](#) documentation Spring Cloud Data Flow allows a user to view Spring Cloud Task App executions. So in this section we will discuss what is required by a Task Application and Spring Cloud Data Flow to share the task execution information.

29.1. A Common DataStore Dependency

Spring Cloud Data Flow supports many databases out-of-the-box, so all the user typically has to do is declare the `spring_datasource_*` environment variables to establish what data store Spring Cloud Data Flow will need. So whatever database you decide to use for Spring Cloud Data Flow make sure that the your task also includes that database dependency in its `pom.xml` or `gradle.build` file. If the database dependency that is used by Spring Cloud Data Flow is not present in the Task Application, the task will fail and the task execution will not be recorded.

29.2. A Common Data Store

Spring Cloud Data Flow and your task application must access the same datastore instance. This is so that the task executions recorded by the task application can be read by Spring Cloud Data Flow to list them in the Shell and Dashboard views. Also the task app must have read & write privileges to the task data tables that are used by Spring Cloud Data Flow.

Given the understanding of Datasource dependency between Task apps and Spring Cloud Data Flow, let's review how to apply them in various Task orchestration scenarios.

29.2.1. Simple Task Launch

When launching a task from Spring Cloud Data Flow, Data Flow adds its datasource properties (`spring.datasource.url` , `spring.datasource.driverClassName` , `spring.datasource.username` , `spring.datasource.password`) to the app properties of the task being launched. Thus a task application will record its task execution information to the Spring Cloud Data Flow repository.

29.2.2. Composed Task Runner

Spring Cloud Data Flow allows a user to create a directed graph where each node of the graph is a task application and this is done via the [Composed Task Runner](#). In this case the rules that applied to a [Simple Task Launch](#) or Task Launcher Sink apply to the composed task runner as well. All child apps must also have access to the datastore that is being used by the composed task runner. Also, All child apps must have the same database dependency as the composed task runner enumerated in their `pom.xml` or `gradle.build` file.

29.2.3. Launching a task externally from Spring Cloud Data Flow

Users may wish to launch Spring Cloud Task applications via another method (scheduler for example) but still track the task execution via Spring Cloud Data Flow. This can be done so long as the task applications observe the rules specified [here](#) and [here](#).

If a user wishes to use Spring Cloud Data Flow to view their [Spring Batch](#) jobs, the user must make sure that their batch application use the `@EnableTask` annotation and follow the rules enumerated [here](#) and [here](#). More information is available [here](#).

30. Scheduling Tasks

Spring Cloud Data Flow lets a user schedule the execution of tasks via a cron expression. A schedule can be created through the RESTful API or the Spring Cloud Data Flow UI.

30.1. The Scheduler

Spring Cloud Data Flow will schedule the execution of its tasks via a scheduling agent that is available on the cloud platform. When using the Cloud Foundry platform Spring Cloud Data Flow will use the [PCF Scheduler](#). When using Kubernetes, a [CronJob](#) will be used. The way that Spring Cloud Data Flow does this is by deploying a `SchedulerTaskLauncher` application to the platform and then the scheduling agent (e.g., `CronJob` in Kubernetes) is notified to schedule the `SchedulerTaskLauncher`. When the scheduling agent launches the [SchedulerTaskLauncher](#), the `SchedulerTaskLauncher` will utilize Spring Cloud Data Flow's RESTful API to launch the task.

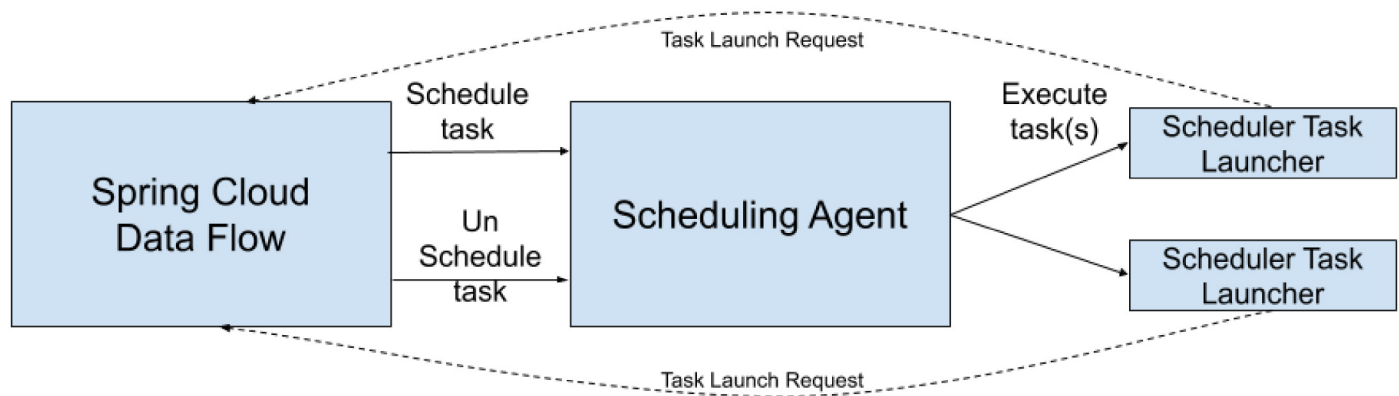


Figure 9. Architectural Overview

30.2. Enabling Scheduling

By default the Spring Cloud Data Flow leaves the scheduling feature disabled. To enable the scheduling feature the following feature properties must be set to `true` :

- `spring.cloud.dataflow.features.schedules-enabled`
- `spring.cloud.dataflow.features.tasks-enabled`

30.3. The Lifecycle of a Schedule

The lifecycle of a schedule has 2 parts:

- [Scheduling a Task Execution](#)
- [Deleting a Schedule](#)
- [Listing Schedules](#)

30.3.1. Scheduling a Task Execution

You can schedule a task execution via the:

- RESTful API
- Spring Cloud Data Flow Dashboard

To schedule a task from the UI click the Tasks tab at the top of the screen, this will take you to the Task Definitions screen. From the Task Definition that you wish to schedule, please click the dropdown icon and then select the `Schedule Task` option. This will lead you to a `Create Schedule(s)` screen, where you will create a unique name for the schedule and enter the associated cron expression. Keep in mind you can always create multiple schedules for a single task definition.

30.3.2. Deleting a Schedule

You can delete a schedule via the:

- RESTful API
- Spring Cloud Data Flow Dashboard

To delete a schedule through the dashboard, select the Schedule tab under Tasks tab and click the dropdown icon next to the schedule you wish to delete and select the `Delete schedule` option.

Any currently running tasks that were run by the scheduling agent will not be stopped if the schedule is deleted. It only prevents future executions.

30.3.3. Listing Schedules

To view your schedules click the Tasks tab and then click the Schedules tab on the Tasks page. From here you will see your available schedules. You will note that all schedules created by Spring Cloud Data Flow will have a suffix appended to it consisting of `-scdf-<task definition name>`. For example if I create a schedule name of `my-schedule` for a task definition of `tstamp` the schedule name will look like `my-schedule-scdf-tstamp`.

The `scdf-` separator is configurable by setting the `spring.cloud.dataflow.task.scheduleNamePrefix` property with the value you prefer.

31. Continuous Deployment

As task applications evolve, you want to get your updates to production. This section walks through the capabilities that Spring Cloud Data Flow provides around being able to update task applications.

When a task application is registered ([Registering a Task Application](#)), a version is associated with it. A task application can have multiple versions associated with it, with one selected as the default. The following image illustrates an application with multiple versions associated with it (see the timestamp entry).

Data Flow

Quick Search

Apps

Runtime

Streams

Tasks

Jobs

Audit Records

About & Docs

Applications + Add Application(s)

This section lists all the available applications and provides the control to register/unregister them (if applicable).

Actions Filter items All types Refresh

Name	Type	Uri	Versions
composed-task-runner	TASK	maven://org.springframework.cloud.task.app:composedtaskrunner-task:2.1.0.RELEASE	2.1.0.RELEASE
timestamp	TASK	maven://org.springframework.cloud.task.app:timestamp-task:2.1.0.RELEASE	2.1.0.RELEASE 2
timestamp-batch	TASK	maven://org.springframework.cloud.task.app:timestamp-batch-task:2.1.0.RELEASE	2.1.0.RELEASE

items per page: 30 | 1-3 applications of 3 applications

Versions of an application are managed by registering multiple applications with the same name and coordinates, *except* the version. For example, if you were to register an application with the following values, you would get one application registered with two versions (2.0.0.RELEASE and 2.1.0.RELEASE):

- Application 1
 - Name: timestamp
 - Type: task
 - URI: maven://org.springframework.cloud.task.app:timestamp-task:2.0.0.RELEASE
- Application 2
 - Name: timestamp
 - Type: task
 - URI: maven://org.springframework.cloud.task.app:timestamp-task:2.1.0.RELEASE

Besides having multiple versions, Spring Cloud Data Flow needs to know which version to run on the next launch. This is indicated by setting a version to be the default version. Whatever version of a task application is configured as the default version is the one to be run on the next launch request. You can see which version is the default in the UI as this image shows:

Manage versions of the application **timestamp** TASK

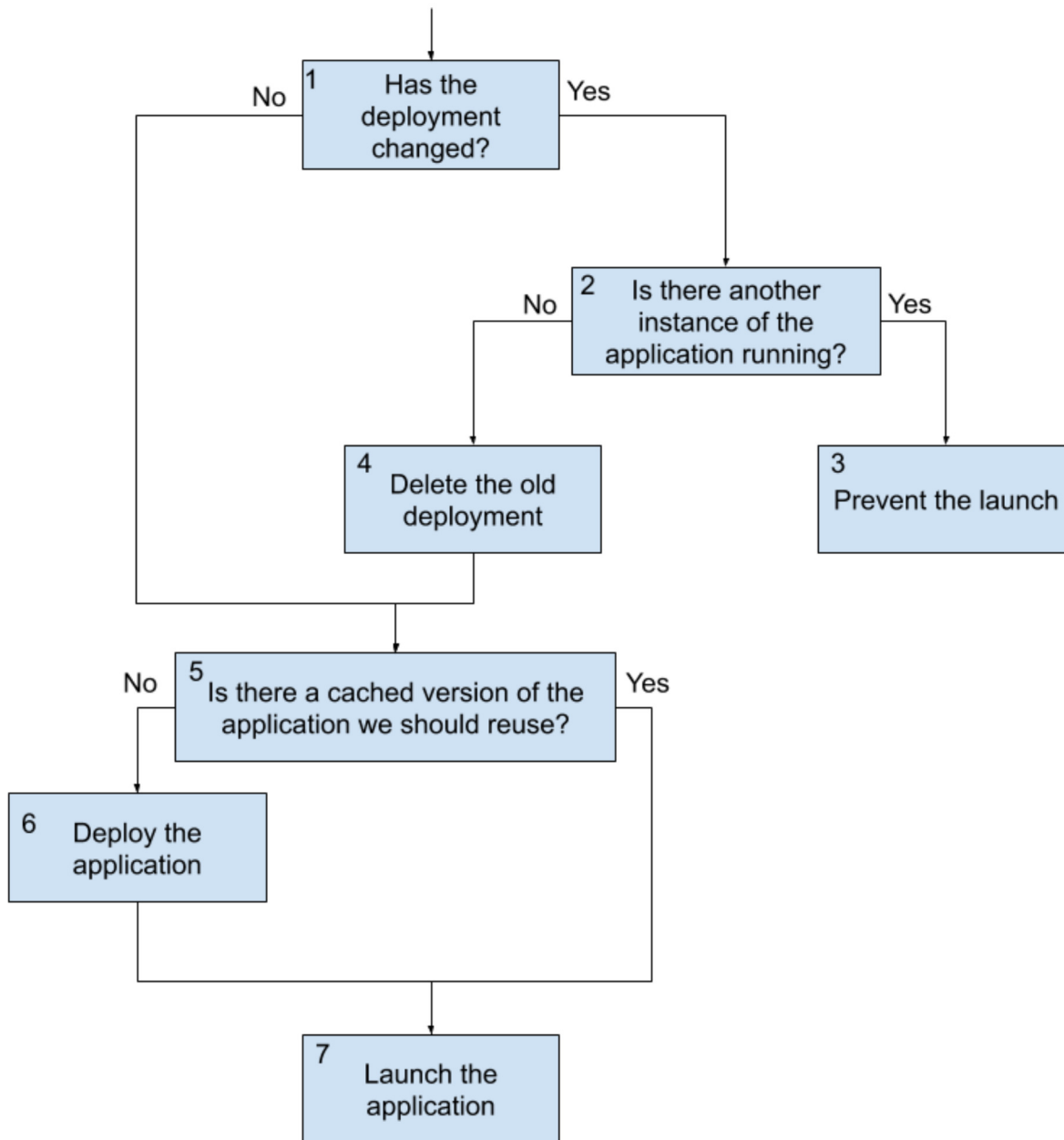
This section lists all the **versions available for an application**. You can **unregister** an app or **switch the default version** of the application.

Version	Uri		
2.0.0.RELEASE	maven://org.springframework.cloud.task.app:timestamp-task:2.0.0.RELEASE	★	🗑️
★ 2.1.0.RELEASE	maven://org.springframework.cloud.task.app:timestamp-task:2.1.0.RELEASE	★	🗑️

Cancel

31.1. Task Launch Lifecycle

In previous versions of Spring Cloud Data Flow, when the request to launch a task was received, Spring Cloud Data Flow would deploy the application (if needed) and run it. If the application was being run on a platform that did not need to have the application deployed every time (CloudFoundry for example), the previously deployed application was used. This flow has changed in 2.3. The following image shows what happens when a task launch request comes in now:



There are three main flows to consider in the preceding diagram. Launching the first time or launching with no changes is one. The other is launching when there are changes. We look at the flow with no changes first.

31.1.1. Launch a Task With No Changes

1. A launch request comes into to Data Flow. Data Flow determines that an upgrade is not required, since nothing has changed (no properties, deployment properites, or versions have changed since the last execution).
5. On platforms that cache a deployed artifact (CloudFoundry at the writing of this documentation), Data Flow checks whether the application was previously deployed.
6. If the application needs to be deployed, Data Flow does the deployment of the task application.
7. Data Flow launches the application.

That flow is the default behavior and occurs every time a request comes in if nothing has changed. It is important to note that this is the same flow that Data Flow has always executed for launching of tasks.

31.1.2. Launch a Task With Changes That Is Not Currently Running

The second flow to consider when launching a task is whether there was a change in any of the task application version, application properties, or deployment properties. In this case, the following flow is executed:

1. A launch request comes into Data Flow. Data Flow determines that an upgrade is required since there was a change in either task application version, application properties, or deployment properties.
2. Data Flow checks to see whether another instance of the task definition is currently running.
4. If there is not another instance of the task definition currently running, the old deployment is deleted.
5. On platforms that cache a deployed artifact (CloudFoundry at the writing of this documentation), Data Flow checks whether the application was previously deployed (this check will evaluate to `false` in this flow since the old deployment was deleted).
6. Data Flow does the deployment of the task application with the updated values (new application version, new merged properties, and new merged deployment properties).
7. Data Flow launches the application.

This flow is what fundamentally enables continuous deployment for Spring Cloud Data Flow.

31.1.3. Launch a Task With Changes While Another Instance Is Running

The last main flow is when a launch request comes to Spring Cloud Data Flow to do an upgrade but the task definition is currently running. In this case, the launch is blocked due to the requirement to delete the current application. On some platforms (CloudFoundry at the writing of this document), deleting the application causes all currently running applications to be shut down. This feature prevents that from happening. The following process describes what happens when a task changes while another instance is running:

1. A launch request comes into to Data Flow. Data Flow determines that an upgrade is required, since there was a change in either task application version, application properties, or deployment properties.
2. Data Flow checks to see whether another instance of the task definition is currently running.
3. Data Flow prevents the launch from happening because other instances of the task definition are running.

Any launch that requires an upgrade of a task definition that is running at the time of the request is blocked from executing due to the need to delete any currently running tasks.

Task Developer Guide

Please see the [Batch Developer](#) section of the microsite for more information on how to create, test, and run Spring Cloud Task applications on your local machine.

Task Monitoring

Please see the [Task Monitoring](#) Guide of the microsite for more information on how to monitor the applications that were deployed as part of a Task.

Dashboard

This section describes how to use the dashboard of Spring Cloud Data Flow.

32. Introduction

Spring Cloud Data Flow provides a browser-based GUI called the dashboard to manage the following information:

- **Apps:** The Apps tab lists all available applications and provides the controls to register/unregister them.
- **Runtime:** The Runtime tab provides the list of all running applications.
- **Streams:** The Streams tab lets you list, design, create, deploy, and destroy Stream Definitions.
- **Tasks:** The Tasks tab lets you list, create, launch, schedule and, destroy Task Definitions.
- **Jobs:** The Jobs tab lets you perform batch job related functions.

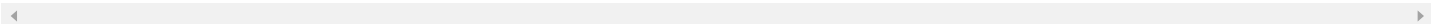
Upon starting Spring Cloud Data Flow, the dashboard is available at:

`<host>:<port>/dashboard`

For example, if Spring Cloud Data Flow is running locally, the dashboard is available at `http://localhost:9393/dashboard`.

If you have enabled https, then the dashboard will be located at `https://localhost:9393/dashboard`. If you have enabled security, a login form is available at `http://localhost:9393/dashboard/#/login`.

The default Dashboard server port is 9393.



The following image shows the opening page of the Spring Cloud Data Flow dashboard:

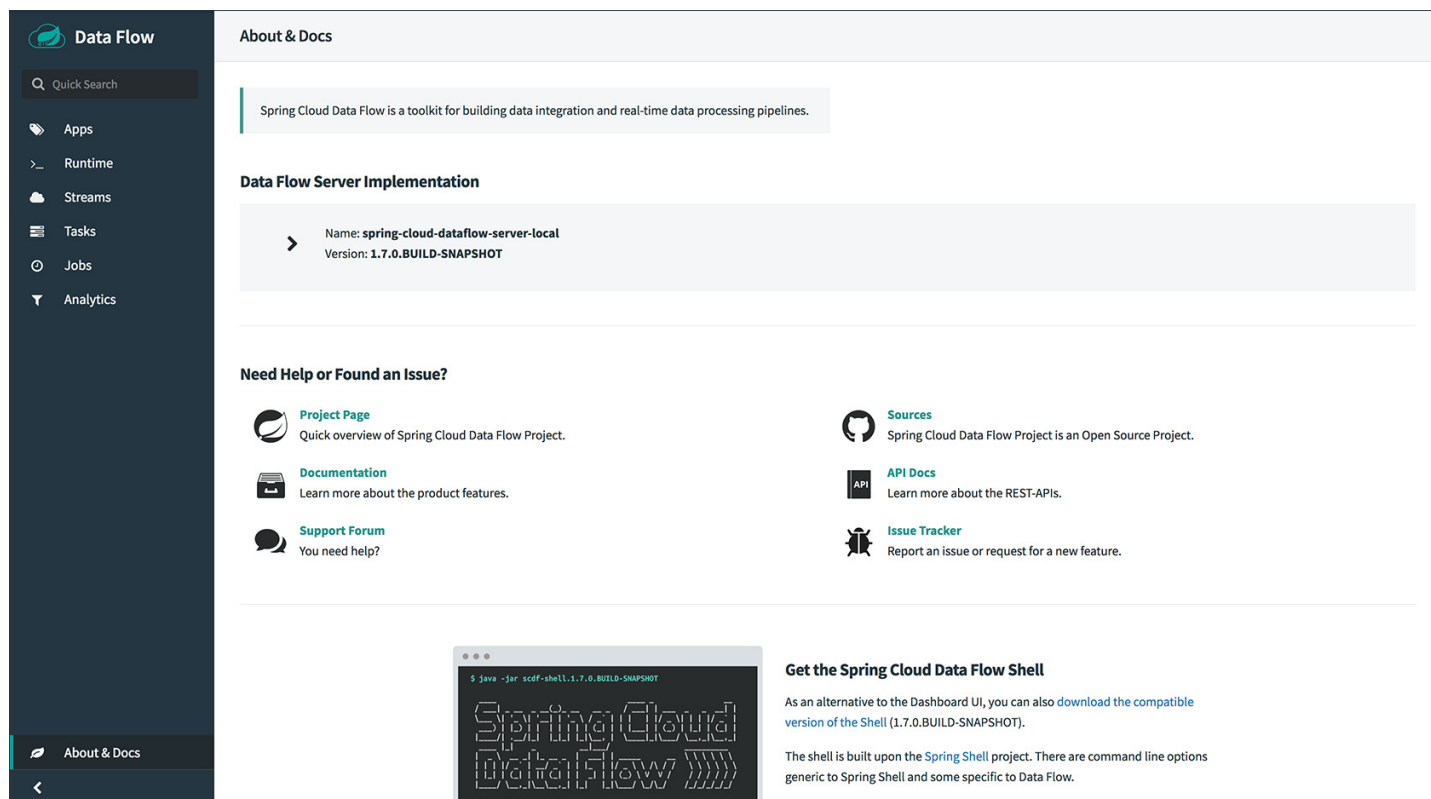


Figure 10. The Spring Cloud Data Flow Dashboard

33. Apps

The Apps section of the dashboard lists all the available applications and provides the controls to register and unregister them (if applicable). It is possible to import a number of applications at once by using the Bulk Import Applications action.

The following image shows a typical list of available apps within the dashboard:

☐	Name	Type	Uri	Info
☐	aggregate-counter	SINK	maven://org.springframework.cloud.stream.app:aggregate-counter-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	aggregator	PROCESSOR	maven://org.springframework.cloud.stream.app:aggregator-processor-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	bridge	PROCESSOR	maven://org.springframework.cloud.stream.app:bridge-processor-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	cassandra	SINK	maven://org.springframework.cloud.stream.app:cassandra-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	composed-task-runner	TASK	maven://org.springframework.cloud.task.app:composedtaskrunner-task:1.1.1.BUILD-SNAPSHOT	i
☐	counter	SINK	maven://org.springframework.cloud.stream.app:counter-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	field-value-counter	SINK	maven://org.springframework.cloud.stream.app:field-value-counter-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	file	SINK	maven://org.springframework.cloud.stream.app:file-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	file	SOURCE	maven://org.springframework.cloud.stream.app:file-source-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	filter	PROCESSOR	maven://org.springframework.cloud.stream.app:filter-processor-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	ftp	SOURCE	maven://org.springframework.cloud.stream.app:ftp-source-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	ftp	SINK	maven://org.springframework.cloud.stream.app:ftp-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	gemfire	SINK	maven://org.springframework.cloud.stream.app:gemfire-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	gemfire	SOURCE	maven://org.springframework.cloud.stream.app:gemfire-source-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	gemfire-cq	SOURCE	maven://org.springframework.cloud.stream.app:gemfire-cq-source-kafka-10:1.3.1.BUILD-SNAPSHOT	i
☐	gpfdist	SINK	maven://org.springframework.cloud.stream.app:gpfdist-sink-kafka-10:1.3.1.BUILD-SNAPSHOT	i

Figure 11. List of Available Applications

33.1. Bulk Import of Applications

The Bulk Import Applications page provides numerous options for defining and importing a set of applications all at once. For bulk import, the application definitions are expected to be expressed in a properties style, as follows:

```
<type>.<name> = <coordinates>
```

The following examples show a typical application definitions:

```
task.timestamp=maven://org.springframework.cloud.task.app:timestamp-task:1.2.0.RELEASE
processor.transform=maven://org.springframework.cloud.stream.app:transform-processor-rabbit:1.2.0.RELEASE
```

At the top of the bulk import page, a URI can be specified that points to a properties file stored elsewhere, it should contain properties formatted as shown in the previous example. Alternatively, by using the textbox labeled “Apps as Properties”, you can directly list each property string. Finally, if the properties are stored in a local file, the “Select Properties File” option opens a local file browser to select the file. After setting your definitions through one of these routes, click Import.

The following image shows the Bulk Import Applications page:

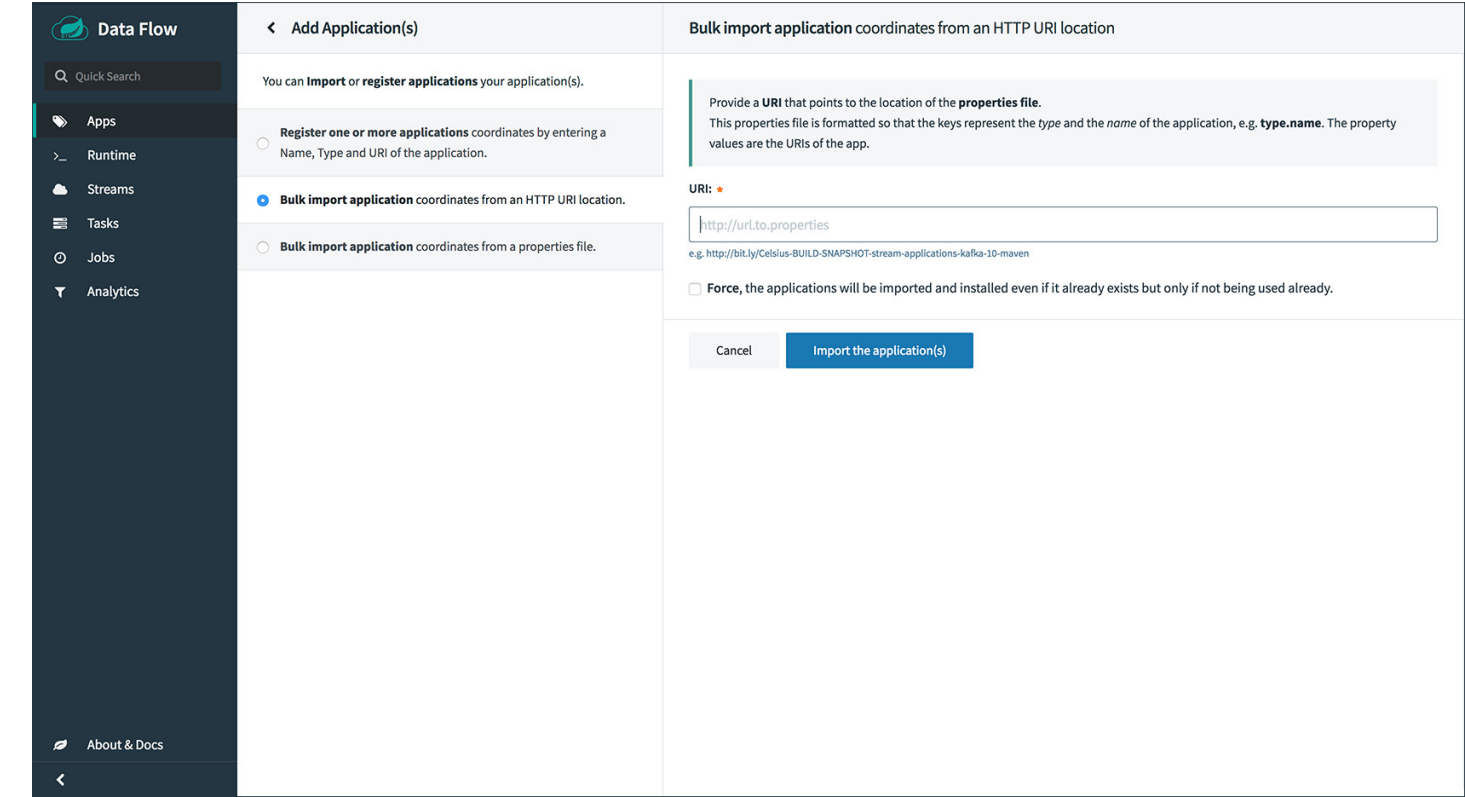


Figure 12. Bulk Import Applications

34. Runtime

The Runtime section of the Dashboard application shows the list of all running applications. For each runtime app, the state of the deployment and the number of deployed instances is shown. A list of the used deployment properties is available by clicking on the App Id.

The following image shows an example of the Runtime tab in use:

App Id	Status	# of Instances
foo_4peima4wu1.file	DEPLOYED	1 instance
foo_4peima4wu1.log	DEPLOYED	1 instance

Figure 13. List of Running Applications

35. Streams

The Streams tab has two child tabs: Definitions and Create Stream. The following topics describe how to work with each one:

- [Working with Stream Definitions](#)
- [Creating a Stream](#)
- [Deploying a Stream](#)
- [Accessing Stream Logs](#)

35.1. Working with Stream Definitions

The Streams section of the Dashboard includes the Definitions tab that provides a listing of Stream definitions. There you have the option to deploy or undeploy those stream definitions. Additionally, you can remove the definition by clicking on Destroy. Each row includes an arrow on the left, which you can click to see a visual representation of the definition. Hovering over the boxes in the visual representation shows more details about the apps, including any options passed to them.

In the following screenshot, the `timer` stream has been expanded to show the visual representation:

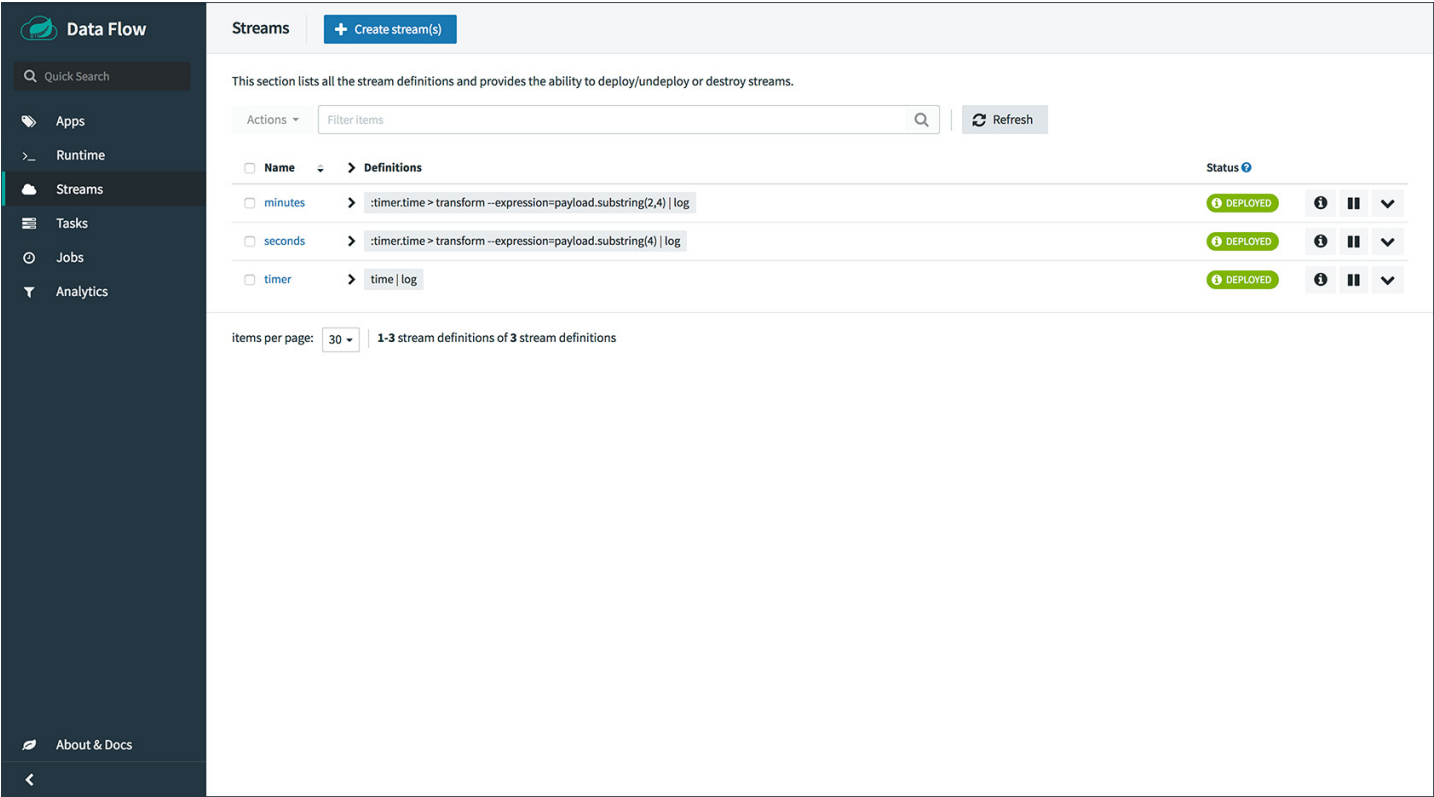


Figure 14. List of Stream Definitions

If you click the details button, the view changes to show a visual representation of that stream and any related streams. In the preceding example, if you click details for the `timer` stream, the view changes to the following view, which clearly shows the relationship between the three streams (two of them are tapping into the `timer` stream):

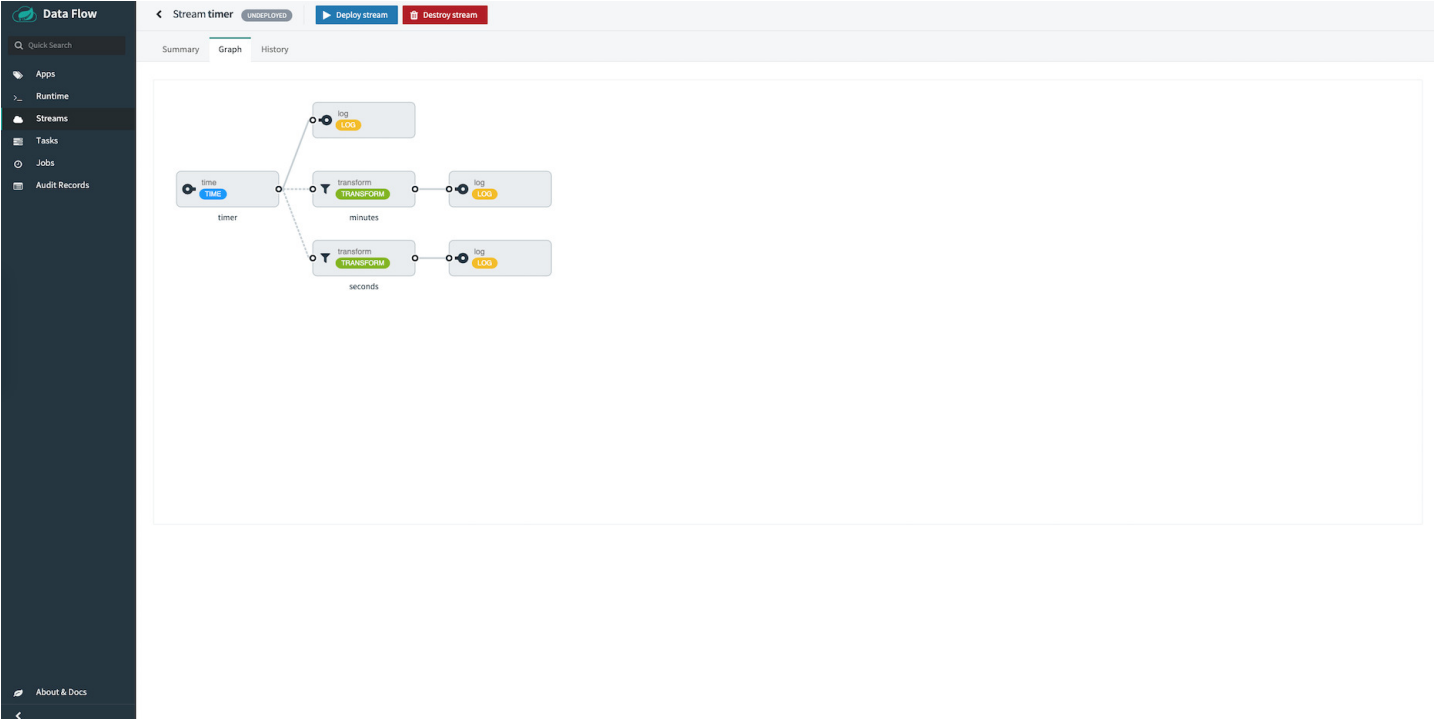


Figure 15. Stream Details Page

35.2. Creating a Stream

The Streams section of the Dashboard includes the Create Stream tab, which makes available the [Spring Flo](#) designer: a canvas application that offers an interactive graphical interface for creating data pipelines.

In this tab, you can:

- Create, manage, and visualize stream pipelines using DSL, a graphical canvas, or both
- Write pipelines via DSL with content-assist and auto-complete
- Use auto-adjustment and grid-layout capabilities in the GUI for simpler and interactive organization of pipelines

You should watch this [screencast](#) that highlights some of the "Flo for Spring Cloud Data Flow" capabilities. The [Spring Flo wiki](#) includes more detailed content on core Flo capabilities.

The following image shows the Flo designer in use:

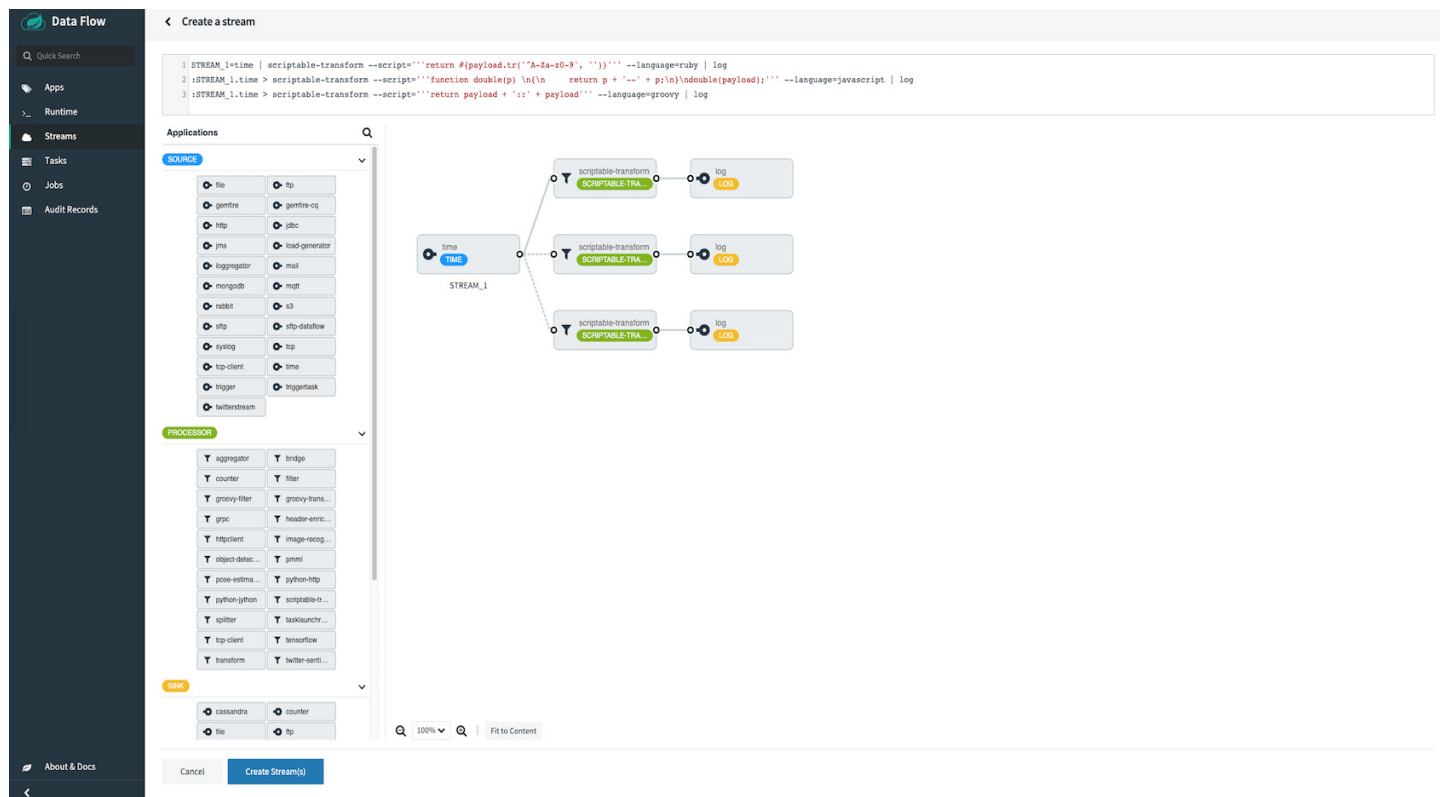


Figure 16. Flo for Spring Cloud Data Flow

35.3. Deploying a Stream

The stream deploy page includes tabs that provide different ways to setup the deployment properties and deploy the stream. The following screenshots show the stream deploy page for `foobar (time | log)`.

You can define deployments properties using:

- Form builder tab: a builder which help you to define deployment properties (deployer, application properties...)
- Free text tab: a free text area (key/value pairs)

You can switch between the both views, the form builder provides a more stronger validation of the inputs.

Data Flow

Quick Search

Apps

Runtime

Streams

Tasks

Jobs

Analytics

About & Docs

<

Deploy Stream Definition foobar

Please specify any **optional** deployment properties. You can either enter deployment properties directly into the text-area field below. Alternatively, you can point to an external **properties file**, which will be used to populate the text-area field. For more information please see the [Technical Documentation](#).

Builder

Freetext

	Global	time SOURCE	file SINK
Generic Deployer Properties			
memory	720 MB	Enter a number MB	720 MB
cpu	Enter a number Core(s)	Enter a number Core(s)	Enter a number Core(s)
disk	Enter a number MB	Enter a number MB	Enter a number MB
count	Enter a number	Enter a number	Enter a number
Deployer Properties			
Enter a value	Enter a value	Enter a value	Enter a value
Applications Properties			
Properties		1 / 6 properties	0 / 8 properties
mykey	A	B	Enter a value
Enter a value	Enter a value	Enter a value	Enter a value

Cancel

Export

Deploy the stream

Figure 17. The following image shows the form builder

Data Flow

Quick Search

Apps

Runtime

Streams

Tasks

Jobs

Analytics

About & Docs

<

Deploy Stream Definition foobar

Please specify any **optional** deployment properties. You can either enter deployment properties directly into the text-area field below. Alternatively, you can point to an external **properties file**, which will be used to populate the text-area field. For more information please see the [Technical Documentation](#).

Builder

Freetext

Enter the list of properties into the text area field below. Alternatively, you can also select a file in your local file system, which is used to populate the text area field.

1 app.*.mykey=A
2 app.time.mykey=B
3 app.time.trigger.time-unit=MILLISECONDS
4 deployer.*.memory=720
5 deployer.file.memory=720

Import a local file

Cancel

Export

Deploy the stream

Figure 18. The following image shows the same properties in the free text

35.4. Accessing Stream Logs

Once the stream applications are deployed, their logs can be accessed from the Stream summary page as below:

https://docs.spring.io/spring-cloud-dataflow/docs/current/reference/htmlsingle/

D2.1.5-159

'1

Data Flow

Quick Search

Apps

Runtime

Streams

Tasks

Jobs

Audit Records

About & Docs

<

Stream ticktock

DEPLOYED

Undeploy stream

Update stream

Destroy stream

Summary

Graph

History

Definition:

time | log

Status:

DEPLOYED

Applications:

Name	Type	
time	SOURCE	i
log	SINK	i

Deployment Info:

log

resource	maven://org.springframework.cloud.stream.app:log-sink-rabbitjar
spring.cloud.deployer.group	ticktock
version	2.1.0.RELEASE

time

resource	maven://org.springframework.cloud.stream.app:time-source-rabbitjar
spring.cloud.deployer.group	ticktock
version	2.1.0.RELEASE

Runtimes:

App Id	# of Instances
ticktock.log-v1	1 instance
ticktock.time-v1	1 instance

Logs:

ticktock.log-v1

Refresh

2019-07-26 14:16:09.092 INFO 12827 --- [main] trationDelegate\$BeanPostProcessorChecker : Bean 'org.springframework.cloud.autoconfigure.ConfigurationProperties
:: Spring Boot :: (v2.1.3.RELEASE)
2019-07-26 14:16:09.439 INFO 12827 --- [main] c.c.c.ConfigServicePropertySourceLocator : Fetching config from server at : http://localhost:8888

35.5. Creating Fan-In/Fan-Out Streams

In chapter [Fan-in and Fan-out](#) you learned how we can support fan-in and fan-out use cases using [named destinations](#). The UI provides dedicated support for named destinations as well:

Data Flow

Quick Search

Apps

Runtime

Streams

Tasks

Jobs

Audit Records

About & Docs

<

Create a stream

1 http > isharedData
2 jdbc > isharedData
3 isharedData > file
4 isharedData > cassandra

Applications

SOURCE

file, gemfire, http, jms, aggregator, mongodb, rabbit, slp, syslog, tcp-client, trigger, twitterstream, tp, gemfire-cq, jdbc, load-generator, mail, mqtt, s3, stp-dataflow, time, triggerTask

PROCESSOR

aggregator, counter, groovy-filter, gpx, httpclient, object-detect..., pose-estima..., python-gython, splitter, tcp-client, transform, bridge, filter, groovy-trans..., header-entri..., image-recog..., pmml, python-http, scriptable-tr..., tasklauncher..., tensorflow, twitter-senti...

SINK

cassandra, file, counter, tp

100%

Fit to Content

Cancel

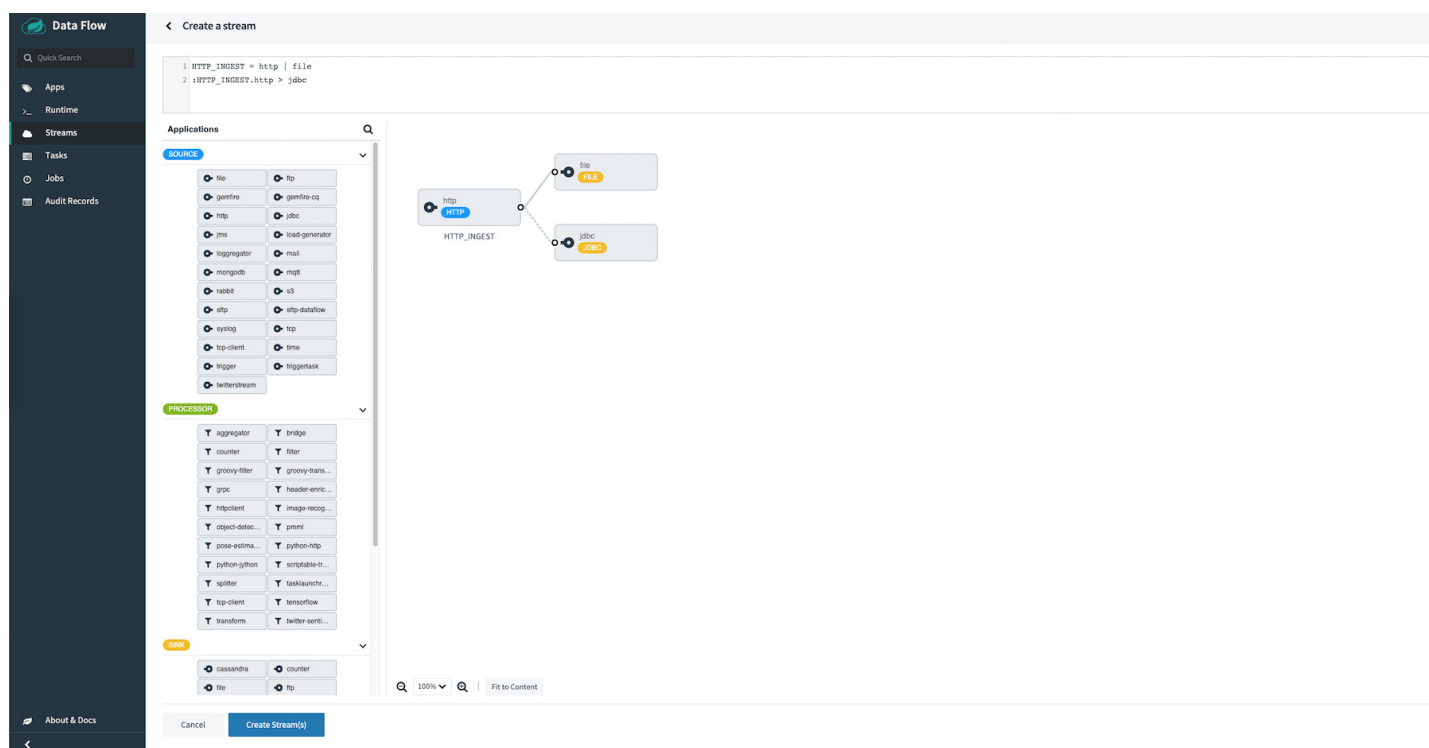
Create Stream(s)

Figure 19. Flo for Spring Cloud Data Flow

In this example we have data from an *HTTP Source* and a *JDBC Source* that is being sent to the *sharedData* channel which represents a **Fan-in** use case. On the other end we have a *Cassandra Sink* and a *File Sink* subscribed to the *sharedData* channel which represents a **Fan-out** use case.

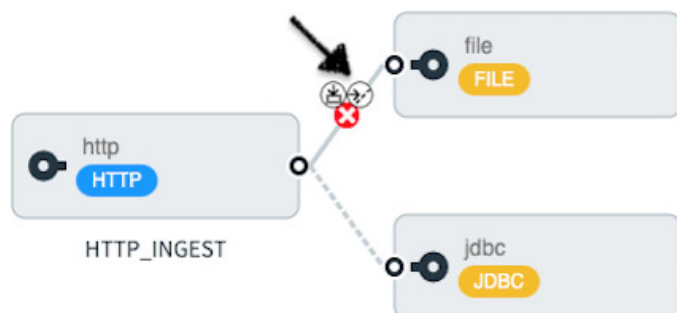
35.6. Creating a Tap Stream

Creating Taps using the Dashboard is straightforward. Let's say you have stream consisting of an *HTTP Source* and a *File Sink* and you would like to tap into the stream to also send data to a *JDBC Sink*. In order to create the tap stream simply connect the output connector of the *HTTP Source* to the *JDBC Sink*. The connection will be displayed as a dotted line, indicating that you created a tap stream.

**Figure 20. Creating a Tap Stream**

The primary stream (*HTTP Source* to *File Sink*) will be automatically named, in case you did not provide a name for the stream, yet. When creating tap streams, the primary stream must always be explicitly named. In the picture above, the primary stream was named *HTTP_INGEST*.

Using the Dashboard, you can also switch the primary stream to become the secondary tap stream.

**Figure 21. Change Primary Stream to Secondary Tap Stream**

Simply hover over the existing primary stream, the line between *HTTP Source* and *File Sink*. Several control icons will appear, and by clicking on the icon labeled *Switch to/from tap*, you change the primary stream into a tap stream. Do the same for the tap stream and switch it to a primary stream.



Figure 22. End Result of Switching the Primary Stream

When interacting directly with [named destinations](#), there can be "n" combinations (Inputs/Outputs). This allows you to create complex topologies involving a wide variety of data sources and destinations.

35.7. Import/Export Streams

The Streams section of the Dashboard includes a new Utils page that provides the option to import and export streams.

The following image shows the streams export page:

Figure 23. Stream Utils Export page

On the Streams import page, you have to import from a valid JSON file. You can either manually draft the file or export the file from the Streams export page.

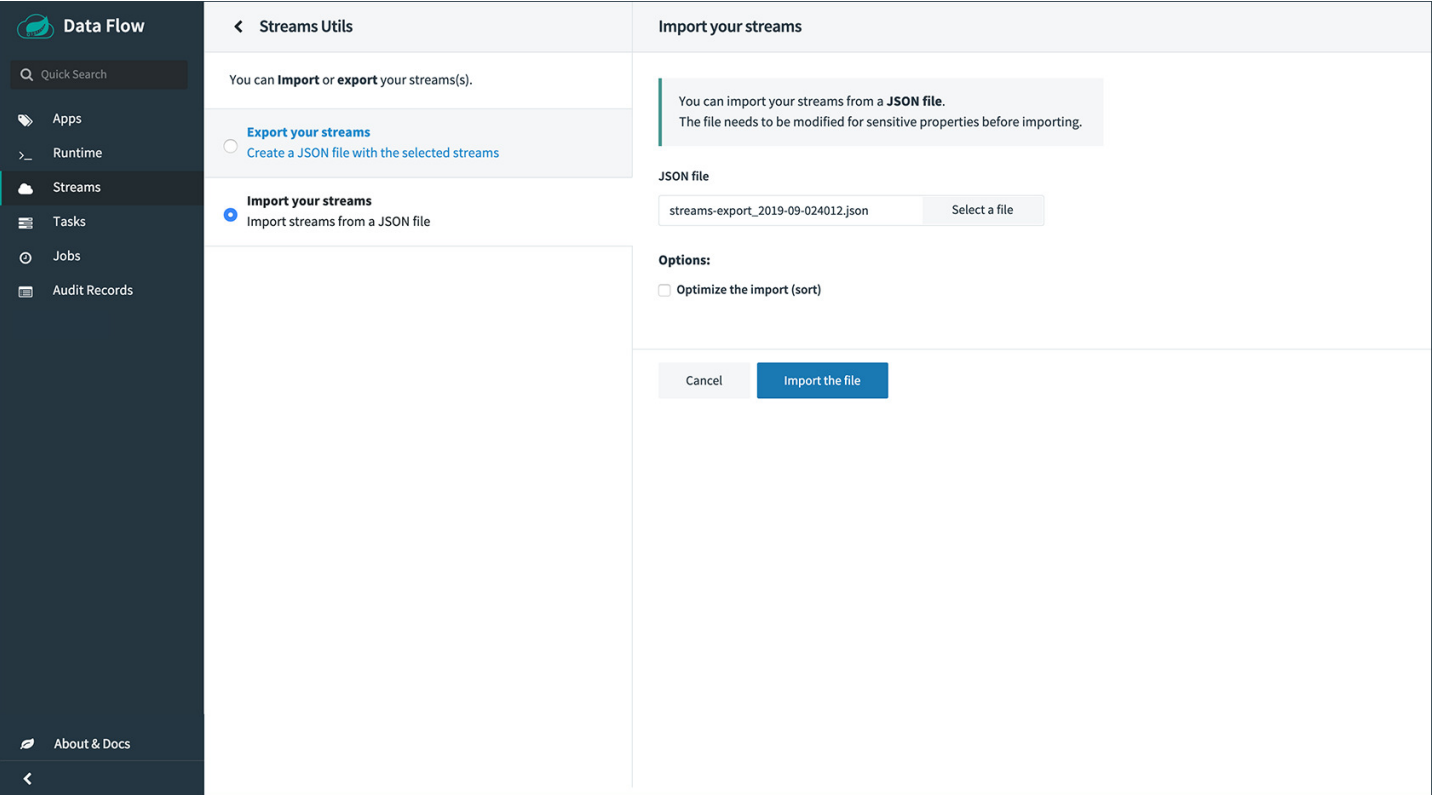


Figure 24. Stream Utils Import page

After importing the file, you will get confirmation on whether the operation completed successfully.

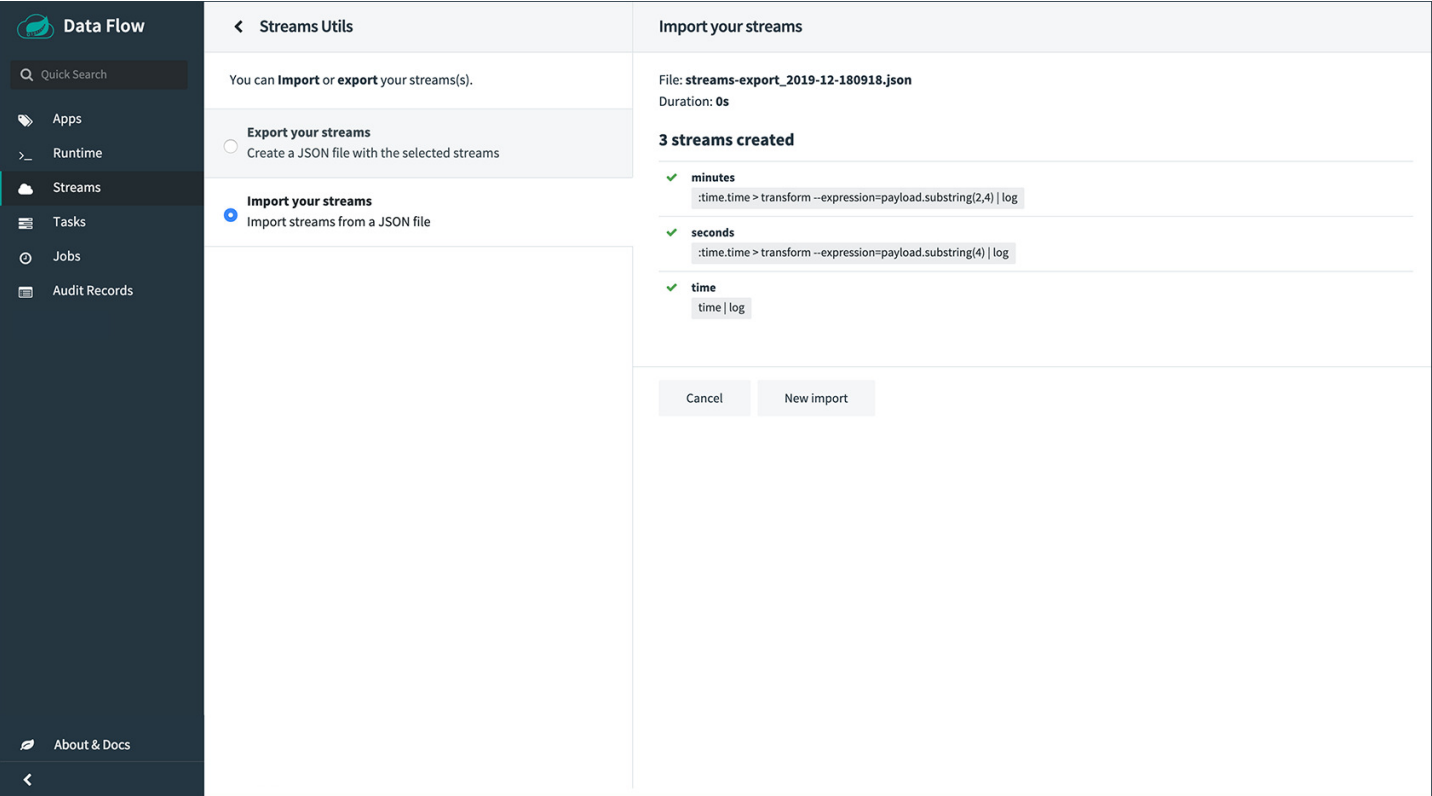


Figure 25. Stream Utils Import Result page

36. Tasks

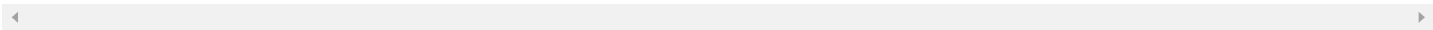
The Tasks section of the Dashboard currently has three tabs:

- [Apps](#)
- [Definitions](#)
- [Executions](#)
- [Scheduling](#)

36.1. Apps

Each app encapsulates a unit of work into a reusable component. Within the Data Flow runtime environment, apps let users create definitions for streams as well as tasks. Consequently, the Apps tab within the Tasks section lets users create task definitions.

You can also use this tab to create Batch Jobs.



The following image shows a typical list of task apps:

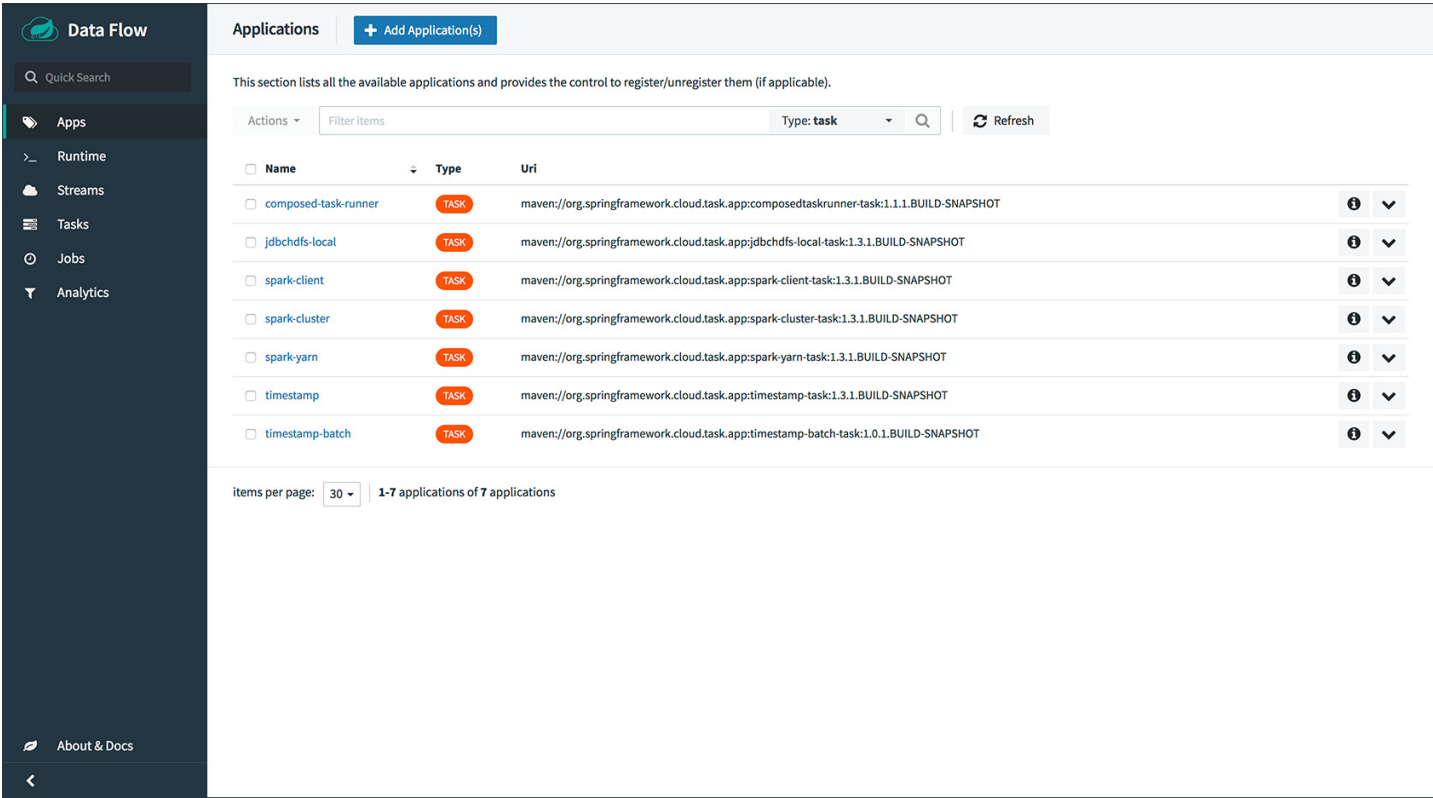


Figure 26. List of Task Apps

On this screen, you can perform the following actions:

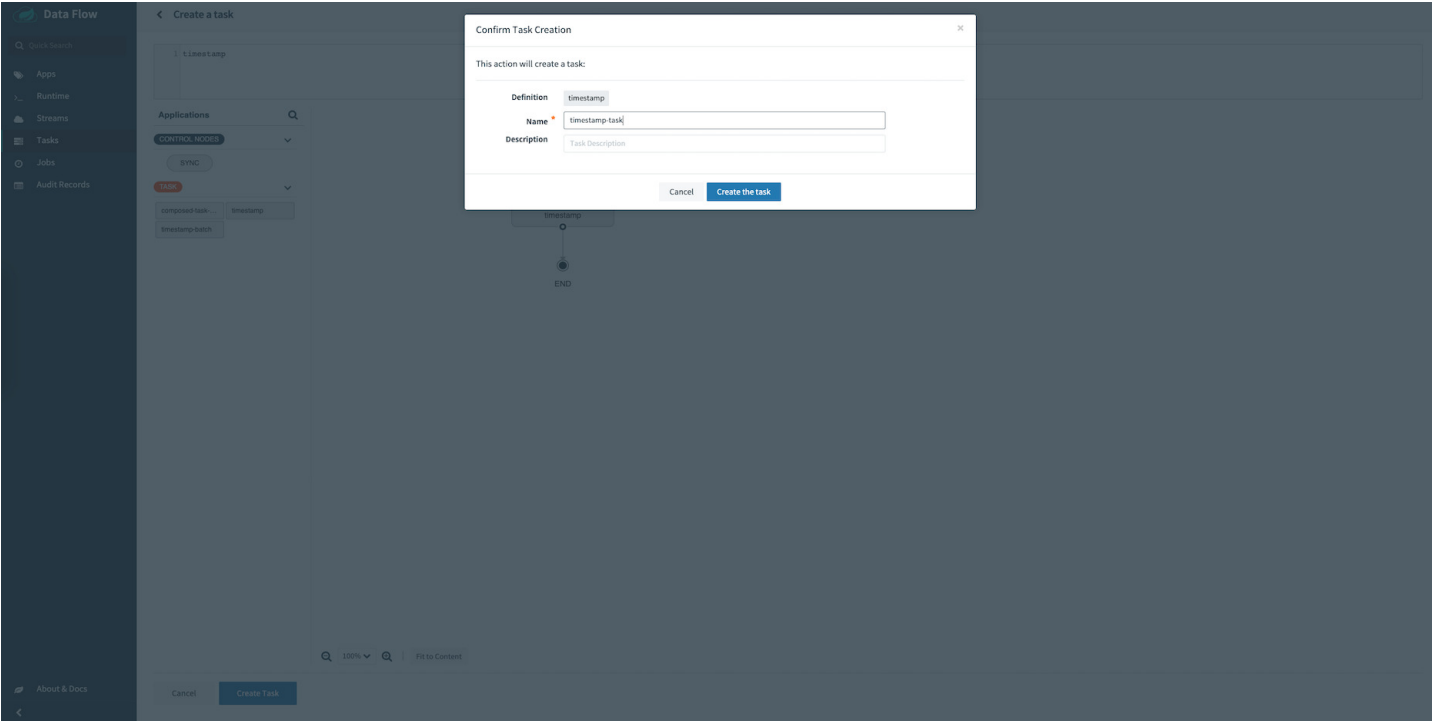
- View details, such as the task app options.

- Create a task definition from the respective app.

36.1.1. View Task App Details

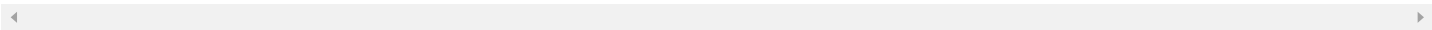
On this page you can view the details of a selected task app, including the list of available options (properties) for that app.

36.1.2. Create a Task Definition



At a minimum, you must provide a name for the new definition. You also have the option to specify various properties that are used during the deployment of the app.

Each parameter is included only if the Include checkbox is selected.



36.2. Definitions

This page lists the Data Flow task definitions and provides actions to launch or destroy those tasks. It also provides a shortcut operation to define one or more tasks with simple textual input, indicated by the Bulk Define Tasks button.

The following image shows the Definitions page:

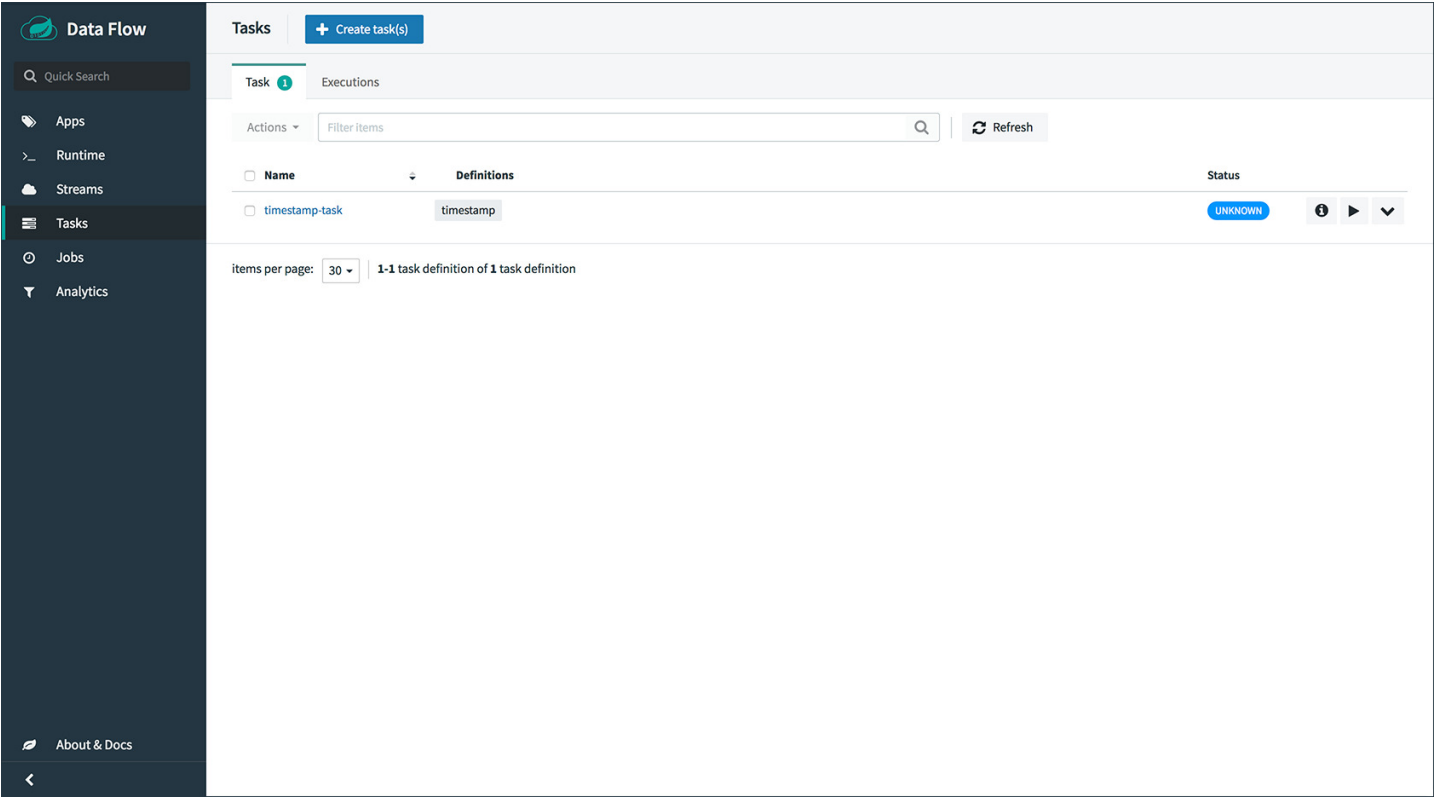


Figure 27. List of Task Definitions

36.2.1. Creating Composed Task Definitions

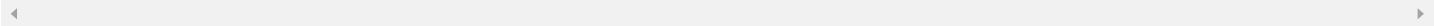
The dashboard includes the Create Composed Task tab, which provides an interactive graphical interface for creating composed tasks.

In this tab, you can:

- Create and visualize composed tasks using DSL, a graphical canvas, or both.
- Use auto-adjustment and grid-layout capabilities in the GUI for simpler and interactive organization of the composed task.

On the Create Composed Task screen, you can define one or more task parameters by entering both the parameter key and the parameter value.

Task parameters are not typed.



The following image shows the composed task designer:

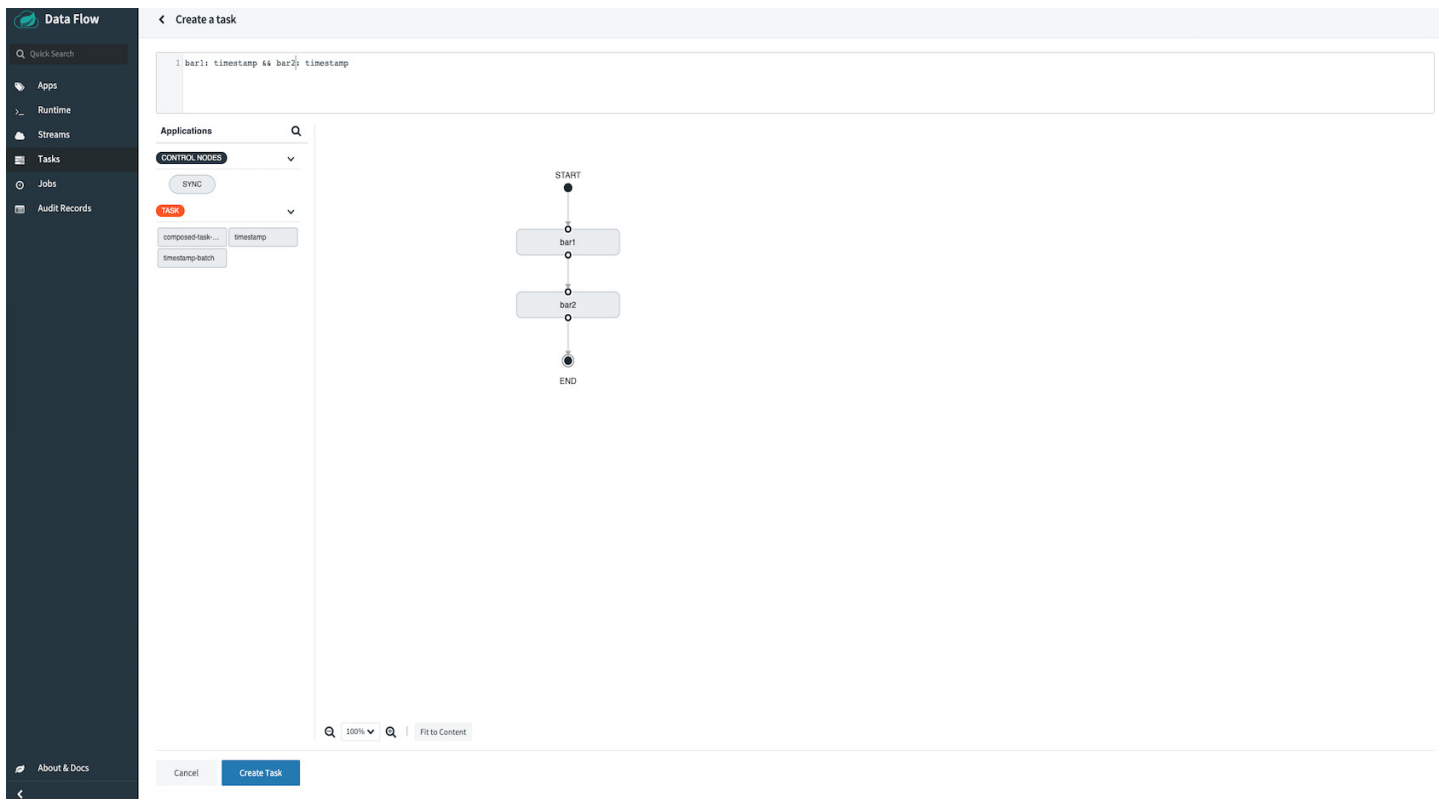


Figure 28. Composed Task Designer

36.2.2. Launching Tasks

Once the task definition has been created, the tasks can be launched through the dashboard. To do so, click the Definitions tab and select the task you want to launch by pressing **Launch**.

36.2.3. Import/Export Tasks

The Tasks section of the Dashboard includes a new Utils page that provides the option to import and export tasks.

The following image shows the tasks export page:

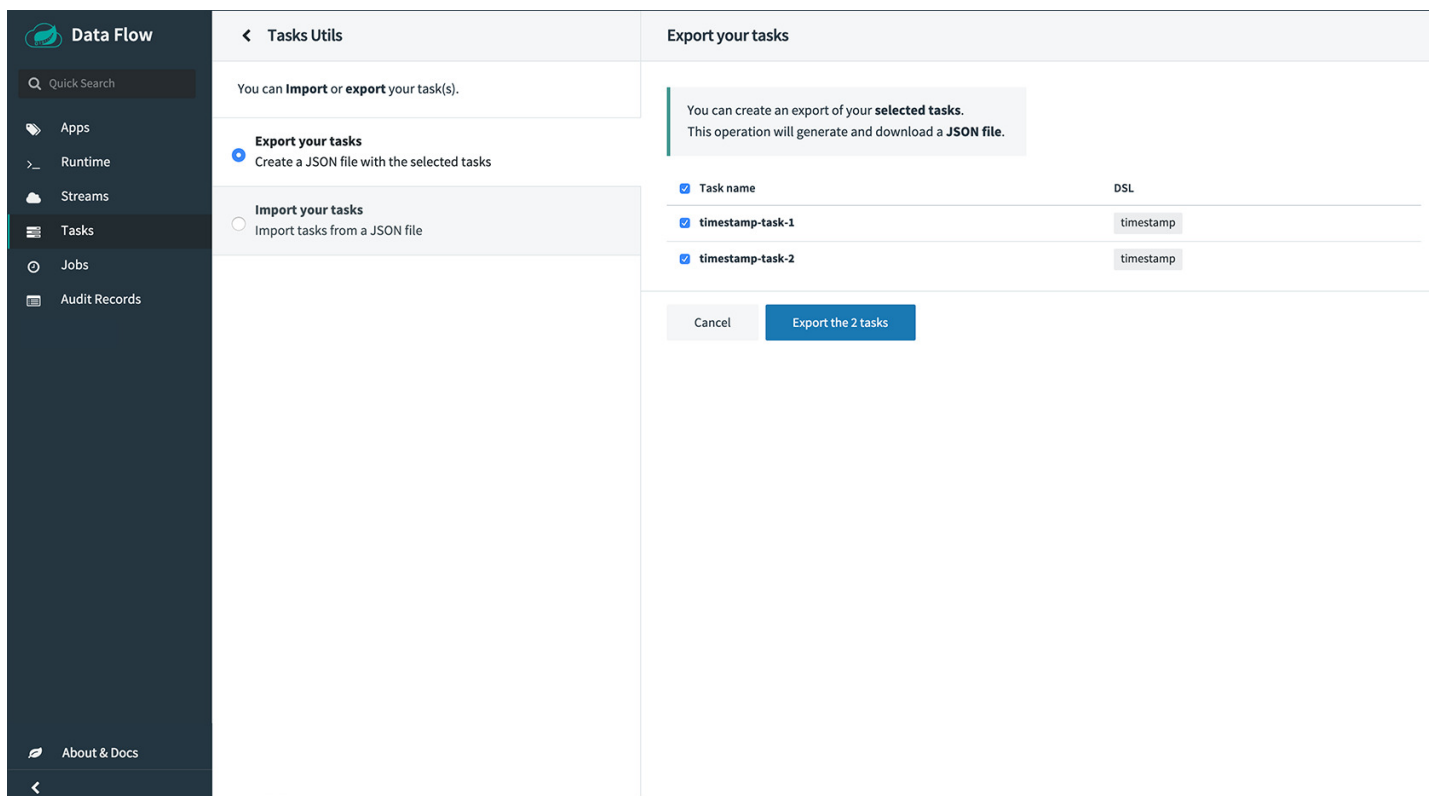


Figure 29. Tasks Utils Export page

On the Tasks import page, you have to import from a valid JSON file. You can either manually draft the file or export the file from the Tasks export page.

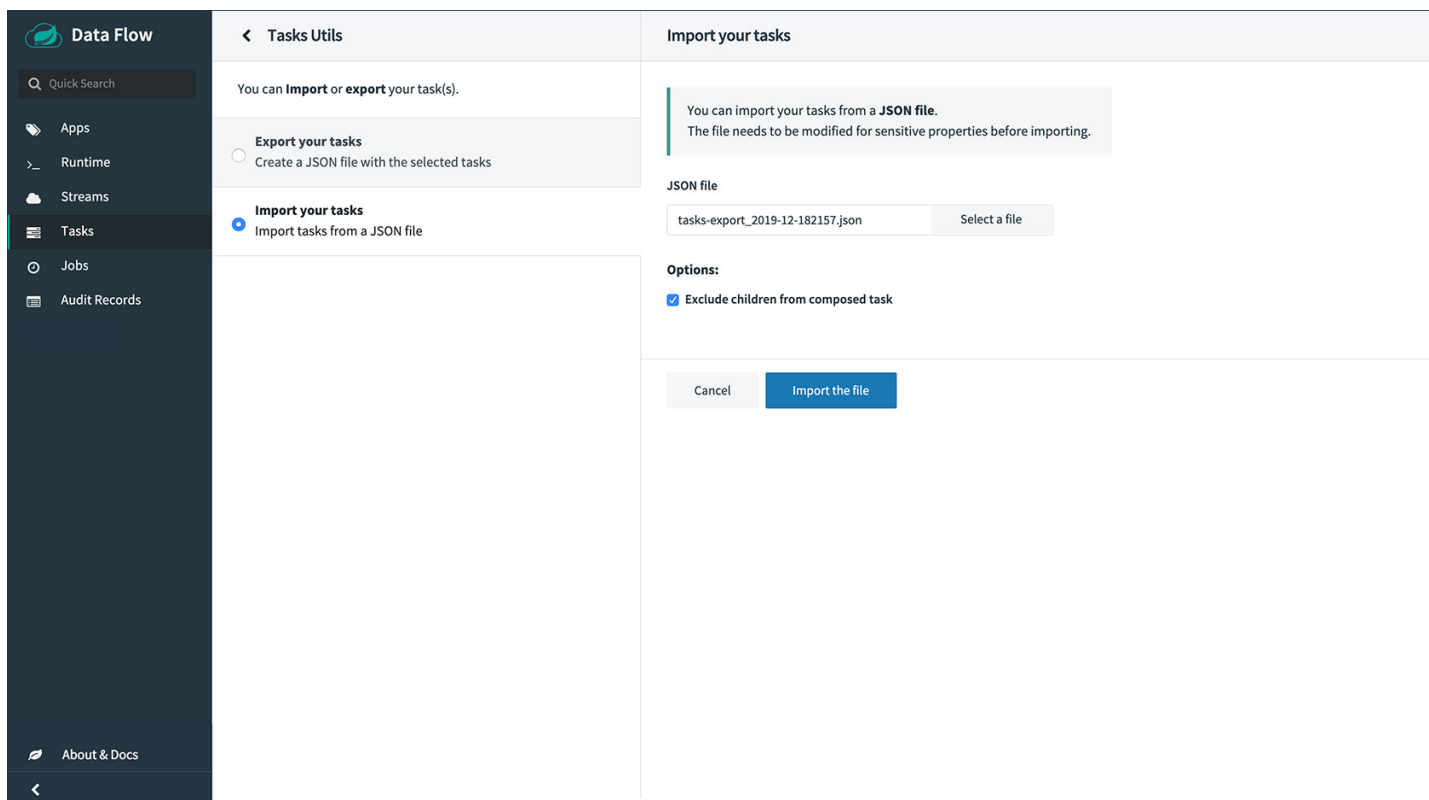


Figure 30. Tasks Utils Import page

After importing the file, you will get confirmation on whether the operation completed successfully.

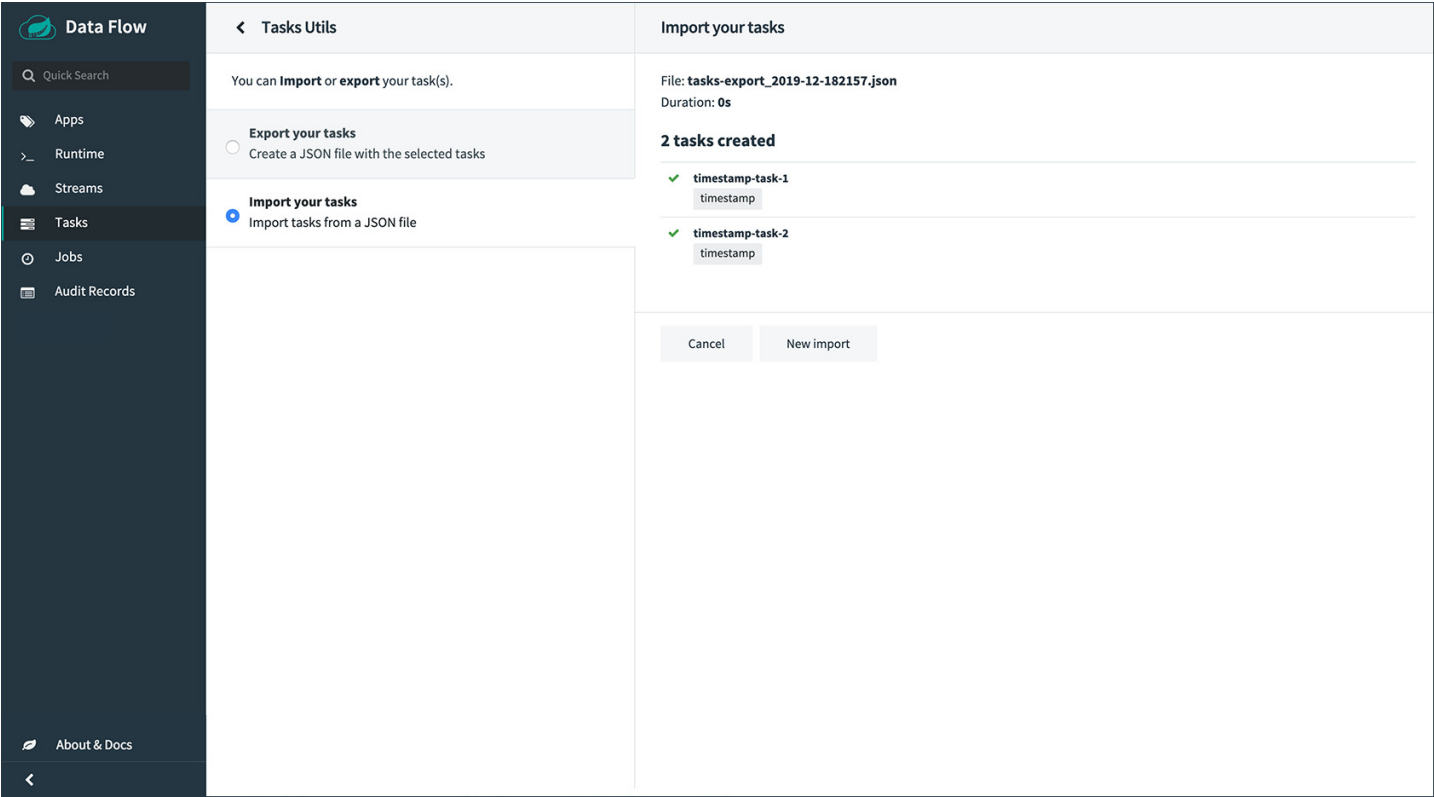


Figure 31. Tasks Utils Import Result page

36.3. Executions

The Executions tab shows the current running and completed task executions. From this page you can drill down into the Task Execution details page. Furthermore, you can relaunch a Task Execution or stop a running execution.

Lastly, you can clean up 1 or more Task Executions. This operation will remove any associated Task and/or Batch Job from the underlying persistence store. This operation can only be triggered for *parent* Task Executions and will cascade down to the child Task Executions (if there are any).

The following image shows the Executions tab:

Data Flow

Apps

Runtime

Streams

Tasks

Jobs

Audit Records

About & Docs

Tasks

+ Create task(s)

Task 4

Executions 11

Actions

Refresh

Task	Execution Id	Task Name	Start Date	End Date	Exit Code	
	#18	ComposedTask1-timestamp-batch	2019-07-18 23:59:10,855[-10]	2019-07-18 23:59:10,916[-10]	0	
	#17	ComposedTask1-timestamp	2019-07-18 23:59:00,446[-10]	2019-07-18 23:59:00,496[-10]	0	
☐	#16	ComposedTask1	2019-07-18 23:58:56,605[-10]	2019-07-18 23:59:17,230[-10]	0	
☐	#15	ComposedTask1	2019-07-18 23:58:24,044[-10]	2019-07-18 23:58:24,182[-10]	0	
	#14	ComposedTask1-timestamp-batch	2019-07-18 23:57:07,470[-10]	2019-07-18 23:57:07,550[-10]	0	
	#13	ComposedTask1-timestamp	2019-07-18 23:56:57,019[-10]	2019-07-18 23:56:57,050[-10]	0	
☐	#12	ComposedTask1	2019-07-18 23:56:53,567[-10]	2019-07-18 23:57:14,188[-10]	0	
☐	#8	ts-batch1	2019-07-18 23:55:46,120[-10]	2019-07-18 23:55:46,254[-10]	0	
☐	#3	ts-batch1	2019-07-18 23:41:57,922[-10]	2019-07-18 23:41:58,072[-10]	0	
☐	#2	ComposedTask1-timestamp	2019-07-18 23:41:53,603[-10]	2019-07-18 23:41:53,632[-10]	0	
☐	#1	ComposedTask1	2019-07-18 23:41:49,991[-10]	2019-07-18 23:42:10,619[-10]	0	

items per page: 30

1-11 task executions of 11 task executions

Figure 32. List of Task Executions

36.4. Execution Detail

For each task execution on the Executions page, a user can retrieve detailed information about a specific execution by clicking the information icon located to the right of the task execution.

Data Flow

Quick Search

- Apps
- Runtime
- Streams
- Tasks**
- Jobs
- Audit Records

About & Docs

Task Execution Details - Execution ID: 16

Details Relaunch Stop Task Execution cleanup

Property	Value
Execution Id	16
Task Name	ComposedTask1
Arguments	foo: bar4 --spring.cloud.data.flow.platformname: default --spring.cloud.task.executionid: 16
External Execution Id	ComposedTask1-53ccda28-0daf-49e5-9feb-a7df3499c259
Batch Job	✓
Job Execution Ids	11
Start Time	2019-07-18 23:58:56,605[-10]
End Time	2019-07-18 23:59:17,230[-10]
Duration	00:00:20.625
Exit Code	0
Exit Message	N/A

Logs

```

2019-07-18 23:58:53.429 INFO 36918 --- [          main] trationDelegates$BeanPostProcessorChecker : Bean 'org.springframework.cloud.aut
  
```

On this screen the user can view not only the information from the Task Executions page but also:

- Task Arguments
- External Execution Id
- Batch Job Indicator (indicates if the task execution contained Spring Batch jobs.)
- Job Execution Ids links (Clicking the Job Execution Id will take you to the [Job Execution Details](#) for that Job Execution Id.)
- Task Execution Duration
- Task Execution Exit Message
- Logging output from the Task Execution

Additionally, users can trigger the following operations:

- Relaunch the Task
- Stop a running Task
- Task Execution cleanup (For parent Task Executions only)

36.4.1. Stop Executing Tasks

To submit a stop task execution request to the platform, click the drop down button next to the task execution that needs to be stopped. Now click the `Stop task` option. The dashboard will present a dialog box asking if you are sure that you want to stop the task execution, click `Stop Task Execution(s)`.

Task Execution Id	Task Name	Start Date	End Date	Exit Code
#19	ComposedTask1	N/A	N/A	
#18	ComposedTask1-timestamp-batch	2019-07-18 23:59:10,855[-10]	2019-07-18 23:59:10,916[-10]	
#17	ComposedTask1-timestamp	2019-07-18 23:59:00,446[-10]	2019-07-18 23:59:00,496[-10]	
#16	ComposedTask1	2019-07-18 23:58:56,605[-10]	2019-07-18 23:59:17,230[-10]	
#15	ComposedTask1	2019-07-18 23:58:24,044[-10]	2019-07-18 23:58:24,182[-10]	
#14	ComposedTask1-timestamp-batch	2019-07-18 23:57:07,470[-10]	2019-07-18 23:57:07,550[-10]	0
#13	ComposedTask1-timestamp	2019-07-18 23:56:57,019[-10]	2019-07-18 23:56:57,050[-10]	0
#12	ComposedTask1	2019-07-18 23:56:53,567[-10]	2019-07-18 23:57:14,188[-10]	0
#8	ts-batch1	2019-07-18 23:55:46,120[-10]	2019-07-18 23:55:46,254[-10]	0

Child Spring Cloud Task applications launched via Spring Batch applications that utilize remote partitioning will not be stopped.

37. Jobs

The Jobs section of the Dashboard lets you inspect batch jobs. The main section of the screen provides a list of job executions. Batch jobs are tasks that each execute one or more batch jobs. Each job execution has a reference to the task execution ID (in the Task Id column).

The list of Job Executions also shows the state of the underlying Job Definition. Thus, if the underlying definition has been deleted, “No definition found” appears in the Status column.

You can take the following actions for each job:

- Restart (for failed jobs).
- Stop (for running jobs).
- View execution details.

Note: Clicking the stop button actually sends a stop request to the running job, which may not immediately stop.

The following image shows the Jobs page:

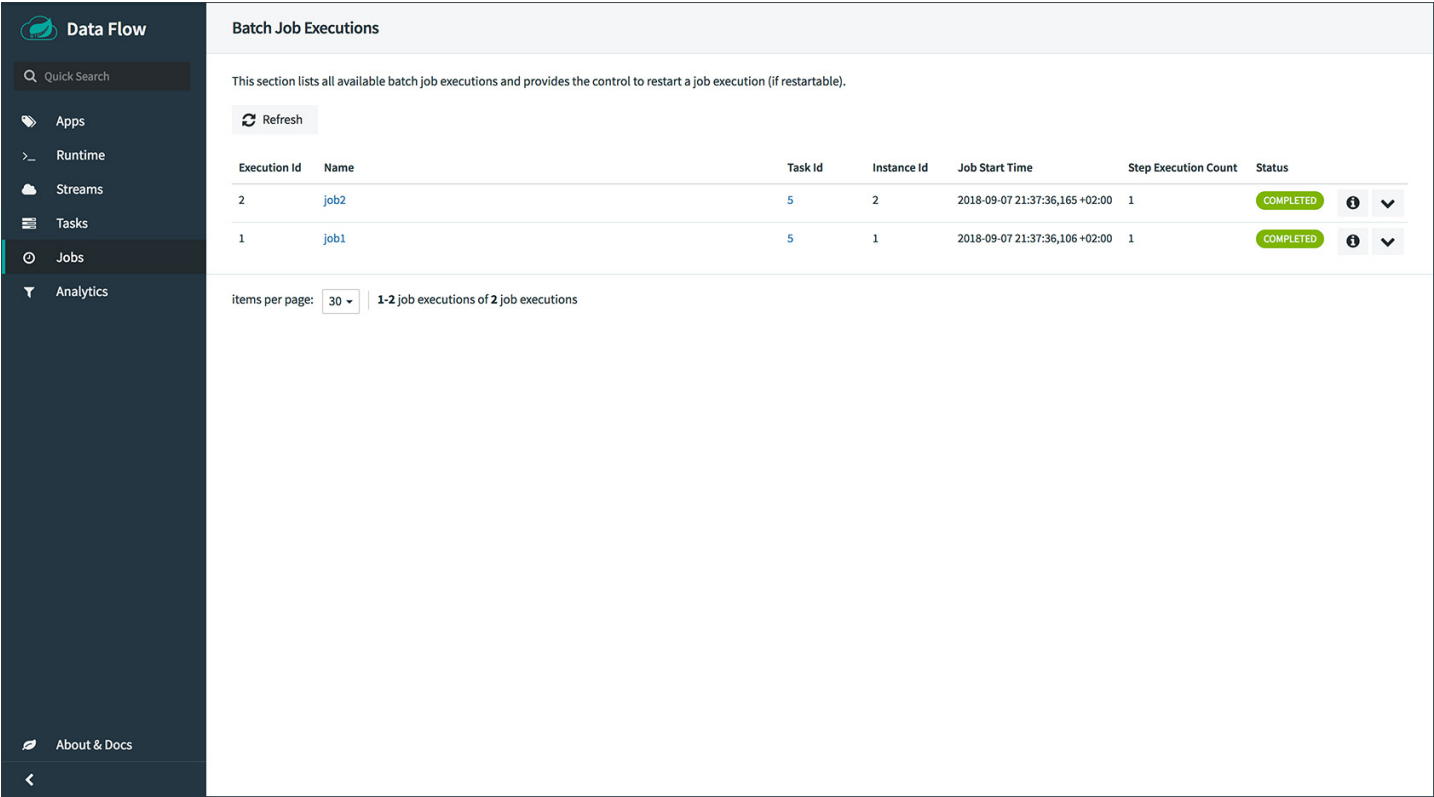


Figure 33. List of Job Executions

37.1. Job Execution Details

After having launched a batch job, the Job Execution Details page will show information about the job.

The following image shows the Job Execution Details page:

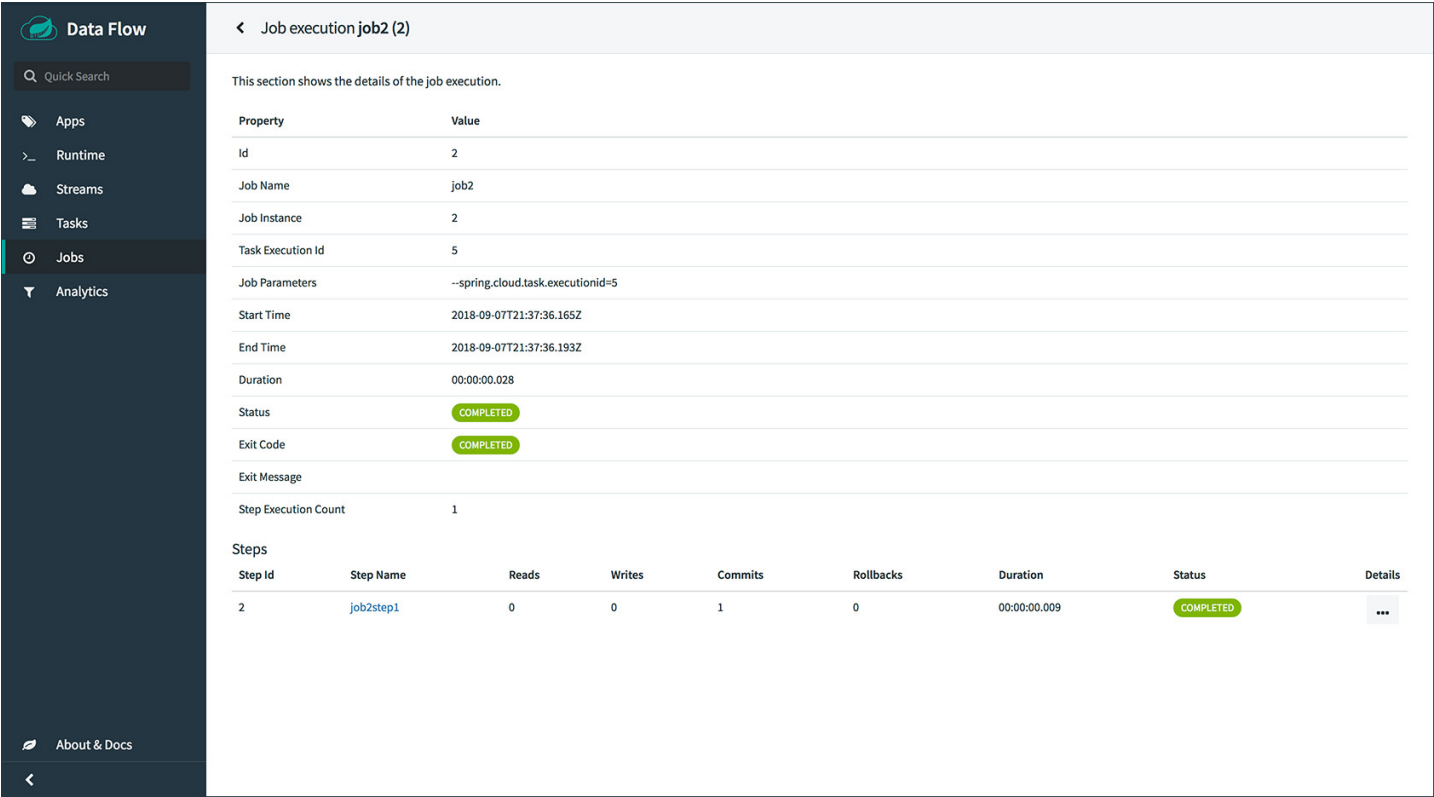


Figure 34. Job Execution Details

The Job Execution Details page contains a list of the executed steps. You can further drill into the details of each step's execution by clicking the magnifying glass icon.

37.2. Step Execution Details

The Step Execution Details page provides information about an individual step within a job.

The following image shows the Step Execution Details page:

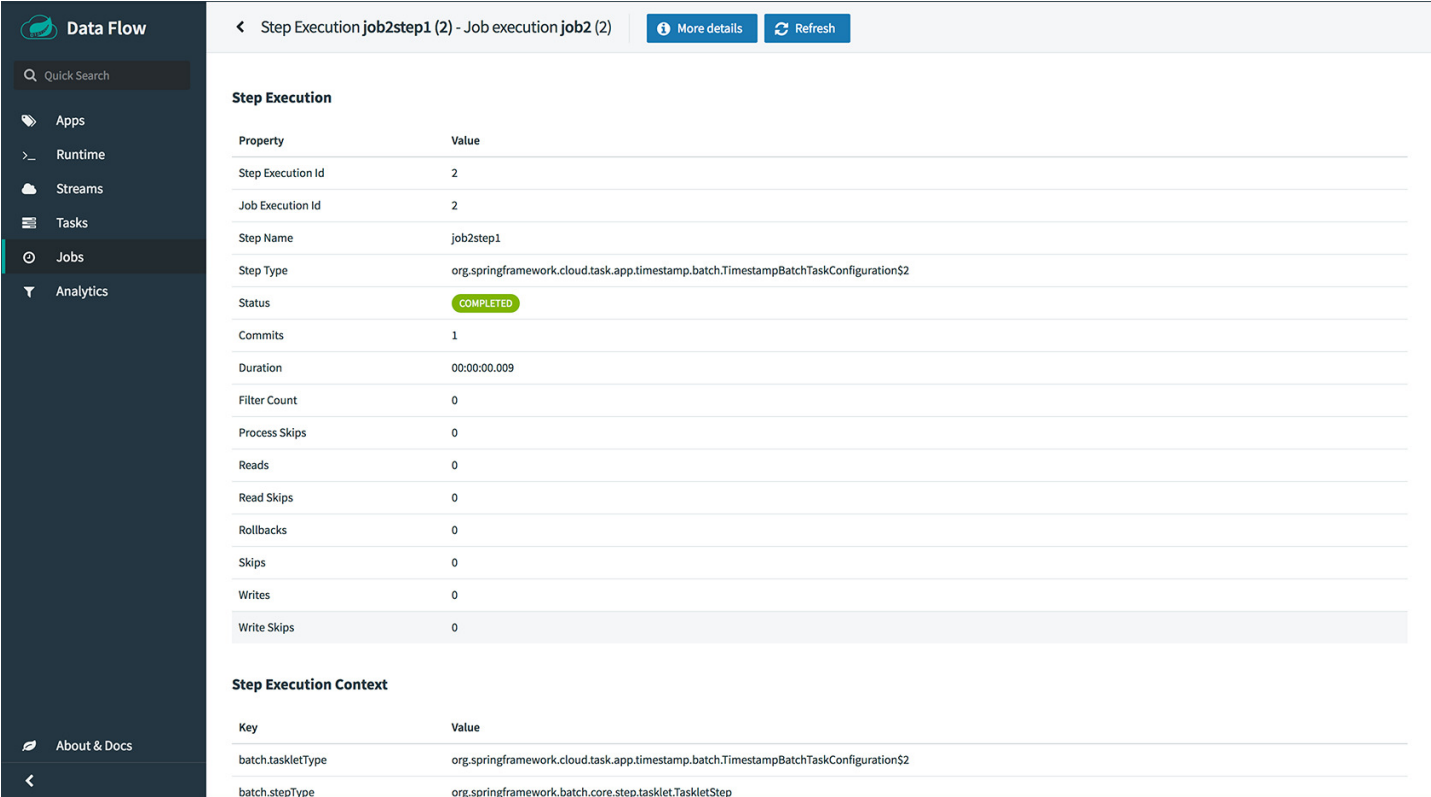


Figure 35. Step Execution Details

On the top of the page, you can see a progress indicator the respective step, with the option to refresh the indicator. A link is provided to view the step execution history.

The Step Execution Details screen provides a complete list of all Step Execution Context key/value pairs.

For exceptions, the Exit Description field contains additional error information. However, this field can have a maximum of 2500 characters. Therefore, in the case of long exception stack traces, trimming of error messages may occur. When that happens, refer to the server log files for further details.

37.3. Step Execution Progress

On this screen, you can see a progress bar indicator in regards to the execution of the current step. Under the Step Execution History, you can also view various metrics associated with the selected step, such as duration, read counts, write counts, and others.

38. Scheduling

You can create schedules from the SCDF Dashboard for the Task Definitions. Please see the [Scheduling Batch Jobs](#) section of the microsite for more information.

39. Auditing

The Auditing page of the Dashboard gives you access to recorded audit events. Currently audit event are recorded for:

- Streams
 - Create
 - Delete
 - Deploy
 - Undeploy
- Tasks
 - Create
 - Delete
 - Launch
- Scheduling of Tasks
 - Create Schedule
 - Delete Schedule

Q

Audit Records

This section gives you access to recorded audit events.

All operations

All actions

Refresh

ID	Created On	Operation	Action	Correlation Id	Created By	Data
4	2018-10-12T08:53:32.717Z	TASK	DELETE	test-timestamp	N/A	timestamp
3	2018-10-12T08:53:11.696Z	TASK	DEPLOY	test-timestamp	N/A	...rgs":[]
2	2018-10-12T08:52:48.488Z	TASK	CREATE	test-timestamp	N/A	timestamp
1	2018-10-12T08:52:08.221Z	STREAM	CREATE	foo-test	N/A	http log

items per page: 30

1-4 audit records of 4 audit records

Figure 36. List Overview of Audit Records

By clicking on the *Show Details* icon, you can obtain further details regarding the auditing details:

The screenshot shows a sidebar with navigation icons and a main content area titled 'Audit Records Details (4)'. The details are as follows:

Id:	4
Created On:	2018-10-12T08:53:32.717Z
Operation:	TASK
Action:	DELETE
Correlation Id:	test-timestamp
Created By:	N/A
Data:	"timestamp"

Figure 37. List Details of an Audit Record

Generally, auditing provides the following information:

- When was the record created?
- Username that triggered the audit event (if security is enabled)
- Audit operation (Schedule, Stresm, Task)
- Performed action (Create, Delete, Deploy, Rollback, Undeploy, Update)
- Correlation Id, e.g. Stream/Task name
- Audit Data

The written value of the property *Audit Data* depends on the performed *Audit Operation* and the *ActionType*. For example, when a Schedule is being created, the name of the task definition, task definition properties, deployment properties, as well as command line arguments are written to the persistence store.

Sensitive information is sanitized prior to saving the Audit Record, in an *best-effort-manner*. Any of the following keys are being detected and its sensitive values are masked:

- password
- secret
- key
- token
- .*credentials.*
- vcap_services

Samples

This section shows the available samples.

40. Links

Several samples have been created to help you get started on implementing higher-level use cases than the basic Streams and Tasks shown in the reference guide. The samples are part of a separate [repository](#) and have their own [reference documentation](#).

- [Java DSL](#)
- [HTTP to Cassandra](#)
- [HTTP to MySQL](#)
- [HTTP to Gemfire](#)
- [Gemfire CQ to Log Demo](#)
- [Gemfire to Log Demo](#)
- [Custom Processor](#)

Task and Batch

- [Batch Job on Cloud Foundry](#)
- [Batch File Ingest](#)

Data Science

- [Species Prediction](#)

Functions

- [Using Spring Cloud Function](#)

REST API Guide

This section describes the Spring Cloud Data Flow REST API.

41. Overview

Spring Cloud Data Flow provides a REST API that lets you access all aspects of the server. In fact, the Spring Cloud Data Flow shell is a first-class consumer of that API.

If you plan to use the REST API with Java, you should consider using the provided Java client (`DataflowTemplate`) that uses the REST API internally.

41.1. HTTP verbs

Spring Cloud Data Flow tries to adhere as closely as possible to standard HTTP and REST conventions in its use of HTTP verbs, as described in the following table:

Verb	Usage
GET	Used to retrieve a resource
POST	Used to create a new resource
PUT	Used to update an existing resource, including partial updates. Also used for resources that imply the concept of restarts, such as Tasks.
DELETE	Used to delete an existing resource.

41.2. HTTP Status Codes

Spring Cloud Data Flow tries to adhere as closely as possible to standard HTTP and REST conventions in its use of HTTP status codes, as shown in the following table:

Status code	Usage
200 OK	The request completed successfully.
201 Created	A new resource has been created successfully. The resource's URI is available from the response's Location header.
204 No Content	An update to an existing resource has been applied successfully.
400 Bad Request	The request was malformed. The response body includes an error description that provides further information.
404 Not Found	The requested resource did not exist.
409 Conflict	The requested resource already exists. For example, the task already exists or the stream was already being deployed
422 Unprocessable Entity	Returned in cases where the Job Execution cannot be stopped or restarted.

41.3. Headers

Every response has the following header(s):

Name	Description
Content-Type	The Content-Type of the payload, e.g. <code>application/hal+json</code>

41.4. Errors

Path	Type	Description
error	String	The HTTP error that occurred, e.g. Bad Request
message	String	A description of the cause of the error
path	String	The path to which the request was made
status	Number	The HTTP status code, e.g. 400
timestamp	String	The time, in milliseconds, at which the error occurred

41.5. Hypermedia

Spring Cloud Data Flow uses hypermedia, and resources include links to other resources in their responses. Responses are in the [Hypertext Application from resource to resource Language \(HAL\)](#) format. Links can be found beneath the `_links` key. Users of the API should not create URIs themselves. Instead, they should use the above-described links to navigate.

42. Resources

The API includes the following resources:

- [Index](#)
- [Server Meta Information](#)
- [Audit Records](#)
- [Registered Applications](#)
- [Stream Definitions](#)
- [Stream Deployments](#)
- [Stream Validation](#)
- [Task Definitions](#)
- [Task Executions](#)
- [Task Scheduler](#)
- [Task Validation](#)

- [Job Executions](#)
- [Job Instances](#)
- [Job Step Executions](#)
- [Runtime Information about Applications](#)
- [Stream Logs](#)
- [Task Logs](#)

42.1. Index

The index provides the entry point into Spring Cloud Data Flow’s REST API. The following topics provide more details:

- [Accessing the index](#)
- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)
- [Example Response](#)
- [Links](#)

42.1.1. Accessing the index

Use a `GET` request to access the index.

Request Structure

```
GET / HTTP/1.1
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/' -i -X GET
```

BASH

Response Structure

Path	Type	Description
_links	Object	Links to other resources
['api.revision']	Number	Incremented each time a change is implemented in this REST API
_links.audit-records.href	String	Link to the audit records

Path	Type	Description
<code>_links.dashboard.href</code>	String	Link to the dashboard
<code>_links.streams/definitions.href</code>	String	Link to the streams/definitions
<code>_links.streams/definitions/definition.href</code>	String	Link to the streams/definitions/definition
<code>_links.streams/definitions/definition.templated</code>	Boolean	Link streams/definitions/definition is templated
<code>_links.runtime/apps.href</code>	String	Link to the runtime/apps
<code>_links.runtime/apps/{appId}.href</code>	String	Link to the runtime/apps/{appId}
<code>_links.runtime/apps/{appId}.templated</code>	Boolean	Link runtime/apps is templated
<code>_links.runtime/apps/{appId}/instances.href</code>	String	Link to the runtime/apps/{appId}/instances
<code>_links.runtime/apps/{appId}/instances.templated</code>	Boolean	Link runtime/apps/{appId}/instances is templated
<code>_links.runtime/apps/{appId}/instances/{instanceId}.href</code>	String	Link to the runtime/apps/{appId}/instances/{instanceId}
<code>_links.runtime/apps/{appId}/instances/{instanceId}.templated</code>	Boolean	Link runtime/apps/{appId}/instances/{instanceId} is templated
<code>_links.runtime/streams.href</code>	String	Link to the runtime/streams
<code>_links.runtime/streams.templated</code>	Boolean	Link runtime/streams is templated
<code>_links.runtime/streams/{streamNames}.href</code>	String	Link to the runtime/streams/{streamNames}

Path	Type	Description
<code>_links.runtime/streams/{streamNames}.templated</code>	Boolean	Link runtime/streams/{streamNames} is templated
<code>_links.streams/logs.href</code>	String	Link to the streams/logs
<code>_links.streams/logs/{streamName}.href</code>	String	Link to the streams/logs/{streamName}
<code>_links.streams/logs/{streamName}/{appName}.href</code>	String	Link to the streams/logs/{streamName}/{appName}
<code>_links.streams/logs/{streamName}.templated</code>	Boolean	Link streams/logs/{streamName} is templated
<code>_links.streams/logs/{streamName}/{appName}.templated</code>	Boolean	Link streams/logs/{streamName}/{appName} is templated
<code>_links.streams/deployments.href</code>	String	Link to the streams/deployments
<code>_links.streams/deployments/{name}.href</code>	String	Link to the streams/deployments/{name}
<code>_links.streams/deployments/{name}.templated</code>	Boolean	Link streams/deployments/{name} is templated
<code>_links.streams/deployments/deployment.href</code>	String	Link to the streams/deployments/deployment
<code>_links.streams/deployments/deployment.templated</code>	Boolean	Link streams/deployments/deployment is templated
<code>_links.streams/deployments/manifest/{name}/{version}.href</code>	String	Link to the streams/deployments/manifest/{name}/{version}
<code>_links.streams/deployments/manifest/{name}/{version}.templated</code>	Boolean	Link streams/deployments/manifest/{name}/{version} is templated

Path	Type	Description
<code>_links.streams/deployments/history/{name}.href</code>	String	Link to the streams/deployments/history/{name}
<code>_links.streams/deployments/history/{name}.templated</code>	Boolean	Link streams/deployments/history is templated
<code>_links.streams/deployments/rollback/{name}/{version}.href</code>	String	Link to the streams/deployments/rollback/{name}/{version}
<code>_links.streams/deployments/rollback/{name}/{version}.templated</code>	Boolean	Link streams/deployments/rollback/{name}/{version} is templated
<code>_links.streams/deployments/update/{name}.href</code>	String	Link to the streams/deployments/update/{name}
<code>_links.streams/deployments/update/{name}.templated</code>	Boolean	Link streams/deployments/update/{name} is templated
<code>_links.streams/deployments/platform/list.href</code>	String	Link to the streams/deployments/platform/list
<code>_links.streams/deployments/scale/{streamName}/{appName}/instances/{count}.href</code>	String	Link to the streams/deployments/scale/{streamName}/{appName}/instances/{count}
<code>_links.streams/deployments/scale/{streamName}/{appName}/instances/{count}.templated</code>	Boolean	Link streams/deployments/scale/{streamName}/{appName}/instances/{count} is templated
<code>_links.streams/validation.href</code>	String	Link to the streams/validation
<code>_links.streams/validation.templated</code>	Boolean	Link streams/validation is templated
<code>_links.tasks/platforms.href</code>	String	Link to the tasks/platforms

Path	Type	Description
<code>_links.tasks/definitions.href</code>	String	Link to the tasks/definitions
<code>_links.tasks/definitions/definition.href</code>	String	Link to the tasks/definitions/definition
<code>_links.tasks/definitions/definition.templated</code>	Boolean	Link tasks/definitions/definition is templated
<code>_links.tasks/executions.href</code>	String	Link to the tasks/executions
<code>_links.tasks/executions/name.href</code>	String	Link to the tasks/executions/name
<code>_links.tasks/executions/name.templated</code>	Boolean	Link tasks/executions/name is templated
<code>_links.tasks/executions/current.href</code>	String	Link to the tasks/executions/current
<code>_links.tasks/executions/execution.href</code>	String	Link to the tasks/executions/execution
<code>_links.tasks/executions/execution.templated</code>	Boolean	Link tasks/executions/execution is templated
<code>_links.tasks/logs.href</code>	String	Link to the tasks/logs
<code>_links.tasks/logs.templated</code>	Boolean	Link tasks/logs is templated
<code>_links.tasks/schedules.href</code>	String	Link to the tasks/executions/schedules
<code>_links.tasks/schedules/instances.href</code>	String	Link to the tasks/schedules/instances
<code>_links.tasks/schedules/instances.templated</code>	Boolean	Link tasks/schedules/instances is templated
<code>_links.tasks/validation.href</code>	String	Link to the tasks/validation

Path	Type	Description
<code>_links.tasks/validation.templated</code>	Boolean	Link tasks/validation is templated
<code>_links.jobs/executions.href</code>	String	Link to the jobs/executions
<code>_links.jobs/thinexecutions.href</code>	String	Link to the jobs/thinexecutions
<code>_links.jobs/executions/name.href</code>	String	Link to the jobs/executions/name
<code>_links.jobs/executions/name.templated</code>	Boolean	Link jobs/executions/name is templated
<code>_links.jobs/thinexecutions/name.href</code>	String	Link to the jobs/thinexecutions/name
<code>_links.jobs/thinexecutions/name.templated</code>	Boolean	Link jobs/executions/name is templated
<code>_links.jobs/executions/execution.href</code>	String	Link to the jobs/executions/execution
<code>_links.jobs/executions/execution.templated</code>	Boolean	Link jobs/executions/execution is templated
<code>_links.jobs/executions/execution/steps.href</code>	String	Link to the jobs/executions/execution/steps
<code>_links.jobs/executions/execution/steps.templated</code>	Boolean	Link jobs/executions/execution/steps is templated
<code>_links.jobs/executions/execution/steps/step.href</code>	String	Link to the jobs/executions/execution/steps/step
<code>_links.jobs/executions/execution/steps/step.templated</code>	Boolean	Link jobs/executions/execution/steps/step is templated
<code>_links.jobs/executions/execution/steps/step/progress.href</code>	String	Link to the jobs/executions/execution/steps/step/progress

Path	Type	Description
_links.jobs/executions/execution/steps/step/progress.templated	Boolean	Link jobs/executions/execution/steps/step/progress is templated
_links.jobs/instances/name.href	String	Link to the jobs/instances/name
_links.jobs/instances/name.templated	Boolean	Link jobs/instances/name is templated
_links.jobs/instances/instance.href	String	Link to the jobs/instances/instance
_links.jobs/instances/instance.templated	Boolean	Link jobs/instances/instance is templated
_links.tools/parseTaskTextToGraph.href	String	Link to the tools/parseTaskTextToGraph
_links.tools/convertTaskGraphToText.href	String	Link to the tools/convertTaskGraphToText
_links.apps.href	String	Link to the apps
_links.about.href	String	Link to the about
_links.completions/stream.href	String	Link to the completions/stream
_links.completions/stream.templated	Boolean	Link completions/stream is templated
_links.completions/task.href	String	Link to the completions/task
_links.completions/task.templated	Boolean	Link completions/task is templated

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 6289

{
  "_links" : {
    "dashboard" : {
      "href" : "http://localhost:9393/dashboard"
    }
  },
```

HTTP

```

"audit-records" : {
  "href" : "http://localhost:9393/audit-records"
},
"streams/definitions" : {
  "href" : "http://localhost:9393/streams/definitions"
},
"streams/definitions/definition" : {
  "href" : "http://localhost:9393/streams/definitions/{name}",
  "templated" : true
},
"streams/validation" : {
  "href" : "http://localhost:9393/streams/validation/{name}",
  "templated" : true
},
"runtime/streams" : {
  "href" : "http://localhost:9393/runtime/streams?names={names}",
  "templated" : true
},
"runtime/streams/{streamNames}" : {
  "href" : "http://localhost:9393/runtime/streams/{streamNames}",
  "templated" : true
},
"runtime/apps" : {
  "href" : "http://localhost:9393/runtime/apps"
},
"runtime/apps/{appId}" : {
  "href" : "http://localhost:9393/runtime/apps/{appId}",
  "templated" : true
},
"runtime/apps/{appId}/instances" : {
  "href" : "http://localhost:9393/runtime/apps/{appId}/instances",
  "templated" : true
},
"runtime/apps/{appId}/instances/{instanceId}" : {
  "href" : "http://localhost:9393/runtime/apps/{appId}/instances/{instanceId}",
  "templated" : true
},
"streams/deployments" : {
  "href" : "http://localhost:9393/streams/deployments"
},
"streams/deployments/{name}" : {
  "href" : "http://localhost:9393/streams/deployments/{name}",
  "templated" : true
},
"streams/deployments/history/{name}" : {
  "href" : "http://localhost:9393/streams/deployments/history/{name}",
  "templated" : true
},
"streams/deployments/manifest/{name}/{version}" : {
  "href" : "http://localhost:9393/streams/deployments/manifest/{name}/{version}",
  "templated" : true
},
"streams/deployments/platform/list" : {
  "href" : "http://localhost:9393/streams/deployments/platform/list"
},
"streams/deployments/rollback/{name}/{version}" : {
  "href" : "http://localhost:9393/streams/deployments/rollback/{name}/{version}",
  "templated" : true
},
"streams/deployments/update/{name}" : {
  "href" : "http://localhost:9393/streams/deployments/update/{name}",
  "templated" : true
},
"streams/deployments/deployment" : {
  "href" : "http://localhost:9393/streams/deployments/{name}",
  "templated" : true
},
"streams/deployments/scale/{streamName}/{appName}/instances/{count}" : {
  "href" : "http://localhost:9393/streams/deployments/scale/{streamName}/{appName}/instances/{count}",
  "templated" : true
},
"streams/logs" : {
  "href" : "http://localhost:9393/streams/logs"
},
"streams/logs/{streamName}" : {
  "href" : "http://localhost:9393/streams/logs/{streamName}",
  "templated" : true
}

```

```

},
"streams/logs/{streamName}/{appName}" : {
  "href" : "http://localhost:9393/streams/logs/{streamName}/{appName}",
  "templated" : true
},
"tasks/platforms" : {
  "href" : "http://localhost:9393/tasks/platforms"
},
"tasks/definitions" : {
  "href" : "http://localhost:9393/tasks/definitions"
},
"tasks/definitions/definition" : {
  "href" : "http://localhost:9393/tasks/definitions/{name}",
  "templated" : true
},
"tasks/executions" : {
  "href" : "http://localhost:9393/tasks/executions"
},
"tasks/executions/name" : {
  "href" : "http://localhost:9393/tasks/executions/{?name}",
  "templated" : true
},
"tasks/executions/current" : {
  "href" : "http://localhost:9393/tasks/executions/current"
},
"tasks/executions/execution" : {
  "href" : "http://localhost:9393/tasks/executions/{id}",
  "templated" : true
},
"tasks/validation" : {
  "href" : "http://localhost:9393/tasks/validation/{name}",
  "templated" : true
},
"tasks/logs" : {
  "href" : "http://localhost:9393/tasks/logs/{taskExternalExecutionId}{?platformName}",
  "templated" : true
},
"tasks/schedules" : {
  "href" : "http://localhost:9393/tasks/schedules"
},
"tasks/schedules/instances" : {
  "href" : "http://localhost:9393/tasks/schedules/instances/{taskDefinitionName}",
  "templated" : true
},
"jobs/executions" : {
  "href" : "http://localhost:9393/jobs/executions"
},
"jobs/executions/name" : {
  "href" : "http://localhost:9393/jobs/executions/{?name}",
  "templated" : true
},
"jobs/executions/execution" : {
  "href" : "http://localhost:9393/jobs/executions/{id}",
  "templated" : true
},
"jobs/executions/execution/steps" : {
  "href" : "http://localhost:9393/jobs/executions/{jobExecutionId}/steps",
  "templated" : true
},
"jobs/executions/execution/steps/step" : {
  "href" : "http://localhost:9393/jobs/executions/{jobExecutionId}/steps/{stepId}",
  "templated" : true
},
"jobs/executions/execution/steps/step/progress" : {
  "href" : "http://localhost:9393/jobs/executions/{jobExecutionId}/steps/{stepId}/progress",
  "templated" : true
},
"jobs/instances/name" : {
  "href" : "http://localhost:9393/jobs/instances/{?name}",
  "templated" : true
},
"jobs/instances/instance" : {
  "href" : "http://localhost:9393/jobs/instances/{id}",
  "templated" : true
},
"tools/parseTaskTextToGraph" : {
  "href" : "http://localhost:9393/tools"
}

```



```
    },
    "tools/convertTaskGraphToText" : {
      "href" : "http://localhost:9393/tools"
    },
    "jobs/thinexecutions" : {
      "href" : "http://localhost:9393/jobs/thinexecutions"
    },
    "jobs/thinexecutions/name" : {
      "href" : "http://localhost:9393/jobs/thinexecutions{?name}",
      "templated" : true
    },
    "apps" : {
      "href" : "http://localhost:9393/apps"
    },
    "about" : {
      "href" : "http://localhost:9393/about"
    },
    "completions/stream" : {
      "href" : "http://localhost:9393/completions/stream{?start,detailLevel}",
      "templated" : true
    },
    "completions/task" : {
      "href" : "http://localhost:9393/completions/task{?start,detailLevel}",
      "templated" : true
    }
  },
  "api.revision" : 14
}
```

Links

The main element of the index are the links, as they let you traverse the API and execute the desired functionality:

Relation	Description
about	Access meta information, including enabled features, security info, version information
dashboard	Access the dashboard UI
audit-records	Provides audit trail information
apps	Handle registered applications
completions/stream	Exposes the DSL completion features for Stream
completions/task	Exposes the DSL completion features for Task
jobs/executions	Provides the JobExecution resource
jobs/thinexecutions	Provides the JobExecution thin resource with no step executions included
jobs/executions/execution	Provides details for a specific JobExecution
jobs/executions/execution/steps	Provides the steps for a JobExecution
jobs/executions/execution/steps/step	Returns the details for a specific step
jobs/executions/execution/steps/step/progress	Provides progress information for a specific step

Relation	Description
jobs/executions/name	Retrieve Job Executions by Job name
jobs/thinexecutions/name	Retrieve Job Executions by Job name with no step executions included
jobs/instances/instance	Provides the job instance resource for a specific job instance
jobs/instances/name	Provides the Job instance resource for a specific job name
runtime/streams	Exposes stream runtime status
runtime/streams/{streamNames}	Exposes streams runtime status
runtime/apps	Provides the runtime application resource
runtime/apps/{appId}	Exposes the runtime status for a specific app
runtime/apps/{appId}/instances	Provides the status for app instances
runtime/apps/{appId}/instances/{instanceId}	Provides the status for specific app instance
tasks/definitions	Provides the task definition resource
tasks/definitions/definition	Provides details for a specific task definition
tasks/validation	Provides the validation for a task definition
tasks/executions	Returns Task executions and allows launching of tasks
tasks/executions/current	Provides the current count of running tasks
tasks/schedules	Provides schedule information of tasks
tasks/schedules/instances	Provides schedule information of a specific task
tasks/executions/name	Returns all task executions for a given Task name
tasks/executions/execution	Provides details for a specific task execution
tasks/platforms	Provides platform accounts for launching tasks
tasks/logs	Retrieve the task application log
streams/definitions	Exposes the Streams resource
streams/definitions/definition	Handle a specific Stream definition
streams/validation	Provides the validation for a stream definition
streams/deployments	Provides Stream deployment operations

Relation	Description
streams/deployments/{name}	Request un-deployment of an existing stream
streams/deployments/deployment	Request (un-)deployment of an existing stream definition
streams/deployments/manifest/{name}/{version}	Return a manifest info of a release version
streams/deployments/history/{name}	Get stream's deployment history as list or Releases for this release
streams/deployments/rollback/{name}/{version}	Rollback the stream to the previous or a specific version of the stream
streams/deployments/update/{name}	Update the stream.
streams/deployments/platform/list	List of supported deployment platforms
streams/deployments/scale/{streamName}/{appName}/instances/{count}	Scale up or down number of application instances for a selected stream
streams/logs	Retrieve application logs of the stream
streams/logs/{streamName}	Retrieve application logs of the stream
streams/logs/{streamName}/{appName}	Retrieve a specific application log of the stream
tools/parseTaskTextToGraph	Parse a task definition into a graph structure
tools/convertTaskGraphToText	Convert a graph format into DSL text format

42.2. Server Meta Information

The server meta information endpoint provides more information about the server itself. The following topics provide more details:

- [Retrieving information about the server](#)
- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

42.2.1. Retrieving information about the server

A `GET` request returns meta information for Spring Cloud Data Flow, including:

- Runtime environment information
- Information regarding which features are enabled
- Dependency information of Spring Cloud Data Flow Server
- Security information

Request Structure

```
GET /about HTTP/1.1
Accept: application/json
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/about' -i -X GET \
-H 'Accept: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 2638
```

HTTP

```
{
  "featureInfo" : {
    "analyticsEnabled" : true,
    "streamsEnabled" : true,
    "tasksEnabled" : true,
    "schedulesEnabled" : true,
    "grafanaEnabled" : false
  },
  "versionInfo" : {
    "implementation" : {
      "name" : "${info.app.name}",
      "version" : "${info.app.version}"
    },
    "core" : {
      "name" : "Spring Cloud Data Flow Core",
      "version" : "2.4.0.RELEASE"
    },
    "dashboard" : {
      "name" : "Spring Cloud Dataflow UI",
      "version" : "2.4.0.RELEASE"
    },
    "shell" : {
      "name" : "Spring Cloud Data Flow Shell",
      "version" : "2.4.0.RELEASE",
      "url" : "https://repo.spring.io/libs-release/org/springframework/cloud/spring-cloud-dataflow-shell/2.4.0.RELEASE/spri
    }
  },
  "securityInfo" : {
    "authenticationEnabled" : false,
    "authenticated" : false,
    "username" : null,
    "roles" : [ ]
  },
  "runtimeEnvironment" : {
    "appDeployer" : {
      "deployerImplementationVersion" : "Test Version",
      "deployerName" : "Test Server",
      "deployerSpiVersion" : "2.3.0.RELEASE",
      "javaVersion" : "1.8.0_232",
      "platformApiVersion" : "",
      "platformClientVersion" : "",
      "platformHostVersion" : "",
      "platformSpecificInfo" : {
        "default" : "local"
      },
      "platformType" : "Skipper Managed",
      "springBootVersion" : "2.2.4.RELEASE",
      "springVersion" : "5.2.3.RELEASE"
    },
    "taskLaunchers" : [ {
```

```

    "deployerImplementationVersion" : "2.2.0.RELEASE",
    "deployerName" : "LocalTaskLauncher",
    "deployerSpiVersion" : "2.2.0.RELEASE",
    "javaVersion" : "1.8.0_232",
    "platformApiVersion" : "Linux 4.15.0-74-generic",
    "platformClientVersion" : "4.15.0-74-generic",
    "platformHostVersion" : "4.15.0-74-generic",
    "platformSpecificInfo" : { },
    "platformType" : "Local",
    "springBootVersion" : "2.2.4.RELEASE",
    "springVersion" : "5.2.3.RELEASE"
  }, {
    "deployerImplementationVersion" : "2.2.0.RELEASE",
    "deployerName" : "LocalTaskLauncher",
    "deployerSpiVersion" : "2.2.0.RELEASE",
    "javaVersion" : "1.8.0_232",
    "platformApiVersion" : "Linux 4.15.0-74-generic",
    "platformClientVersion" : "4.15.0-74-generic",
    "platformHostVersion" : "4.15.0-74-generic",
    "platformSpecificInfo" : { },
    "platformType" : "Local",
    "springBootVersion" : "2.2.4.RELEASE",
    "springVersion" : "5.2.3.RELEASE"
  } ]
},
"grafanaInfo" : {
  "url" : "",
  "refreshInterval" : 15
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/about"
  }
}
}

```

42.3. Registered Applications

The registered applications endpoint provides information about the applications that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [Listing Applications](#)
- [Getting Information on a Particular Application](#)
- [Registering a New Application](#)
- [Registering a New Application with version](#)
- [Registering Applications in Bulk](#)
- [Set the default application version](#)
- [Unregistering an Application](#)
- [Unregistering all Applications](#)

42.3.1. Listing Applications

A `GET` request will list all applications known to Spring Cloud Data Flow. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)

- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /apps?search=&type=source&page=0&size=10&sort=name%2CASC&search= HTTP/1.1
Accept: application/json
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
search	The search string performed on the name (optional)
type	Restrict the returned apps to the type of the app. One of [app, source, processor, sink, task]
page	The zero-based page number (optional)
sort	The sort on the list (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/apps?search=&type=source&page=0&size=10&sort=name%2CASC&search=' -i -X GET \
-H 'Accept: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 973

{
  "_embedded" : {
    "appRegistrationResourceList" : [ {
      "name" : "http",
      "type" : "source",
      "uri" : "maven://org.springframework.cloud.stream.app:http-source-rabbit:1.2.0.RELEASE",
      "version" : "1.2.0.RELEASE",
      "defaultVersion" : true,
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/apps/source/http/1.2.0.RELEASE"
        }
      }
    }, {
      "name" : "time",
      "type" : "source",
      "uri" : "maven://org.springframework.cloud.stream.app:time-source-rabbit:1.2.0.RELEASE",
      "version" : "1.2.0.RELEASE",
      "defaultVersion" : true,
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/apps/source/time/1.2.0.RELEASE"
        }
      }
    }
  ]
}
```

HTTP

```
    }
  }
} ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/apps?page=0&size=10&sort=name,asc"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 2,
  "totalPages" : 1,
  "number" : 0
}
}
```

42.3.2. Getting Information on a Particular Application

A GET request on /apps/<type>/<name> gets info on a particular application. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /apps/source/http?exhaustive=false HTTP/1.1
Accept: application/json
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
exhaustive	Return all application properties, including common Spring Boot properties

Path Parameters

```
/apps/{type}/{name}
```

Parameter	Description
type	The type of application to query. One of [app, source, processor, sink, task]
name	The name of the application to query

Example Request

```
$ curl 'http://localhost:9393/apps/source/http?exhaustive=false' -i -X GET \
-H 'Accept: application/json'
```

Response Structure

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 1980

```
{
  "name" : "http",
  "type" : "source",
  "uri" : "maven://org.springframework.cloud.stream.app:http-source-rabbit:1.2.0.RELEASE",
  "version" : "1.2.0.RELEASE",
  "defaultVersion" : true,
  "options" : [ {
    "id" : "http.path-pattern",
    "name" : "path-pattern",
    "type" : "java.lang.String",
    "description" : "An Ant-Style pattern to determine which http requests will be captured.",
    "shortDescription" : "An Ant-Style pattern to determine which http requests will be captured.",
    "defaultValue" : "/",
    "hints" : {
      "keyHints" : [ ],
      "keyProviders" : [ ],
      "valueHints" : [ ],
      "valueProviders" : [ ]
    },
    "deprecation" : null,
    "deprecated" : false
  }, {
    "id" : "http.mapped-request-headers",
    "name" : "mapped-request-headers",
    "type" : "java.lang.String[]",
    "description" : "Headers that will be mapped.",
    "shortDescription" : "Headers that will be mapped.",
    "defaultValue" : null,
    "hints" : {
      "keyHints" : [ ],
      "keyProviders" : [ ],
      "valueHints" : [ ],
      "valueProviders" : [ ]
    },
    "deprecation" : null,
    "deprecated" : false
  }, {
    "id" : "http.secured",
    "name" : "secured",
    "type" : "java.lang.Boolean",
    "description" : "Secure or not HTTP source path.",
    "shortDescription" : "Secure or not HTTP source path.",
    "defaultValue" : false,
    "hints" : {
      "keyHints" : [ ],
      "keyProviders" : [ ],
      "valueHints" : [ ],
      "valueProviders" : [ ]
    },
    "deprecation" : null,
    "deprecated" : false
  }, {
    "id" : "server.port",
    "name" : "port",
    "type" : "java.lang.Integer",
    "description" : "Server HTTP port.",
    "shortDescription" : "Server HTTP port.",
    "defaultValue" : null,
    "hints" : {
      "keyHints" : [ ],
      "keyProviders" : [ ],
      "valueHints" : [ ],
      "valueProviders" : [ ]
    },
    "deprecation" : null,
    "deprecated" : false
  }
]
```



```
    },
    "deprecation" : null,
    "deprecated" : false
  } ],
  "shortDescription" : null
}
```

42.3.3. Registering a New Application

A `POST` request on `/apps/<type>/<name>` allows registration of a new application. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /apps/source/http HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

uri=maven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE
```

HTTP

Request Parameters

Parameter	Description
uri	URI where the application bits reside
metadata-uri	URI where the application metadata jar can be found
force	Must be true if a registration with the same name and type already exists, otherwise an error will occur

Path Parameters

`/apps/{type}/{name}`

Parameter	Description
type	The type of application to register. One of [app, source, processor, sink, task]
name	The name of the application to register

Example Request

```
$ curl 'http://localhost:9393/apps/source/http' -i -X POST \
  -d 'uri=maven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE'
```

Response Structure

HTTP/1.1 201 Created

HTTP

42.3.4. Registering a New Application with version

A POST request on /apps/<type>/<name>/<version> allows registration of a new application The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

POST /apps/source/http/1.1.0.RELEASE HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

uri=maven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE

HTTP

Request Parameters

Parameter	Description
uri	URI where the application bits reside
metadata-uri	URI where the application metadata jar can be found
force	Must be true if a registration with the same name and type already exists, otherwise an error will occur

Path Parameters

/apps/{type}/{name}/{version:.*}

Parameter	Description
type	The type of application to register. One of [app, source, processor, sink, task] (optional)

Parameter	Description
name	The name of the application to register
version	The version of the application to register

Example Request

```
$ curl 'http://localhost:9393/apps/source/http/1.1.0.RELEASE' -i -X POST \  
  -d 'uri=maven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE'
```

BASH

Response Structure

```
HTTP/1.1 201 Created
```

HTTP

42.3.5. Registering Applications in Bulk

A POST request on /apps allows registering multiple applications at once. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /apps HTTP/1.1  
Host: localhost:9393  
Content-Type: application/x-www-form-urlencoded
```

HTTP

```
apps=source.http%3Dmaven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE&force=false
```

Request Parameters

Parameter	Description
uri	URI where a properties file containing registrations can be fetched. Exclusive with apps.
apps	Inline set of registrations. Exclusive with uri.
force	Must be true if a registration with the same name and type already exists, otherwise an error will occur

Example Request

BASH

```
$ curl 'http://localhost:9393/apps' -i -X POST \
-d 'apps=source.http%3Dmaven%3A%2F%2Forg.springframework.cloud.stream.app%3Ahttp-source-rabbit%3A1.1.0.RELEASE&force=fa'
```

Response Structure

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/hal+json
Content-Length: 611
```

```
{
  "_embedded" : {
    "appRegistrationResourceList" : [ {
      "name" : "http",
      "type" : "source",
      "uri" : "maven://org.springframework.cloud.stream.app:http-source-rabbit:1.1.0.RELEASE",
      "version" : "1.1.0.RELEASE",
      "defaultVersion" : true,
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/apps/source/http/1.1.0.RELEASE"
        }
      }
    } ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/apps?page=0&size=20"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

42.3.6. Set the default application version

For an application with the `name` and `type`, multiple versions can be registered. In this case, one of the versions can be chosen as the default application.

The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

HTTP

```
PUT /apps/source/http/1.2.0.RELEASE HTTP/1.1
Accept: application/json
Host: localhost:9393
```

Path Parameters

/apps/{type}/{name}/{version:.*}

Parameter	Description
type	The type of application. One of [app, source, processor, sink, task]
name	The name of the application
version	The version of the application

Example Request

```
$ curl 'http://localhost:9393/apps/source/http/1.2.0.RELEASE' -i -X PUT \
-H 'Accept: application/json'
```

BASH

Response Structure

```
HTTP/1.1 202 Accepted
```

HTTP

42.3.7. Unregistering an Application

A DELETE request on /apps/<type>/<name> unregisters a previously registered application. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /apps/source/http/1.2.0.RELEASE HTTP/1.1
Host: localhost:9393
```

HTTP

Path Parameters

/apps/{type}/{name}/{version}

Parameter	Description
type	The type of application to unregister. One of [app, source, processor, sink, task]
name	The name of the application to unregister

Parameter	Description
version	The version of the application to unregister (optional)

Example Request

```
$ curl 'http://localhost:9393/apps/source/http/1.2.0.RELEASE' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.3.8. Unregistering all Applications

A `DELETE` request on `/apps` unregisters all the applications. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /apps HTTP/1.1  
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/apps' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.4. Audit Records

The audit records endpoint provides information about the audit records. The following topics provide more details:

- [List All Audit Records](#)
- [Retrieve Audit Record Detail](#)
- [List all the Audit Action Types](#)
- [List all the Audit Operation Types](#)

42.4.1. List All Audit Records

The audit records endpoint lets you retrieve audit trail information.

The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
HTTP
GET /audit-records?page=0&size=10&operations=STREAM&actions=CREATE&fromDate=2000-01-01T00%3A00%3A00&toDate=2099-01-01T00%3A00%3A00
Host: localhost:9393
```

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)
operations	Comma-separated list of Audit Operations (optional)
actions	Comma-separated list of Audit Actions (optional)
fromDate	From date filter (ex.: 2019-02-03T00:00:30) (optional)
toDate	To date filter (ex.: 2019-02-03T00:00:30) (optional)

Example Request

```
BASH
$ curl 'http://localhost:9393/audit-records?page=0&size=10&operations=STREAM&actions=CREATE&fromDate=2000-01-01T00%3A00%3A00&toDate=2099-01-01T00%3A00%3A00'
```

Response Structure

```
HTTP
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 680

{
  "_embedded" : {
    "auditRecordResourceList" : [ {
      "auditRecordId" : 5,
      "createdBy" : null,
      "correlationId" : "timelog",
      "auditData" : "time --format='YYYY MM DD' | log",
    },
    ...
  ]
}
```

```
"createdOn" : "2020-01-31T12:17:05.160Z",
"auditAction" : "CREATE",
"auditOperation" : "STREAM",
"platformName" : null,
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/audit-records/5"
  }
}
} ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/audit-records?page=0&size=10"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 1,
  "totalPages" : 1,
  "number" : 0
}
}
```

42.4.2. Retrieve Audit Record Detail

The audit record endpoint lets you get a single audit record. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /audit-records/5 HTTP/1.1
Host: localhost:9393
```

HTTP

Path Parameters

/audit-records/{id}

Parameter	Description
id	The id of the audit record to query (required)

Example Request

```
$ curl 'http://localhost:9393/audit-records/5' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
```

HTTP

Content-Length: 354

```
{
  "auditRecordId" : 5,
  "createdBy" : null,
  "correlationId" : "timelog",
  "auditData" : "time --format='YYYY MM DD' | log",
  "createdOn" : "2020-01-31T12:17:05.160Z",
  "auditAction" : "CREATE",
  "auditOperation" : "STREAM",
  "platformName" : null,
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/audit-records/5"
    }
  }
}
```

42.4.3. List all the Audit Action Types

The audit record endpoint lets you get the action types. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /audit-records/audit-action-types HTTP/1.1
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/audit-records/audit-action-types' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 934
```

HTTP

```
[ {
  "id" : 100,
  "name" : "Create",
  "description" : "Create an Entity",
  "nameWithDescription" : "Create (Create an Entity)",
  "key" : "CREATE"
}, {
  "id" : 200,
  "name" : "Delete",
  "description" : "Delete an Entity",
  "nameWithDescription" : "Delete (Delete an Entity)",
  "key" : "DELETE"
}, {
  "id" : 300,
  "name" : "Deploy",
  "description" : "Deploy an Entity",
  "nameWithDescription" : "Deploy (Deploy an Entity)",
  "key" : "DEPLOY"
}, {
```

```

    "id" : 400,
    "name" : "Rollback",
    "description" : "Rollback an Entity",
    "nameWithDescription" : "Rollback (Rollback an Entity)",
    "key" : "ROLLBACK"
  }, {
    "id" : 500,
    "name" : "Undeploy",
    "description" : "Undeploy an Entity",
    "nameWithDescription" : "Undeploy (Undeploy an Entity)",
    "key" : "UNDEPLOY"
  }, {
    "id" : 600,
    "name" : "Update",
    "description" : "Update an Entity",
    "nameWithDescription" : "Update (Update an Entity)",
    "key" : "UPDATE"
  } ]

```

42.4.4. List all the Audit Operation Types

The audit record endpoint lets you get the operation types. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```

GET /audit-records/audit-operation-types HTTP/1.1
Host: localhost:9393

```

HTTP

Example Request

```
$ curl 'http://localhost:9393/audit-records/audit-operation-types' -i -X GET
```

BASH

Response Structure

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 258

```

HTTP

```

[ {
  "id" : 100,
  "name" : "App Registration",
  "key" : "APP_REGISTRATION"
}, {
  "id" : 200,
  "name" : "Schedule",
  "key" : "SCHEDULE"
}, {
  "id" : 300,
  "name" : "Stream",
  "key" : "STREAM"
}, {
  "id" : 400,
  "name" : "Task",
  "key" : "TASK"
} ]

```

42.5. Stream Definitions

The registered applications endpoint provides information about the stream definitions that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [Creating a New Stream Definition](#)
- [List All Stream Definitions](#)
- [List Related Stream Definitions](#)
- [Retrieve Stream Definition Detail](#)
- [Delete a Single Stream Definition](#)
- [Delete All Stream Definitions](#)
- [Deploying Stream Definition](#)
- [Undeploy Stream Definition](#)
- [Undeploy All Stream Definitions](#)

42.5.1. Creating a New Stream Definition

Creating a stream definition is achieved by creating a POST request to the stream definitions endpoint. A curl request for a `ticktock` stream might resemble the following:

```
curl -X POST -d "name=ticktock&definition=time | log" localhost:9393/streams/definitions?deploy=false
```

BASH

A stream definition can also contain additional parameters. For instance, in the example shown under “[Request Structure](#)”, we also provide the date-time format.

The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /streams/definitions HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

name=timelog&definition=time+--format%3D%27YYYY+MM+DD%27+%7C+log&description=Demo+stream+for+testing&deploy=false
```

HTTP

Request Parameters

Parameter	Description

Parameter	Description
name	The name for the created task definitions
definition	The definition for the stream, using Data Flow DSL
description	The description of the stream definition
deploy	If true, the stream is deployed upon creation (default is false)

Example Request

```
$ curl 'http://localhost:9393/streams/definitions' -i -X POST \
  -d 'name=timelog&definition=time--format%3D%27YYYY+MM+DD%27+%7C+log&description=Demo+stream+for+testing&deploy=false'
```

BASH

Response Structure

```
HTTP/1.1 201 Created
Content-Type: application/hal+json
Content-Length: 391

{
  "name" : "timelog",
  "dslText" : "time --format='YYYY MM DD' | log",
  "originalDslText" : "time --format='YYYY MM DD' | log",
  "status" : "unknown",
  "description" : "Demo stream for testing",
  "statusDescription" : "The app or group deployment is not known to the system",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/streams/definitions/timelog"
    }
  }
}
```

HTTP

42.5.2. List All Stream Definitions

The streams endpoint lets you list all the stream definitions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/definitions?page=0&sort=name%2CASC&search=&size=10&search= HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
search	The search string performed on the name (optional)
sort	The sort on the list (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/streams/definitions?page=0&sort=name%2CASC&search=&size=10&search=' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 734

{
  "_embedded" : {
    "streamDefinitionResourceList" : [ {
      "name" : "timelog",
      "dslText" : "time --format='YYYY MM DD' | log",
      "originalDslText" : "time --format='YYYY MM DD' | log",
      "status" : "unknown",
      "description" : "Demo stream for testing",
      "statusDescription" : "The app or group deployment is not known to the system",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/streams/definitions/timelog"
        }
      }
    }
  ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/streams/definitions?page=0&size=10&sort=name,asc"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 1,
  "totalPages" : 1,
  "number" : 0
}
}
```

HTTP

42.5.3. List Related Stream Definitions

The streams endpoint lets you list related stream definitions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/definitions/timelog/related?page=0&sort=name%2CASC&search=&size=10&nested=true&search= HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
nested	Should we recursively findByTaskNameContains for related stream definitions (optional)
page	The zero-based page number (optional)
search	The search string performed on the name (optional)
sort	The sort on the list (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/streams/definitions/timelog/related?page=0&sort=name%2CASC&search=&size=10&nested=true&search=
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 750

{
  "_embedded" : {
    "streamDefinitionResourceList" : [ {
      "name" : "timelog",
      "dslText" : "time --format='YYYY MM DD' | log",
      "originalDslText" : "time --format='YYYY MM DD' | log",
      "status" : "unknown",
      "description" : "Demo stream for testing",
      "statusDescription" : "The app or group deployment is not known to the system",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/streams/definitions/timelog"
        }
      }
    }
  ],
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/streams/definitions/timelog/related?page=0&size=10&sort=name,asc"
    }
  },
  "page" : {
    "size" : 10,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

HTTP

42.5.4. Retrieve Stream Definition Detail

The stream definition endpoint lets you get a single stream definition. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/definitions/timelog HTTP/1.1
Host: localhost:9393
```

HTTP

Path Parameters

/streams/definitions/{name}

Parameter	Description
name	The name of the stream definition to query (required)

Example Request

```
$ curl 'http://localhost:9393/streams/definitions/timelog' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 391

{
  "name" : "timelog",
  "dslText" : "time --format='YYYY MM DD' | log",
  "originalDslText" : "time --format='YYYY MM DD' | log",
  "status" : "unknown",
  "description" : "Demo stream for testing",
  "statusDescription" : "The app or group deployment is not known to the system",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/streams/definitions/timelog"
    }
  }
}
```

HTTP

42.5.5. Delete a Single Stream Definition

The streams endpoint lets you delete a single stream definition. (See also: [Delete All Stream Definitions](#).) The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /streams/definitions/timelog HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/definitions/timelog' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.5.6. Delete All Stream Definitions

The streams endpoint lets you delete all single stream definitions. (See also: [Delete a Single Stream Definition](#).) The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /streams/definitions HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/definitions' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.6. Stream Validation

The stream validation endpoint lets you validate the apps in a stream definition. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

42.6.1. Request Structure

```
GET /streams/validation/timelog HTTP/1.1
Host: localhost:9393
```

HTTP

42.6.2. Path Parameters

/streams/validation/{name}

Parameter	Description
name	The name of a stream definition to be validated (required)

42.6.3. Example Request

```
$ curl 'http://localhost:9393/streams/validation/timelog' -i -X GET
```

BASH

42.6.4. Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 197

{
  "appName" : "timelog",
  "dsl" : "time --format='YYYY MM DD' | log",
  "description" : "Demo stream for testing",
  "appStatuses" : {
    "source:time" : "valid",
```

HTTP

```
"sink:log" : "valid"
}
}
```

42.7. Stream Deployments

The deployment definitions endpoint provides information about the deployments that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [Deploying Stream Definition](#)
- [Undeploy Stream Definition](#)
- [Undeploy All Stream Definitions](#)
- [Update Deployed Stream](#)
- [Rollback Stream Definition](#)
- [Get Manifest](#)
- [Get Deployment History](#)
- [Get Deployment Platforms](#)
- [Scale Stream Definition](#)

42.7.1. Deploying Stream Definition

The stream definition endpoint lets you deploy a single stream definition. Optionally, you can pass application parameters as properties in the request body. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /streams/deployments/timelog HTTP/1.1
Content-Type: application/json
Content-Length: 36
Host: localhost:9393

{"app.time.timestamp.format":"YYYY"}
```

HTTP

/streams/deployments/{timelog}

Parameter	Description
timelog	The name of an existing stream definition (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/timelog' -i -X POST \
-H 'Content-Type: application/json' \
-d '{"app.time.timestamp.format":"YYYY"}'
```

BASH

Response Structure

```
HTTP/1.1 201 Created
```

HTTP

42.7.2. Undeploy Stream Definition

The stream definition endpoint lets you undeploy a single stream definition. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /streams/deployments/timelog HTTP/1.1
Host: localhost:9393
```

HTTP

/streams/deployments/{timelog}

Parameter	Description
timelog	The name of an existing stream definition (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/timelog' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.7.3. Undeploy All Stream Definitions

The stream definition endpoint lets you undeploy all single stream definitions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /streams/deployments HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.7.4. Update Deployed Stream

Thanks to Skipper, you can update deployed streams, and provide additional deployment properties.

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /streams/deployments/update/timelog1 HTTP/1.1
Content-Type: application/json
Content-Length: 196
Host: localhost:9393
```

HTTP

```
{"releaseName": "timelog1", "packageIdentifier": {"repositoryName": "test", "packageName": "timelog1", "packageVersion": "1.0.0"}, "
```

Parameter	Description
timelog1	The name of an existing stream definition (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/update/timelog1' -i -X POST \  
  -H 'Content-Type: application/json' \  
  -d '{"releaseName":"timelog1","packageIdentifier":{"repositoryName":"test","packageName":"timelog1","packageVersion":"1
```

BASH

Response Structure

HTTP/1.1 201 Created

HTTP

42.7.5. Rollback Stream Definition

Rollback the stream to the previous or a specific version of the stream.

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /streams/deployments/rollback/timelog1/1 HTTP/1.1  
Content-Type: application/json  
Host: localhost:9393
```

HTTP

/streams/deployments/rollback/{name}/{version}

Parameter	Description
name	The name of an existing stream definition (required)
version	The version to rollback to

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/rollback/timelog1/1' -i -X POST \
-H 'Content-Type: application/json'
```

BASH

Response Structure

```
HTTP/1.1 201 Created
```

HTTP

42.7.6. Get Manifest

Return a manifest of a released version. For packages with dependencies, the manifest includes the contents of those dependencies.

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/deployments/manifest/timelog1/1 HTTP/1.1
Content-Type: application/json
Host: localhost:9393
```

HTTP

/streams/deployments/manifest/{name}/{version}

Parameter	Description
name	The name of an existing stream definition (required)
version	The version of the stream

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/manifest/timelog1/1' -i -X GET \
-H 'Content-Type: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.7.7. Get Deployment History

Get the stream's deployment history.

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/deployments/history/timelog1 HTTP/1.1
Content-Type: application/json
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/history/timelog1' -i -X GET \
-H 'Content-Type: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 162
```

HTTP

```
[ {
  "name" : null,
  "version" : 0,
  "info" : null,
  "pkg" : null,
  "configValues" : {
    "raw" : null
  },
  "manifest" : null,
  "platformName" : null
} ]
```

42.7.8. Get Deployment Platforms

Retrieve a list of supported deployment platforms.

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/deployments/platform/list HTTP/1.1
Content-Type: application/json
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/platform/list' -i -X GET \
-H 'Content-Type: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 106

[ {
  "id" : null,
  "name" : "default",
  "type" : "local",
  "description" : null,
  "options" : [ ]
} ]
```

HTTP

42.7.9. Scale Stream Definition

The stream definition endpoint lets you scale a single app in a stream definition. Optionally, you can pass application parameters as properties in the request body. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /streams/deployments/scale/timelog/log/instances/1 HTTP/1.1
Content-Type: application/json
Content-Length: 36
Host: localhost:9393

{"app.time.timestamp.format":"YYYY"}
```

HTTP

/streams/deployments/scale/{streamName}/{appName}/instances/{count}

Parameter	Description
streamName	the name of an existing stream definition (required)
appName	in stream application name to scale
count	number of instances for the selected stream application (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/streams/deployments/scale/timelog/log/instances/1' -i -X POST \
-H 'Content-Type: application/json' \
-d '{"app.time.timestamp.format":"YYYY"}'
```

BASH

Response Structure

```
HTTP/1.1 201 Created
```

HTTP

42.8. Task Definitions

The task definitions endpoint provides information about the task definitions that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [Creating a New Task Definition](#)
- [List All Task Definitions](#)
- [Retrieve Task Definition Detail](#)
- [Delete Task Definition](#)

42.8.1. Creating a New Task Definition

The task definition endpoint lets you create a new task definition. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /tasks/definitions HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

name=my-task&definition=timestamp+--format%3D%27YYYY+MM+DD%27&description=Demo+task+definition+for+testing
```

HTTP

Request Parameters

Parameter	Description
name	The name for the created task definition
definition	The definition for the task, using Data Flow DSL

Parameter	Description
description	The description of the task definition

Example Request

```
$ curl 'http://localhost:9393/tasks/definitions' -i -X POST \
  -d 'name=my-task&definition=timestamp+--format%3D%27YYYY+MM+DD%27&description=Demo+task+definition+for+testing'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 309

{
  "name" : "my-task",
  "dslText" : "timestamp --format='YYYY MM DD'",
  "description" : "Demo task definition for testing",
  "composed" : false,
  "lastTaskExecution" : null,
  "status" : "UNKNOWN",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/definitions/my-task"
    }
  }
}
```

HTTP

42.8.2. List All Task Definitions

The task definition endpoint lets you get all task definitions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/definitions?page=0&size=10&sort=taskName%2CASC&search=&search= HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Parameter	Description
search	The search string performed on the name (optional)
sort	The sort on the list (optional)

Example Request

```
$ curl 'http://localhost:9393/tasks/definitions?page=0&size=10&sort=taskName%2CASC&search=&search=' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 652
```

HTTP

```
{
  "_embedded" : {
    "taskDefinitionResourceList" : [ {
      "name" : "my-task",
      "dslText" : "timestamp --format='YYYY MM DD'",
      "description" : "Demo task definition for testing",
      "composed" : false,
      "lastTaskExecution" : null,
      "status" : "UNKNOWN",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/tasks/definitions/my-task"
        }
      }
    }
  ]
},
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/definitions?page=0&size=10&sort=taskName,asc"
    }
  },
  "page" : {
    "size" : 10,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

42.8.3. Retrieve Task Definition Detail

The task definition endpoint lets you get a single task definition. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/definitions/my-task HTTP/1.1
Host: localhost:9393
```

/tasks/definitions/{my-task}

Parameter	Description
my-task	The name of an existing task definition (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/tasks/definitions/my-task' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 309

{
  "name" : "my-task",
  "dslText" : "timestamp --format='YYYY MM DD'",
  "description" : "Demo task definition for testing",
  "composed" : false,
  "lastTaskExecution" : null,
  "status" : "UNKNOWN",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/definitions/my-task"
    }
  }
}
```

HTTP

42.8.4. Delete Task Definition

The task definition endpoint lets you delete a single task definition. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /tasks/definitions/my-task HTTP/1.1
Host: localhost:9393
```

HTTP

/tasks/definitions/{my-task}

Parameter	Description
my-task	The name of an existing task definition (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/tasks/definitions/my-task' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.9. Task Scheduler

The task scheduler endpoint provides information about the task schedules that are registered with the Scheduler Implementation. The following topics provide more details:

- [Creating a New Task Schedule](#)
- [List All Schedules](#)
- [List Filtered Schedules](#)
- [Delete Task Schedule](#)

42.9.1. Creating a New Task Schedule

The task schedule endpoint lets you create a new task schedule. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
POST /tasks/schedules HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

scheduleName=myschedule&taskDefinitionName=mytaskname&properties=scheduler.cron.expression%3D00+22+17+%3F+*&arguments=--foo
```

HTTP

Request Parameters

Parameter	Description
scheduleName	The name for the created schedule
taskDefinitionName	The name of the task definition to be scheduled
properties	the properties that are required to schedule and launch the task
arguments	the command line arguments to be used for launching the task

Example Request

```
$ curl 'http://localhost:9393/tasks/schedules' -i -X POST \
  -d 'scheduleName=myschedule&taskDefinitionName=mytaskname&properties=scheduler.cron.expression%3D00+22+17+%3F+*&argumen
```

BASH

Response Structure

HTTP/1.1 201 Created

HTTP

42.9.2. List All Schedules

The task schedules endpoint lets you get all task schedules. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

GET /tasks/schedules?page=0&size=10 HTTP/1.1
Host: localhost:9393

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/tasks/schedules?page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 587

{
  "_embedded" : {
    "scheduleInfoResourceList" : [ {
      "scheduleName" : "FOO",
      "taskDefinitionName" : "BAR",
      "scheduleProperties" : {
        "scheduler.AAA.spring.cloud.scheduler.cron.expression" : "00 41 17 ? * *"
      },
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/tasks/schedules/FOO"
        }
      }
    } ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/schedules?page=0&size=10"
    }
  },
  "page" : {
    "size" : 10,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

HTTP

42.9.3. List Filtered Schedules

The task schedules endpoint lets you get all task schedules that have the specified task definition name. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/schedules/instances/FOO?page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

/tasks/schedules/instances/{task-definition-name}

Parameter	Description
-----------	-------------

Parameter	Description
task-definition-name	Filter schedules based on the specified task definition (required)

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/tasks/schedules/instances/F00?page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 599

{
  "_embedded" : {
    "scheduleInfoResourceList" : [ {
      "scheduleName" : "F00",
      "taskDefinitionName" : "BAR",
      "scheduleProperties" : {
        "scheduler.AAA.spring.cloud.scheduler.cron.expression" : "00 41 17 ? * *"
      },
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/tasks/schedules/F00"
        }
      }
    } ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/schedules/instances/F00?page=0&size=1"
    }
  },
  "page" : {
    "size" : 1,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

HTTP

42.9.4. Delete Task Schedule

The task schedule endpoint lets you delete a single task schedule. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)

- [Example Request](#)
- [Response Structure](#)

Request Structure

```
DELETE /tasks/schedules/mytestschedule HTTP/1.1
Host: localhost:9393
```

HTTP

/tasks/schedules/{scheduleName}

Parameter	Description
scheduleName	The name of an existing schedule (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/tasks/schedules/mytestschedule' -i -X DELETE
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.10. Task Validation

The task validation endpoint lets you validate the apps in a task definition. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Example Request](#)
- [Response Structure](#)

42.10.1. Request Structure

```
GET /tasks/validation/taskC HTTP/1.1
Host: localhost:9393
```

HTTP

42.10.2. Path Parameters

/tasks/validation/{name}

Parameter	Description
name	The name of a task definition to be validated (required)

42.10.3. Example Request

```
$ curl 'http://localhost:9393/tasks/validation/taskC' -i -X GET
```

BASH

42.10.4. Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 144

{
  "appName" : "taskC",
  "dsl" : "timestamp --format='yyyy MM dd'",
  "description" : "",
  "appStatuses" : {
    "task:taskC" : "valid"
  }
}
```

HTTP

42.11. Task Executions

The task executions endpoint provides information about the task executions that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [Launching a Task](#)
- [Stopping a Task](#)
- [List All Task Executions](#)
- [List All Task Executions With a Specified Task Name](#)
- [Task Execution Detail](#)
- [Delete Task Execution](#)
- [Task Execution Current Count](#)

42.11.1. Launching a Task

Launching a task is done by requesting the creation of a new task execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

2/18/2020Spring Cloud Data Flow Reference GuideHTTP

```
POST /tasks/executions HTTP/1.1
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

name=taskA&properties=app.my-task.foo%3Dbar%2Cdeployer.my-task.something-else%3D3&arguments=--server.port%3D8080+--foo%3Dba
```

Request Parameters

Parameter	Description
name	The name of the task definition to launch
properties	Application and Deployer properties to use while launching
arguments	Command line arguments to pass to the task

Example Request

\$ curl 'http://localhost:9393/tasks/executions' -i -X POST \BASH

-d 'name=taskA&properties=app.my-task.foo%3Dbar%2Cdeployer.my-task.something-else%3D3&arguments=--server.port%3D8080+--

Response Structure

HTTP/1.1 201 CreatedHTTP

Content-Type: application/json

Content-Length: 1

1

42.11.2. Stopping a Task

Stopping a task is done by posting the id of an existing task execution. The following topics provide more details:

- [Request Structure](#)
- [Path Parameters](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

POST /tasks/executions/1 HTTP/1.1HTTP

Host: localhost:9393

Content-Type: application/x-www-form-urlencoded

platform=default

Path Parameters

/tasks/executions/{id}

Parameter	Description
id	The ids of an existing task execution (required)

Request Parameters

Parameter	Description
platform	The platform associated with the task execution(optional)

Example Request

```
$ curl 'http://localhost:9393/tasks/executions/1' -i -X POST \
  -d 'platform=default'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.11.3. List All Task Executions

The task executions endpoint lets you list all task executions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/executions?page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/tasks/executions?page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 2263
```

HTTP

```
{
  "_embedded" : {
    "taskExecutionResourceList" : [ {
      "executionId" : 2,
      "exitCode" : null,
      "taskName" : "taskB",
      "startTime" : null,
      "endTime" : null,
      "exitMessage" : null,
      "arguments" : [ ],
      "jobExecutionIds" : [ ],
      "errorMessage" : null,
      "externalExecutionId" : "taskB-b8f8e8fc-f54d-4336-a85f-c63c936e51f0",
      "parentExecutionId" : null,
      "resourceUrl" : "org.springframework.cloud.task.app:timestamp-task:jar:1.2.0.RELEASE",
      "appProperties" : {
        "timestamp.format" : "yyyy MM dd",
        "spring.datasource.username" : null,
        "spring.cloud.task.name" : "taskB",
        "spring.datasource.url" : null,
        "spring.datasource.driverClassName" : null
      },
      "deploymentProperties" : {
        "app.my-task.foo" : "bar",
        "deployer.my-task.something-else" : "3"
      },
      "taskExecutionStatus" : "UNKNOWN",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/tasks/executions/2"
        }
      }
    }, {
      "executionId" : 1,
      "exitCode" : null,
      "taskName" : "taskA",
      "startTime" : null,
      "endTime" : null,
      "exitMessage" : null,
      "arguments" : [ ],
      "jobExecutionIds" : [ ],
      "errorMessage" : null,
      "externalExecutionId" : "taskA-55855539-a799-4f65-bd3c-7ea679c2093d",
      "parentExecutionId" : null,
      "resourceUrl" : "org.springframework.cloud.task.app:timestamp-task:jar:1.2.0.RELEASE",
      "appProperties" : {
        "timestamp.format" : "yyyy MM dd",
        "spring.datasource.username" : null,
        "spring.cloud.task.name" : "taskA",
        "spring.datasource.url" : null,
        "spring.datasource.driverClassName" : null
      },
      "deploymentProperties" : {
        "app.my-task.foo" : "bar",
        "deployer.my-task.something-else" : "3"
      },
      "taskExecutionStatus" : "UNKNOWN",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/tasks/executions/1"
        }
      }
    }
  ]
}
```

```
    }
  } ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/tasks/executions?page=0&size=10"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 2,
  "totalPages" : 1,
  "number" : 0
}
}
```

42.11.4. List All Task Executions With a Specified Task Name

The task executions endpoint lets you list task executions with a specified task name. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/executions?name=taskB&page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)
name	The name associated with the task execution

Example Request

```
$ curl 'http://localhost:9393/tasks/executions?name=taskB&page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 1268

{
```

HTTP

```

    "_embedded" : {
      "taskExecutionResourceList" : [ {
        "executionId" : 2,
        "exitCode" : null,
        "taskName" : "taskB",
        "startTime" : null,
        "endTime" : null,
        "exitMessage" : null,
        "arguments" : [ ],
        "jobExecutionIds" : [ ],
        "errorMessage" : null,
        "externalExecutionId" : "taskB-b8f8e8fc-f54d-4336-a85f-c63c936e51f0",
        "parentExecutionId" : null,
        "resourceUrl" : "org.springframework.cloud.task.app:timestamp-task:jar:1.2.0.RELEASE",
        "appProperties" : {
          "timestamp.format" : "yyyy MM dd",
          "spring.datasource.username" : null,
          "spring.cloud.task.name" : "taskB",
          "spring.datasource.url" : null,
          "spring.datasource.driverClassName" : null
        },
        "deploymentProperties" : {
          "app.my-task.foo" : "bar",
          "deployer.my-task.something-else" : "3"
        },
        "taskExecutionStatus" : "UNKNOWN",
        "_links" : {
          "self" : {
            "href" : "http://localhost:9393/tasks/executions/2"
          }
        }
      } ]
    },
    "_links" : {
      "self" : {
        "href" : "http://localhost:9393/tasks/executions?page=0&size=10"
      }
    },
    "page" : {
      "size" : 10,
      "totalElements" : 1,
      "totalPages" : 1,
      "number" : 0
    }
  }
}

```

42.11.5. Task Execution Detail

The task executions endpoint lets you get the details about a task execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```

GET /tasks/executions/1 HTTP/1.1
Host: localhost:9393

```

HTTP

/tasks/executions/{id}

Parameter	Description
id	The id of an existing task execution (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/tasks/executions/1' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 873
```

HTTP

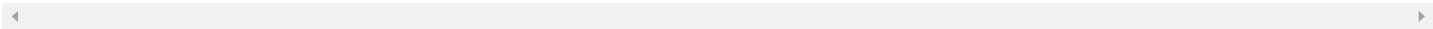
```
{
  "executionId" : 1,
  "exitCode" : null,
  "taskName" : "taskA",
  "startTime" : null,
  "endTime" : null,
  "exitMessage" : null,
  "arguments" : [ ],
  "jobExecutionIds" : [ ],
  "errorMessage" : null,
  "externalExecutionId" : "taskA-55855539-a799-4f65-bd3c-7ea679c2093d",
  "parentExecutionId" : null,
  "resourceUrl" : "org.springframework.cloud.task.app:timestamp-task:jar:1.2.0.RELEASE",
  "appProperties" : {
    "timestamp.format" : "yyyy MM dd",
    "spring.datasource.username" : null,
    "spring.cloud.task.name" : "taskA",
    "spring.datasource.url" : null,
    "spring.datasource.driverClassName" : null
  },
  "deploymentProperties" : {
    "app.my-task.foo" : "bar",
    "deployer.my-task.something-else" : "3"
  },
  "taskExecutionStatus" : "UNKNOWN",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/tasks/executions/1"
    }
  }
}
```

42.11.6. Delete Task Execution

The task executions endpoint lets you:

- Clean up resources used to deploy the task
- Remove relevant task data as well as possibly associated Spring Batch job data from the persistence store

The cleanup implementation (first option) is platform specific. Both operations can be triggered at once or separately.



The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Please refer to the following section in regards to [Deleting Task Execution Data](#).

Request Structure

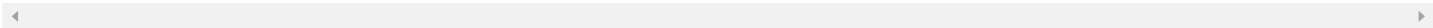
```
DELETE /tasks/executions/1,2?action=CLEANUP,REMOVE_DATA HTTP/1.1
Host: localhost:9393
```

HTTP

/tasks/executions/{ids}

Parameter	Description
ids	Providing 2 comma separated task execution id values.

You must ensure to only provide task execution ids that actually exist. Otherwise a 404 (Not Found) HTTP status is returned. In case of submitting multiple task execution ids, the invalidity of a single task execution id will cause the entire request to fail, without performing any operation.



Request Parameters

This endpoint supports 1 optional request parameter named **action**. It is an enumeration and supports the following 2 values:

- CLEANUP
- REMOVE_DATA

Parameter	Description
action	Using both actions CLEANUP and REMOVE_DATA simultaneously.

Example Request

```
$ curl 'http://localhost:9393/tasks/executions/1,2?action=CLEANUP,REMOVE_DATA' -i -X DELETE
```

BASH

Response Structure

HTTP/1.1 200 OK

HTTP

42.11.7. Deleting Task Execution Data

Not only can you clean up resources that were used to deploy tasks but you can also delete the data associated with task executions from the underlying persistence store. Also, if a task execution is associated with one or more batch job executions, these will be removed as well.

The following example illustrates how a request can be made using multiple task execution ids and multiple actions:

```
$ curl 'http://localhost:9393/tasks/executions/1,2?action=CLEANUP,REMOVE_DATA' -i -X DELETE
```

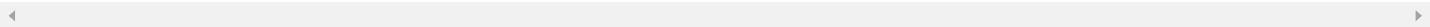
BASH

/tasks/executions/{ids}

Parameter	Description
ids	Providing 2 comma separated task execution id values.

Parameter	Description
action	Using both actions CLEANUP and REMOVE_DATA simultaneously.

When deleting data from the persistence store using the `REMOVE_DATA` action parameter, you must ensure to provide only task execution ids that represent parent task executions. When providing child task executions (Executed as part of a composed task), then a `400` (Bad Request) HTTP status is returned.



42.11.8. Task Execution Current Count

The task executions current endpoint lets you retrieve the current number of running executions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/executions/current HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/tasks/executions/current' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 111
```

HTTP

```
[ {
  "name" : "default",
  "type" : "Local",
  "maximumTaskExecutions" : 20,
  "runningExecutionCount" : 0
} ]
```

42.12. Job Executions

The job executions endpoint provides information about the job executions that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [List All Job Executions](#)
- [List All Job Executions Without Step Executions Included](#)
- [List All Job Executions With a Specified Job Name](#)
- [List All Job Executions With a Specified Job Name Without Step Executions Included](#)
- [Job Execution Detail](#)
- [Stop Job Execution](#)
- [Restart Job Execution](#)

42.12.1. List All Job Executions

The job executions endpoint lets you list all job executions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

HTTP

```
GET /jobs/executions?page=0&size=10 HTTP/1.1
Host: localhost:9393
```

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

BASH

```
$ curl 'http://localhost:9393/jobs/executions?page=0&size=10' -i -X GET
```

Response Structure

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 3965
```

```
{
  "_embedded" : {
    "jobExecutionResourceList" : [ {
      "executionId" : 2,
      "stepExecutionCount" : 0,
      "jobId" : 2,
      "taskExecutionId" : 2,
      "name" : "DOCJOB_1",
      "startDate" : "2020-01-31",
      "startTime" : "12:17:48",
      "duration" : "00:00:01",
      "jobExecution" : {
        "id" : 2,
        "version" : 1,
        "jobParameters" : {
          "parameters" : {
            "-spring.cloud.data.flow.platformname" : {
              "identifying" : true,
              "value" : "default",
              "type" : "STRING"
            }
          }
        },
        "empty" : false
      },
      "jobInstance" : {
        "id" : 2,
        "version" : null,
        "jobName" : "DOCJOB_1",
        "instanceId" : 2
      },
      "stepExecutions" : [ ],
      "status" : "STOPPED",
      "startTime" : "2020-01-31T12:17:48.927+0000",
      "createTime" : "2020-01-31T12:17:48.924+0000",
      "endTime" : null,
      "lastUpdated" : "2020-01-31T12:17:48.927+0000",
      "exitStatus" : {
        "exitCode" : "UNKNOWN",
        "exitDescription" : ""
      }
    }
  ]
}
```

```

    "running" : true
  },
  "executionContext" : {
    "dirty" : false,
    "empty" : true,
    "values" : [ ]
  },
  "failureExceptions" : [ ],
  "jobConfigurationName" : null,
  "running" : true,
  "jobId" : 2,
  "stopping" : false,
  "allFailureExceptions" : [ ]
},
"jobParameters" : {
  "-spring.cloud.data.flow.platformname" : "default"
},
"jobParametersString" : "-spring.cloud.data.flow.platformname=default",
"restartable" : true,
"abandonable" : true,
"stoppable" : false,
"defined" : true,
"timeZone" : "UTC",
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions/2"
  }
}
}, {
  "executionId" : 1,
  "stepExecutionCount" : 0,
  "jobId" : 1,
  "taskExecutionId" : 1,
  "name" : "DOCJOB",
  "startDate" : "2020-01-31",
  "startTime" : "12:17:48",
  "duration" : "00:00:01",
  "jobExecution" : {
    "id" : 1,
    "version" : 2,
    "jobParameters" : {
      "parameters" : {
        "-spring.cloud.data.flow.platformname" : {
          "identifying" : true,
          "value" : "default",
          "type" : "STRING"
        }
      }
    },
    "empty" : false
  },
  "jobInstance" : {
    "id" : 1,
    "version" : null,
    "jobName" : "DOCJOB",
    "instanceId" : 1
  },
  "stepExecutions" : [ ],
  "status" : "STOPPING",
  "startTime" : "2020-01-31T12:17:48.913+0000",
  "createTime" : "2020-01-31T12:17:48.902+0000",
  "endTime" : null,
  "lastUpdated" : "2020-01-31T12:17:49.242+0000",
  "exitStatus" : {
    "exitCode" : "UNKNOWN",
    "exitDescription" : "",
    "running" : true
  },
  "executionContext" : {
    "dirty" : false,
    "empty" : true,
    "values" : [ ]
  },
  "failureExceptions" : [ ],
  "jobConfigurationName" : null,
  "running" : true,
  "jobId" : 1,
  "stopping" : true,

```

```
    "allFailureExceptions" : [ ]
  },
  "jobParameters" : {
    "-spring.cloud.data.flow.platformname" : "default"
  },
  "jobParametersString" : "-spring.cloud.data.flow.platformname=default",
  "restartable" : false,
  "abandonable" : true,
  "stoppable" : false,
  "defined" : false,
  "timeZone" : "UTC",
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/jobs/executions/1"
    }
  }
} ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions?page=0&size=10"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 2,
  "totalPages" : 1,
  "number" : 0
}
}
```

42.12.2. List All Job Executions Without Step Executions Included

The job executions endpoint lets you list all job executions without step executions included. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/thinexecutions?page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/jobs/thinexecutions?page=0&size=10' -i -X GET
```

BASH

Response Structure

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 1823

{
  "_embedded" : {
    "jobExecutionThinResourceList" : [ {
      "executionId" : 2,
      "stepExecutionCount" : 0,
      "jobId" : 2,
      "taskExecutionId" : 2,
      "instanceId" : 2,
      "name" : "DOCJOB_1",
      "startDate" : "2020-01-31",
      "startTime" : "12:17:48",
      "startDateTime" : "2020-01-31T12:17:48.927+0000",
      "duration" : "00:00:00",
      "jobParameters" : {
        "-spring.cloud.data.flow.platformname" : "default"
      },
      "jobParametersString" : "-spring.cloud.data.flow.platformname=default",
      "restartable" : true,
      "abandonable" : true,
      "stoppable" : false,
      "defined" : true,
      "timeZone" : "UTC",
      "status" : "STOPPED",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/jobs/thinexecutions/2"
        }
      }
    }, {
      "executionId" : 1,
      "stepExecutionCount" : 0,
      "jobId" : 1,
      "taskExecutionId" : 1,
      "instanceId" : 1,
      "name" : "DOCJOB",
      "startDate" : "2020-01-31",
      "startTime" : "12:17:48",
      "startDateTime" : "2020-01-31T12:17:48.913+0000",
      "duration" : "00:00:00",
      "jobParameters" : {
        "-spring.cloud.data.flow.platformname" : "default"
      },
      "jobParametersString" : "-spring.cloud.data.flow.platformname=default",
      "restartable" : false,
      "abandonable" : false,
      "stoppable" : true,
      "defined" : false,
      "timeZone" : "UTC",
      "status" : "STARTED",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/jobs/thinexecutions/1"
        }
      }
    }
  ]
},
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/jobs/thinexecutions?page=0&size=10"
    }
  },
  "page" : {
    "size" : 10,
    "totalElements" : 2,
    "totalPages" : 1,
    "number" : 0
  }
}

```

42.12.3. List All Job Executions With a Specified Job Name

The job executions endpoint lets you list all job executions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/executions?name=DOCJOB&page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)
name	The name associated with the job execution

Example Request

```
$ curl 'http://localhost:9393/jobs/executions?name=DOCJOB&page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 2117

{
  "_embedded" : {
    "jobExecutionResourceList" : [ {
      "executionId" : 1,
      "stepExecutionCount" : 0,
      "jobId" : 1,
      "taskExecutionId" : 1,
      "name" : "DOCJOB",
      "startDate" : "2020-01-31",
      "startTime" : "12:17:48",
      "duration" : "00:00:01",
      "jobExecution" : {
        "id" : 1,
        "version" : 2,
        "jobParameters" : {
          "parameters" : {
```

HTTP


```

        "-spring.cloud.data.flow.platformname" : {
            "identifying" : true,
            "value" : "default",
            "type" : "STRING"
        }
    },
    "empty" : false
},
"jobInstance" : {
    "id" : 1,
    "version" : null,
    "jobName" : "DOCJOB",
    "instanceId" : 1
},
"stepExecutions" : [ ],
"status" : "STOPPING",
"startTime" : "2020-01-31T12:17:48.913+0000",
"createTime" : "2020-01-31T12:17:48.902+0000",
"endTime" : null,
"lastUpdated" : "2020-01-31T12:17:49.242+0000",
"exitStatus" : {
    "exitCode" : "UNKNOWN",
    "exitDescription" : "",
    "running" : true
},
"executionContext" : {
    "dirty" : false,
    "empty" : true,
    "values" : [ ]
},
"failureExceptions" : [ ],
"jobConfigurationName" : null,
"running" : true,
"jobId" : 1,
"stopping" : true,
"allFailureExceptions" : [ ]
},
"jobParameters" : {
    "-spring.cloud.data.flow.platformname" : "default"
},
"jobParametersString" : "-spring.cloud.data.flow.platformname=default",
"restartable" : false,
"abandonable" : true,
"stoppable" : false,
"defined" : false,
"timeZone" : "UTC",
"_links" : {
    "self" : {
        "href" : "http://localhost:9393/jobs/executions/1"
    }
}
} ]
},
"_links" : {
    "self" : {
        "href" : "http://localhost:9393/jobs/executions?page=0&size=10"
    }
},
"page" : {
    "size" : 10,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
}
}

```

42.12.4. List All Job Executions With a Specified Job Name Without Step Executions Included

The job executions endpoint lets you list all job executions. The following topics provide more details:

- [Request Structure](#)

- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/thinexecutions?name=DOCJOB&page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)
name	The name associated with the job execution

Example Request

```
$ curl 'http://localhost:9393/jobs/thinexecutions?name=DOCJOB&page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 1052

{
  "_embedded" : {
    "jobExecutionThinResourceList" : [ {
      "executionId" : 1,
      "stepExecutionCount" : 0,
      "jobId" : 1,
      "taskExecutionId" : 1,
      "instanceId" : 1,
      "name" : "DOCJOB",
      "startDate" : "2020-01-31",
      "startTime" : "12:17:48",
      "startDateTime" : "2020-01-31T12:17:48.913+0000",
      "duration" : "00:00:01",
      "jobParameters" : {
        "-spring.cloud.data.flow.platformname" : "default"
      },
      "jobParametersString" : "-spring.cloud.data.flow.platformname=default",
      "restartable" : false,
      "abandonable" : true,
      "stoppable" : false,
      "defined" : false,
      "timeZone" : "UTC",
      "status" : "STOPPING",
      "_links" : {
        "self" : {
          "href" : "http://localhost:9393/jobs/thinexecutions/1"
        }
      }
    }
  ]
}
```

HTTP

```
    },
    "_links" : {
      "self" : {
        "href" : "http://localhost:9393/jobs/thinexecutions?page=0&size=10"
      }
    },
    "page" : {
      "size" : 10,
      "totalElements" : 1,
      "totalPages" : 1,
      "number" : 0
    }
  }
}
```

42.12.5. Job Execution Detail

The job executions endpoint lets you get the details about a job execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/executions/2 HTTP/1.1
Host: localhost:9393
```

HTTP

/jobs/executions/{id}

Parameter	Description
id	The id of an existing job execution (required)

Request Parameters

There are no request parameter for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/jobs/executions/2' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 1583

{
  "executionId" : 2,
  "stepExecutionCount" : 0,
  "jobId" : 2,
  "taskExecutionId" : 2,
```

HTTP

```

"name" : "DOCJOB_1",
"startDate" : "2020-01-31",
"startTime" : "12:17:48",
"duration" : "00:00:01",
"jobExecution" : {
  "id" : 2,
  "version" : 1,
  "jobParameters" : {
    "parameters" : {
      "-spring.cloud.data.flow.platformname" : {
        "identifying" : true,
        "value" : "default",
        "type" : "STRING"
      }
    }
  },
  "empty" : false
},
"jobInstance" : {
  "id" : 2,
  "version" : 0,
  "jobName" : "DOCJOB_1",
  "instanceId" : 2
},
"stepExecutions" : [ ],
"status" : "STOPPED",
"startTime" : "2020-01-31T12:17:48.927+0000",
"createTime" : "2020-01-31T12:17:48.924+0000",
"endTime" : null,
"lastUpdated" : "2020-01-31T12:17:48.927+0000",
"exitStatus" : {
  "exitCode" : "UNKNOWN",
  "exitDescription" : "",
  "running" : true
},
"executionContext" : {
  "dirty" : false,
  "empty" : true,
  "values" : [ ]
},
"failureExceptions" : [ ],
"jobConfigurationName" : null,
"running" : true,
"jobId" : 2,
"stopping" : false,
"allFailureExceptions" : [ ]
},
"jobParameters" : {
  "-spring.cloud.data.flow.platformname" : "default"
},
"jobParametersString" : "-spring.cloud.data.flow.platformname=default",
"restartable" : true,
"abandonable" : true,
"stoppable" : false,
"defined" : true,
"timeZone" : "UTC",
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions/2"
  }
}
}
}

```

42.12.6. Stop Job Execution

The job executions endpoint lets you stop a job execution. The following topics provide more details:

- [Request structure](#)
- [Request parameters](#)
- [Example request](#)

- [Response structure](#)

Request structure

```
PUT /jobs/executions/1 HTTP/1.1
Accept: application/json
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded

stop=true
```

HTTP

/jobs/executions/{id}

Parameter	Description
id	The id of an existing job execution (required)

Request parameters

Parameter	Description
stop	Sends signal to stop the job if set to true

Example request

```
$ curl 'http://localhost:9393/jobs/executions/1' -i -X PUT \
-H 'Accept: application/json' \
-d 'stop=true'
```

BASH

Response structure

```
HTTP/1.1 200 OK
```

HTTP

42.12.7. Restart Job Execution

The job executions endpoint lets you restart a job execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
PUT /jobs/executions/2 HTTP/1.1
Accept: application/json
Host: localhost:9393
Content-Type: application/x-www-form-urlencoded
```

HTTP

```
restart=true
```

/jobs/executions/{id}

Parameter	Description
id	The id of an existing job execution (required)

Request Parameters

Parameter	Description
restart	Sends signal to restart the job if set to true

Example Request

```
$ curl 'http://localhost:9393/jobs/executions/2' -i -X PUT \
-H 'Accept: application/json' \
-d 'restart=true'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
```

HTTP

42.13. Job Instances

The job instances endpoint provides information about the job instances that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [List All Job Instances](#)
- [Job Instance Detail](#)

42.13.1. List All Job Instances

The job instances endpoint lets you list all job instances. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/instances?name=DOCJOB&page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)
name	The name associated with the job instance

Example Request

```
$ curl 'http://localhost:9393/jobs/instances?name=DOCJOB&page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 2014

{
  "_embedded" : {
    "jobInstanceResourceList" : [ {
      "jobName" : "DOCJOB",
      "jobInstanceId" : 1,
      "jobExecutions" : [ {
        "executionId" : 1,
        "stepExecutionCount" : 0,
        "jobId" : 1,
        "taskExecutionId" : 1,
        "name" : "DOCJOB",
        "startDate" : "2020-01-31",
        "startTime" : "12:17:34",
        "duration" : "00:00:00",
        "jobExecution" : {
          "id" : 1,
          "version" : 1,
          "jobParameters" : {
            "parameters" : { },
            "empty" : true
          },
          "jobInstance" : {
            "id" : 1,
            "version" : 0,
            "jobName" : "DOCJOB",
            "instanceId" : 1
          },
          "stepExecutions" : [ ],
          "status" : "STARTED",
          "startTime" : "2020-01-31T12:17:34.585+0000",
          "createTime" : "2020-01-31T12:17:34.575+0000",
          "endTime" : null,
          "lastUpdated" : "2020-01-31T12:17:34.585+0000",
          "exitStatus" : {
            "exitCode" : "UNKNOWN",
            "exitDescription" : "",
            "running" : true
          },
          "executionContext" : {
            "dirty" : false,
            "empty" : true,
            "values" : [ ]
          }
        }
      ]
    }
  }
}
```

HTTP

```
    "failureExceptions" : [ ],
    "jobConfigurationName" : null,
    "running" : true,
    "allFailureExceptions" : [ ],
    "jobId" : 1,
    "stopping" : false
  },
  "jobParameters" : { },
  "jobParametersString" : "",
  "restartable" : false,
  "abandonable" : false,
  "stoppable" : true,
  "defined" : false,
  "timeZone" : "UTC"
} ],
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/instances/1"
  }
}
} ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/instances?page=0&size=10"
  }
}
},
"page" : {
  "size" : 10,
  "totalElements" : 1,
  "totalPages" : 1,
  "number" : 0
}
}
```

42.13.2. Job Instance Detail

The job instances endpoint lets you list all job instances. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/instances/1 HTTP/1.1
Host: localhost:9393
```

HTTP

/jobs/instances/{id}

Parameter	Description
id	The id of an existing job instance (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/jobs/instances/1' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 1499
```

HTTP

```
{
  "jobName" : "DOCJOB",
  "jobInstanceId" : 1,
  "jobExecutions" : [ {
    "executionId" : 1,
    "stepExecutionCount" : 0,
    "jobId" : 1,
    "taskExecutionId" : 1,
    "name" : "DOCJOB",
    "startDate" : "2020-01-31",
    "startTime" : "12:17:34",
    "duration" : "00:00:00",
    "jobExecution" : {
      "id" : 1,
      "version" : 1,
      "jobParameters" : {
        "parameters" : { },
        "empty" : true
      },
      "jobInstance" : {
        "id" : 1,
        "version" : 0,
        "jobName" : "DOCJOB",
        "instanceId" : 1
      },
      "stepExecutions" : [ ],
      "status" : "STARTED",
      "startTime" : "2020-01-31T12:17:34.585+0000",
      "createTime" : "2020-01-31T12:17:34.575+0000",
      "endTime" : null,
      "lastUpdated" : "2020-01-31T12:17:34.585+0000",
      "exitStatus" : {
        "exitCode" : "UNKNOWN",
        "exitDescription" : "",
        "running" : true
      },
      "executionContext" : {
        "dirty" : false,
        "empty" : true,
        "values" : [ ]
      },
      "failureExceptions" : [ ],
      "jobConfigurationName" : null,
      "running" : true,
      "allFailureExceptions" : [ ],
      "jobId" : 1,
      "stopping" : false
    },
    "jobParameters" : { },
    "jobParametersString" : "",
    "restartable" : false,
    "abandonable" : false,
    "stoppable" : true,
    "defined" : false,
    "timeZone" : "UTC"
  } ],
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/jobs/instances/1"
    }
  }
}
```

42.14. Job Step Executions

The job step executions endpoint provides information about the job step executions that are registered with the Spring Cloud Data Flow server. The following topics provide more details:

- [List All Step Executions For a Job Execution](#)
- [Job Step Execution Detail](#)
- [Job Step Execution Progress](#)

42.14.1. List All Step Executions For a Job Execution

The job step executions endpoint lets you list all job step executions. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/executions/1/steps?page=0&size=10 HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
page	The zero-based page number (optional)
size	The requested page size (optional)

Example Request

```
$ curl 'http://localhost:9393/jobs/executions/1/steps?page=0&size=10' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 1677

{
  "_embedded" : {
    "stepExecutionResourceList" : [ {
```

HTTP

```

"jobExecutionId" : 1,
"stepExecution" : {
  "id" : 1,
  "version" : 0,
  "stepName" : "DOCJOB_STEP",
  "status" : "STARTING",
  "readCount" : 0,
  "writeCount" : 0,
  "commitCount" : 0,
  "rollbackCount" : 0,
  "readSkipCount" : 0,
  "processSkipCount" : 0,
  "writeSkipCount" : 0,
  "startTime" : "2020-01-31T12:16:50.742+0000",
  "endTime" : null,
  "lastUpdated" : "2020-01-31T12:16:50.742+0000",
  "executionContext" : {
    "dirty" : false,
    "empty" : true,
    "values" : [ ]
  },
  "exitStatus" : {
    "exitCode" : "EXECUTING",
    "exitDescription" : "",
    "running" : true
  },
  "terminateOnly" : false,
  "filterCount" : 0,
  "failureExceptions" : [ ],
  "jobParameters" : {
    "parameters" : { },
    "empty" : true
  },
  "jobExecutionId" : 1,
  "skipCount" : 0,
  "summary" : "StepExecution: id=1, version=0, name=DOCJOB_STEP, status=STARTING, exitStatus=EXECUTING, readCount=0,
},
"stepType" : "",
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions/1/steps/1"
  }
}
} ]
},
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions/1/steps?page=0&size=10"
  }
},
"page" : {
  "size" : 10,
  "totalElements" : 1,
  "totalPages" : 1,
  "number" : 0
}
}

```

42.14.2. Job Step Execution Detail

The job step executions endpoint lets you get details about a job step execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/executions/1/steps/1 HTTP/1.1
Host: localhost:9393
```

HTTP

/jobs/executions/{id}/steps/{stepid}

Parameter	Description
id	The id of an existing job execution (required)
stepid	The id of an existing step execution for a specific job execution (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/jobs/executions/1/steps/1' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
Content-Length: 1219

{
  "jobExecutionId" : 1,
  "stepExecution" : {
    "id" : 1,
    "version" : 0,
    "stepName" : "DOCJOB_STEP",
    "status" : "STARTING",
    "readCount" : 0,
    "writeCount" : 0,
    "commitCount" : 0,
    "rollbackCount" : 0,
    "readSkipCount" : 0,
    "processSkipCount" : 0,
    "writeSkipCount" : 0,
    "startTime" : "2020-01-31T12:16:50.742+0000",
    "endTime" : null,
    "lastUpdated" : "2020-01-31T12:16:50.742+0000",
    "executionContext" : {
      "dirty" : false,
      "empty" : true,
      "values" : [ ]
    },
    "exitStatus" : {
      "exitCode" : "EXECUTING",
      "exitDescription" : "",
      "running" : true
    },
    "terminateOnly" : false,
    "filterCount" : 0,
    "failureExceptions" : [ ],
    "jobParameters" : {
      "parameters" : { }
    }
  }
}
```

HTTP

2/18/2020Spring Cloud Data Flow Reference Guide

```
"empty" : true
},
"jobExecutionId" : 1,
"skipCount" : 0,
"summary" : "StepExecution: id=1, version=0, name=DOCJOB_STEP, status=STARTING, exitStatus=EXECUTING, readCount=0, filt
},
"stepType" : "",
"_links" : {
  "self" : {
    "href" : "http://localhost:9393/jobs/executions/1/steps/1"
  }
}
}
```

42.14.3. Job Step Execution Progress

The job step executions endpoint lets you get details about the progress of a job step execution. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /jobs/executions/1/steps/1/progress HTTP/1.1
Host: localhost:9393
```

HTTP

/jobs/executions/{id}/steps/{stepid}/progress

Parameter	Description
id	The id of an existing job execution (required)
stepid	The id of an existing step execution for a specific job execution (required)

Request Parameters

There are no request parameters for this endpoint.

Example Request

```
$ curl 'http://localhost:9393/jobs/executions/1/steps/1/progress' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
```

HTTP

Content-Length: 2722

```

{
  "stepExecution" : {
    "id" : 1,
    "version" : 0,
    "stepName" : "DOCJOB_STEP",
    "status" : "STARTING",
    "readCount" : 0,
    "writeCount" : 0,
    "commitCount" : 0,
    "rollbackCount" : 0,
    "readSkipCount" : 0,
    "processSkipCount" : 0,
    "writeSkipCount" : 0,
    "startTime" : "2020-01-31T12:16:50.742+0000",
    "endTime" : null,
    "lastUpdated" : "2020-01-31T12:16:50.742+0000",
    "executionContext" : {
      "dirty" : false,
      "empty" : true,
      "values" : [ ]
    },
    "exitStatus" : {
      "exitCode" : "EXECUTING",
      "exitDescription" : "",
      "running" : true
    },
    "terminateOnly" : false,
    "filterCount" : 0,
    "failureExceptions" : [ ],
    "jobParameters" : {
      "parameters" : { },
      "empty" : true
    },
    "jobExecutionId" : 1,
    "skipCount" : 0,
    "summary" : "StepExecution: id=1, version=0, name=DOCJOB_STEP, status=STARTING, exitStatus=EXECUTING, readCount=0, filt
  },
  "stepExecutionHistory" : {
    "stepName" : "DOCJOB_STEP",
    "count" : 0,
    "commitCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "rollbackCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "readCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "writeCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "filterCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    }
  }
}

```

```

    "readSkipCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "writeSkipCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "processSkipCount" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "duration" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    },
    "durationPerRead" : {
      "count" : 0,
      "min" : 0.0,
      "max" : 0.0,
      "mean" : 0.0,
      "standardDeviation" : 0.0
    }
  },
  "percentageComplete" : 0.5,
  "finished" : false,
  "duration" : 335.0,
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/jobs/executions/1/steps/1"
    }
  }
}

```

42.15. Runtime Information about Applications

It is possible to get information about running apps known to the system, either globally or individually. The following topics provide more details:

- [Listing All Applications at Runtime](#)
- [Querying All Instances of a Single App](#)
- [Querying a Single Instance of a Single App](#)

42.15.1. Listing All Applications at Runtime

To retrieve information about all instances of all apps, query the `/runtime/apps` endpoint by using `GET`. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)

- [Response Structure](#)

Request Structure

```
GET /runtime/apps HTTP/1.1
Accept: application/json
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/runtime/apps' -i -X GET \
-H 'Accept: application/json'
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 209
```

HTTP

```
{
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/runtime/apps?page=0&size=20"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 0,
    "totalPages" : 0,
    "number" : 0
  }
}
```

42.15.2. Querying All Instances of a Single App

To retrieve information about all instances of a particular app, query the `/runtime/apps/<appId>/instances` endpoint by using `GET`. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /runtime/apps HTTP/1.1
Accept: application/json
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/runtime/apps' -i -X GET \
-H 'Accept: application/json'
```

BASH

Response Structure

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 209

{
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/runtime/apps?page=0&size=20"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 0,
    "totalPages" : 0,
    "number" : 0
  }
}

```

HTTP

42.15.3. Querying a Single Instance of a Single App

Finally, to retrieve information about a particular instance of a particular app, query the `/runtime/apps/<appId>/instances/<instanceId>` endpoint by using `GET`. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```

GET /runtime/apps HTTP/1.1
Accept: application/json
Host: localhost:9393

```

HTTP

Example Request

```

$ curl 'http://localhost:9393/runtime/apps' -i -X GET \
  -H 'Accept: application/json'

```

BASH

Response Structure

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 209

{
  "_links" : {
    "self" : {
      "href" : "http://localhost:9393/runtime/apps?page=0&size=20"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 0,
    "totalPages" : 0,
  }
}

```

HTTP

```

    "number" : 0
  }
}

```

42.16. Stream Logs

It is possible to get the application logs of the stream for the entire stream or a specific application inside the stream. The following topics provide more details:

- [Get the applications' logs by the stream name](#)
- [Get the logs of a specific application from the stream](#)

42.16.1. Get the applications' logs by the stream name

Use the HTTP GET method with the `/streams/logs/<streamName>` REST endpoint to retrieve all the applications' logs for the given stream name. The following topics provide more details:

- [Request Structure](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```

GET /streams/logs/ticktock HTTP/1.1
Host: localhost:9393

```

HTTP

Example Request

```
$ curl 'http://localhost:9393/streams/logs/ticktock' -i -X GET
```

BASH

Response Structure

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 93

```

HTTP

```

{
  "logs" : {
    "ticktock-time-v1" : "Logs-time",
    "ticktock-log-v1" : "Logs-log"
  }
}

```

42.16.2. Get the logs of a specific application from the stream

To retrieve the logs of a specific application from the stream, query the `/streams/logs/<streamName>/<appName>` endpoint using the GET HTTP method. The following topics provide more details:

- [Request Structure](#)

- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /streams/logs/ticktock/ticktock-log-v1 HTTP/1.1
Host: localhost:9393
```

HTTP

Example Request

```
$ curl 'http://localhost:9393/streams/logs/ticktock/ticktock-log-v1' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 55

{
  "logs" : {
    "ticktock-log-v1" : "Logs-log"
  }
}
```

HTTP

42.17. Task Logs

It is possible to get the task execution log for a specific task execution.

The following topic provides more details:

- [Get the task execution log](#)

42.17.1. Get the task execution log

To retrieve the logs of the task execution, query the `/tasks/logs/<ExternalTaskExecutionId>` endpoint using the HTTP `GET` method.. The following topics provide more details:

- [Request Structure](#)
- [Request Parameters](#)
- [Example Request](#)
- [Response Structure](#)

Request Structure

```
GET /tasks/logs/taskA-1aa02294-9011-4e51-bde3-06d888a54c8a?platformName=default HTTP/1.1
Host: localhost:9393
```

HTTP

Request Parameters

Parameter	Description
platformName	The name of the platform the task is launched.

Example Request

```
$ curl 'http://localhost:9393/tasks/logs/taskA-1aa02294-9011-4e51-bde3-06d888a54c8a?platformName=default' -i -X GET
```

BASH

Response Structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 10059

"2020-01-31 12:13:33.783 INFO 26257 --- [main] s.c.a.AnnotationConfigApplicationContext : Refreshing org.spring
```

HTTP

Appendices

Having trouble with Spring Cloud Data Flow, We'd like to help!

- Ask a question - we monitor [stackoverflow.com](#) for questions tagged with `spring-cloud-dataflow`.
- Report bugs with Spring Cloud Data Flow at [github.com/spring-cloud/spring-cloud-dataflow/issues](#).

Appendix A: Data Flow Template

As described in the previous chapter, Spring Cloud Data Flow's functionality is completely exposed through REST endpoints. While you can use those endpoints directly, Spring Cloud Data Flow also provides a Java-based API, which makes using those REST endpoints even easier.

The central entry point is the `DataFlowTemplate` class in the `org.springframework.cloud.dataflow.rest.client` package.

This class implements the `DataFlowOperations` interface and delegates to the following sub-templates that provide the specific functionality for each feature-set:

Interface	Description
StreamOperations	REST client for stream operations
CounterOperations	REST client for counter operations
FieldValueCounterOperations	REST client for field value counter operations
AggregateCounterOperations	REST client for aggregate counter operations

Interface	Description
TaskOperations	REST client for task operations
JobOperations	REST client for job operations
AppRegistryOperations	REST client for app registry operations
CompletionOperations	REST client for completion operations
RuntimeOperations	REST Client for runtime operations

When the `DataFlowTemplate` is being initialized, the sub-templates can be discovered through the REST relations, which are provided by HATEOAS.^[1]

If a resource cannot be resolved, the respective sub-template results in NULL. A common cause is that Spring Cloud Data Flow offers for specific sets of features to be enabled/disabled when launching. For more information, see [local](#), [Cloud Foundry](#), [Kubernetes](#).

A.1. Using the Data Flow Template

When you use the Data Flow Template, the only needed Data Flow dependency is the Spring Cloud Data Flow Rest Client, as shown in the following Maven snippet:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-dataflow-rest-client</artifactId>
  <version>2.4.0.RELEASE</version>
</dependency>
```

XML

With that dependency, you get the `DataFlowTemplate` class as well as all the dependencies needed to make calls to a Spring Cloud Data Flow server.

When instantiating the `DataFlowTemplate`, you also pass in a `RestTemplate`. Please be aware that the needed `RestTemplate` requires some additional configuration to be valid in the context of the `DataFlowTemplate`. When declaring a `RestTemplate` as a bean, the following configuration suffices:

JAVA

```
@Bean
public static RestTemplate restTemplate() {
    RestTemplate restTemplate = new RestTemplate();
    restTemplate.setErrorHandler(new VndErrorResponseErrorHandler(restTemplate.getMessageConverters()));
    for (HttpMessageConverter<> converter : restTemplate.getMessageConverters()) {
        if (converter instanceof MappingJackson2HttpMessageConverter) {
            final MappingJackson2HttpMessageConverter jacksonConverter =
                (MappingJackson2HttpMessageConverter) converter;
            jacksonConverter.getObjectMapper()
                .registerModule(new Jackson2HalModule())
                .addMixIn(JobExecution.class, JobExecutionJacksonMixIn.class)
                .addMixIn(JobParameters.class, JobParametersJacksonMixIn.class)
                .addMixIn(JobParameter.class, JobParameterJacksonMixIn.class)
                .addMixIn(JobInstance.class, JobInstanceJacksonMixIn.class)
                .addMixIn(ExitStatus.class, ExitStatusJacksonMixIn.class)
        }
    }
}
```

```

        .addMixIn(StepExecution.class, StepExecutionJacksonMixIn.class)
        .addMixIn(ExecutionContext.class, ExecutionContextJacksonMixIn.class)
        .addMixIn(StepExecutionHistory.class, StepExecutionHistoryJacksonMixIn.class);
    }
}
return restTemplate;
}

```

You can also get a pre-configured `RestTemplate` using `DataFlowTemplate.getDefaultDataflowRestTemplate()`;

Now you can instantiate the `DataFlowTemplate` with the following code:

```

DataFlowTemplate dataFlowTemplate = new DataFlowTemplate(
    new URI("http://localhost:9393/"), restTemplate);
1

```

JAVA

- 1 The `URI` points to the ROOT of your Spring Cloud Data Flow Server.

Depending on your requirements, you can now make calls to the server. For instance, if you want to get a list of currently available applications you can run the following code:

```

PagedResources<AppRegistrationResource> apps = dataFlowTemplate.appRegistryOperations().list();

System.out.println(String.format("Retrieved %s application(s)",
    apps.getContent().size()));

for (AppRegistrationResource app : apps.getContent()) {
    System.out.println(String.format("App Name: %s, App Type: %s, App URI: %s",
        app.getName(),
        app.getType(),
        app.getUri()));
}

```

JAVA

A.2. Data Flow Template and Security

When using the `DataFlowTemplate`, you can also provide all the security-related options similar to as if you were using the *Data Flow Shell*. In fact the *Data Flow Shell* uses the `DataFlowTemplate` for all its operations.

In order to get started, we provide a `HttpClientConfigurer` that uses the builder pattern to set the various security-related options:

```

HttpClientConfigurer
    .create(targetUri)
    .basicAuthCredentials(username, password)
    .skipTlsCertificateVerification()
    .withProxyCredentials(proxyUri, proxyUsername, proxyPassword)
    .addInterceptor(interceptor)
    .buildClientHttpRequestFactory()
1
2
3
4
5
6

```

JAVA

- 1 Creates a `HttpClientConfigurer` with the provided target URI.
- 2 Sets the credentials for basic authentication (Using OAuth2 Password Grant)
- 3 Skip SSL certificate verification (**Use for DEVELOPMENT ONLY!**)

- 4 Configure any Proxy settings
- 5 Add a custom interceptor e.g. to set the OAuth2 Authorization header. This allows you to pass an OAuth2 Access Token instead of username/password credentials.
- 6 Builds the `ClientHttpRequestFactory` that can be set on the `RestTemplate`.

Once the `HttpClientConfigurer` is configured, its `buildClientHttpRequestFactory` can be used to build the `ClientHttpRequestFactory` and you would set the corresponding property on the `RestTemplate`. You can then instantiate the actual `DataFlowTemplate` using that `RestTemplate`.

In order to configure *Basic Authentication*, the following setup is required:

```
RestTemplate restTemplate = DataFlowTemplate.getDefaultDataflowRestTemplate();
HttpClientConfigurer httpClientConfigurer = HttpClientConfigurer.create("http://localhost:9393");

httpClientConfigurer.basicAuthCredentials("my_username", "my_password");
restTemplate.setRequestFactory(httpClientConfigurer.buildClientHttpRequestFactory());

DataFlowTemplate dataFlowTemplate = new DataFlowTemplate("http://localhost:9393", restTemplate);
```

JAVA

You can find a sample application as part of the [spring-cloud-dataflow-samples](#) on GitHub.

Appendix B: “How-to” guides

This section provides answers to some common ‘how do I do that...’ type of questions that often arise when using Spring Cloud Data Flow.

If you are having a specific problem that we do not cover here, you might want to check out [stackoverflow.com](#) to see if someone has already provided an answer. That is also a great place to ask new questions (please use the `spring-cloud-dataflow` tag).

We are also more than happy to extend this section. If you want to add a “how-to”, you can send us a [pull request](#).

B.1. Configure Maven Properties

You can set the maven properties such as local maven repository location, remote maven repositories, authentication credentials, and proxy server properties through command line properties when starting the Data Flow server. Alternatively, you can set the properties using `SPRING_APPLICATION_JSON` environment property for the Data Flow server.

The remote maven repositories need to be configured explicitly if the apps are resolved using maven repository, except for a `local` Data Flow server. The other Data Flow server implementations (that use maven resources for app artifacts resolution) have no default value for remote repositories. The `local` server has [repo.spring.io/libs-snapshot](#) as the default remote repository.

To pass the properties as commandline options, run the server with a command similar to the following:

```
$ java -jar <dataflow-server>.jar --maven.localRepository=mylocal
--maven.remote-repositories.repo1.url=https://repo1
--maven.remote-repositories.repo1.auth.username=repo1user
--maven.remote-repositories.repo1.auth.password=repo1pass
--maven.remote-repositories.repo2.url=https://repo2 --maven.proxy.host=proxyhost
```

BASH

```
--maven.proxy.port=9018 --maven.proxy.auth.username=proxyuser
--maven.proxy.auth.password=proxypass
```

You can also use the `SPRING_APPLICATION_JSON` environment property:

```
export SPRING_APPLICATION_JSON='{ "maven": { "local-repository": "local", "remote-repositories": { "repo1": { "url": "https:
"repo2": { "url": "https://repo2" } }, "proxy": { "host": "proxyhost", "port": 9018, "auth": { "username": "proxyuser", "pa
```

Here is the same content in nicely formatted JSON:

```
SPRING_APPLICATION_JSON='{
  "maven": {
    "local-repository": "local",
    "remote-repositories": {
      "repo1": {
        "url": "https://repo1",
        "auth": {
          "username": "repo1user",
          "password": "repo1pass"
        }
      },
      "repo2": {
        "url": "https://repo2"
      }
    }
  },
  "proxy": {
    "host": "proxyhost",
    "port": 9018,
    "auth": {
      "username": "proxyuser",
      "password": "proxypass"
    }
  }
}
```

JSON

Depending on the Spring Cloud Data Flow server implementation, you may have to pass the environment properties by using the platform specific environment-setting capabilities. For instance, in Cloud Foundry, you would pass them as `cf set-env SPRING_APPLICATION_JSON`.

B.2. Troubleshooting

This section covers how to troubleshoot Spring Cloud Data Flow on your platform of choice. Please visit the Troubleshooting sections of the microsite for [Stream](#) and [Batch](#) processing.

B.3. Frequently Asked Questions

In this section, we review the frequently asked questions in Spring Cloud Data Flow. Please see the [Frequently Asked Questions](#) section of the microsite for more information.

Appendix C: Building

To build the source, you need to install JDK 1.8.

The build uses the Maven wrapper so that you do not have to install a specific version of Maven.

The main build command is as follows:

```
$ ./mvnw clean install
```

If you like, you can add '-DskipTests' to avoid running the tests.

You can also install Maven (>=3.3.3) yourself and run the `mvn` command in place of `./mvnw` in the examples below. If you do that, you also might need to add `-P spring` if your local Maven settings do not contain repository declarations for Spring pre-release artifacts.

You might need to increase the amount of memory available to Maven by setting a `MAVEN_OPTS` environment variable with a value similar to `-Xmx512m -XX:MaxPermSize=128m`. We try to cover this in the `.mvn` configuration, so, if you find you have to do it to make a build succeed, please raise a ticket to get the settings added to source control.

C.1. Documentation

There is a `full` profile that generates documentation. You can build only the documentation by using the following command:

```
$ ./mvnw clean package -DskipTests -P full -pl spring-cloud-dataflow-docs -am
```

C.2. Working with the Code

If you do not have an IDE preference, we recommend that you use [Spring Tools Suite](#) or [Eclipse](#) when working with the code. We use the [m2eclipse](#) Eclipse plugin for Maven support. Other IDEs and tools generally also work without issue.

C.2.1. Importing into Eclipse with m2eclipse

We recommend the [m2eclipse](#) eclipse plugin when working with Eclipse. If you do not already have m2eclipse installed, it is available from the Eclipse marketplace.

Unfortunately, m2e does not yet support Maven 3.3. Consequently, once the projects are imported into Eclipse, you also need to tell m2eclipse to use the `.settings.xml` file for the projects. If you do not do this, you may see many different errors related to the POMs in the projects. To do so:

1. Open your Eclipse preferences.
2. Expand the Maven preferences.
3. Select User Settings.

4. In the User Settings field, click Browse and navigate to the Spring Cloud project you imported.
5. Select the `.settings.xml` file in that project.
6. Click Apply.
7. Click OK.

Alternatively, you can copy the repository settings from Spring Cloud's `.settings.xml` file into your own `~/.m2/settings.xml`.

C.2.2. Importing into Eclipse without m2eclipse

If you prefer not to use m2eclipse, you can generate Eclipse project metadata by using the following command:

```
$ ./mvnw eclipse:eclipse
```

The generated Eclipse projects can be imported by selecting `Import existing projects` from the `File` menu.

Appendix D: Contributing

Spring Cloud is released under the non-restrictive Apache 2.0 license and follows a very standard Github development process, using Github tracker for issues and merging pull requests into master. If you want to contribute even something trivial, please do not hesitate, but follow the guidelines below.

D.1. Sign the Contributor License Agreement

Before we accept a non-trivial patch or pull request, we need you to sign the [contributor's agreement](#). Signing the contributor's agreement does not grant anyone commit rights to the main repository, but it does mean that we can accept your contributions, and you will get an author credit if we do. Active contributors might be asked to join the core team and be given the ability to merge pull requests.

D.2. Code Conventions and Housekeeping

None of the following guidelines is essential for a pull request, but they all help your fellow developers understand and work with your code. They can also be added after the original pull request but before a merge.

- Use the Spring Framework code format conventions. If you use Eclipse, you can import formatter settings by using the `eclipse-code-formatter.xml` file from the [Spring Cloud Build](#) project. If using IntelliJ, you can use the [Eclipse Code Formatter Plugin](#) to import the same file.
- Make sure all new `.java` files have a simple Javadoc class comment with at least an `@author` tag identifying you, and preferably at least a paragraph describing the class's purpose.
- Add the ASF license header comment to all new `.java` files (to do so, copy from existing files in the project).
- Add yourself as an `@author` to the `.java` files that you modify substantially (more than cosmetic changes).

- Add some Javadocs and, if you change the namespace, some XSD doc elements.
- A few unit tests would help a lot as well. Someone has to do it, and your fellow developers appreciate the effort.
- If no one else uses your branch, rebase it against the current master (or other target branch in the main project).
- When writing a commit message, follow [these conventions](#). If you fix an existing issue, add `Fixes gh-XXXX` (where XXXX is the issue number) at the end of the commit message.

1. HATEOAS stands for Hypermedia as the Engine of Application State

Last updated 2020-01-31 12:19:38 UTC